Spectral matching, step by step

A slow textbook on detecting and naming sub-pixel materials
in the Allotrope residual

Companion to `research/math/spectral-matching/spectral_matching.tex`

Allotrope research note, teaching edition

29 October 2026; Part VII added 31 October 2026

Contents

How to read this book ix

I Foundations 1

1 Spectra as vectors 2

1.1 The problem: a pixel is not a colour . . . 2
1.2 A three-band world . . . 3
1.3 Pictures: a curve and an arrow . . . 3
1.4 Adding vectors and scaling them . . . 4
1.5 Mixing: the line between two spectra . . . 4
1.6 The difference vector . . . 5

2 The dot product and length 7

2.1 The problem: you need numbers, not lists . . . 7
2.2 The summation sign . . . 7
2.3 The dot product . . . 7
2.4 Length . . . 8
2.5 Expanding the squared length of a sum . . . 9
2.6 Mean absolute value and root mean square . . . 9

3 Angles and the spectral angle mapper 12

3.1 The problem: brightness is a nuisance . . . 12
3.2 Cosine and sine . . . 12
3.3 The angle between two vectors . . . 13
3.4 The spectral angle mapper (SAM) . . . 13
3.5 Angles in radians, and small angles . . . 14

4 Matrices as machines 16

4.1 The problem: you need to transform spectra . . . 16
4.2 What a matrix is . . . 16
4.3 Chaining machines: the matrix product . . . 17
4.4 The transpose . . . 17
4.5 The inverse: undoing a machine . . . 18

4.6 Quadratic forms . . . 19

5 Mean, variance and covariance from a tiny data set 21
5.1 The problem: what counts as unusual? . . . 21
5.2 A tiny data set . . . 21
5.3 Covariance: wobbling together . . . 22
5.4 The covariance matrix as a sum of outer products . . . 23
5.5 The wobble of a weighted sum . . . 23
5.6 Samples and populations . . . 24

6 Eigenvectors and principal components by hand 26
6.1 The problem: which directions matter? . . . 26
6.2 Directions a matrix only stretches . . . 26
6.3 Finding eigenvalues by hand . . . 26
6.4 Eigenvalues are variances . . . 27
6.5 Taking the matrix apart . . . 28
6.6 Principal components and energy shares . . . 28
6.7 An uncentred version, for library families . . . 28

7 Whitening and the Mahalanobis distance 31
7.1 The problem: a fair yardstick for surprise . . . 31
7.2 One band first: standard scores . . . 31
7.3 Whitening in two steps . . . 31
7.4 The Mahalanobis distance . . . 33
7.5 A new price list: the S^{-1} inner product . . . 33
7.6 Throwing directions away, and the ridge . . . 34

8 Slopes and minima: the calculus you need 36
8.1 The problem: “best” means “smallest error” . . . 36
8.2 Slope and the derivative . . . 36
8.3 Minimum where the slope is zero . . . 37
8.4 Two or more unknowns: the gradient . . . 38
8.5 Maximising over a variance . . . 38

9 Projections onto a line and onto a subspace 41
9.1 The problem: how much of the residual looks like the target? . . . 41
9.2 Projection onto a line . . . 41
9.3 The projection as a matrix . . . 42
9.4 Subspaces . . . 43
9.5 Projection onto a subspace . . . 43
9.6 Projecting in the whitened metric . . . 44
9.7 The angle between two subspaces . . . 44

10 Probability, the Gaussian, likelihood and the logarithm **46**
10.1 The problem: noise is random . . . 46
10.2 Probability, random variables, expectation . . . 46
10.3 Densities and areas . . . 47
10.4 The Gaussian . . . 47
10.5 The average size of a Gaussian wobble . . . 48
10.6 Many bands: the multivariate Gaussian . . . 48
10.7 The logarithm . . . 49
10.8 Likelihood . . . 49

11 Hypothesis tests, thresholds, false alarms and detections **51**
11.1 The problem: every rule makes two kinds of mistake . . . 51
11.2 The parts of a test . . . 51
11.3 Required signal-to-noise ratio . . . 52
11.4 Thresholds by percentile, and false alarms per scene . . . 52
11.5 The ROC curve and pAUC . . . 53
11.6 Likelihood ratios and the generalised likelihood ratio test . . . 53
11.7 The chi-square family . . . 54

II The problem **56**

12 The mixing model and the fill **57**
12.1 The problem: targets are smaller than pixels . . . 57
12.2 The model . . . 57
12.3 The difference form . . . 58
12.4 What you know and what you do not . . . 58
12.5 The fill is not the area fraction . . . 58

13 The point-spread function **60**
13.1 The problem: pixels are not sharp squares . . . 60
13.2 One dimension first . . . 60
13.3 Two dimensions: multiply . . . 61
13.4 Width: standard deviation and FWHM . . . 61
13.5 The project's implant weights . . . 62

14 Resampling the library to a sensor **64**
14.1 The problem: the library and the sensor speak different languages . . . 64
14.2 The spectral response of a band . . . 64
14.3 Resampling as a matrix, and why it commutes with mixing . . . 65
14.4 What if the band is not where you think? . . . 65

15 Normalisation by wavelength **67**

15.1 The problem: one set of units for everyone . . . 67
15.2 The normalisation . . . 67
15.3 Mixing survives normalisation . . . 68
15.4 Why per-scene min-max scaling fails . . . 68
15.5 Noise in normalised units . . . 69

III The learned background 70

16 The residual 71
16.1 The problem: the background is unknown . . . 71
16.2 How Indradhanu predicts the background . . . 71
16.3 The residual and the model's error . . . 72
16.4 Clutter-limited, not noise-limited . . . 73

17 Residual Mahalanobis: surprise in any direction 75
17.1 The problem: one number per pixel . . . 75
17.2 The recipe . . . 75
17.3 Worked on three bands . . . 76
17.4 RM is a likelihood-ratio test . . . 76
17.5 The price of not knowing the direction . . . 77
17.6 What RM does not see . . . 77

IV Matching on the residual 79

18 The fill estimate 80
18.1 The problem: how much metal is in this pixel? . . . 80
18.2 The noise model . . . 80
18.3 Deriving the estimate, one step at a time . . . 80
18.4 Is it right on average? Unbiasedness . . . 82
18.5 How precise is it? The variance . . . 82
18.6 Could any estimate do better? The Cramér-Rao bound . . . 82
18.7 Keeping the fill between 0 and 1 . . . 83
18.8 A wrong library spectrum . . . 83

19 The matched filter and the smallest detectable fill 86
19.1 The problem: is the material there? . . . 86
19.2 The statistic . . . 86
19.3 Detection rate and the smallest detectable fill . . . 87
19.4 A real example: galvanised metal over grass . . . 87
19.5 The reality check . . . 88

20 ACE: matching the shape, ignoring the scale **91**
20.1 The problem: the clutter's loudness varies . . . 91
20.2 The model with unknown scale . . . 91
20.3 Deriving the GLRT, one step at a time . . . 91
20.4 Why the scale drops out . . . 92
20.5 How ACE behaves on background . . . 93
20.6 The shipped matcher . . . 93

21 Families as subspaces **95**
21.1 The problem: one material, many spectra . . . 95
21.2 How many directions does a family need? . . . 95
21.3 The subspace detector . . . 96
21.4 Explained energy, unexplained energy and the projection angle . . . 96
21.5 Why raw family scores cannot be compared . . . 96
21.6 How far apart are families? . . . 96

22 Nuisances and the family cone **99**
22.1 The problem: the target is not exactly the library . . . 99
22.2 Matched subspace detection with interference . . . 99
22.3 Brightness and haze are a known interference subspace . . . 100
22.4 Simplex ACE: the family as a cone . . . 101

23 Why SAM fails on mixed pixels **103**
23.1 The problem . . . 103
23.2 The geometry of a two-component mixture . . . 103
23.3 The real example: galvanised metal over grass and soil . . . 104
23.4 The fix: match the residual . . . 105

V What goes wrong **107**

24 Leakage **108**
24.1 The problem: the painter can peek . . . 108
24.2 Scalar leakage: the model . . . 108
24.3 Three consequences . . . 109
24.4 Leakage as a matrix . . . 109
24.5 The 40% halo, re-derived . . . 110
24.6 From halo exposure to a leakage fraction . . . 111
24.7 When do wider masks pay? . . . 111
24.8 A part of the residual that cannot leak . . . 112
24.9 The classical ring leaks too . . . 113

25 Atmospheric gain **114**

25.1 The problem: reflectance is itself an estimate . . . 114
25.2 The offset cancels; the gain does not . . . 114
25.3 How far can a gain turn a vector? Two bands first . . . 114
25.4 How big are the gains we actually have? . . . 116

26 Why the Gaussian theory is too optimistic 118
26.1 The problem: theory and measurement disagree . . . 118
26.2 Reason 1: heavy tails . . . 118
26.3 Reason 2: the error is largest where targets are . . . 119
26.4 Reason 3: real objects hide in the error's own directions . . . 119
26.5 Reason 4: leakage and spread . . . 119
26.6 The largest error of all: the library is not the material . . . 119
26.7 The decoder's output floor . . . 119
26.8 All the error sources in one table . . . 120

VI Deciding and measuring 122

27 Abstention and per-scene calibration 123
27.1 The problem: when to say "I don't know" . . . 123
27.2 The ideal: posterior probabilities . . . 123
27.3 Per-scene calibration . . . 124
27.4 The false-name bound, derived . . . 124
27.5 How precise is the bar? . . . 125
27.6 The bound says nothing about false alarms . . . 125
27.7 Why a fixed absolute threshold fails . . . 126
27.8 Open-set naming: three tests . . . 126

28 The learned namer 128
28.1 The problem: could a network do the naming? . . . 128
28.2 The generator . . . 128
28.3 What cross-entropy training learns . . . 129
28.4 What the input must contain . . . 129
28.5 The gap between the generator and reality . . . 129

29 The benchmark and the paired bootstrap 132
29.1 The problem: how do you know a namer is better? . . . 132
29.2 An OSM-labelled benchmark . . . 132
29.3 Wrong labels shrink differences . . . 133
29.4 The paired bootstrap . . . 133

30 The minimum detectable improvement 136
30.1 The problem: planning a benchmark . . . 136

30.2 The per-object difference . . . 136
30.3 From standard error to the smallest detectable difference . . . 136
30.4 Objects from the same scene are not independent . . . 137
30.5 Where to go from here . . . 138

VII Natural first, then the class 140

31 Two questions instead of one 141
31.1 Why the families had to change . . . 141
31.2 The two levels . . . 141
31.3 Conditional probability, and the product rule . . . 142
31.4 From probabilities to the three answers . . . 142
31.5 The identity on measured counts . . . 143

32 Level 1 as an open set: not explained by nature 145
32.1 The problem: the man-made side of any library is incomplete . . . 145
32.2 Near the average is not the same as explained . . . 145
32.3 Setting the threshold without man-made examples . . . 146
32.4 What the separability check measures, and what it does not . . . 147

33 Matching instead of classifying: the dual encoder 149
33.1 The question . . . 149
33.2 A closed classifier . . . 149
33.3 A matching model . . . 149
33.4 ACE is a matching model . . . 150
33.5 Why the fill drops out . . . 150
33.6 Leave one spectrum out . . . 151

34 Training the matcher: the contrastive objective 153
34.1 The problem . . . 153
34.2 Softmax with a temperature . . . 153
34.3 The gradient: probability minus truth . . . 154
34.4 A batch: every other spectrum is a negative . . . 155
34.5 What carries over from Chapter 28 . . . 155
34.6 Where Part VII leaves the design . . . 156

35 The namer as built 157
35.1 What was built, in one table . . . 157
35.2 The model as two functions . . . 158
35.2.1 Counting the parameters as a formula . . . 158
35.3 The stem ϕ_θ, step by step . . . 160
35.4 The class score . . . 162

35.5 The gradient of the class loss . . . 163
35.6 Level 1 by matching . . . 164
35.6.1 The evidence that forced the change . . . 164
35.6.2 The rule . . . 164
35.6.3 The gradient through the logsumexp . . . 165
35.6.4 Why matching helps: three reasons and an offset . . . 166
35.6.5 What did not change . . . 167
35.7 The generator as a sampling law . . . 167
35.8 One training step, as shapes . . . 168
35.9 The split as a coin, and the open-set class . . . 169
35.10 The pass test and inference . . . 169
35.11 Where Part VII leaves the namer . . . 171

Glossary **172**

Symbol table **178**

Index of worked examples **182**

Annotated bibliography **183**

Index of defined terms **185**

How to read this book

Who this is for

This book teaches the mathematics behind one question: *a pixel has been flagged as unusual; what material is it?* The original research note, `research/math/spectral-matching/spectral_matching.tex`, answers that question in twenty dense pages written for a specialist. This book covers the same ground for a reader who knows school arithmetic and not much more, and who wants to go slowly.

You need to be able to add, subtract, multiply and divide decimals, work with fractions, and take a square root on a calculator. Everything else is built here, from the start: vectors, matrices, covariance, eigenvectors, probability, the Gaussian curve, logarithms, derivatives, hypothesis tests. Nothing is assumed because it is "standard".

How each chapter is built

Every chapter has the same shape, so you always know where you are.

1. A blue box, *The one thing this chapter teaches*, in one sentence. If you remember nothing else, remember that.
2. The explanation. It starts with the problem (why you need the idea) before the mechanism (how it works).
3. Worked examples in light-blue boxes, with every arithmetic step on its own line. They use two or three bands and 2×2 matrices, small enough to do on paper in a few minutes. **Do them on paper.** Reading arithmetic is not the same as doing it, and the later chapters assume your hands know the earlier ones.
4. One analogy, in a green box, that does real work.
5. *Common confusions*: mistakes people actually make, as questions with answers.
6. *Check yourself*: about five questions, at least one needing hand arithmetic, followed by full worked answers. Try them before reading the answers.

When an algebra step uses a rule (for example "the transpose of a product reverses the order"), the rule is stated in a yellow box and proved on a 2×2 example the first time it appears. After that it is used by name.

Where the numbers come from

The book uses two kinds of numbers, and you should never confuse them.

Toy numbers are invented to make the arithmetic easy: a "grass" spectrum of (0.1, 0.1, 0.5) in three bands, a 2×2 covariance of $\left(\begin{smallmatrix} 2.5 & 2 \\ 2 & 2.5 \end{smallmatrix}\right)$. They teach the method. They are not measurements of anything.

Project numbers come from the Allotrope repository and carry the same three tags as the original note:

- [M] **measured** in a project experiment; the path of its `README.md` or output file is given;
- [C] **computed** for the note from the shipped spectral library and constants, by `compute_numbers.py` (output in `compute_numbers_output.txt`), or, for the few extra numbers the step-by-step derivations need, by `compute_step_by_step.py` (output in `compute_step_by_step_output.txt`); both scripts sit next to the note;
- [A] **assumed**: a modelling choice, not a fact about the world.

Every project number in this book matches the original note or one of the two output files. The new script also re-checks the toy examples the derivations build on; the remaining exercise arithmetic was checked separately by calculator.

What this book is not

It is study material. The authoritative sources are, in order: the code, the original note, and the low-level design `docs/lld/spectral-family-match.md`. If this book and the code disagree, the code wins. Some values quoted here will drift as the project moves: the measured experiment results (pAUC, objects caught, decoder floors), the code constants (the 30 kept components, the 10,000-pixel null sample, the ring sizes) and the library counts (411 spectra). Chapters 16, 17, 19, 24, 26 and 27 lean on such values; re-check them against the cited files before relying on them.

The chapters

	Chapter	What it is for
Part I. Foundations		
1	Spectra as vectors	A pixel's spectrum is a list of numbers you can add and scale.
2	The dot product and length	One number for "how much two spectra agree" and one for "how big".
3	Angles and the spectral angle mapper	Comparing spectra by direction, ignoring brightness.
4	Matrices as machines	A matrix turns one vector into another; transpose and inverse.
5	Mean, variance and covariance	Measuring how background residuals usually wobble, band by band and together.
6	Eigenvectors and principal components	The long and short axes of a data cloud, found by hand.
7	Whitening and the Mahalanobis distance	Measuring surprise in units of the usual wobble.
8	Slopes and minima	The calculus needed to find a best estimate.
9	Projections	Splitting a vector into "looks like the target" and "the rest".
10	Probability, the Gaussian, likelihood	How likely a residual is, and which explanation fits best.
11	Hypothesis tests	Thresholds, false alarms, detections and the chi-square curve.
Part II. The problem		
12	The mixing model and the fill	A target smaller than a pixel mixes with its background.
13	The point-spread function	One object spreads its light over a neighbourhood of pixels.
14	Resampling the library	Turning a lab spectrum into what the sensor's bands would see.
15	Normalisation by wavelength	Putting image and library into the same units without breaking mixing.
Part III. The learned background		
16	The residual	Indradhanu predicts the background; the residual is what it cannot explain.

17	Residual Mahalanobis	The product's anomaly score: surprise in any direction.
Part IV. Matching on the residual		
18	The fill estimate	How much of a known material is in the pixel, and how precisely.
19	The matched filter	Detecting a known material, and the smallest fill you can detect.
20	ACE	Matching shape while ignoring the overall scale of the clutter.
21	Families as subspaces	A material family is a few directions; how far apart families are.
22	Nuisances and the family cone	Brightness, offset and mixtures inside a family.
23	Why SAM fails on mixed pixels	The galvanised-metal-over-grass example, re-derived.
Part V. What goes wrong		
24	Leakage	The background prediction partly copies the target; the 40% halo, re-derived.
25	Atmospheric gain	A gain error rotates the target; the 2.7° bound, re-derived.
26	Why the Gaussian theory is too optimistic	Tails, uneven clutter, alignment, and the size of each error.
Part VI. Deciding and measuring		
27	Abstention and per-scene calibration	When to name a family and when to say "unconfirmed".
28	The learned namer	What a trained classifier learns, and how its world differs from ours.
29	The benchmark and the paired bootstrap	Measuring naming accuracy and its uncertainty.
30	The minimum detectable improvement	How many objects a benchmark needs.
Part VII. Natural first, then the class (added 31 October 2026)		
31	Two questions instead of one	Natural or man-made, then which class; $P(k) = P(M)\,P(k \mid M)$.
32	Level 1 as an open set	Calling man-made whatever nature cannot explain; the separability check.
33	Matching instead of classifying	The dual encoder; why a new library needs no retraining; why the fill drops out.
34	Training the matcher	Softmax, temperature, the contrastive loss and its gradient.
35	The namer as built	Two towers, the stem, class scores, level 1 by matching and its gradient, the sampling law.
	Glossary, symbol table, index of worked examples	Look-ups.

How the chapters depend on each other

The book is strictly linear: chapter N uses only chapters 1 to $N-1$. If you already know some of the background, these are the safe shortcuts.

- Chapters 1 to 4 (vectors, dot products, angles, matrices) are needed by everything. Skip them only if you can invert a 2×2 matrix and explain why $(\mathbf{AB})^\top = \mathbf{B}^\top\mathbf{A}^\top$.
- Chapters 5 to 7 (covariance, eigenvectors, whitening) are needed from Chapter 17 on. Chapter 8 (calculus) and chapter 9 (projections) are needed from Chapter 18 on. Chapters 10 and 11 (probability, tests) are needed from Chapter 17 on.
- Part II (chapters 12 to 15) is the physics; Part III (16, 17) the learned background. Both are needed for Part IV.
- Part IV is the core. Chapter 18 (the fill estimate) is the derivation every later chapter reuses; read it slowly.
- Parts V and VI can be read in either order after Part IV, except that Chapter 26 refers back to Chapters 24 and 25.

- Part VII (chapters 31 to 35) is a design discussed on 31 October 2026 (chapters 31 to 34) and the namer built from it the same day (chapter 35). It needs ACE (Chapter 20), the family angles (Chapter 21), abstention (Chapter 27) and the learned namer (Chapter 28).

One picture of the whole problem

Here is the pipeline the book walks through, with the chapter where each step is explained.

```
satellite pixel x  (165 numbers, ch.1, 15)
       |
       |   Indradhanu predicts the background b_hat from the pixel's neighbours (ch.16)
       v
residual r = x - b_hat            (what the model could not explain)
       |
       |-- RM: is r unusual in ANY direction?  -> anomaly flag      (ch.17)
       |
       |   for each library material t (resampled, normalised: ch.14, 15)
       |-- fill estimate, matched filter, ACE against d = t - b_hat  (ch.18-20)
       |-- family subspaces, nuisances, cone                          (ch.21-22)
       v
per-family scores  --> calibrate against this scene's ordinary pixels (ch.27)
       |
       v
name a family, or say "man-made, type unconfirmed"
       |
  what can go wrong: leakage (ch.24), gain (ch.25), heavy tails (ch.26)
  how we measure it: benchmark, bootstrap, sample size (ch.29-30)
  the redesign: natural first, then the class; matching (ch.31-34)
```

Part I

Foundations

Chapter 1

Spectra as vectors

The one thing this chapter teaches

A pixel's spectrum is an ordered list of numbers, one per band, and you can add such lists and multiply them by a number one entry at a time; that is all a "vector" is.

1.1 The problem: a pixel is not a colour

Your phone camera records three numbers per pixel: how much red, green and blue light arrived. A hyperspectral satellite such as PRISMA or EnMAP records a few hundred numbers per pixel, each for a narrow slice of the spectrum. The question this book is about, "what material is in this pixel?", is a question about those few hundred numbers. Before you can ask it you need a way to write them down, compare them and combine them.

Some words first.

Definition: wavelength, band, reflectance, spectrum

Light is described by its **wavelength**, measured in nanometres (nm; a nanometre is a millionth of a millimetre). Blue light is about 450 nm, red about 650 nm. Wavelengths from about 700 nm to 2500 nm are invisible to your eye but the satellite sees them; they are called near-infrared and short-wave infrared.

A **band** is one narrow slice of wavelength that the sensor measures, for example "550 nm, about 10 nm wide".

Reflectance is the fraction of the sunlight arriving at the ground that a surface sends back, in one band. It is a number between 0 (reflects nothing) and 1 (reflects everything). Fresh grass reflects about 0.05 in red light and about 0.5 in near-infrared light.

A **spectrum** is the list of a surface's reflectances, one per band, in band order.

The Allotrope product puts every sensor onto a *common grid* of 165 bands, 10 nm apart, from 460 nm to 2450 nm, with the wavelengths where the atmosphere absorbs almost everything (water vapour and carbon dioxide windows) left out (`app/models/dataset/vendables.py`, `build_common_wavelength_grid`). The sensors' own band counts differ: PRISMA has 234 bands in the training shards, EnMAP 224, AVIRIS-NG 372 good bands. So a project spectrum is a list of 165 numbers (or 224, or 234). That is too many to write by hand, so this book works with three.

1.2 A three-band world

Throughout the book you will meet the same three toy materials. Each has three bands, which you can think of as green, red and near-infrared. The numbers are invented to keep the arithmetic easy; they are roughly the right shape for the real materials.

name	symbol	band 1 (green)	band 2 (red)	band 3 (near-infrared)
grass	**g**	0.1	0.1	0.5
metal roof	**t**	0.3	0.3	0.3
soil	**s**	0.1	0.2	0.3

Grass is dark in green and red and bright in near-infrared (living leaves reflect near-infrared strongly). Metal is the same in every band: a flat grey. Soil rises slowly with wavelength.

Definition: vector, component, dimension

A **vector** is an ordered list of numbers. Each number is a **component**. The number of components is the vector's **dimension**, written N. Vectors are printed in bold: $\mathbf{g} = (0.1,\ 0.1,\ 0.5)$. The n-th component of $\mathbf{g}$ is written g_n (not bold, because it is a single number): $g_1 = 0.1$, $g_2 = 0.1$, $g_3 = 0.5$. The small number n is called the **index**.

"Ordered" matters. $(0.1,\ 0.1,\ 0.5)$ and $(0.5,\ 0.1,\ 0.1)$ are different vectors: the first is grass, the second would be something bright green and dark in near-infrared, which no healthy plant is.

1.3 Pictures: a curve and an arrow

You can draw a spectrum in two ways. The first is a curve: wavelength along the bottom, reflectance up the side. That is how spectra are usually shown. The second is an arrow. If you keep only two bands, say red and near-infrared, each spectrum is a point on a flat sheet, and you can draw an arrow from the origin $(0, 0)$ to it. Figure 1.1 does this for grass and metal.

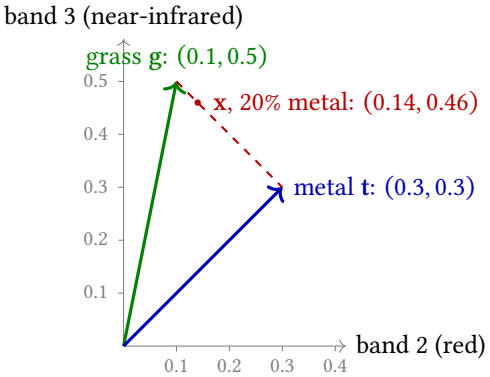


Figure 1.1: Two bands of the toy spectra drawn as arrows. The dashed segment is every possible mixture of grass and metal (Section 1.5); the dot is a pixel that is 20% metal and 80% grass.

The arrow picture is the one this book uses, because it turns questions about spectra into questions about geometry: how long is the arrow, which way does it point, how far apart are two arrow tips. With three bands the arrows live in three-dimensional space; with 165 bands, in 165-dimensional space. You cannot picture 165 dimensions and nobody can. The trick is that every calculation you do in two or three dimensions works unchanged in 165: the formulas only ever add up over the bands, and adding up 165 terms is no harder in principle than adding up three.

1.4 Adding vectors and scaling them

Two operations are allowed on vectors, and both work one component at a time.

Definition: vector addition and scalar multiplication

Addition: $\mathbf{u} + \mathbf{v}$ is the vector whose n-th component is $u_n + v_n$.
Scalar multiplication: for a single number c (called a **scalar** to distinguish it from a vector), $c\,\mathbf{u}$ is the vector whose n-th component is $c\,u_n$.
Subtraction combines the two: $\mathbf{u} - \mathbf{v} = \mathbf{u} + (-1)\mathbf{v}$, so its n-th component is $u_n - v_n$.

Scaling has a physical meaning for spectra. A metal roof in shadow reflects less light in every band by about the same factor. If the shadow halves the light, the roof shows $0.5\,\mathbf{t}$. The *shape* of the spectrum is unchanged; only its brightness is.

Worked example 1.1: A metal roof in half shadow

Compute $0.5\,\mathbf{t}$ with $\mathbf{t} = (0.3,\ 0.3,\ 0.3)$. Multiply each component by 0.5:

$$\begin{aligned} \text{band 1:} &\quad 0.5 \times 0.3 = 0.15 \\ \text{band 2:} &\quad 0.5 \times 0.3 = 0.15 \\ \text{band 3:} &\quad 0.5 \times 0.3 = 0.15 \end{aligned}$$

So $0.5\,\mathbf{t} = (0.15,\ 0.15,\ 0.15)$: still flat grey, half as bright.

1.5 Mixing: the line between two spectra

A PRISMA pixel covers a square of ground 30 m on a side, 900 square metres. A small metal shed of 10 m by 10 m covers 100 of those 900 square metres, about 11%. The pixel does not see "a shed" or "grass"; it sees a blend. The simplest model of the blend says that each band's reflectance is the area-weighted average of the two materials' reflectances. You will meet this model properly in Chapter 12; here is the arithmetic.

Definition: linear combination

A **linear combination** of vectors $\mathbf{u}$ and $\mathbf{v}$ is $c_1\mathbf{u} + c_2\mathbf{v}$ for two numbers c_1, c_2. A **mixture** is a linear combination whose two numbers are a and $1 - a$ with a between 0 and 1. The number a is called the **fill**: the share of the pixel taken by the first material.

Worked example 1.2: A pixel that is 20% metal and 80% grass

Compute $\mathbf{x} = 0.2\,\mathbf{t} + 0.8\,\mathbf{g}$. First scale each vector, one component at a time.

$$\begin{aligned} 0.2\,\mathbf{t} &= (0.2 \times 0.3,\ 0.2 \times 0.3,\ 0.2 \times 0.3) = (0.06,\ 0.06,\ 0.06) \\ 0.8\,\mathbf{g} &= (0.8 \times 0.1,\ 0.8 \times 0.1,\ 0.8 \times 0.5) = (0.08,\ 0.08,\ 0.40) \end{aligned}$$

Then add, one component at a time.

$$\begin{aligned} \text{band 1:} &\quad 0.06 + 0.08 = 0.14 \\ \text{band 2:} &\quad 0.06 + 0.08 = 0.14 \\ \text{band 3:} &\quad 0.06 + 0.40 = 0.46 \end{aligned}$$

So $\mathbf{x} = (0.14,\ 0.14,\ 0.46)$. In Figure 1.1 (bands 2 and 3) it is the dot at $(0.14,\ 0.46)$, one fifth of the way along the dashed line from grass to metal.

As the fill a runs from 0 to 1, the mixture $a\,\mathbf{t} + (1-a)\,\mathbf{g}$ slides along the straight segment from $\mathbf{g}$ (at $a = 0$) to $\mathbf{t}$ (at $a = 1$). That segment is the **mixing line**. It is a straight line because each component is a straight-line function of a: band 3, for instance, is $0.3a + 0.5(1-a) = 0.5 - 0.2a$.

1.6 The difference vector

Subtracting one spectrum from another gives a vector that is *not* a spectrum: it can have negative components. It describes how the two differ, band by band.

Worked example 1.3: Metal minus grass, and the mixed pixel minus grass

Compute $\mathbf{t} - \mathbf{g}$:

$$\begin{aligned} \text{band 1:} \quad & 0.3 - 0.1 = 0.2 \\ \text{band 2:} \quad & 0.3 - 0.1 = 0.2 \\ \text{band 3:} \quad & 0.3 - 0.5 = -0.2 \end{aligned}$$

So $\mathbf{t} - \mathbf{g} = (0.2,\ 0.2,\ -0.2)$. Now compute $\mathbf{x} - \mathbf{g}$ for the mixed pixel of worked example 1.2:

$$\begin{aligned} \text{band 1:} \quad & 0.14 - 0.1 = 0.04 \\ \text{band 2:} \quad & 0.14 - 0.1 = 0.04 \\ \text{band 3:} \quad & 0.46 - 0.5 = -0.04 \end{aligned}$$

So $\mathbf{x} - \mathbf{g} = (0.04,\ 0.04,\ -0.04)$. Compare it with $0.2\,(\mathbf{t} - \mathbf{g}) = (0.2 \times 0.2,\ 0.2 \times 0.2,\ 0.2 \times (-0.2)) = (0.04,\ 0.04,\ -0.04)$. They are the same.

That last line is the seed of the whole book. **If you subtract the background from a mixed pixel, what is left is the fill times the difference between target and background.** Here is why, in symbols. Start from $\mathbf{x} = a\,\mathbf{t} + (1-a)\,\mathbf{g}$ and subtract $\mathbf{g}$ from both sides:

$$\begin{aligned} \mathbf{x} - \mathbf{g} &= a\,\mathbf{t} + (1-a)\,\mathbf{g} - \mathbf{g} && \\ &= a\,\mathbf{t} + \mathbf{g} - a\,\mathbf{g} - \mathbf{g} && \textit{multiply out } (1-a)\mathbf{g} = \mathbf{g} - a\mathbf{g} \\ &= a\,\mathbf{t} - a\,\mathbf{g} && \mathbf{g} - \mathbf{g} \textit{ cancels} \\ &= a\,(\mathbf{t} - \mathbf{g}) && \textit{take out the common factor } a \end{aligned}$$

Each line uses only the component-by-component rules above, so each holds in every band at once.

Analogy: a recipe card

A spectrum is like a recipe card that lists quantities of each ingredient in a fixed order: flour, sugar, butter. Doubling a recipe doubles every quantity (scalar multiplication). If you blend two doughs, 20% of dough A and 80% of dough B, each ingredient in the blend is 20% of A's amount plus 80% of B's amount, ingredient by ingredient (a mixture). And the "difference" between two recipes, "50 g more flour, 20 g less butter", is a perfectly useful list even though you could not bake it (negative components).

Common confusions

Is the curve the vector? The curve is a drawing of the vector's components against wavelength. The vector is the list. Two different pictures, one object.

Does n count pixels or bands? In this book a subscript n always counts bands, from 1 to N. Pixels get the letter p (as in $\mathbf{x}_p$, the spectrum at pixel p), introduced in Chapter 12.

Is $0.5\,\mathbf{t}$ "half a roof"? No. It is the whole roof at half the brightness. "Half the pixel is roof" is the mixture $0.5\,\mathbf{t} + 0.5\,\mathbf{b}$ with some background $\mathbf{b}$. Both reduce the roof's contribution and you will see

in Chapter 22 that from one pixel you often cannot tell them apart.
Can a difference vector be a spectrum? Not usually. A spectrum's components are reflectances between 0 and 1. A difference such as $(0.2,\ 0.2,\ -0.2)$ has a negative component. It is still a vector, and most of this book is about difference vectors.

Check yourself

1. What is s_2, the second component of soil?
2. Compute $0.5\,\mathbf{s} + 0.5\,\mathbf{g}$ by hand. Is it a mixture?
3. Compute the mixture with 30% metal and 70% grass, $0.3\,\mathbf{t} + 0.7\,\mathbf{g}$, showing each step.
4. Could $(-0.1,\ 0.2,\ 0.0)$ be the spectrum of a surface? Could it be a difference of two spectra?
5. A pixel shows $(0.24,\ 0.24,\ 0.24)$. Give two different explanations involving the metal roof.

Check yourself: worked answers

1. $\mathbf{s} = (0.1,\ 0.2,\ 0.3)$, so $s_2 = 0.2$.
2. $0.5\,\mathbf{s} = (0.05,\ 0.10,\ 0.15)$ and $0.5\,\mathbf{g} = (0.05,\ 0.05,\ 0.25)$. Adding: band 1: $0.05 + 0.05 = 0.10$; band 2: $0.10 + 0.05 = 0.15$; band 3: $0.15 + 0.25 = 0.40$. So $(0.10,\ 0.15,\ 0.40)$. Yes, it is a mixture with fill $a = 0.5$ (the two numbers are 0.5 and $1 - 0.5$).
3. $0.3\,\mathbf{t} = (0.09,\ 0.09,\ 0.09)$. $0.7\,\mathbf{g} = (0.07,\ 0.07,\ 0.35)$. Band 1: $0.09 + 0.07 = 0.16$. Band 2: $0.09 + 0.07 = 0.16$. Band 3: $0.09 + 0.35 = 0.44$. So $(0.16,\ 0.16,\ 0.44)$.
4. Not a surface: reflectance cannot be negative. Yes as a difference: for example soil minus metal is $(0.1 - 0.3,\ 0.2 - 0.3,\ 0.3 - 0.3) = (-0.2,\ -0.1,\ 0.0)$, the same kind of object.
5. It is $0.8\,\mathbf{t}$. Either the whole pixel is metal roof at 80% brightness (a shadow or a tilted roof), or the pixel is 80% metal mixed with 20% of something that reflects nothing: $0.8\,\mathbf{t} + 0.2 \times (0, 0, 0)$. From this one pixel you cannot tell which.

Next: Chapter 2, one number for how much two spectra agree.

Chapter 2

The dot product and length

The one thing this chapter teaches

The dot product multiplies two vectors band by band and adds the results; it measures how much they agree, and the dot product of a vector with itself is its squared length.

2.1 The problem: you need numbers, not lists

A list of 165 reflectances is hard to reason about. To decide "is this pixel metal?" you will want a single number that says how strongly the pixel's spectrum agrees with metal's, and another that says how big a spectrum is. Both come from one operation.

2.2 The summation sign

You will add up over bands constantly, so there is a shorthand.

Definition: summation sign $\sum$

$\sum_{n=1}^{N} u_n$ means $u_1 + u_2 + \cdots + u_N$: start with $n = 1$, add the term, increase n by one, and stop after $n = N$.
The letter n is a counter and has no meaning outside the sum.

For $\mathbf{g} = (0.1,\ 0.1,\ 0.5)$, $\sum_{n=1}^{3} g_n = 0.1 + 0.1 + 0.5 = 0.7$.

2.3 The dot product

Definition: dot product

The **dot product** of two vectors with the same number of components is

$$\mathbf{u} \cdot \mathbf{v} = \sum_{n=1}^{N} u_n v_n = u_1 v_1 + u_2 v_2 + \cdots + u_N v_N.$$

Multiply the matching components, then add. The result is a single number.

Worked example 2.1: The dot product of metal and grass

$\mathbf{t} = (0.3,\ 0.3,\ 0.3)$ and $\mathbf{g} = (0.1,\ 0.1,\ 0.5)$. Multiply band by band:

$$\begin{aligned} \text{band 1:} \quad & 0.3 \times 0.1 = 0.03 \\ \text{band 2:} \quad & 0.3 \times 0.1 = 0.03 \\ \text{band 3:} \quad & 0.3 \times 0.5 = 0.15 \end{aligned}$$

Add: $0.03 + 0.03 = 0.06$, then $0.06 + 0.15 = 0.21$. So $\mathbf{t} \cdot \mathbf{g} = 0.21$.

A big dot product means the two vectors have large components in the same bands. A dot product of zero means that, across the bands, the agreements and disagreements cancel exactly.

Three properties follow straight from the definition. Check each on the toy numbers if you doubt it.

- **Order does not matter**: $\mathbf{u} \cdot \mathbf{v} = \mathbf{v} \cdot \mathbf{u}$, because $u_n v_n = v_n u_n$.
- **Scaling comes out**: $(c\,\mathbf{u}) \cdot \mathbf{v} = c\,(\mathbf{u} \cdot \mathbf{v})$, because every term $c\,u_n v_n$ has the factor c.
- **Sums split**: $\mathbf{u} \cdot (\mathbf{v} + \mathbf{w}) = \mathbf{u} \cdot \mathbf{v} + \mathbf{u} \cdot \mathbf{w}$, because $u_n(v_n + w_n) = u_n v_n + u_n w_n$ in every band.

2.4 Length

In two dimensions the length of the arrow from $(0, 0)$ to $(3, 4)$ is 5, by Pythagoras' theorem: the two components are the sides of a right-angled triangle and the arrow is the long side, so length$^2 = 3^2 + 4^2 = 9 + 16 = 25$ and length $= \sqrt{25} = 5$.

In three dimensions you apply Pythagoras twice. For $(1, 2, 2)$: the shadow of the arrow on the floor runs from $(0, 0)$ to $(1, 2)$ and has squared length $1^2 + 2^2 = 5$. The arrow itself is the long side of a second right-angled triangle whose other sides are that shadow and the height 2. So length$^2 = 5 + 2^2 = 9$, length $= 3$. The pattern continues to any number of bands: square each component, add, take the square root.

Definition: length (norm)

The **length** or **norm** of $\mathbf{u}$ is

$$\|\mathbf{u}\| = \sqrt{\sum_{n=1}^{N} u_n^2} = \sqrt{\mathbf{u} \cdot \mathbf{u}}.$$

So the squared length is the dot product of a vector with itself: $\|\mathbf{u}\|^2 = \mathbf{u} \cdot \mathbf{u}$.

Worked example 2.2: The lengths of metal and grass

For $\mathbf{t} = (0.3,\ 0.3,\ 0.3)$:

$$\begin{aligned} 0.3^2 &= 0.09 \quad \text{(each band)} \\ \|\mathbf{t}\|^2 &= 0.09 + 0.09 + 0.09 = 0.27 \\ \|\mathbf{t}\| &= \sqrt{0.27} = 0.5196 \end{aligned}$$

For $\mathbf{g} = (0.1,\ 0.1,\ 0.5)$:

$$\begin{aligned} 0.1^2 = 0.01, \quad 0.1^2 = 0.01, \quad 0.5^2 &= 0.25 \\ \|\mathbf{g}\|^2 &= 0.01 + 0.01 + 0.25 = 0.27 \\ \|\mathbf{g}\| &= \sqrt{0.27} = 0.5196 \end{aligned}$$

Metal and grass have exactly the same length.

That coincidence is worth a moment. Two completely different materials can have the same length, so **length tells you about brightness, not about material**. You need something else to tell materials apart; Chapter 3 supplies it.

Definition: distance, unit vector, orthogonal

The **distance** between $\mathbf{u}$ and $\mathbf{v}$ is the length of their difference, $\|\mathbf{u}-\mathbf{v}\|$.
A **unit vector** has length 1. Any non-zero $\mathbf{u}$ can be turned into one by dividing by its length: $\mathbf{u}/\|\mathbf{u}\|$.
Two vectors are **orthogonal** (perpendicular) when their dot product is zero.

The distance from metal to grass is $\|\mathbf{t}-\mathbf{g}\| = \|(0.2,\ 0.2,\ -0.2)\| = \sqrt{0.04+0.04+0.04} = \sqrt{0.12} = 0.3464$. The unit vector along metal is $\mathbf{t}/0.5196 = (0.5774,\ 0.5774,\ 0.5774)$. The vectors $(1,1)$ and $(1,-1)$ are orthogonal: $1\times 1 + 1\times(-1) = 1-1 = 0$.

2.5 Expanding the squared length of a sum

One identity is used in nearly every derivation later on, so it gets its own box.

Rule 2.1: Squared length of a sum or difference

$$\|\mathbf{u}+\mathbf{v}\|^2 = \|\mathbf{u}\|^2 + 2\,\mathbf{u}\cdot\mathbf{v} + \|\mathbf{v}\|^2, \qquad \|\mathbf{u}-\mathbf{v}\|^2 = \|\mathbf{u}\|^2 - 2\,\mathbf{u}\cdot\mathbf{v} + \|\mathbf{v}\|^2.$$

Proof in two bands. Write out the components and multiply out each bracket using $(p+q)^2 = p^2+2pq+q^2$:

$$\begin{aligned}\|\mathbf{u}+\mathbf{v}\|^2 &= (u_1+v_1)^2 + (u_2+v_2)^2 \\ &= u_1^2 + 2u_1v_1 + v_1^2 + u_2^2 + 2u_2v_2 + v_2^2 \\ &= (u_1^2+u_2^2) + 2(u_1v_1+u_2v_2) + (v_1^2+v_2^2) \qquad \textit{regroup} \\ &= \|\mathbf{u}\|^2 + 2\,\mathbf{u}\cdot\mathbf{v} + \|\mathbf{v}\|^2.\end{aligned}$$

With more bands there are more terms of the same kind and the regrouping is identical. For the difference, replace $\mathbf{v}$ by $-\mathbf{v}$; the middle term changes sign because $\mathbf{u}\cdot(-\mathbf{v}) = -\mathbf{u}\cdot\mathbf{v}$.

Worked example 2.3: Checking the rule on metal and grass

Directly: $\|\mathbf{t}-\mathbf{g}\|^2 = 0.12$ (above). By the rule:

$$\begin{aligned}\|\mathbf{t}\|^2 - 2\,\mathbf{t}\cdot\mathbf{g} + \|\mathbf{g}\|^2 &= 0.27 - 2\times 0.21 + 0.27 \\ &= 0.27 - 0.42 + 0.27 \\ &= 0.12.\end{aligned}$$

The two agree.

2.6 Mean absolute value and root mean square

The original note sometimes quotes a difference as “0.170 reflectance per band” and sometimes as a squared length such as “904”. They are different summaries of one vector, and you need to be able to move between them.

Definition: mean absolute value, root mean square

For a vector $\mathbf{u}$ with N components, the **mean absolute value** is $\frac{1}{N}\sum_n |u_n|$ (drop the signs, average). The **root mean square** (RMS) is $\sqrt{\frac{1}{N}\sum_n u_n^2} = \|\mathbf{u}\|/\sqrt{N}$ (square, average, square-root).

For $\mathbf{t}-\mathbf{g} = (0.2,\ 0.2,\ -0.2)$ both are 0.2, because every component has the same size. For a real difference they differ a little. The galvanised-metal-over-grass difference used in Chapter 19 has a mean absolute value of 2.18 in normalised units and a squared length of 904.1 over 165 bands[C]. Its RMS is $\sqrt{904.1/165}$: first $904.1/165 = 5.479$, then $\sqrt{5.479} = 2.341$. Same order of size as 2.18, slightly bigger, as an RMS always is (squaring gives the big components more say).

Analogy: a shopping bill

Think of $\mathbf{u}$ as quantities bought (3 apples, 2 loaves, 1 jar) and $\mathbf{v}$ as prices per item. The dot product $\mathbf{u}\cdot\mathbf{v}$ is the bill: each quantity times its price, added up. The squared length $\mathbf{u}\cdot\mathbf{u}$ is the bill you would get if each item's price equalled the quantity bought. Keep this picture: in Chapter 7 you will change the price list, charging more for bands where surprises are rare, and the "bill" becomes the Mahalanobis distance.

Common confusions

Is the dot product a vector? No, it is one number. Multiplying component by component *without* adding gives a vector, but that is a different (and here unused) operation.
Is $\|\mathbf{u}\|^2$ the same as $\|\mathbf{u}\|$ squared component by component? $\|\mathbf{u}\|^2$ is one number, the sum of the squares. "Squaring a vector" component by component is not something this book does.
Distance or length? Length belongs to one vector. Distance belongs to a pair and is the length of their difference.
Why not compare materials by distance? Distance mixes brightness and shape: a metal roof in shadow, $0.5\,\mathbf{t}$, is at distance $\|0.5\,\mathbf{t}\| = 0.26$ from the bright roof $\mathbf{t}$, farther than many unrelated materials might be. The next chapter removes brightness.

Check yourself

1. Compute $\mathbf{s}\cdot\mathbf{g}$ for soil and grass by hand.
2. Compute $\|\mathbf{s}\|$, the length of soil.
3. Are $(2,\ 1)$ and $(-1,\ 2)$ orthogonal?
4. Compute $\|\mathbf{s}-\mathbf{g}\|^2$ two ways: directly, and by rule 2.1.
5. Why can length not tell metal from grass?

Check yourself: worked answers

1. Band 1: $0.1\times0.1 = 0.01$. Band 2: $0.2\times0.1 = 0.02$. Band 3: $0.3\times0.5 = 0.15$. Sum: $0.01+0.02+0.15 = 0.18$.
2. Squares: 0.01, 0.04, 0.09. Sum: 0.14. Square root: $\sqrt{0.14} = 0.3742$.
3. $2\times(-1) + 1\times 2 = -2+2 = 0$. Yes.
4. Directly: $\mathbf{s}-\mathbf{g} = (0.1-0.1,\ 0.2-0.1,\ 0.3-0.5) = (0,\ 0.1,\ -0.2)$; squares $0,\ 0.01,\ 0.04$; sum 0.05. By the rule: $\|\mathbf{s}\|^2 - 2\,\mathbf{s}\cdot\mathbf{g} + \|\mathbf{g}\|^2 = 0.14 - 2\times0.18 + 0.27 = 0.14 - 0.36 + 0.27 = 0.05$. They agree.
5. Both have squared length 0.27 (worked example 2.2). Length measures overall brightness, and two different materials can be equally bright.

Next: Chapter 3, comparing spectra by direction.

Chapter 3

Angles and the spectral angle mapper

The one thing this chapter teaches

The angle between two spectra, computed from their dot product and lengths, measures how different their shapes are and ignores brightness; the spectral angle mapper names a pixel after the library spectrum at the smallest angle.

3.1 The problem: brightness is a nuisance

The same roof looks brighter in full sun than in shadow, and brighter when it faces the sun than when it is tilted away. All of these scale every band by about the same factor (worked example 1.1). A comparison that is upset by brightness will call a shaded roof a different material. You want a comparison that sees only the *shape* of the spectrum. In the arrow picture, scaling changes the arrow's length but not the way it points. So: compare directions.

3.2 Cosine and sine

Definition: cosine and sine

Draw a circle of radius 1 centred on the origin. Walk anticlockwise around it from the point $(1, 0)$ through an angle θ. The point you reach has coordinates $(\cos\theta,\ \sin\theta)$. That defines the **cosine** and **sine** of θ. Because the point is at distance 1 from the origin, Pythagoras gives $\cos^2\theta + \sin^2\theta = 1$ for every angle.

A few values you will use:

angle θ	0°	30°	45°	60°	90°
$\cos\theta$	1	0.866	0.707	0.5	0
$\sin\theta$	0	0.5	0.707	0.866	1

Cosine is 1 when the angle is zero and falls to 0 at a right angle. A point at angle θ on a circle of radius L instead of 1 has coordinates $(L\cos\theta,\ L\sin\theta)$: everything scales by L.

Definition: arccos

$\arccos(c)$, also written $\cos^{-1}(c)$, is the angle between 0° and 180° whose cosine is c. It undoes the cosine. On a calculator it is the "$\cos^{-1}$" key; make sure the calculator is in degree mode.

3.3 The angle between two vectors

Two vectors $\mathbf{u}$ and $\mathbf{v}$ that both start at the origin lie in one flat plane, even if they have 165 components. (Two arrows from one point always do, the same way two pencils held at their tips span a sheet of paper.) Choose coordinates in that plane so that $\mathbf{u}$ lies along the first axis. Then, writing θ for the angle between the arrows,

$$\mathbf{u} = (\|\mathbf{u}\|,\ 0), \qquad \mathbf{v} = (\|\mathbf{v}\|\cos\theta,\ \|\mathbf{v}\|\sin\theta).$$

The dot product in these coordinates is

$$\begin{aligned}\mathbf{u}\cdot\mathbf{v} &= \|\mathbf{u}\| \times \|\mathbf{v}\|\cos\theta + 0 \times \|\mathbf{v}\|\sin\theta \\ &= \|\mathbf{u}\|\,\|\mathbf{v}\|\cos\theta.\end{aligned}$$

The dot product and the lengths do not depend on which coordinates you use to write the vectors (turning the axes does not change how long an arrow is or how much two arrows agree), so this holds in the original 165 coordinates too. Divide both sides by $\|\mathbf{u}\|\,\|\mathbf{v}\|$:

Rule 3.1: The cosine of the angle between two vectors

$$\cos\theta = \frac{\mathbf{u}\cdot\mathbf{v}}{\|\mathbf{u}\|\,\|\mathbf{v}\|}, \qquad \theta = \arccos\!\left(\frac{\mathbf{u}\cdot\mathbf{v}}{\|\mathbf{u}\|\,\|\mathbf{v}\|}\right).$$

Because the cosine of any angle lies between -1 and 1, so does this ratio. (That fact has a name, the Cauchy-Schwarz inequality: $|\mathbf{u}\cdot\mathbf{v}| \le \|\mathbf{u}\|\,\|\mathbf{v}\|$.)

Worked example 3.1: The angle between metal and grass

From worked examples 2.1 and 2.2: $\mathbf{t}\cdot\mathbf{g} = 0.21$, $\|\mathbf{t}\| = 0.5196$, $\|\mathbf{g}\| = 0.5196$.

$$\begin{aligned}\|\mathbf{t}\|\,\|\mathbf{g}\| &= 0.5196 \times 0.5196 = 0.27 \\ \cos\theta &= 0.21/0.27 = 0.7778 \\ \theta &= \arccos(0.7778) = 38.94^\circ\end{aligned}$$

Worked example 3.2: Brightness does not change the angle

Take the roof at double brightness, $2\mathbf{t} = (0.6,\ 0.6,\ 0.6)$, and compare it with $\mathbf{t}$.

$$\begin{aligned}(2\mathbf{t})\cdot\mathbf{t} &= 0.6\times0.3 + 0.6\times0.3 + 0.6\times0.3 = 0.18 + 0.18 + 0.18 = 0.54 \\ \|2\mathbf{t}\| &= \sqrt{0.36 + 0.36 + 0.36} = \sqrt{1.08} = 1.0392 \\ \|2\mathbf{t}\|\,\|\mathbf{t}\| &= 1.0392 \times 0.5196 = 0.54 \\ \cos\theta &= 0.54/0.54 = 1, \qquad \theta = 0^\circ.\end{aligned}$$

In general, for any positive number c: $(c\mathbf{u})\cdot\mathbf{v} = c\,(\mathbf{u}\cdot\mathbf{v})$ and $\|c\mathbf{u}\| = c\|\mathbf{u}\|$, so the factor c appears once on top and once underneath and cancels.

3.4 The spectral angle mapper (SAM)

Definition: spectral angle mapper

Given a pixel spectrum $\mathbf{x}$ and a library of reference spectra, the **spectral angle mapper** computes the angle between $\mathbf{x}$ and each library spectrum and names the pixel after the library spectrum with the smallest angle.

The Allotrope code has an implementation (`app/spectral_match/sam.py`). The library is the shipped family library, 411 lab spectra in 17 families (Chapter 14).

Worked example 3.3: SAM on a soil-like pixel

Library: $\mathbf{g}$, $\mathbf{t}$, $\mathbf{s}$. Pixel: $\mathbf{p} = (0.12,\ 0.20,\ 0.32)$.
Length of the pixel: $0.12^2 = 0.0144$, $0.20^2 = 0.04$, $0.32^2 = 0.1024$; sum 0.1568; $\|\mathbf{p}\| = \sqrt{0.1568} = 0.3960$.
Against soil, $\|\mathbf{s}\| = 0.3742$:

$$\mathbf{p} \cdot \mathbf{s} = 0.12 \times 0.1 + 0.20 \times 0.2 + 0.32 \times 0.3 = 0.012 + 0.040 + 0.096 = 0.148$$
$$\cos = 0.148/(0.3960 \times 0.3742) = 0.148/0.14818 = 0.9989, \qquad \text{angle} = 2.68^\circ$$

Against grass, $\|\mathbf{g}\| = 0.5196$:

$$\mathbf{p} \cdot \mathbf{g} = 0.012 + 0.020 + 0.160 = 0.192$$
$$\cos = 0.192/(0.3960 \times 0.5196) = 0.192/0.20576 = 0.9331, \qquad \text{angle} = 21.07^\circ$$

Against metal, $\|\mathbf{t}\| = 0.5196$:

$$\mathbf{p} \cdot \mathbf{t} = 0.036 + 0.060 + 0.096 = 0.192, \qquad \text{angle} = 21.07^\circ \text{ (same numbers as grass)}$$

Smallest angle: soil, at 2.68°. SAM names the pixel soil.

3.5 Angles in radians, and small angles

Some formulas later on need angles in **radians** rather than degrees. A full turn is 360° or 2π radians, so

$$\text{radians} = \text{degrees} \times \frac{\pi}{180}, \qquad \pi = 3.14159.$$

For example 2.73° is $2.73 \times 3.14159/180 = 0.04765$ radians.

For small angles measured in radians there are three handy approximations:

$$\sin\theta \approx \theta, \qquad \tan\theta \approx \theta, \qquad \cos\theta \approx 1 - \tfrac{1}{2}\theta^2.$$

(Here $\tan\theta = \sin\theta/\cos\theta$, the **tangent**.) How good are they?

angle	in radians	$\sin\theta$	$\tan\theta$	$\cos\theta$ vs $1 - \theta^2/2$
1°	0.017453	0.017452	0.017455	0.999848 vs 0.999848
3°	0.052360	0.052336	0.052408	0.998630 vs 0.998629
10°	0.174533	0.173648	0.176327	0.984808 vs 0.984769

Very good up to a few degrees. The last column also tells you something important: **near zero, angles hide in the third or fourth decimal of the cosine.** A cosine of 0.99 is an angle of 8.1°; 0.999 is 2.56°; 0.9999 is 0.81°. Materials in the same family differ by a few degrees (Chapter 21), so you will spend a lot of time in that fourth decimal.

Analogy: a compass bearing

Two hikers leave camp on the same compass bearing; one walks 2 km, the other 5 km. They are in very different places but heading in the same direction. Brightness is the distance walked; material is the bearing. SAM compares bearings only. Keep the analogy in mind for its limit too: a hiker who walks partly north and partly east ends up on an in-between bearing, and you cannot tell from the bearing alone how much of each leg they walked. That is the mixed-pixel problem of Chapter 23.

Common confusions

My calculator says $\arccos(0.7778) = 0.6797$**.** It is in radian mode. $0.6797 \times 180/\pi = 38.94^\circ$.
Does angle 0° **mean the spectra are equal?** No, it means one is a positive multiple of the other: same shape, possibly different brightness (worked example 3.2).
Does SAM ignore everything except material? It ignores a common brightness factor only. Adding a constant to every band (a haze offset) does change the angle. For grass, $\mathbf{g} + 0.1 = (0.2,\ 0.2,\ 0.6)$ sits 9.45° away from $\mathbf{g}$. So does mixing with another material, as Chapter 23 shows in detail.
Is a smaller angle always a better match? Only between pure spectra. A pixel that is 80% grass and 20% metal is much closer in angle to grass than to metal, which is right as a description and wrong if the question was "is there metal here?".

Check yourself

1. Compute the angle between soil and grass by hand. (Use $\mathbf{s}\cdot\mathbf{g} = 0.18$, $\|\mathbf{s}\| = 0.3742$, $\|\mathbf{g}\| = 0.5196$.)
2. What is the angle between $(1,\ 0)$ and $(1,\ 1)$?
3. What is the angle between $0.5\,\mathbf{s}$ and $3\,\mathbf{s}$?
4. Convert 5.22° to radians and check that cos of it is about $1 - \theta^2/2$.
5. A cosine of 0.5 corresponds to which angle?

Check yourself: worked answers

1. $\|\mathbf{s}\|\,\|\mathbf{g}\| = 0.3742 \times 0.5196 = 0.19443$. cos $= 0.18/0.19443 = 0.9258$. $\arccos(0.9258) = 22.21^\circ$.
2. Dot product $1 \times 1 + 0 \times 1 = 1$. Lengths 1 and $\sqrt{2} = 1.4142$. cos $= 1/1.4142 = 0.7071$, which is 45°.
3. 0°: both are positive multiples of $\mathbf{s}$.
4. $5.22 \times 3.14159/180 = 0.09111$ radians. $\theta^2 = 0.008301$, half of it 0.004151, so $1 - \theta^2/2 = 0.995849$. The calculator gives $\cos(5.22^\circ) = 0.995853$. Close.
5. 60° (from the table).

Next: Chapter 4, machines that turn one vector into another.

Chapter 4

Matrices as machines

The one thing this chapter teaches

A matrix is a machine that turns one vector into another by taking a dot product with each of its rows; the transpose flips it, the inverse undoes it, and a quadratic form turns a vector into one number.

4.1 The problem: you need to transform spectra

Later chapters do things to spectra: scale each band by a different gain, rotate onto new axes, stretch the directions where surprises are rare, keep only the part that lies in some subspace. Each of those operations takes a vector in and gives a vector out, and each is *linear*: it respects addition and scaling. All such operations can be written the same way, as a matrix.

4.2 What a matrix is

Definition: matrix

A **matrix** is a rectangular table of numbers with m rows and k columns, called an $m \times k$ matrix. Matrices are printed as bold capitals, $\mathbf{A}$. The entry in row i, column j is written A_{ij}. A vector with N components is treated as an $N \times 1$ matrix: a single **column**.

Definition: matrix times vector

If $\mathbf{A}$ is $m \times k$ and $\mathbf{v}$ has k components, $\mathbf{Av}$ is the vector with m components whose i-th component is the dot product of row i of $\mathbf{A}$ with $\mathbf{v}$:

$$(\mathbf{Av})_i = \sum_{j=1}^{k} A_{ij} v_j.$$

Worked example 4.1: A 2×2 machine

$\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$, $\mathbf{v} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$.

$$\text{row } 1 \cdot \mathbf{v} = 2 \times 1 + 1 \times 2 = 2 + 2 = 4$$
$$\text{row } 2 \cdot \mathbf{v} = 0 \times 1 + 3 \times 2 = 0 + 6 = 6$$

So $\mathbf{Av} = \begin{pmatrix} 4 \\ 6 \end{pmatrix}$.

Two special kinds of matrix appear everywhere.

Definition: identity and diagonal matrices

The **identity I** has 1 on its diagonal (the entries $A_{11}, A_{22}, \dots$) and 0 elsewhere. It leaves every vector unchanged: $\mathbf{I}\mathbf{v} = \mathbf{v}$.
A **diagonal matrix** $\mathrm{diag}(d_1, \dots, d_N)$ has $d_1, \dots, d_N$ on the diagonal and 0 elsewhere. It multiplies component n by d_n and does nothing else.

A diagonal matrix is exactly what a per-band calibration error does: band 1 is read 10% too high, band 2 10% too low. With $\mathbf{G} = \mathrm{diag}(1.1,\ 0.9)$ and the flat metal spectrum $(0.3,\ 0.3)$ in two bands, $\mathbf{G}\mathbf{t} = (1.1 \times 0.3,\ 0.9 \times 0.3) = (0.33,\ 0.27)$. The roof is no longer flat; its direction has turned. Chapter 25 measures by how much.

4.3 Chaining machines: the matrix product

Definition: matrix product

If $\mathbf{A}$ is $m \times k$ and $\mathbf{B}$ is $k \times l$, the product $\mathbf{AB}$ is the $m \times l$ matrix whose entry (i, j) is the dot product of row i of $\mathbf{A}$ with column j of $\mathbf{B}$. It is the machine "first apply $\mathbf{B}$, then apply $\mathbf{A}$": $(\mathbf{AB})\mathbf{v} = \mathbf{A}(\mathbf{B}\mathbf{v})$.

Worked example 4.2: Products in both orders

$\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$, $\mathbf{B} = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$.

$$(\mathbf{AB})_{11} = 2 \times 1 + 1 \times 1 = 3, \qquad (\mathbf{AB})_{12} = 2 \times 0 + 1 \times 1 = 1,$$
$$(\mathbf{AB})_{21} = 0 \times 1 + 3 \times 1 = 3, \qquad (\mathbf{AB})_{22} = 0 \times 0 + 3 \times 1 = 3.$$

So $\mathbf{AB} = \begin{pmatrix} 3 & 1 \\ 3 & 3 \end{pmatrix}$. In the other order:

$$(\mathbf{BA})_{11} = 1 \times 2 + 0 \times 0 = 2, \qquad (\mathbf{BA})_{12} = 1 \times 1 + 0 \times 3 = 1,$$
$$(\mathbf{BA})_{21} = 1 \times 2 + 1 \times 0 = 2, \qquad (\mathbf{BA})_{22} = 1 \times 1 + 1 \times 3 = 4.$$

So $\mathbf{BA} = \begin{pmatrix} 2 & 1 \\ 2 & 4 \end{pmatrix} \neq \mathbf{AB}$. **Order matters.**
Check the chaining with $\mathbf{v} = (1,\ 2)$: $\mathbf{B}\mathbf{v} = (1 \times 1 + 0 \times 2,\ 1 \times 1 + 1 \times 2) = (1,\ 3)$, then $\mathbf{A}(1,\ 3) = (2 \times 1 + 1 \times 3,\ 0 \times 1 + 3 \times 3) = (5,\ 9)$. And $(\mathbf{AB})\mathbf{v} = (3 \times 1 + 1 \times 2,\ 3 \times 1 + 3 \times 2) = (5,\ 9)$. The same.

4.4 The transpose

Definition: transpose

The **transpose** $\mathbf{A}^\top$ swaps rows and columns: $(\mathbf{A}^\top)_{ij} = A_{ji}$. A column vector $\mathbf{v}$ transposes to a row $\mathbf{v}^\top$. A matrix with $\mathbf{A}^\top = \mathbf{A}$ is **symmetric**.

With this notation the dot product is a matrix product of a row with a column: $\mathbf{u}^\top\mathbf{v} = \sum_n u_n v_n = \mathbf{u} \cdot \mathbf{v}$. From here on the book writes dot products as $\mathbf{u}^\top\mathbf{v}$, because that is how they appear in formulas with matrices in between.

Rule 4.1: The transpose of a product reverses the order

$$(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top.$$

Proof for 2×2. Let $\mathbf{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$ and $\mathbf{B} = \begin{pmatrix} e & f \\ g & h \end{pmatrix}$. Then

$$\mathbf{AB} = \begin{pmatrix} ae+bg & af+bh \\ ce+dg & cf+dh \end{pmatrix}, \qquad (\mathbf{AB})^\top = \begin{pmatrix} ae+bg & ce+dg \\ af+bh & cf+dh \end{pmatrix}.$$

On the other side, $\mathbf{B}^\top = \begin{pmatrix} e & g \\ f & h \end{pmatrix}$ and $\mathbf{A}^\top = \begin{pmatrix} a & c \\ b & d \end{pmatrix}$, so

$$\mathbf{B}^\top \mathbf{A}^\top = \begin{pmatrix} ea+gb & ec+gd \\ fa+hb & fc+hd \end{pmatrix}.$$

Entry by entry the two are equal. Larger matrices work the same way, entry by entry.

Rule 4.2: A single number is its own transpose

A 1×1 matrix (a single number) is unchanged by transposing. So for vectors $\mathbf{u}, \mathbf{v}$ and a matrix $\mathbf{A}$, using rule 4.1 twice,

$$\mathbf{u}^\top \mathbf{A} \mathbf{v} = (\mathbf{u}^\top \mathbf{A} \mathbf{v})^\top = \mathbf{v}^\top \mathbf{A}^\top \mathbf{u},$$

and if $\mathbf{A}$ is symmetric, $\mathbf{u}^\top \mathbf{A} \mathbf{v} = \mathbf{v}^\top \mathbf{A} \mathbf{u}$.
Check. $\mathbf{u} = (1, 0)$, $\mathbf{v} = (0, 1)$, $\mathbf{A} = \left(\begin{smallmatrix} 2 & 1 \\ 0 & 3 \end{smallmatrix}\right)$. $\mathbf{Av} = (1,\ 3)$, so $\mathbf{u}^\top \mathbf{Av} = 1$. $\mathbf{A}^\top = \left(\begin{smallmatrix} 2 & 0 \\ 1 & 3 \end{smallmatrix}\right)$, $\mathbf{A}^\top \mathbf{u} = (2,\ 1)$, so $\mathbf{v}^\top \mathbf{A}^\top \mathbf{u} = 1$. Equal.

You will use rule 4.2 in almost every derivation of Part IV: it is why two "cross terms" always merge into one.

4.5 The inverse: undoing a machine

Definition: inverse

The **inverse** of a square matrix $\mathbf{A}$ is the matrix $\mathbf{A}^{-1}$ with $\mathbf{A}^{-1}\mathbf{A} = \mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$. It undoes $\mathbf{A}$: if $\mathbf{w} = \mathbf{Av}$ then $\mathbf{A}^{-1}\mathbf{w} = \mathbf{v}$.

Rule 4.3: The inverse of a 2×2 matrix

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}^{-1} = \frac{1}{ad-bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix},$$

provided the **determinant** $ad - bc$ is not zero. *Proof:* multiply out.

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix} = \begin{pmatrix} ad-bc & -ab+ba \\ cd-dc & -cb+da \end{pmatrix} = (ad-bc)\,\mathbf{I},$$

so dividing by $ad - bc$ leaves $\mathbf{I}$.

Worked example 4.3: Inverting and undoing

$\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$. Determinant: $2 \times 3 - 1 \times 0 = 6$. So

$$\mathbf{A}^{-1} = \frac{1}{6}\begin{pmatrix} 3 & -1 \\ 0 & 2 \end{pmatrix} = \begin{pmatrix} 0.5 & -0.1667 \\ 0 & 0.3333 \end{pmatrix}.$$

In worked example 4.1, $\mathbf{A}$ sent $(1,\ 2)$ to $(4,\ 6)$. Undo it:

$$\text{component 1:} \quad 0.5 \times 4 + (-0.1667) \times 6 = 2 - 1 = 1$$
$$\text{component 2:} \quad 0 \times 4 + 0.3333 \times 6 = 2$$

Back to $(1,\ 2)$.

Four facts about inverses are used later.

- **Diagonal**: $\text{diag}(d_1, \ldots, d_N)^{-1} = \text{diag}(1/d_1, \ldots, 1/d_N)$. Undo each band's scaling by dividing.
- **Product**: $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$. To undo "first $\mathbf{B}$, then $\mathbf{A}$" you undo $\mathbf{A}$ first, like taking off shoes before socks. Check: $(\mathbf{AB})(\mathbf{B}^{-1}\mathbf{A}^{-1}) = \mathbf{A}(\mathbf{B}\mathbf{B}^{-1})\mathbf{A}^{-1} = \mathbf{A}\mathbf{I}\mathbf{A}^{-1} = \mathbf{I}$.
- **Symmetric**: the inverse of a symmetric matrix is symmetric. In rule 4.3, if $b = c$ then the inverse has $-b$ in both off-diagonal places.
- **Singular**: if the determinant is zero there is no inverse. $\left(\begin{smallmatrix} 1 & 2 \\ 2 & 4 \end{smallmatrix}\right)$ has determinant $4 - 4 = 0$, and it sends $(2,\ -1)$ to $(2 - 2,\ 4 - 4) = (0,\ 0)$. Once a direction has been crushed to zero, nothing can bring it back.

The last fact has a near miss that matters. $\left(\begin{smallmatrix} 1 & 2 \\ 2 & 4.0001 \end{smallmatrix}\right)$ has determinant $4.0001 - 4 = 0.0001$, so by rule 4.3 its inverse has entries around $4/0.0001 = 40{,}000$. A covariance matrix that is almost singular has an inverse that blows tiny wobbles up into enormous numbers. This is why plain RX on full hyperspectral bands was dropped from the product in 2026-05: the residual covariance was near-singular and its distances ran to 10^{11}.

4.6 Quadratic forms

Definition: quadratic form

For a square matrix $\mathbf{A}$ and a vector $\mathbf{u}$, the number $\mathbf{u}^\top\mathbf{A}\mathbf{u}$ is a **quadratic form**. For a 2×2 symmetric $\mathbf{A}$ it expands to $A_{11}u_1^2 + 2A_{12}u_1u_2 + A_{22}u_2^2$.

Worked example 4.4: A quadratic form two ways

$\mathbf{u} = (1,\ 2)$, $\mathbf{A} = \left(\begin{smallmatrix} 2 & 1 \\ 1 & 3 \end{smallmatrix}\right)$. First $\mathbf{A}\mathbf{u} = (2 \times 1 + 1 \times 2,\ 1 \times 1 + 3 \times 2) = (4,\ 7)$. Then $\mathbf{u}^\top(\mathbf{A}\mathbf{u}) = 1 \times 4 + 2 \times 7 = 4 + 14 = 18$. By the expansion: $2 \times 1^2 + 2 \times 1 \times 1 \times 2 + 3 \times 2^2 = 2 + 4 + 12 = 18$.

The squared length is the quadratic form with $\mathbf{A} = \mathbf{I}$. The Mahalanobis distance of Chapter 7 is the quadratic form with $\mathbf{A} = \mathbf{S}^{-1}$, the inverse of a covariance.

Analogy: a mixing desk

A sound engineer's mixing desk takes several input channels and produces several output channels. Each output is a weighted sum of the inputs, with the weights set by a row of faders. That row of faders is a row of the matrix; matrix times vector is "run the inputs through the desk". Plugging one desk's outputs into a second desk is the matrix product, and the order of the desks matters. The

inverse is a desk that exactly undoes the first, which is possible only if the first desk never merged two channels into one (a zero determinant).

Common confusions

Is AB the same as BA? Almost never (worked example 4.2).
Is the inverse made of the reciprocals of the entries? Only for a diagonal matrix. For $\left(\begin{smallmatrix}2&1\\0&3\end{smallmatrix}\right)$ the inverse's top-right entry is $-1/6$, not $1/1$.
Rows or columns? Matrix times vector takes dot products with rows. The columns of $\mathbf{A}$ are where the basic vectors $(1,0)$ and $(0,1)$ are sent: $\mathbf{A}(1,0) = (A_{11}, A_{21})$, the first column.
Is $\mathbf{u}^\top\mathbf{v}$ a matrix? It is 1×1, which is a single number. $\mathbf{u}\mathbf{v}^\top$, the other order, is an $N \times N$ matrix (the "outer product", Chapter 5).

Check yourself

1. Compute $\left(\begin{smallmatrix}1&2\\3&4\end{smallmatrix}\right)\left(\begin{smallmatrix}1\\1\end{smallmatrix}\right)$.
2. Write the transpose of $\left(\begin{smallmatrix}1&2\\3&4\end{smallmatrix}\right)$. Is the matrix symmetric?
3. Find the inverse of $\left(\begin{smallmatrix}3&1\\2&1\end{smallmatrix}\right)$ and check it by multiplying.
4. Compute the quadratic form $\mathbf{u}^\top\mathbf{S}\mathbf{u}$ for $\mathbf{u} = (1,\ -1)$ and $\mathbf{S} = \left(\begin{smallmatrix}2.5&2\\2&2.5\end{smallmatrix}\right)$.
5. Why does $\left(\begin{smallmatrix}1&2\\2&4\end{smallmatrix}\right)$ have no inverse?

Check yourself: worked answers

1. Row 1: $1+2=3$. Row 2: $3+4=7$. Result $(3,\ 7)$.
2. $\left(\begin{smallmatrix}1&3\\2&4\end{smallmatrix}\right)$. Not symmetric: the off-diagonal entries 2 and 3 differ.
3. Determinant $3\times 1 - 1\times 2 = 1$. Inverse $\left(\begin{smallmatrix}1&-1\\-2&3\end{smallmatrix}\right)$. Check: $\left(\begin{smallmatrix}3&1\\2&1\end{smallmatrix}\right)\left(\begin{smallmatrix}1&-1\\-2&3\end{smallmatrix}\right) = \left(\begin{smallmatrix}3-2&-3+3\\2-2&-2+3\end{smallmatrix}\right) = \left(\begin{smallmatrix}1&0\\0&1\end{smallmatrix}\right)$.
4. $\mathbf{S}\mathbf{u} = (2.5-2,\ 2-2.5) = (0.5,\ -0.5)$. Then $\mathbf{u}^\top(\mathbf{S}\mathbf{u}) = 1\times 0.5 + (-1)\times(-0.5) = 0.5 + 0.5 = 1$.
5. Its determinant is $1\times 4 - 2\times 2 = 0$. It sends $(2,\ -1)$ to $(0,\ 0)$, so no machine can recover $(2,\ -1)$.

Next: Chapter 5, measuring how residuals usually wobble.

Chapter 5

Mean, variance and covariance from a tiny data set

The one thing this chapter teaches

The covariance matrix records how much each band wobbles and how pairs of bands wobble together, and it tells you the wobble of any weighted sum of bands through the formula $\mathbf{c}^\top \mathbf{S}\mathbf{c}$.

5.1 The problem: what counts as unusual?

The anomaly detector of Chapter 17 asks "is this pixel's residual unusual?". Unusual compared with what? With the residuals of ordinary background pixels in the same scene. To make that comparison you need a compact description of how ordinary residuals vary. For one band, "they are usually within about 0.1 of zero" would do. With many bands there is a second thing to know: whether bands move together. If band 1 and band 2 always rise and fall together, a residual where band 1 is high and band 2 is low is strange even if each value on its own is ordinary.

5.2 A tiny data set

Four background pixels, two bands each. The values are residuals in normalised units (Chapter 15); here they are toy numbers.

pixel	band 1	band 2
1	2	1
2	−2	−1
3	1	2
4	−1	−2

Definition: mean, deviation

The **mean** of numbers $y_1, \dots, y_n$ is $\bar{y} = \frac{1}{n}\sum_i y_i$. The **mean vector** of a set of vectors is the vector of the band-by-band means. The **deviation** of a value is its distance from the mean, $y_i - \bar{y}$.

Band 1 mean: $(2 + (-2) + 1 + (-1))/4 = 0/4 = 0$. Band 2 mean: $(1 + (-1) + 2 + (-2))/4 = 0/4 = 0$. The mean vector is $(0,\ 0)$, so here the deviations are the values themselves. (Real residuals have a mean close to zero too, if the background model is good.)

Definition: variance, standard deviation

The **variance** of $y_1, \dots, y_n$ is the average squared deviation,

$$\frac{1}{n}\sum_{i=1}^{n}(y_i - \bar{y})^2.$$

The **standard deviation** (sd) is its square root, back in the original units. The symbol σ (sigma) is the usual name for a standard deviation and σ^2 for a variance.

Worked example 5.1: Variance of band 1

Deviations: $2,\ -2,\ 1,\ -1$. Squares:

$$2^2 = 4, \quad (-2)^2 = 4, \quad 1^2 = 1, \quad (-1)^2 = 1$$

$$\text{sum} = 4 + 4 + 1 + 1 = 10$$

$$\text{variance} = 10/4 = 2.5$$

$$\text{sd} = \sqrt{2.5} = 1.581$$

Band 2 has the same four squares in a different order, so its variance is also 2.5.

5.3 Covariance: wobbling together

Definition: covariance, correlation

The **covariance** of two bands is the average product of their deviations,

$$\frac{1}{n}\sum_{i=1}^{n}(y_i - \bar{y})(z_i - \bar{z}).$$

It is positive when the two tend to be above their means together, negative when one is high while the other is low, and near zero when there is no pattern. The **correlation** is the covariance divided by the product of the two standard deviations; it lies between -1 and 1.

Worked example 5.2: Covariance of band 1 with band 2

Products of deviations, pixel by pixel:

$$2 \times 1 = 2, \quad (-2) \times (-1) = 2, \quad 1 \times 2 = 2, \quad (-1) \times (-2) = 2$$

$$\text{sum} = 2 + 2 + 2 + 2 = 8$$

$$\text{covariance} = 8/4 = 2$$

$$\text{correlation} = 2/(1.581 \times 1.581) = 2/2.5 = 0.8$$

Definition: covariance matrix

The **covariance matrix** $\mathbf{S}$ of N-band vectors is the $N \times N$ matrix with the variance of band i in position (i, i) and the covariance of bands i and j in position (i, j). It is symmetric, because the covariance of i with j equals that of j with i.

For the four pixels,

$$\mathbf{S} = \begin{pmatrix} 2.5 & 2 \\ 2 & 2.5 \end{pmatrix}.$$

Keep this matrix: Chapters 6 and 7 take it apart and Chapter 17 uses it to score residuals.

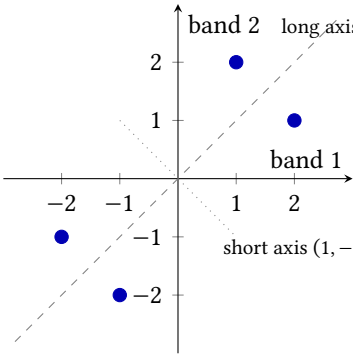


Figure 5.1: The four pixels. The cloud is stretched along the direction $(1, 1)$ (bands rise together) and thin along $(1, -1)$. Chapter 6 finds those two directions from S alone.

5.4 The covariance matrix as a sum of outer products

There is a compact way to build S in one step, and it is how code computes it.

Definition: outer product

For column vectors $\mathbf{u}$ (with m components) and $\mathbf{v}$ (with k), the **outer product** $\mathbf{u}\mathbf{v}^\top$ is the $m \times k$ matrix with entry (i, j) equal to $u_i v_j$.

Worked example 5.3: Building S from outer products

Pixel 1, $(2,\ 1)$: $\left(\begin{smallmatrix}2\\1\end{smallmatrix}\right)\left(\begin{smallmatrix}2&1\end{smallmatrix}\right) = \left(\begin{smallmatrix}2\times2 & 2\times1\\ 1\times2 & 1\times1\end{smallmatrix}\right) = \left(\begin{smallmatrix}4&2\\2&1\end{smallmatrix}\right)$. Pixel 2, $(-2,\ -1)$: the signs cancel in every product, giving $\left(\begin{smallmatrix}4&2\\2&1\end{smallmatrix}\right)$ again. Pixels 3 and 4 each give $\left(\begin{smallmatrix}1&2\\2&4\end{smallmatrix}\right)$. Add the four:

$$\begin{pmatrix} 4+4+1+1 & 2+2+2+2 \\ 2+2+2+2 & 1+1+4+4 \end{pmatrix} = \begin{pmatrix} 10 & 8 \\ 8 & 10 \end{pmatrix},$$

and divide by $n = 4$: $\left(\begin{smallmatrix}2.5&2\\2&2.5\end{smallmatrix}\right)$, as before.

In symbols: $\mathrm{S} = \frac{1}{n}\sum_i (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^\top$.

5.5 The wobble of a weighted sum

Here is the reason covariance matrices are worth the trouble. Every detector in this book forms a weighted sum of bands, $\mathbf{c}^\top\mathbf{w} = c_1 w_1 + c_2 w_2 + \dots$, and you need to know how much that sum wobbles from pixel to pixel.

Rule 5.1: Variance of a weighted sum

If the vectors $\mathbf{w}$ have mean zero and covariance S, then for any fixed weights $\mathbf{c}$,

$$\mathrm{Var}(\mathbf{c}^\top\mathbf{w}) = \mathbf{c}^\top\mathrm{S}\mathbf{c}.$$

More generally, for two weight vectors, the covariance of $\mathbf{c}^\top\mathbf{w}$ with $\mathbf{q}^\top\mathbf{w}$ is $\mathbf{c}^\top\mathrm{S}\mathbf{q}$.

Proof in two bands. The mean of $c_1 w_1 + c_2 w_2$ is zero, so its variance is the average of its square. Expand the square:

$$(c_1 w_1 + c_2 w_2)^2 = c_1^2 w_1^2 + 2 c_1 c_2 w_1 w_2 + c_2^2 w_2^2.$$

Average over pixels. The average of w_1^2 is S_{11}, of $w_1 w_2$ is S_{12}, of w_2^2 is S_{22}, and the fixed numbers c_1, c_2 come outside the averages:

$$\mathrm{Var}(c_1 w_1 + c_2 w_2) = c_1^2 S_{11} + 2 c_1 c_2 S_{12} + c_2^2 S_{22},$$

which is the expansion of the quadratic form $\mathbf{c}^\top \mathbf{S}\mathbf{c}$ (Chapter 4). The two-vector version is the same calculation with $(c_1 w_1 + c_2 w_2)(q_1 w_1 + q_2 w_2)$.

Worked example 5.4: Checking the rule on the four pixels

Take $\mathbf{c} = (1, -1)$, so the sum is band 1 minus band 2. Directly from the data:

$$2 - 1 = 1, \quad -2 - (-1) = -1, \quad 1 - 2 = -1, \quad -1 - (-2) = 1$$

$$\text{squares } 1, 1, 1, 1; \quad \text{variance} = 4/4 = 1.$$

By the rule: $1^2 \times 2.5 + 2 \times 1 \times (-1) \times 2 + (-1)^2 \times 2.5 = 2.5 - 4 + 2.5 = 1$. Equal.
Take $\mathbf{c} = (1, 1)$. Directly: $3, -3, 3, -3$; squares $9, 9, 9, 9$; variance $36/4 = 9$. By the rule: $2.5 + 2 \times 2 + 2.5 = 9$. Equal.

The difference of the bands wobbles with variance 1, the sum with variance 9. Same data, same bands, a factor of nine between two directions. That is what "the cloud is thin along $(1, -1)$" means in numbers.

Rule 5.2: Covariance after a matrix

If $\mathbf{w}$ has covariance $\mathbf{S}$, then $\mathbf{A}\mathbf{w}$ has covariance $\mathbf{A}\mathbf{S}\mathbf{A}^\top$.
Why. Output i is (row i of $\mathbf{A}$) $\cdot\, \mathbf{w}$. By the two-vector form of rule 5.1, the covariance of outputs i and j is (row i) $\mathbf{S}$ (row j)$^\top$, which is entry (i, j) of $\mathbf{A}\mathbf{S}\mathbf{A}^\top$.

Worked example 5.5: Covariance of the sum and the difference together

$\mathbf{A} = \left(\begin{smallmatrix} 1 & 1 \\ 1 & -1 \end{smallmatrix}\right)$ turns (w_1, w_2) into (sum, difference). First $\mathbf{S}\mathbf{A}^\top = \left(\begin{smallmatrix} 2.5 & 2 \\ 2 & 2.5 \end{smallmatrix}\right)\left(\begin{smallmatrix} 1 & 1 \\ 1 & -1 \end{smallmatrix}\right) = \left(\begin{smallmatrix} 2.5+2 & 2.5-2 \\ 2+2.5 & 2-2.5 \end{smallmatrix}\right) = \left(\begin{smallmatrix} 4.5 & 0.5 \\ 4.5 & -0.5 \end{smallmatrix}\right)$. Then

$$\mathbf{A}(\mathbf{S}\mathbf{A}^\top) = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}\begin{pmatrix} 4.5 & 0.5 \\ 4.5 & -0.5 \end{pmatrix} = \begin{pmatrix} 4.5 + 4.5 & 0.5 - 0.5 \\ 4.5 - 4.5 & 0.5 + 0.5 \end{pmatrix} = \begin{pmatrix} 9 & 0 \\ 0 & 1 \end{pmatrix}.$$

Variances 9 and 1 as in worked example 5.4, and covariance 0: the sum and the difference do not wobble together. Directly: products of sum and difference are 3×1, $(-3)(-1)$, $3(-1)$, $(-3)(1)$, i.e. $3 + 3 - 3 - 3 = 0$.

5.6 Samples and populations

Two small points of bookkeeping.

Dividing by n or $n - 1$. Many programs, including `numpy.cov`, divide by $n - 1$ instead of n (it corrects a small bias when the mean is estimated from the same data). For the four pixels that gives $\left(\begin{smallmatrix} 3.33 & 2.67 \\ 2.67 & 3.33 \end{smallmatrix}\right)$, the same matrix times $4/3$. Nothing in this book depends on the choice: the detectors use ratios and percentiles, and the product estimates covariances from up to 200,000 pixels, where the two differ by $1/200{,}000$.

Sample versus population. $\mathbf{S}$ computed from data is a *sample* covariance. The idealised "true" covariance of a process, which the data estimate, is written Σ (capital sigma) with a subscript: Σ_n for sensor noise, Σ_e for background-prediction error, Σ_r for residuals (Chapter 16). The **expectation** $\mathbb{E}[\,\cdot\,]$ is the long-run average over many draws; Var and Cov in formulas mean the long-run versions. Chapter 10 makes this precise.

Analogy: heights and weights

Measure the heights and weights of a hundred adults. Taller people tend to be heavier: the covariance is positive. A person 10 cm taller and 10 kg heavier than average is unremarkable. A person 10 cm taller and 10 kg *lighter* is unusual, although they are no farther from average in either measurement taken alone. The covariance matrix is what lets a computer know the difference. Background residuals behave the same way: bands rise and fall together, and a residual that breaks the usual pattern is the interesting one.

Common confusions

Is variance in the same units as the data? No, in squared units. The standard deviation is back in the data's units. A variance of 2.5 is a standard deviation of 1.58.
Does zero covariance mean the bands are unrelated? It means there is no straight-line tendency. For the Gaussian models of this book that is the same as unrelated; in general it is weaker.
Why is S symmetric? Because $y_i z_i = z_i y_i$: the average product of band i with band j is the same as of band j with band i.
Do I subtract the mean before forming outer products? For a covariance, yes. In Chapter 6 you will meet an "uncentred" version that does not, used for library families; it answers a different question.

Check yourself

1. Compute the mean of $0.2,\ 0.4,\ 0.9$.
2. Compute the variance (dividing by n) of $1,\ 3,\ 5$, showing each step.
3. Four pixels $(1,1),\ (-1,-1),\ (1,-1),\ (-1,1)$. Write their covariance matrix.
4. With $\mathbf{S} = \begin{pmatrix} 2.5 & 2 \\ 2 & 2.5 \end{pmatrix}$ and $\mathbf{c} = (2,\ -1)$, compute $\mathbf{c}^\top \mathbf{S}\mathbf{c}$, then check it on the four pixels of this chapter.
5. In words, what does a positive covariance between two bands mean?

Check yourself: worked answers

1. $0.2 + 0.4 + 0.9 = 1.5$; $1.5/3 = 0.5$.
2. Mean $(1 + 3 + 5)/3 = 9/3 = 3$. Deviations $-2,\ 0,\ 2$. Squares $4,\ 0,\ 4$. Sum 8. Variance $8/3 = 2.667$.
3. Means are 0. Band 1 squares: $1, 1, 1, 1$, variance 1. Band 2: variance 1. Products: $1 \times 1 = 1$, $(-1)(-1) = 1$, $1 \times (-1) = -1$, $(-1) \times 1 = -1$, sum 0, covariance 0. So $\mathbf{S} = \mathbf{I}$. (You will meet this cloud again: it is the whitened version of this chapter's data.)
4. $c_1^2 S_{11} = 4 \times 2.5 = 10$; $2c_1 c_2 S_{12} = 2 \times 2 \times (-1) \times 2 = -8$; $c_2^2 S_{22} = 1 \times 2.5 = 2.5$. Total $10 - 8 + 2.5 = 4.5$. Data: $2w_1 - w_2$ is $4 - 1 = 3$, $-4 + 1 = -3$, $2 - 2 = 0$, $-2 + 2 = 0$; squares $9, 9, 0, 0$; average $18/4 = 4.5$.
5. When one band is above its mean, the other tends to be above its mean too.

Next: Chapter 6, the long and short axes of the cloud.

Chapter 6

Eigenvectors and principal components by hand

The one thing this chapter teaches

An eigenvector of a covariance matrix is a direction along which the data's wobble is pure stretching, and its eigenvalue is the variance along that direction; together they describe the cloud's axes.

6.1 The problem: which directions matter?

Figure 5.1 shows a cloud stretched along $(1, 1)$ and thin along $(1, -1)$. You found that by looking. With 165 bands you cannot look, so you need a calculation that finds the long and short axes from $\mathbf{S}$ alone and says how long each is. The product's anomaly score (Chapter 17) keeps the 30 longest axes of the residual cloud; the family analysis (Chapter 21) asks how many axes a material family needs.

6.2 Directions a matrix only stretches

Feed some vectors to the machine $\mathbf{S} = \left(\begin{smallmatrix} 2.5 & 2 \\ 2 & 2.5 \end{smallmatrix}\right)$:

$\mathbf{S}(1,\ 0) = (2.5,\ 2)$	a different direction
$\mathbf{S}(1,\ 1) = (2.5 + 2,\ 2 + 2.5) = (4.5,\ 4.5) = 4.5\,(1,\ 1)$	same direction, 4.5 times longer
$\mathbf{S}(1,\ -1) = (2.5 - 2,\ 2 - 2.5) = (0.5,\ -0.5) = 0.5\,(1,\ -1)$	same direction, half as long

Definition: eigenvector, eigenvalue

A non-zero vector $\mathbf{v}$ is an **eigenvector** of a square matrix $\mathbf{A}$ if $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$ for some number λ (lambda), called its **eigenvalue**. The machine only stretches $\mathbf{v}$ (by λ), without turning it.

So $(1, 1)$ is an eigenvector with eigenvalue 4.5, and $(1, -1)$ one with eigenvalue 0.5. Any multiple of an eigenvector is also one, so they are usually scaled to length 1: $\mathbf{e}_1 = (1, 1)/\sqrt{2} = (0.7071,\ 0.7071)$ and $\mathbf{e}_2 = (1, -1)/\sqrt{2} = (0.7071,\ -0.7071)$.

6.3 Finding eigenvalues by hand

Rearrange $\mathbf{S}\mathbf{v} = \lambda\mathbf{v}$. Write $\lambda\mathbf{v}$ as $\lambda\mathbf{I}\mathbf{v}$ and subtract it from both sides:

$$\mathbf{S}\mathbf{v} - \lambda\mathbf{I}\mathbf{v} = \mathbf{0}$$

$$(\mathbf{S} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0} \qquad \textit{take out the common factor } \mathbf{v}$$

The machine $\mathbf{S} - \lambda\mathbf{I}$ sends the non-zero $\mathbf{v}$ to zero, so (from Chapter 4) it must be singular: its determinant is zero. That gives an equation for λ alone.

Worked example 6.1: Eigenvalues and eigenvectors of S

$\mathbf{S} - \lambda\mathbf{I} = \begin{pmatrix} 2.5-\lambda & 2 \\ 2 & 2.5-\lambda \end{pmatrix}$. Determinant set to zero:

$$\begin{aligned} (2.5-\lambda)(2.5-\lambda) - 2\times 2 &= 0 \\ (2.5-\lambda)^2 &= 4 && \textit{add 4 to both sides} \\ 2.5-\lambda = 2 \quad \text{or} \quad 2.5-\lambda &= -2 && \textit{square root; both signs} \\ \lambda = 0.5 \quad \text{or} \quad \lambda &= 4.5. \end{aligned}$$

Eigenvector for $\lambda = 4.5$: $\mathbf{S} - 4.5\mathbf{I} = \left(\begin{smallmatrix} -2 & 2 \\ 2 & -2 \end{smallmatrix}\right)$. Row 1 says $-2v_1 + 2v_2 = 0$, so $v_1 = v_2$: the direction $(1, 1)$.
Eigenvector for $\lambda = 0.5$: $\mathbf{S} - 0.5\mathbf{I} = \left(\begin{smallmatrix} 2 & 2 \\ 2 & 2 \end{smallmatrix}\right)$. Row 1 says $2v_1 + 2v_2 = 0$, so $v_1 = -v_2$: the direction $(1, -1)$.

For any symmetric 2×2 matrix there is a ready formula. Expanding the determinant of $\left(\begin{smallmatrix} a-\lambda & b \\ b & d-\lambda \end{smallmatrix}\right)$:

$$(a-\lambda)(d-\lambda) - b^2 = \lambda^2 - (a+d)\lambda + (ad - b^2) = 0.$$

Call $T = a + d$ the **trace** (sum of the diagonal) and $D = ad - b^2$ the determinant. The school formula for a quadratic equation gives

$$\lambda = \frac{T \pm \sqrt{T^2 - 4D}}{2}.$$

Check on S: $T = 5$, $D = 6.25 - 4 = 2.25$, $T^2 - 4D = 25 - 9 = 16$, $\sqrt{16} = 4$, so $\lambda = (5 \pm 4)/2$, which is 4.5 or 0.5.

6.4 Eigenvalues are variances

By rule 5.1, the variance of the data along a unit direction $\mathbf{u}$ (the variance of $\mathbf{u}^\top\mathbf{w}$) is $\mathbf{u}^\top\mathbf{S}\mathbf{u}$. For an eigenvector, $\mathbf{S}\mathbf{u} = \lambda\mathbf{u}$, so $\mathbf{u}^\top\mathbf{S}\mathbf{u} = \lambda\,\mathbf{u}^\top\mathbf{u} = \lambda \times 1 = \lambda$. **The eigenvalue is the variance of the data along its eigenvector.**

Worked example 6.2: Variance along the long axis, from the data

Project each pixel onto $\mathbf{e}_1 = (0.7071,\ 0.7071)$:

$$(2, 1) \to 0.7071 \times 2 + 0.7071 \times 1 = 2.1213$$
$$(-2, -1) \to -2.1213, \qquad (1, 2) \to 2.1213, \qquad (-1, -2) \to -2.1213$$

Squares: 4.5 each. Average: 4.5. The eigenvalue.

What about other directions? A unit vector at angle θ is $(\cos\theta,\ \sin\theta)$, and

$$\mathbf{u}^\top\mathbf{S}\mathbf{u} = 2.5\cos^2\theta + 2 \times 2\cos\theta\sin\theta + 2.5\sin^2\theta = 2.5 + 4\cos\theta\sin\theta,$$

using $\cos^2\theta + \sin^2\theta = 1$. At $\theta = 0°$: 2.5. At 45°: $2.5 + 4 \times 0.7071 \times 0.7071 = 2.5 + 2 = 4.5$. At 90°: 2.5. At 135°: $2.5 + 4 \times (-0.7071) \times 0.7071 = 2.5 - 2 = 0.5$. The largest and smallest variances occur exactly at the two eigenvectors. That is true for every symmetric matrix.

6.5 Taking the matrix apart

The two eigenvectors are perpendicular: $\mathbf{e}_1^\top \mathbf{e}_2 = 0.7071 \times 0.7071 + 0.7071 \times (-0.7071) = 0$. That is also true for every symmetric matrix (eigenvectors with different eigenvalues are perpendicular). Put them side by side as the columns of a matrix $\mathbf{U}$, and the eigenvalues on the diagonal of $\mathbf{\Lambda}$ (capital lambda):

$$\mathbf{U} = \begin{pmatrix} 0.7071 & 0.7071 \\ 0.7071 & -0.7071 \end{pmatrix}, \qquad \mathbf{\Lambda} = \begin{pmatrix} 4.5 & 0 \\ 0 & 0.5 \end{pmatrix}.$$

Because the columns are perpendicular unit vectors, $\mathbf{U}^\top\mathbf{U} = \mathbf{I}$: each entry of $\mathbf{U}^\top\mathbf{U}$ is the dot product of two columns, which is 1 for a column with itself and 0 for two different columns. Such a $\mathbf{U}$ is called **orthonormal**; as a machine it is a rotation (possibly with a flip), and it changes no lengths and no angles.

Rule 6.1: Eigendecomposition of a symmetric matrix

$$\mathbf{S} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top,$$

with the eigenvectors as the columns of $\mathbf{U}$ and the eigenvalues on the diagonal of $\mathbf{\Lambda}$.
Check on $\mathbf{S}$. $\mathbf{U\Lambda}$ multiplies column 1 of $\mathbf{U}$ by 4.5 and column 2 by 0.5: $\mathbf{U\Lambda} = \left(\begin{smallmatrix} 3.182 & 0.354 \\ 3.182 & -0.354 \end{smallmatrix}\right)$. Then $(\mathbf{U\Lambda})\mathbf{U}^\top$ has entry $(1,1)$ equal to $3.182 \times 0.7071 + 0.354 \times 0.7071 = 2.25 + 0.25 = 2.5$, entry $(1,2)$ equal to $3.182 \times 0.7071 - 0.354 \times 0.7071 = 2.25 - 0.25 = 2$, and by the same steps entry $(2,2)$ equal to 2.5. That is $\mathbf{S}$.

Read $\mathbf{S} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$ right to left as a machine: rotate onto the eigen-axes ($\mathbf{U}^\top$), stretch each axis by its eigenvalue ($\mathbf{\Lambda}$), rotate back ($\mathbf{U}$). Every covariance matrix is "a stretch along perpendicular axes".

6.6 Principal components and energy shares

Definition: principal components, energy share, rank

The eigenvectors of a covariance matrix, sorted by decreasing eigenvalue, are its **principal components**. The **energy share** of a component is its eigenvalue divided by the sum of all eigenvalues. The sum of the eigenvalues equals the trace, the total variance. The **rank** is the number of non-zero eigenvalues: how many independent directions the data really use.

For $\mathbf{S}$: total variance $4.5 + 0.5 = 5$ (the trace, $2.5 + 2.5$). The first component's share is $4.5/5 = 0.9$. Keeping only the top component keeps 90% of the wobble. **Principal component analysis** (PCA) is the habit of describing data by its top few components and discarding the rest. The anomaly score of Chapter 17 keeps the top 30 of 165.

6.7 An uncentred version, for library families

For a material family the question is different: "do all these lab spectra point the same way?". The original note answers it by forming $\sum_i \mathbf{h}_i\mathbf{h}_i^\top$ over the family's spectra *without* subtracting their mean ("uncentred"), so that the common brightness direction itself counts. The eigenvectors of that matrix are the family's **singular directions**, and its eigenvalues are the squares of the **singular values** of the data matrix; you do not need more than that name.

Worked example 6.3: A two-spectrum family is nearly one direction

Two spectra of one material, in two normalised bands: $\mathbf{h}_1 = (2,\ 1)$ and a brighter, slightly different $\mathbf{h}_2 = (4,\ 2.2)$.

$$\mathbf{h}_1\mathbf{h}_1^\top = \begin{pmatrix} 4 & 2 \\ 2 & 1 \end{pmatrix}, \qquad \mathbf{h}_2\mathbf{h}_2^\top = \begin{pmatrix} 16 & 8.8 \\ 8.8 & 4.84 \end{pmatrix}$$

$$\mathbf{B} = \mathbf{h}_1\mathbf{h}_1^\top + \mathbf{h}_2\mathbf{h}_2^\top = \begin{pmatrix} 20 & 10.8 \\ 10.8 & 5.84 \end{pmatrix}$$

Trace $T = 20 + 5.84 = 25.84$. Determinant $D = 20 \times 5.84 - 10.8^2 = 116.8 - 116.64 = 0.16$.

$$T^2 = 667.7056, \qquad 4D = 0.64, \qquad T^2 - 4D = 667.0656, \qquad \sqrt{667.0656} = 25.8276$$
$$\lambda_1 = (25.84 + 25.8276)/2 = 25.8338, \qquad \lambda_2 = (25.84 - 25.8276)/2 = 0.0062$$

Top share: $25.8338/25.84 = 0.99976$. The family is one direction (one shape at two brightnesses) plus a sliver.

The shipped library's man-made families behave like this: the top direction holds 0.89 to 0.99 of the energy[C](Table 21.1).

Analogy: a rugby ball

A cloud of residuals is like a rugby ball floating in space. Its eigenvectors are the ball's axes: one long, two short. Its eigenvalues are how far the ball extends along each axis (squared). Rotating your head to line up with the axes is $\mathbf{U}^\top$; the ball's shape along those axes is $\mathbf{\Lambda}$. A cloud in 165 bands is a 165-dimensional rugby ball, very long in a few directions and very thin in most.

Common confusions

Is the eigenvector $(1, 1)$ or $(0.7071, 0.7071)$ or $(-1, -1)$? All are the same direction. Programs return unit length and an arbitrary sign.
Is the eigenvalue a standard deviation? No, a variance. The extent along $(1, 1)$ in data units is $\sqrt{4.5} = 2.12$.
Centred or uncentred? A covariance subtracts the mean first and describes the wobble around it. The family analysis does not subtract, and describes where the spectra point. They answer different questions and give different numbers.
Are eigenvectors always perpendicular? For symmetric matrices, which is every covariance matrix in this book, yes. For general matrices, no.

Check yourself

1. Verify by multiplication that $(1, -1)$ is an eigenvector of S and find its eigenvalue.
2. Find the eigenvalues and eigenvectors of $\left(\begin{smallmatrix} 3 & 1 \\ 1 & 3 \end{smallmatrix}\right)$ by hand.
3. What is the variance of this chapter's four pixels along the direction $(1, 0)$?
4. For $\left(\begin{smallmatrix} 3 & 1 \\ 1 & 3 \end{smallmatrix}\right)$, what share of the total variance does the top component hold?
5. What are the eigenvalues and eigenvectors of diag$(4,\ 1)$?

Check yourself: worked answers

1. $\mathbf{S}(1, -1) = (2.5 \times 1 + 2 \times (-1),\ 2 \times 1 + 2.5 \times (-1)) = (0.5,\ -0.5) = 0.5\,(1, -1)$. Eigenvalue 0.5.
2. $T = 6$, $D = 9 - 1 = 8$, $T^2 - 4D = 36 - 32 = 4$, $\sqrt{4} = 2$, $\lambda = (6 \pm 2)/2$, so 4 and 2. For 4: row 1 of

$\left(\begin{smallmatrix}-1 & 1\\ 1 & -1\end{smallmatrix}\right)$ gives $v_1 = v_2$, direction $(1, 1)$. For 2: row 1 of $\left(\begin{smallmatrix}1 & 1\\ 1 & 1\end{smallmatrix}\right)$ gives $v_1 = -v_2$, direction $(1, -1)$.

3. $(1, 0)^\top \mathbf{S}(1, 0) = S_{11} = 2.5$, the variance of band 1.
4. $4/(4 + 2) = 4/6 = 0.667$.
5. 4 and 1, with eigenvectors $(1, 0)$ and $(0, 1)$. A diagonal matrix only stretches along the axes.

Next: Chapter 7, measuring surprise in units of the usual wobble.

Chapter 7

Whitening and the Mahalanobis distance

The one thing this chapter teaches

Whitening rotates the data onto its principal axes and divides each axis by its standard deviation, so that the usual wobble becomes the same size in every direction; the squared length after whitening is the Mahalanobis distance $\mathbf{r}^\top \mathbf{S}^{-1}\mathbf{r}$.

7.1 The problem: a fair yardstick for surprise

Two residuals from the four-pixel world of Chapter 5: $(1,\ 1)$ and $(1,\ -1)$. Both have length $\sqrt{2}$. But the first lies along the long axis of the cloud, where wobbles of that size are routine, and the second along the short axis, where the usual wobble has variance only 0.5. The second is far more surprising. You need a measure of size that knows this.

7.2 One band first: standard scores

In one dimension the answer is familiar. A mark of 70 in an exam whose class mean is 60 and standard deviation 5 is $(70 - 60)/5 = 10/5 = 2$ standard deviations above average. A mark of 80 in an exam with mean 70 and standard deviation 20 is $(80 - 70)/20 = 10/20 = 0.5$. The second mark is higher but less remarkable.

Definition: standard score

The **standard score** or **z-score** of a value y is $(y - \bar{y})/\sigma$: its distance from the mean in units of standard deviation.

With several bands you cannot divide band by band, because bands are correlated: the long axis of the cloud is not a band. So first rotate onto the cloud's own axes, where the wobbles are uncorrelated, then divide each axis by its own standard deviation.

7.3 Whitening in two steps

Worked example 7.1: Whitening the pixel $(2,\ 1)$ by hand

Step 1, rotate. The coordinates along the eigenvectors are the dot products with $\mathbf{e}_1 = (0.7071,\ 0.7071)$ and $\mathbf{e}_2 = (0.7071,\ -0.7071)$:

$$\mathbf{e}_1^\top (2, 1) = 0.7071 \times 2 + 0.7071 \times 1 = 2.1213$$

$$\mathbf{e}_2^\top(2,1) = 0.7071 \times 2 - 0.7071 \times 1 = 0.7071$$

Step 2, divide by the standard deviation along each axis, $\sqrt{\lambda_1} = \sqrt{4.5} = 2.1213$ and $\sqrt{\lambda_2} = \sqrt{0.5} = 0.7071$:

$$2.1213/2.1213 = 1$$
$$0.7071/0.7071 = 1$$

The whitened pixel is $(1,\ 1)$.

Definition: whitening matrix

The two steps together form the **whitening matrix**

$$\mathbf{W} = \mathbf{\Lambda}^{-1/2}\mathbf{U}^\top,$$

where $\mathbf{\Lambda}^{-1/2} = \text{diag}(1/\sqrt{\lambda_1}, \ldots, 1/\sqrt{\lambda_N})$. $\mathbf{U}^\top$ rotates; $\mathbf{\Lambda}^{-1/2}$ divides each axis by its standard deviation.

For our S, row 1 of $\mathbf{W}$ is $\mathbf{e}_1^\top/\sqrt{4.5}$ and row 2 is $\mathbf{e}_2^\top/\sqrt{0.5}$. Since $\mathbf{e}_1 = (1,1)/\sqrt{2}$:

$$\text{row 1} = \frac{(1,\ 1)}{\sqrt{2}\sqrt{4.5}} = \frac{(1,\ 1)}{\sqrt{9}} = \left(\tfrac{1}{3},\ \tfrac{1}{3}\right)$$
$$\text{row 2} = \frac{(1,\ -1)}{\sqrt{2}\sqrt{0.5}} = \frac{(1,\ -1)}{\sqrt{1}} = (1,\ -1)$$

$$\mathbf{W} = \begin{pmatrix} 1/3 & 1/3 \\ 1 & -1 \end{pmatrix}.$$

Whiten all four pixels: $\mathbf{W}(2,1) = (1,\ 1)$, $\mathbf{W}(-2,-1) = (-1,\ -1)$, $\mathbf{W}(1,2) = (1,\ -1)$, $\mathbf{W}(-1,-2) = (-1,\ 1)$. The four corners of a square. In Chapter 5 (Check yourself, question 3) you computed that this square has covariance $\mathbf{I}$: variance 1 in every direction, no correlation. The stretched cloud has become round (Figure 7.1).

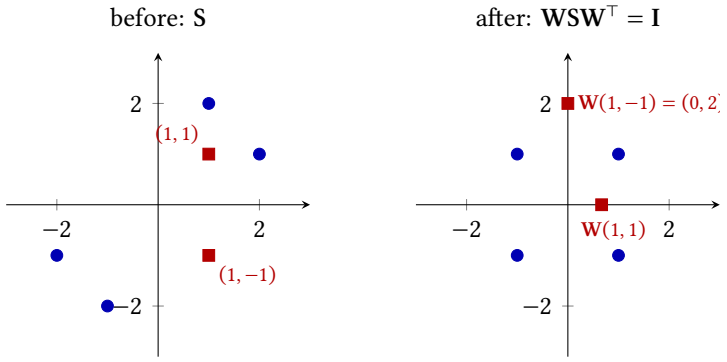


Figure 7.1: Whitening. Blue: the four background pixels. Red: two residuals of equal length. After whitening, the residual across the grain, $(1,-1)$, lies three times farther out than the one along it, $(1,1)$.

You can confirm the roundness with rule 5.2: the covariance of $\mathbf{Ww}$ is $\mathbf{WSW}^\top$. First $\mathbf{WS} = \begin{pmatrix} \frac{1}{3}(2.5+2) & \frac{1}{3}(2+2.5) \\ 2.5-2 & 2-2.5 \end{pmatrix} = \begin{pmatrix} 1.5 & 1.5 \\ 0.5 & -0.5 \end{pmatrix}$. Then multiplying by $\mathbf{W}^\top = \begin{pmatrix} 1/3 & 1 \\ 1/3 & -1 \end{pmatrix}$: entry $(1,1)$ is $1.5/3 + 1.5/3 = 1$, entry $(1,2)$ is $1.5 - 1.5 = 0$, entry $(2,1)$ is $0.5/3 - 0.5/3 = 0$, entry $(2,2)$ is $0.5 + 0.5 = 1$. The identity.

7.4 The Mahalanobis distance

Definition: Mahalanobis distance

The squared **Mahalanobis distance** of a residual $\mathbf{r}$ (from a cloud with mean zero and covariance $\mathbf{S}$) is the squared length of its whitened version,

$$\|\mathbf{W}\mathbf{r}\|^2 = \mathbf{r}^\top \mathbf{S}^{-1}\mathbf{r}.$$

Why the two forms agree: $\|\mathbf{W}\mathbf{r}\|^2 = (\mathbf{W}\mathbf{r})^\top(\mathbf{W}\mathbf{r}) = \mathbf{r}^\top\mathbf{W}^\top\mathbf{W}\mathbf{r}$ by rule 4.1, so you need $\mathbf{W}^\top\mathbf{W} = \mathbf{S}^{-1}$. Check it on the toy:

$$\mathbf{W}^\top\mathbf{W} = \begin{pmatrix} 1/3 & 1 \\ 1/3 & -1 \end{pmatrix}\begin{pmatrix} 1/3 & 1/3 \\ 1 & -1 \end{pmatrix} = \begin{pmatrix} \frac{1}{9}+1 & \frac{1}{9}-1 \\ \frac{1}{9}-1 & \frac{1}{9}+1 \end{pmatrix} = \frac{1}{9}\begin{pmatrix} 10 & -8 \\ -8 & 10 \end{pmatrix}.$$

And by rule 4.3, with determinant $2.5 \times 2.5 - 2 \times 2 = 6.25 - 4 = 2.25$,

$$\mathbf{S}^{-1} = \frac{1}{2.25}\begin{pmatrix} 2.5 & -2 \\ -2 & 2.5 \end{pmatrix} = \begin{pmatrix} 1.111 & -0.889 \\ -0.889 & 1.111 \end{pmatrix} = \frac{1}{9}\begin{pmatrix} 10 & -8 \\ -8 & 10 \end{pmatrix}.$$

The same. In general, $\mathbf{W}^\top\mathbf{W} = \mathbf{U}\boldsymbol{\Lambda}^{-1/2}\boldsymbol{\Lambda}^{-1/2}\mathbf{U}^\top = \mathbf{U}\boldsymbol{\Lambda}^{-1}\mathbf{U}^\top$, and multiplying by $\mathbf{S} = \mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^\top$ gives $\mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^\top\mathbf{U}\boldsymbol{\Lambda}^{-1}\mathbf{U}^\top = \mathbf{U}\boldsymbol{\Lambda}\boldsymbol{\Lambda}^{-1}\mathbf{U}^\top = \mathbf{U}\mathbf{U}^\top = \mathbf{I}$ (using $\mathbf{U}^\top\mathbf{U} = \mathbf{I}$ twice), so $\mathbf{U}\boldsymbol{\Lambda}^{-1}\mathbf{U}^\top$ is $\mathbf{S}^{-1}$.

Worked example 7.2: Two residuals of equal length

$\mathbf{r} = (1,\ -1)$, across the grain.

$$\mathbf{S}^{-1}\mathbf{r} = \tfrac{1}{9}(10 \times 1 - 8 \times (-1),\ -8 \times 1 + 10 \times (-1)) = \tfrac{1}{9}(18,\ -18) = (2,\ -2)$$
$$\mathbf{r}^\top\mathbf{S}^{-1}\mathbf{r} = 1 \times 2 + (-1) \times (-2) = 4$$

Via whitening: $\mathbf{W}\mathbf{r} = (\frac{1}{3} - \frac{1}{3},\ 1 + 1) = (0,\ 2)$, squared length $0 + 4 = 4$.
$\mathbf{r} = (1,\ 1)$, along the grain.

$$\mathbf{S}^{-1}\mathbf{r} = \tfrac{1}{9}(10 - 8,\ -8 + 10) = \tfrac{1}{9}(2,\ 2)$$
$$\mathbf{r}^\top\mathbf{S}^{-1}\mathbf{r} = \tfrac{2}{9} + \tfrac{2}{9} = \tfrac{4}{9} = 0.444$$

Via whitening: $\mathbf{W}\mathbf{r} = (\frac{2}{3},\ 0)$, squared length $\frac{4}{9}$.
Same Euclidean length $\sqrt{2}$, Mahalanobis distances 4 and 0.444: a factor of nine.

There is a third way to compute it that Chapter 19 relies on. In eigen-coordinates the whitened components are $\mathbf{e}_i^\top\mathbf{r}/\sqrt{\lambda_i}$, so

$$\mathbf{r}^\top\mathbf{S}^{-1}\mathbf{r} = \sum_i \frac{(\mathbf{e}_i^\top\mathbf{r})^2}{\lambda_i} \ :$$

each squared component along an axis, divided by that axis's variance, added up. For $\mathbf{r} = (1, -1)$: $\mathbf{e}_1^\top\mathbf{r} = 0$ and $\mathbf{e}_2^\top\mathbf{r} = 0.7071 + 0.7071 = 1.4142$, so the sum is $0/4.5 + 2/0.5 = 0 + 4 = 4$.

7.5 A new price list: the $\mathbf{S}^{-1}$ inner product

The Mahalanobis idea extends from lengths to dot products. Define

$$\langle \mathbf{u}, \mathbf{v} \rangle_{\mathbf{S}} = \mathbf{u}^\top\mathbf{S}^{-1}\mathbf{v} = (\mathbf{W}\mathbf{u})^\top(\mathbf{W}\mathbf{v}), \qquad \|\mathbf{u}\|_{\mathbf{S}}^2 = \mathbf{u}^\top\mathbf{S}^{-1}\mathbf{u}.$$

It is the ordinary dot product taken after whitening. Every formula of Chapters 2 and 3 works in this new "metric", including angles. In the shopping-bill analogy of Chapter 2, $\mathbf{S}^{-1}$ is a new price list: directions where the background rarely goes are expensive.

7.6 Throwing directions away, and the ridge

Two practical complications, both in the product's code.

Truncation. Keep only the top d rows of $\mathbf{W}$, the ones for the largest eigenvalues. The product's anomaly score keeps $d = 30$ (Chapter 17). With $d = 1$ in the toy, $\mathbf{W}_1 = (\frac{1}{3}, \frac{1}{3})$, and $\mathbf{W}_1(1, -1) = 0$: the most surprising residual becomes invisible. Truncation trades away the thin directions for stability.

The ridge. If an eigenvalue is tiny, dividing by it amplifies small errors enormously: a component of 0.01 along a direction of variance 10^{-6} contributes $0.01^2/10^{-6} = 0.0001/0.000001 = 100$. The family matcher adds a small number to every eigenvalue first, 10^{-3} times the mean eigenvalue (`app/spectral_match/family_match.py`, `RIDGE = 1e-3`). In the toy, the mean eigenvalue is $(4.5 + 0.5)/2 = 2.5$, the ridge is 0.0025, and the eigenvalues become 4.5025 and 0.5025: a negligible change here, decisive when the smallest eigenvalues are noise.

$\mathbf{S}^{-1/2}$ versus $\mathbf{W}$. Some formulas whiten with $\mathbf{S}^{-1/2} = \mathbf{U}\mathbf{\Lambda}^{-1/2}\mathbf{U}^\top$ instead of $\mathbf{W} = \mathbf{\Lambda}^{-1/2}\mathbf{U}^\top$. The two differ by the rotation $\mathbf{U}$ at the front, which changes no lengths or angles, so every distance and angle in this book is the same with either.

Analogy: exam marks in standard deviations

Converting raw exam marks into standard deviations above the class mean lets you compare a maths mark with a history mark. Whitening does the same for a residual, except that the "subjects" are first rearranged into the uncorrelated axes of the cloud. The Mahalanobis distance is then the sum of squared standard scores across all those axes. Remember the height-weight person from Chapter 5: tall and light is a big standard score on the thin axis.

Common confusions

Is whitening the same as the normalisation by wavelength of Chapter 15? No. Normalisation divides each band by a fixed, global standard deviation and never mixes bands. Whitening uses the scene's own covariance, rotates, and divides each *axis*. Both happen in the product, in that order.
Is the Mahalanobis distance the square root or the square? The definition above, and the product's score, use the square. Its square root is in units of standard deviations.
Is W unique? No; any rotation of the whitened space gives another valid whitening. Lengths and angles do not depend on the choice.
Does whitening help if a direction was thrown away? Nothing in a discarded direction can be seen afterwards (truncation, above). Chapter 17 comes back to this.

Check yourself

1. A mark of 55 in an exam with mean 50 and sd 2: what is its z-score?
2. Compute the squared Mahalanobis distance of $\mathbf{r} = (2,\ 1)$ under $\mathbf{S}$ by $\mathbf{S}^{-1}\mathbf{r}$, then check with $\mathbf{W}\mathbf{r}$.
3. Compute $\mathbf{W}(3,\ 0)$ and the squared Mahalanobis distance of $(3,\ 0)$. Compare with its squared Euclidean length.
4. Of $(1, 1)$ and $(1, -1)$, which is the more surprising residual, and by what factor?
5. If only the top whitened component is kept, what is the score of $(1, -1)$? What has been lost?

Check yourself: worked answers

1. $(55 - 50)/2 = 5/2 = 2.5$.
2. $\mathbf{S}^{-1}\mathbf{r} = \frac{1}{9}(10 \times 2 - 8 \times 1,\ -8 \times 2 + 10 \times 1) = \frac{1}{9}(12,\ -6) = (1.333,\ -0.667)$. Then $\mathbf{r}^\top\mathbf{S}^{-1}\mathbf{r} =$

$2 \times 1.333 + 1 \times (-0.667) = 2.667 - 0.667 = 2$. Whitening: $\mathbf{W}(2, 1) = (1,\ 1)$, squared length $1 + 1 = 2$.

3. $\mathbf{W}(3, 0) = (\frac{1}{3} \times 3 + \frac{1}{3} \times 0,\ 3 - 0) = (1,\ 3)$. Squared length $1 + 9 = 10$. Euclidean: 9. Close here, because $(3, 0)$ has parts along both axes.
4. $(1, -1)$: 4 against 0.444, a factor of $4/0.444 = 9$.
5. $\mathbf{W}_1(1, -1) = \frac{1}{3} - \frac{1}{3} = 0$, score 0. Everything across the grain, which was exactly what made it surprising.

Next: Chapter 8, finding the bottom of a valley.

Chapter 8

Slopes and minima: the calculus you need

The one thing this chapter teaches

At the bottom of a smooth valley the slope is zero, so you find a best estimate by writing down its error, computing the slope (the derivative), setting it to zero and solving.

8.1 The problem: "best" means "smallest error"

Chapter 18 asks: given a residual and a candidate material, what fill a fits best? "Fits best" will mean "makes a certain squared error as small as possible". You could try many values of a and keep the smallest error, and worked example 18.1 does exactly that as a check. But there is a faster, exact way, and every estimate in this book uses it. It needs one idea from calculus, the derivative, and a handful of rules. This chapter gives you those and nothing more.

8.2 Slope and the derivative

The straight line $y = 3x + 1$ rises 3 for every 1 you move to the right: its **slope** is 3. A curve such as $y = x^2$ has a different slope at every point. At $x = 3$, move a tiny step to $x = 3.001$:

$$\begin{aligned} f(3.001) &= 3.001^2 = 9.006001 \\ f(3) &= 9 \\ \text{rise} &= 9.006001 - 9 = 0.006001 \\ \text{slope} \approx \text{rise/run} &= 0.006001/0.001 = 6.001. \end{aligned}$$

Make the step smaller and the answer settles on 6.

Definition: derivative

The **derivative** of a function $f(x)$, written $f'(x)$ or $\frac{df}{dx}$, is the slope of the curve at x: the limit of rise over run as the run shrinks to zero. The **second derivative** $f''(x)$ is the derivative of $f'(x)$; it says whether the slope is increasing (curve bending up, like a bowl) or decreasing (bending down, like a hill).

You will not need to compute limits. You need the following rules, each of which you can check numerically the way the slope of x^2 was checked above.

Rule 8.1: Derivative rules

function	derivative	numerical check at one point
a constant c	0	a flat line has no slope
$c\,x$	c	a straight line
x^2	$2x$	at 3: $6.001 \approx 6$ (above)
x^k	$k\,x^{k-1}$	x^3 at 1: $(1.001^3 - 1)/0.001 = 3.003 \approx 3$
$1/x$	$-1/x^2$	at 2: $(1/2.001 - 1/2)/0.001 = -0.24988 \approx -0.25$
$\ln x$	$1/x$	at 2: $(\ln 2.001 - \ln 2)/0.001 = 0.49988 \approx 0.5$

Sum: the derivative of $f + g$ is $f' + g'$. **Constant multiple**: the derivative of $c\,f$ is $c\,f'$. **Chain**: the derivative of $f(g(x))$ is $f'(g(x)) \times g'(x)$; for example $(3x + 1)^2$ has derivative $2(3x + 1) \times 3$. At $x = 1$ that is $2 \times 4 \times 3 = 24$. Check by expanding first: $(3x + 1)^2 = 9x^2 + 6x + 1$, derivative $18x + 6$, at $x = 1$ equal to 24.

(The natural logarithm ln is defined in Chapter 10; it is listed here so all the rules are in one place.)

8.3 Minimum where the slope is zero

At the bottom of a valley the ground is flat: the slope is zero. To the left the slope is negative (going down), to the right positive (going up), so the slope is increasing: the second derivative is positive. That is the test for a minimum. A hilltop also has zero slope, but there the second derivative is negative: a maximum.

Worked example 8.1: The bottom of a quadratic

$J(a) = 9 - 12a + 4a^2$. Derivative, term by term with rule 8.1:

$$J'(a) = 0 - 12 + 4 \times 2a = -12 + 8a.$$

Set it to zero and solve:

$$\begin{aligned} -12 + 8a &= 0 \\ 8a &= 12 && \textit{add 12 to both sides} \\ a &= 1.5 && \textit{divide both sides by 8} \end{aligned}$$

Second derivative: $J''(a) = 8 > 0$, a minimum. Check by trying values:

$$\begin{aligned} J(1) &= 9 - 12 + 4 = 1 \\ J(1.5) &= 9 - 18 + 4 \times 2.25 = 9 - 18 + 9 = 0 \\ J(2) &= 9 - 24 + 16 = 1 \end{aligned}$$

Smallest at 1.5.

This pattern is so common that it deserves a general version. Any quadratic $J(a) = A - 2Ba + Ca^2$ with $C > 0$ has

$$\begin{aligned} J'(a) &= -2B + 2Ca \\ -2B + 2Ca &= 0 && \textit{set to zero} \\ 2Ca &= 2B && \textit{add } 2B \textit{ to both sides} \\ a &= B/C && \textit{divide both sides by } 2C \end{aligned}$$

In worked example 8.1, $A = 9$, $B = 6$, $C = 4$, and $B/C = 1.5$. Remember the shape "minimiser = B/C":

the fill estimate of Chapter 18 is exactly this, with B and C made of dot products.

8.4 Two or more unknowns: the gradient

When the error depends on several numbers $\theta_1, \theta_2, \ldots$ at once, you take the derivative with respect to each one while holding the others fixed. These are **partial derivatives**, written $\partial f/\partial\theta_1$. Stacked into a vector they form the **gradient** ∇f. At a minimum every partial derivative is zero.

Worked example 8.2: A minimum in two unknowns

$f(\theta_1, \theta_2) = \theta_1^2 + \theta_2^2 - 2\theta_1 - 4\theta_2$.

$$\begin{aligned} \partial f/\partial\theta_1 &= 2\theta_1 - 2 && \theta_2 \text{ terms are constants here} \\ \partial f/\partial\theta_2 &= 2\theta_2 - 4 \\ 2\theta_1 - 2 = 0 &\;\Rightarrow\; \theta_1 = 1 \\ 2\theta_2 - 4 = 0 &\;\Rightarrow\; \theta_2 = 2 \end{aligned}$$

With matrices, two gradient rules cover everything in this book.

Rule 8.2: Gradients of a linear term and of a quadratic form

For a fixed vector $\mathbf{b}$ and a fixed symmetric matrix $\mathbf{A}$,

$$\nabla(\mathbf{b}^\top\boldsymbol{\theta}) = \mathbf{b}, \qquad \nabla(\boldsymbol{\theta}^\top\mathbf{A}\boldsymbol{\theta}) = 2\mathbf{A}\boldsymbol{\theta}.$$

Proof for two unknowns. $\mathbf{b}^\top\boldsymbol{\theta} = b_1\theta_1 + b_2\theta_2$, whose partial derivatives are b_1 and b_2. For the quadratic form, with $A_{12} = A_{21}$,

$$\boldsymbol{\theta}^\top\mathbf{A}\boldsymbol{\theta} = A_{11}\theta_1^2 + 2A_{12}\theta_1\theta_2 + A_{22}\theta_2^2.$$

Partial derivative in θ_1: $2A_{11}\theta_1 + 2A_{12}\theta_2$, which is twice the first component of $\mathbf{A}\boldsymbol{\theta} = (A_{11}\theta_1 + A_{12}\theta_2,\ A_{12}\theta_1 + A_{22}\theta_2)$. In θ_2: $2A_{12}\theta_1 + 2A_{22}\theta_2$, twice the second component. So the gradient is $2\mathbf{A}\boldsymbol{\theta}$. *Numerical check.* $\mathbf{A} = \left(\begin{smallmatrix} 2 & 1 \\ 1 & 3 \end{smallmatrix}\right)$, $\boldsymbol{\theta} = (1, 2)$. The form is $2\theta_1^2 + 2\theta_1\theta_2 + 3\theta_2^2$. Its partials at $(1, 2)$: $4\theta_1 + 2\theta_2 = 4 + 4 = 8$ and $2\theta_1 + 6\theta_2 = 2 + 12 = 14$. And $2\mathbf{A}\boldsymbol{\theta} = 2 \times (2 + 2,\ 1 + 6) = 2 \times (4,\ 7) = (8,\ 14)$. Equal.

Use it on the matrix version of the quadratic, $J(\boldsymbol{\theta}) = \boldsymbol{\theta}^\top\mathbf{A}\boldsymbol{\theta} - 2\mathbf{b}^\top\boldsymbol{\theta}$:

$$\begin{aligned} \nabla J &= 2\mathbf{A}\boldsymbol{\theta} - 2\mathbf{b} \\ 2\mathbf{A}\boldsymbol{\theta} - 2\mathbf{b} &= \mathbf{0} && \text{set the gradient to zero} \\ \mathbf{A}\boldsymbol{\theta} &= \mathbf{b} && \text{add } 2\mathbf{b}\text{, divide by } 2 \\ \boldsymbol{\theta} &= \mathbf{A}^{-1}\mathbf{b} && \text{multiply both sides on the left by } \mathbf{A}^{-1} \end{aligned}$$

The same shape as $a = B/C$, with a matrix inverse in place of the division. With $\mathbf{A}$ as above and $\mathbf{b} = (4,\ 7)$, the answer is $\boldsymbol{\theta} = (1,\ 2)$, because $\mathbf{A}(1, 2) = (4, 7)$ (worked example 4.4).

8.5 Maximising over a variance

Chapter 20 has to find the variance that makes the data most likely. The function it meets is

$$f(v) = -\frac{N}{2}\ln v - \frac{Q}{2v},$$

where v is the unknown variance and N, Q are fixed positive numbers.

$$
\begin{aligned}
f'(v) &= -\frac{N}{2}\cdot\frac{1}{v} - \frac{Q}{2}\cdot\left(-\frac{1}{v^2}\right) && \text{rules for } \ln v \text{ and } 1/v \\
&= -\frac{N}{2v} + \frac{Q}{2v^2} \\
-\frac{N}{2v} + \frac{Q}{2v^2} &= 0 && \text{set to zero} \\
-Nv + Q &= 0 && \text{multiply both sides by } 2v^2 \\
v &= Q/N.
\end{aligned}
$$

Worked example 8.3: The best variance

$N = 3$, $Q = 43$ (these come back in Chapter 20). The formula gives $v = 43/3 = 14.333$. Check three values of $f(v) = -1.5\ln v - 21.5/v$:

$$
\begin{aligned}
f(10) &= -1.5 \times 2.3026 - 2.15 = -3.4539 - 2.15 = -5.6039 \\
f(14.333) &= -1.5 \times 2.6626 - 1.5 = -3.9939 - 1.5 = -5.4939 \\
f(20) &= -1.5 \times 2.9957 - 1.075 = -4.4936 - 1.075 = -5.5686
\end{aligned}
$$

The largest (least negative) is at 14.333: a maximum.

Analogy: finding the bottom of a valley in fog

You are in a valley in thick fog and want the lowest point. You cannot see it, but you can feel the slope under your feet. Wherever the ground tilts, the bottom is downhill. When the ground is flat in every direction, you are there. Setting the derivative (or every partial derivative) to zero is "find where the ground is flat"; the second derivative confirms it is a valley and not a hilltop.

Common confusions

Does a zero slope always mean a minimum? No: it can be a maximum or a flat shelf. Check the second derivative, or try values on either side as in worked example 8.1.
Why minimise the squared error rather than the error? Squared error has a single smooth bottom that calculus can find. It also turns out (Chapter 18) to be what the Gaussian model of noise says is most likely.
What if A is not symmetric? Then the gradient of $\boldsymbol{\theta}^\top \mathbf{A}\boldsymbol{\theta}$ is $(\mathbf{A} + \mathbf{A}^\top)\boldsymbol{\theta}$. Every matrix in this book's quadratic forms is a covariance or an inverse covariance, hence symmetric.
Is ∇ a vector or a number? A vector, one partial derivative per unknown.

Check yourself

1. What is the derivative of $5x^2 - 3x + 2$?
2. Find the a that minimises $J(a) = 4 - 8a + 16a^2$ and the minimum value.
3. Minimise $\boldsymbol{\theta}^\top \left(\begin{smallmatrix} 2 & 0 \\ 0 & 4 \end{smallmatrix}\right)\boldsymbol{\theta} - 2\,(2,\ 8)^\top\boldsymbol{\theta}$.
4. Check numerically that the slope of x^3 at $x = 2$ is about 12.
5. Find the v that maximises $f(v) = -2\ln v - 8/v$.

Check yourself: worked answers

1. $5 \times 2x - 3 + 0 = 10x - 3$.
2. $J'(a) = -8{+}32a$. Setting $-8{+}32a = 0$ gives $32a = 8$, $a = 0.25$. $J(0.25) = 4{-}8{\times}0.25{+}16{\times}0.0625 =$

$4 - 2 + 1 = 3$.

3. Here $\mathbf{A} = \mathrm{diag}(2, 4)$ and $\mathbf{b} = (2, 8)$, so $\boldsymbol{\theta} = \mathbf{A}^{-1}\mathbf{b} = (2/2,\ 8/4) = (1,\ 2)$.
4. $2.001^3 = 8.012006$; rise $8.012006 - 8 = 0.012006$; slope $0.012006/0.001 = 12.006 \approx 12 = 3 \times 2^2$.
5. This is the form $-\frac{N}{2}\ln v - \frac{Q}{2v}$ with $N = 4$, $Q = 16$. So $v = 16/4 = 4$. Directly: $f'(v) = -2/v + 8/v^2 = 0$, multiply by v^2: $-2v + 8 = 0$, $v = 4$.

Next: Chapter 9, splitting a vector into a part along a target and the rest.

Chapter 9

Projections onto a line and onto a subspace

The one thing this chapter teaches

Projecting a vector onto a line or a subspace finds the closest point there; the vector splits into that projection plus a perpendicular remainder, and the squared lengths add up (Pythagoras), which gives the "explained" and "unexplained" energies and the projection angle.

9.1 The problem: how much of the residual looks like the target?

A residual is the target's contribution plus a lot of other things. To ask "is there metal here?" you want to pull out the part of the residual that points the way metal would, measure it, and compare it with what is left over. For a single candidate spectrum the part is along a line; for a family of spectra it is in a flat slab of directions called a subspace. Both are projections.

9.2 Projection onto a line

Definition: projection onto a line

The **projection** of a vector $\mathbf{r}$ onto the line through the origin in direction $\mathbf{d}$ is the point $c\,\mathbf{d}$ on that line closest to $\mathbf{r}$.

Find c by minimising the squared distance $J(c) = \|\mathbf{r} - c\,\mathbf{d}\|^2$. Expand with rule 2.1:

$$J(c) = \|\mathbf{r}\|^2 - 2c\,\mathbf{d}^\top\mathbf{r} + c^2\|\mathbf{d}\|^2.$$

This is the quadratic $A - 2Bc + Cc^2$ of Chapter 8 with $B = \mathbf{d}^\top\mathbf{r}$ and $C = \|\mathbf{d}\|^2$, so the minimiser is B/C:

$$c = \frac{\mathbf{d}^\top\mathbf{r}}{\mathbf{d}^\top\mathbf{d}}, \qquad \text{projection} = \frac{\mathbf{d}^\top\mathbf{r}}{\mathbf{d}^\top\mathbf{d}}\,\mathbf{d}.$$

The **remainder** $\mathbf{r} - c\,\mathbf{d}$ is perpendicular to $\mathbf{d}$:

$$\begin{aligned}\mathbf{d}^\top(\mathbf{r} - c\,\mathbf{d}) &= \mathbf{d}^\top\mathbf{r} - c\,\mathbf{d}^\top\mathbf{d} \\ &= \mathbf{d}^\top\mathbf{r} - \frac{\mathbf{d}^\top\mathbf{r}}{\mathbf{d}^\top\mathbf{d}}\,\mathbf{d}^\top\mathbf{d} && \textit{substitute } c \\ &= \mathbf{d}^\top\mathbf{r} - \mathbf{d}^\top\mathbf{r} = 0.\end{aligned}$$

Worked example 9.1: Projecting $(3,1)$ onto the direction $(1,1)$

$\mathbf{d}^\top\mathbf{r} = 1\times 3 + 1\times 1 = 4$. $\mathbf{d}^\top\mathbf{d} = 1+1 = 2$. $c = 4/2 = 2$. Projection $2\,(1,1) = (2,\ 2)$. Remainder $(3-2,\ 1-2) = (1,\ -1)$. Check perpendicular: $(1,1)^\top(1,-1) = 1-1 = 0$.
Squared lengths: $\|\mathbf{r}\|^2 = 9+1 = 10$; projection $4+4 = 8$; remainder $1+1 = 2$. And $8+2 = 10$.

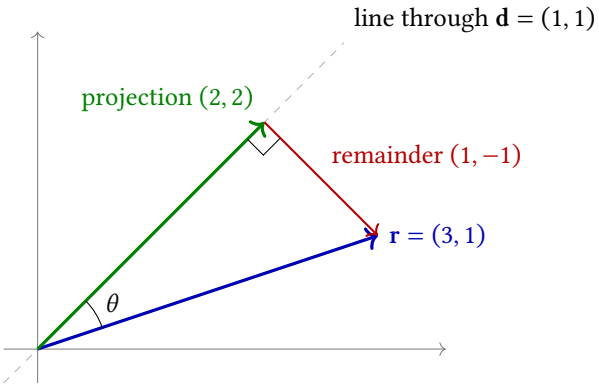


Figure 9.1: Projection onto a line. The remainder meets the line at a right angle, so $\|\mathbf{r}\|^2 = \|\text{projection}\|^2 + \|\text{remainder}\|^2$, here $10 = 8 + 2$. The projection angle θ has $\cos^2\theta = 8/10$.

Because the remainder is perpendicular to the projection, the three vectors form a right-angled triangle and Pythagoras holds: the squared lengths add up. That gives a clean way to describe how well the line explains $\mathbf{r}$.

Definition: explained energy, unexplained energy, projection angle

The squared length of the projection is the **explained energy**; the squared length of the remainder is the **unexplained energy**. They add up to $\|\mathbf{r}\|^2$. The **projection angle** θ between $\mathbf{r}$ and the line has

$$\cos^2\theta = \frac{\text{explained energy}}{\|\mathbf{r}\|^2} = \frac{(\mathbf{d}^\top\mathbf{r})^2}{\|\mathbf{d}\|^2\|\mathbf{r}\|^2}.$$

In worked example 9.1, $\cos^2\theta = 8/10 = 0.8$, so $\cos\theta = 0.894$ and $\theta = 26.6^\circ$. The second form of the formula is the square of rule 3.1: the projection angle onto a line is the ordinary angle between $\mathbf{r}$ and $\mathbf{d}$. The (squared) cosine in that formula will reappear in Chapter 20 under the name ACE.

9.3 The projection as a matrix

The projection is linear in $\mathbf{r}$, so it is a matrix:

$$\mathbf{P} = \frac{\mathbf{d}\mathbf{d}^\top}{\mathbf{d}^\top\mathbf{d}}, \qquad \mathbf{P}\mathbf{r} = \frac{\mathbf{d}(\mathbf{d}^\top\mathbf{r})}{\mathbf{d}^\top\mathbf{d}}.$$

For $\mathbf{d} = (1,1)$: $\mathbf{d}\mathbf{d}^\top = \left(\begin{smallmatrix}1&1\\1&1\end{smallmatrix}\right)$, so $\mathbf{P} = \frac{1}{2}\left(\begin{smallmatrix}1&1\\1&1\end{smallmatrix}\right)$, and $\mathbf{P}(3,1) = \frac{1}{2}(4,\ 4) = (2,\ 2)$.
Two properties identify every projection matrix. **Projecting twice is the same as once**, $\mathbf{P}\mathbf{P} = \mathbf{P}$: a point already on the line stays put. Check: $\mathbf{P}(2,2) = \frac{1}{2}(4,4) = (2,2)$. And **it is symmetric**, $\mathbf{P}^\top = \mathbf{P}$. The matrix that keeps the remainder is $\mathbf{P}^\perp = \mathbf{I} - \mathbf{P}$, read "P-perp".

9.4 Subspaces

Definition: span, subspace, basis, dimension

The **span** of vectors $\mathbf{h}_1, \dots, \mathbf{h}_q$ is the set of all their linear combinations $\theta_1\mathbf{h}_1 + \cdots + \theta_q\mathbf{h}_q$. Such a set is a **subspace**: a flat slab through the origin (a line, a plane, or a higher-dimensional version). If no $\mathbf{h}_j$ is a combination of the others, they form a **basis** and q is the subspace's **dimension**. Writing the $\mathbf{h}_j$ as the columns of a matrix $\mathbf{H}$, the span is all vectors $\mathbf{H}\boldsymbol{\theta}$, also called the **range** of $\mathbf{H}$.

A material family will be a subspace: the span of a few library spectra (Chapter 21). A "nuisance" such as brightness or haze will be another (Chapter 22).

9.5 Projection onto a subspace

Find the $\boldsymbol{\theta}$ for which $\mathbf{H}\boldsymbol{\theta}$ is closest to $\mathbf{r}$. Minimise $J(\boldsymbol{\theta}) = \|\mathbf{r} - \mathbf{H}\boldsymbol{\theta}\|^2$. Expand:

$$\begin{aligned} J(\boldsymbol{\theta}) &= (\mathbf{r} - \mathbf{H}\boldsymbol{\theta})^\top(\mathbf{r} - \mathbf{H}\boldsymbol{\theta}) \\ &= \mathbf{r}^\top\mathbf{r} - \mathbf{r}^\top\mathbf{H}\boldsymbol{\theta} - \boldsymbol{\theta}^\top\mathbf{H}^\top\mathbf{r} + \boldsymbol{\theta}^\top\mathbf{H}^\top\mathbf{H}\boldsymbol{\theta} && \text{multiply out; } (\mathbf{H}\boldsymbol{\theta})^\top = \boldsymbol{\theta}^\top\mathbf{H}^\top \\ &= \mathbf{r}^\top\mathbf{r} - 2(\mathbf{H}^\top\mathbf{r})^\top\boldsymbol{\theta} + \boldsymbol{\theta}^\top(\mathbf{H}^\top\mathbf{H})\boldsymbol{\theta} && \text{the two middle terms are equal numbers (rule 4.2)} \end{aligned}$$

This is $\boldsymbol{\theta}^\top\mathbf{A}\boldsymbol{\theta} - 2\mathbf{b}^\top\boldsymbol{\theta}$ plus a constant, with $\mathbf{A} = \mathbf{H}^\top\mathbf{H}$ (symmetric) and $\mathbf{b} = \mathbf{H}^\top\mathbf{r}$. By Chapter 8 the minimiser solves $\mathbf{A}\boldsymbol{\theta} = \mathbf{b}$:

$$\begin{aligned} \mathbf{H}^\top\mathbf{H}\,\boldsymbol{\theta} &= \mathbf{H}^\top\mathbf{r} && \text{the "normal equations"} \\ \boldsymbol{\theta} &= (\mathbf{H}^\top\mathbf{H})^{-1}\mathbf{H}^\top\mathbf{r} \\ \text{projection} = \mathbf{H}\boldsymbol{\theta} &= \underbrace{\mathbf{H}(\mathbf{H}^\top\mathbf{H})^{-1}\mathbf{H}^\top}_{\mathbf{P}_\mathbf{H}}\,\mathbf{r}. \end{aligned}$$

With one column, $\mathbf{H} = \mathbf{d}$, this is the line formula again.

Worked example 9.2: Projecting onto a plane in three bands

$\mathbf{h}_1 = (1,\ 1,\ 0)$, $\mathbf{h}_2 = (0,\ 1,\ 1)$, $\mathbf{r} = (1,\ 2,\ 3)$.
Step 1: $\mathbf{H}^\top\mathbf{H}$. Its entries are dot products of columns: $\mathbf{h}_1^\top\mathbf{h}_1 = 1 + 1 + 0 = 2$, $\mathbf{h}_1^\top\mathbf{h}_2 = 0 + 1 + 0 = 1$, $\mathbf{h}_2^\top\mathbf{h}_2 = 0 + 1 + 1 = 2$.

$$\mathbf{H}^\top\mathbf{H} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}, \qquad (\mathbf{H}^\top\mathbf{H})^{-1} = \frac{1}{2\times 2 - 1\times 1}\begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix} = \frac{1}{3}\begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}.$$

Step 2: $\mathbf{H}^\top\mathbf{r} = (\mathbf{h}_1^\top\mathbf{r},\ \mathbf{h}_2^\top\mathbf{r}) = (1 + 2 + 0,\ 0 + 2 + 3) = (3,\ 5)$.
Step 3: $\boldsymbol{\theta} = \frac{1}{3}(2\times 3 - 1\times 5,\ -1\times 3 + 2\times 5) = \frac{1}{3}(1,\ 7) = (0.333,\ 2.333)$.
Step 4: projection $\mathbf{H}\boldsymbol{\theta} = 0.333\,(1, 1, 0) + 2.333\,(0, 1, 1) = (0.333,\ 2.667,\ 2.333)$.
Step 5: remainder $\mathbf{r} - \mathbf{H}\boldsymbol{\theta} = (1 - 0.333,\ 2 - 2.667,\ 3 - 2.333) = (0.667,\ -0.667,\ 0.667)$. Check it is perpendicular to both basis vectors: $\mathbf{h}_1$: $0.667 - 0.667 + 0 = 0$; $\mathbf{h}_2$: $0 - 0.667 + 0.667 = 0$.
Step 6: energies. $\|\mathbf{r}\|^2 = 1 + 4 + 9 = 14$. Explained: $(1 + 64 + 49)/9 = 114/9 = 12.667$. Unexplained: $3\times(2/3)^2 = 12/9 = 1.333$. Sum: $12.667 + 1.333 = 14$. Projection angle: $\cos^2\theta = 12.667/14 = 0.905$, $\cos\theta = 0.951$, $\theta = 18.0^\circ$.

The projection matrix for this plane is $\mathbf{P}_\mathbf{H} = \frac{1}{9}\begin{pmatrix} 6 & 3 & -3 \\ 3 & 6 & 3 \\ -3 & 3 & 6 \end{pmatrix}$, and (you can check it) $\mathbf{P}_\mathbf{H}\mathbf{P}_\mathbf{H} = \mathbf{P}_\mathbf{H}$.

9.6 Projecting in the whitened metric

Every projection above uses the ordinary dot product. The detectors of Part IV project in the $\mathbf{S}^{-1}$ metric of Chapter 7: they whiten first ($\tilde{\mathbf{r}} = \mathbf{W}\mathbf{r}$, $\tilde{\mathbf{H}} = \mathbf{W}\mathbf{H}$; a tilde marks a whitened quantity) and then project as above. Writing it out without the whitening gives $\boldsymbol{\theta} = (\mathbf{H}^\top\mathbf{S}^{-1}\mathbf{H})^{-1}\mathbf{H}^\top\mathbf{S}^{-1}\mathbf{r}$, the same formula with $\mathbf{S}^{-1}$ slipped into every dot product.

9.7 The angle between two subspaces

Definition: principal angle

The smallest **principal angle** between two subspaces is the smallest angle between any vector in the first and any vector in the second.

Two lines meeting at 30° have principal angle 30°. Two planes in three dimensions, the floor (span of $(1,0,0)$ and $(0,1,0)$) and a wall (span of $(0,1,0)$ and $(0,0,1)$), share the direction $(0,1,0)$, so their smallest principal angle is 0°. For material families the smallest principal angle says how close the two families come: Chapter 21 finds the shipped man-made families between 0.51° and 5.92° apart[C].

A useful consequence: if a unit vector in one subspace makes angle θ with the other subspace, the share of its energy *outside* the other subspace is $\sin^2\theta$ (the remainder's squared length, since the projection's squared length is $\cos^2\theta$ and they add to 1).

Analogy: a shadow at noon

At noon with the sun overhead, a flagpole leaning in the wind casts its shadow straight down onto the ground. The shadow is the projection of the pole onto the ground (the subspace); the vertical drop from the pole's tip to its shadow is the remainder, and it meets the ground at a right angle. Pole length squared equals shadow squared plus drop squared. A pole nearly flat on the ground has a long shadow and small projection angle: the ground "explains" it well.

Common confusions

Is the projection the coefficient c or the vector $c\mathbf{d}$? The vector. c (or $\boldsymbol{\theta}$) is the set of coordinates of the projection in the basis.
Do the basis vectors need to be perpendicular? No. The formula with $(\mathbf{H}^\top\mathbf{H})^{-1}$ handles any basis. Worked example 9.2 used two that are not perpendicular ($\mathbf{h}_1^\top\mathbf{h}_2 = 1$).
Why do energies add but lengths not? Pythagoras is about squares: $\sqrt{8}+\sqrt{2} = 4.24 \neq \sqrt{10} = 3.16$.
Is a bigger subspace always better at explaining? It always explains at least as much energy, even of pure noise. A 3-dimensional family collects about three times the noise energy of a 1-dimensional one (Chapter 21). That is why raw scores of families of different sizes cannot be compared.

Check yourself

1. Project $(2,\ 0)$ onto the direction $(1,\ 1)$. Give the projection and the remainder.
2. For question 1, check that the energies add up, and give the projection angle.
3. Write the projection matrix onto the direction $(1,\ 0)$. What does it do?
4. Project $\mathbf{r} = (1,\ 0,\ 0)$ onto the plane of worked example 9.2 using $\mathbf{P}_\mathbf{H}$.
5. A unit target direction makes 5.92° with another family's subspace. What share of its energy lies outside that subspace?

Check yourself: worked answers

1. $\mathbf{d}^\top\mathbf{r} = 2$, $\mathbf{d}^\top\mathbf{d} = 2$, $c = 1$. Projection $(1,\ 1)$; remainder $(2-1,\ 0-1) = (1,\ -1)$.
2. $\|\mathbf{r}\|^2 = 4$; projection $1+1=2$; remainder $1+1=2$; $2+2=4$. $\cos^2\theta = 2/4 = 0.5$, $\theta = 45^\circ$.
3. $\mathbf{d}\mathbf{d}^\top/\mathbf{d}^\top\mathbf{d} = \left(\begin{smallmatrix}1 & 0\\ 0 & 0\end{smallmatrix}\right)$. It keeps the first component and sets the second to zero.
4. The first column of $\mathbf{P_H}$: $\frac{1}{9}(6,\ 3,\ -3) = (0.667,\ 0.333,\ -0.333)$.
5. $\sin(5.92^\circ) = 0.1031$; squared, 0.01064, about 1.1%. (This is the paint-versus-plastic number of Chapter 21.)

Next: Chapter 10, how likely a residual is.

Chapter 10

Probability, the Gaussian, likelihood and the logarithm

The one thing this chapter teaches

A Gaussian curve says how likely each value of a noisy quantity is; the likelihood turns that around and asks which explanation makes the observed data most likely, and its logarithm turns products into sums you can differentiate.

10.1 The problem: noise is random

Residuals of ordinary pixels are not zero; they wobble. A residual of 0.3 might be a target, or it might be an unlucky wobble. To decide, you need to say how often a wobble that big happens by chance. That is a question of probability, and the shape of chance used throughout signal processing is the Gaussian bell curve.

10.2 Probability, random variables, expectation

Definition: probability, random variable

A **probability** is a number between 0 and 1 describing how often something happens in the long run: probability 0.01 means about once in a hundred tries. A **random variable** is a quantity whose value varies from try to try, such as the noise in band 1 of a pixel. Capital letters (X, Z) name random variables.

Definition: expectation

The **expectation** $\mathbb{E}[X]$ is the long-run average of X over many tries. Two rules: $\mathbb{E}[X+Y] = \mathbb{E}[X]+\mathbb{E}[Y]$, and $\mathbb{E}[cX] = c\,\mathbb{E}[X]$ for a fixed number c. The variance is $\mathrm{Var}(X) = \mathbb{E}[(X-\mathbb{E}X)^2]$, the long-run version of Chapter 5's formula.

Definition: independence

Two random variables are **independent** if knowing one tells you nothing about the other. Then their covariance is zero and **variances add**: $\mathrm{Var}(X+Y) = \mathrm{Var}(X) + \mathrm{Var}(Y)$. (This is rule 5.1 with $\mathbf{c} = (1,1)$ and a zero off-diagonal.)

10.3 Densities and areas

A noise value can be any number, not one of a list, so you cannot give each value its own probability. Instead you give a curve, the **probability density**, and the probability of landing in a range is the *area* under the curve over that range. The total area under a density is 1.

Definition: integral

The **integral** $\int_u^v f(x)\,dx$ is the area under the curve f between $x = u$ and $x = v$ (area below the axis counting as negative). You will only need two facts about it: areas of neighbouring ranges add up, and if F is a function whose derivative is f, then $\int_u^v f(x)\,dx = F(v) - F(u)$.

10.4 The Gaussian

Definition: standard normal, Gaussian

The **standard normal** density is

$$\varphi(z) = \frac{1}{\sqrt{2\pi}}\, e^{-z^2/2},$$

where $e = 2.71828\ldots$ and $\pi = 3.14159\ldots$. A random variable Z with this density has mean 0 and standard deviation 1, written $Z \sim \mathcal{N}(0,1)$. A **Gaussian** (or normal) variable with mean m and standard deviation σ, $X \sim \mathcal{N}(m, \sigma^2)$, is $X = m + \sigma Z$; its density is $\frac{1}{\sigma}\varphi\big(\frac{x-m}{\sigma}\big)$.

Some values: $\varphi(0) = 1/\sqrt{2\pi} = 1/2.5066 = 0.3989$; $\varphi(1) = 0.3989 \times e^{-0.5} = 0.3989 \times 0.6065 = 0.2420$; $\varphi(2) = 0.3989 \times e^{-2} = 0.3989 \times 0.1353 = 0.0540$. The curve falls off very fast.

Definition: Φ and Q

$\Phi(z)$ is the area under φ to the *left* of z: the probability that $Z \le z$. $Q(z) = 1 - \Phi(z)$ is the area to the *right*: the probability that $Z > z$, called the **tail probability**. $Q^{-1}(p)$ is the z whose right tail is p. By symmetry, $Q(-z) = \Phi(z)$.

There is no simple formula for Φ; you look it up. Every value used in this book is in this table (`compute_step_by_step_output.txt`):

z	0	0.5	1.0	1.28	1.645	1.96	2.0	2.326	3.0	3.09
$\Phi(z)$	0.5	0.6915	0.8413	0.8997	0.9500	0.9750	0.9772	0.9900	0.99865	0.99900
$Q(z)$	0.5	0.3085	0.1587	0.1003	0.0500	0.0250	0.0228	0.0100	0.00135	0.00100

The last column matters most: **a standard normal exceeds 3.09 only once in a thousand tries.** More precisely, $Q^{-1}(0.001) = 3.0902$.

Worked example 10.1: How often does noise exceed 0.3?

Noise in one band is $\mathcal{N}(0,\ 0.1^2)$: mean 0, sd 0.1. The chance of a value above 0.3:

$$\text{standardise: } z = (0.3 - 0)/0.1 = 3$$
$$P(X > 0.3) = Q(3) = 0.00135.$$

About one pixel in 740 ($1/0.00135 = 741$).

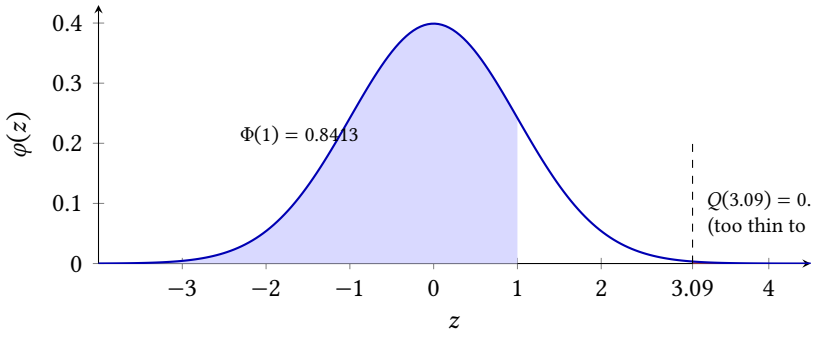


Figure 10.1: The standard normal density. The shaded blue area left of $z = 1$ is $\Phi(1) = 0.8413$. The red area right of 3.09 is 0.001, one part in a thousand.

10.5 The average size of a Gaussian wobble

The original note converts measured *mean absolute* errors into standard deviations with the factor $\sqrt{\pi/2}$. Here is where it comes from. The mean of $|Z|$ is the area under $|z|\,\varphi(z)$; by symmetry, twice the area for $z \geq 0$:

$$\mathrm{E}|Z| = 2\int_0^\infty z\,\varphi(z)\,dz.$$

You need a function whose derivative is $z\varphi(z) = \frac{1}{\sqrt{2\pi}} z e^{-z^2/2}$. Try $F(z) = -\varphi(z) = -\frac{1}{\sqrt{2\pi}} e^{-z^2/2}$. By the chain rule (with the rule that e^u is its own derivative), its derivative is $-\frac{1}{\sqrt{2\pi}} e^{-z^2/2} \times (-z) = z\varphi(z)$. It works. So

$$\int_0^\infty z\varphi(z)\,dz = F(\infty) - F(0) = 0 - (-\varphi(0)) = \varphi(0) = 0.3989$$

$$\mathrm{E}|Z| = 2 \times 0.3989 = 0.7979 = \sqrt{2/\pi}.$$

For $X = \sigma Z$, the mean absolute value is $\sigma \times 0.7979$; turned round, $\sigma = \text{mean}|X| \times 1.2533$ (since $1/0.7979 = 1.2533 = \sqrt{\pi/2}$).

Worked example 10.2: From a mean absolute error to a standard deviation

The original note reads FT04's measured background error, mean absolute 0.10 normalised units per band, as a Gaussian sd:

$$\sigma_e = 0.10 \times 1.2533 = 0.1253.$$

The same for 0.05 and 0.14 gives 0.063 and 0.175, the values in the note[C].

10.6 Many bands: the multivariate Gaussian

For a vector of N noisy bands with mean $\mathbf{m}$ and covariance $\mathbf{S}$, the Gaussian density is

$$p(\mathbf{x}) = \frac{1}{\sqrt{(2\pi)^N \det \mathbf{S}}} \exp\Big(-\tfrac{1}{2}(\mathbf{x}-\mathbf{m})^{\top}\mathbf{S}^{-1}(\mathbf{x}-\mathbf{m})\Big),$$

written $\mathbf{x} \sim \mathcal{N}(\mathbf{m}, \mathbf{S})$. Look at the exponent: it is minus one half of the squared Mahalanobis distance of Chapter 7. **Under a Gaussian model, "unlikely" and "far in Mahalanobis distance" are the same thing.** For a diagonal $\mathbf{S} = \mathrm{diag}(\sigma_1^2, \sigma_2^2)$ the exponent is $-\frac{1}{2}\big(\frac{(x_1-m_1)^2}{\sigma_1^2} + \frac{(x_2-m_2)^2}{\sigma_2^2}\big)$ and the density is the product of two one-band densities: independent bands.

10.7 The logarithm

Definition: natural logarithm

The **natural logarithm** $\ln y$ (for $y > 0$) is the power you must raise e to in order to get y: $e^{\ln y} = y$, and $\ln(e^x) = x$. So $\ln 1 = 0$, $\ln e = 1$, and ln of a number between 0 and 1 is negative.

Rule 10.1: Logarithm rules

$$\ln(ab) = \ln a + \ln b, \qquad \ln(a/b) = \ln a - \ln b, \qquad \ln(a^k) = k \ln a.$$

Check. $\ln 2 = 0.6931$, $\ln 3 = 1.0986$, sum 1.7918; and $\ln 6 = 1.7918$. $\ln(0.001) = \ln(0.1^3) = 3 \times (-2.3026) = -6.9078$.
The logarithm only ever increases: if $a > b$ then $\ln a > \ln b$. So **the value that maximises a positive quantity also maximises its logarithm.**

10.8 Likelihood

So far: given an explanation (a mean, a variance), how likely are the data? **Likelihood** turns the question round: given the data, how well does each candidate explanation account for them?

Definition: likelihood, log-likelihood, maximum likelihood

The **likelihood** of a parameter value is the density of the observed data computed as if that value were true, viewed as a function of the parameter. For independent observations it is the product of their densities. The **log-likelihood** ℓ is its logarithm, a sum by rule 10.1. The **maximum-likelihood estimate** is the parameter value with the largest likelihood (equivalently, the largest log-likelihood).

Worked example 10.3: Which mean explains three readings best?

Three readings, 0.9, 1.1, 1.3, from a Gaussian with known sd $\sigma = 0.2$ and unknown mean m. Try $m = 1.1$. The standardised readings are $(0.9 - 1.1)/0.2 = -1$, 0, $+1$, so the densities are

$$\begin{aligned} \varphi(-1)/0.2 &= 0.2420/0.2 = 1.2099 \\ \varphi(0)/0.2 &= 0.3989/0.2 = 1.9947 \\ \varphi(1)/0.2 &= 1.2099 \\ \text{likelihood} &= 1.2099 \times 1.9947 \times 1.2099 = 2.920. \end{aligned}$$

Try $m = 1.0$. Standardised readings -0.5, 0.5, 1.5; $\varphi(0.5) = 0.3521$, $\varphi(1.5) = 0.1295$:

$$\text{likelihood} = (0.3521/0.2) \times (0.3521/0.2) \times (0.1295/0.2) = 1.7603 \times 1.7603 \times 0.6476 = 2.007.$$

By symmetry $m = 1.2$ also gives 2.007. The best of the three is $m = 1.1$, the average of the readings. Log-likelihoods: $\ln 2.920 = 1.0715$ and $\ln 2.007 = 0.6965$.

That the best mean is the average is no accident. Write the log-likelihood for readings $x_1, \ldots, x_n$ with known σ:

$$\begin{aligned} \ell(m) &= \sum_{i=1}^{n} \ln\Big(\frac{1}{\sigma\sqrt{2\pi}} e^{-(x_i-m)^2/(2\sigma^2)}\Big) \\ &= \sum_{i=1}^{n} \Big(-\ln(\sigma\sqrt{2\pi}) - \frac{(x_i-m)^2}{2\sigma^2}\Big) \qquad \textit{rule 10.1; } \ln e^u = u \end{aligned}$$

$$= \text{const} - \frac{1}{2\sigma^2}\sum_i (x_i - m)^2.$$

Maximising a Gaussian log-likelihood is the same as minimising a sum of squared errors. Differentiate in m (chain rule: the derivative of $(x_i - m)^2$ is $-2(x_i - m)$):

$$\ell'(m) = \frac{1}{\sigma^2}\sum_i (x_i - m) = 0$$

$$\sum_i x_i - n\,m = 0 \qquad \textit{multiply by } \sigma^2;\ \sum_i m = nm$$

$$m = \frac{1}{n}\sum_i x_i.$$

Analogy: a detective weighing suspects

A detective does not ask "how likely is the butler guilty?" directly. She asks, for each suspect, "if this person did it, how unsurprising would the evidence be?" The suspect who makes the evidence least surprising is her best explanation. That number, computed per suspect, is the likelihood. It is not the probability that the suspect is guilty; it is how well the suspect explains what was found.

Common confusions

Can a density be bigger than 1? Yes. Densities are not probabilities; areas are. The likelihood 2.920 above is a product of densities of a narrow curve ($\sigma = 0.2$), and is perfectly fine.
Is the likelihood the probability that the parameter is right? No (see the analogy). Turning one into the other needs Bayes' rule and prior beliefs, which Chapter 27 introduces.
ln **or** log**?** In this book both mean the natural logarithm, base e. A base-10 logarithm would change constants but not where the maximum is.
Is σ or σ^2 in $\mathcal{N}(m, \sigma^2)$? The second slot is the variance. $\mathcal{N}(0,\ 0.1^2)$ has sd 0.1.

Check yourself

1. What is the probability that a standard normal exceeds 2?
2. Noise has sd 0.05. How often does it exceed 0.1?
3. Compute $\ln(e^3)$ and $\ln(0.001)$.
4. The note's smallest background error is a mean absolute value of 0.05. What Gaussian sd is that?
5. Two readings, 2 and 4, with $\sigma = 1$. Compare the log-likelihoods of $m = 2$ and $m = 3$ (the constant can be dropped).

Check yourself: worked answers

1. $Q(2) = 0.0228$, about 1 in 44.
2. $z = 0.1/0.05 = 2$, so again $Q(2) = 0.0228$.
3. $\ln(e^3) = 3$. $\ln(0.001) = -6.9078$.
4. $0.05 \times 1.2533 = 0.0627$, which rounds to the note's 0.063.
5. Drop the constant: $\ell(m) = -\frac{1}{2}[(2-m)^2 + (4-m)^2]$. For $m = 2$: $-\frac{1}{2}(0+4) = -2$. For $m = 3$: $-\frac{1}{2}(1+1) = -1$. $m = 3$, the average, is better.

Next: Chapter 11, deciding between "nothing" and "something".

Chapter 11

Hypothesis tests, thresholds, false alarms and detections

The one thing this chapter teaches

A detector computes a number from the pixel and compares it with a threshold; the threshold fixes how often ordinary pixels are flagged (the false-alarm rate), and the signal strength then decides how often real targets are caught (the detection rate).

11.1 The problem: every rule makes two kinds of mistake

For each pixel you must decide "target" or "no target". Set the bar low and you catch every target but also flag thousands of ordinary pixels. Set it high and the false alarms vanish, along with the faint targets. There is no free lunch. Hypothesis testing is the bookkeeping that tells you exactly what you are trading.

11.2 The parts of a test

Definition: hypotheses, test statistic, threshold

The **null hypothesis** H_0 is "nothing there": the pixel is background. The **alternative** H_1 is "something there". A **test statistic** T is a number computed from the pixel, larger when H_1 is more plausible. The **threshold** η (eta) is the bar: declare a detection when $T > \eta$.

Definition: false-alarm rate, detection rate

The **false-alarm rate** $P_{\text{FA}} = P(T > \eta \mid H_0)$ is the share of background pixels flagged. The **detection rate** $P_D = P(T > \eta \mid H_1)$ is the share of target pixels flagged. (The bar "|" reads "given".)

The simplest case is the one every later detector reduces to: under H_0, $T \sim \mathcal{N}(0, 1)$; under H_1, $T \sim \mathcal{N}(\mu, 1)$, the same bell shifted right by μ (mu).

Worked example 11.1: False-alarm and detection rates

Choose $\eta = 3.09$. Under H_0: $P_{\text{FA}} = Q(3.09) = 0.001$.
If $\mu = 4$: $T - 4 \sim \mathcal{N}(0, 1)$, so

$$P_D = P(T > 3.09) = P(T - 4 > 3.09 - 4) = Q(-0.91) = \Phi(0.91) = 0.819.$$

If $\mu = 3.09$: $P_D = Q(3.09 - 3.09) = Q(0) = 0.5$, a coin toss.
If $\mu = 6$: $P_D = Q(3.09 - 6) = Q(-2.91) = \Phi(2.91) = 0.998$.

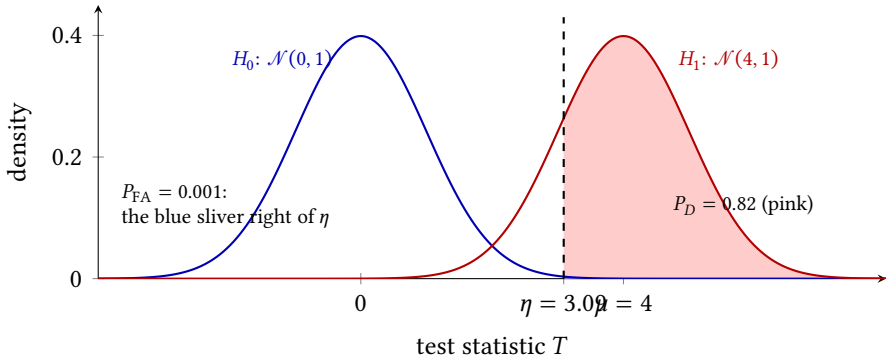


Figure 11.1: A threshold $\eta = 3.09$ flags 0.1% of background pixels (the blue sliver, too thin to see) and 82% of targets whose statistic has mean 4 (pink).

The general formula, used in Chapter 19, is

$$P_D = Q\big(Q^{-1}(P_{\mathrm{FA}}) - \mu\big).$$

11.3 Required signal-to-noise ratio

Turn the formula round: how big must μ be to reach a given P_D at a given P_{FA}? Apply Q^{-1} to both sides, $Q^{-1}(P_D) = Q^{-1}(P_{\mathrm{FA}}) - \mu$, and add $\mu - Q^{-1}(P_D)$ to both sides:

$$\mu = Q^{-1}(P_{\mathrm{FA}}) - Q^{-1}(P_D).$$

The square μ^2 is called the **signal-to-noise ratio** (SNR) of the detection.

Worked example 11.2: The required SNR of 9.55

$P_{\mathrm{FA}} = 10^{-3}$: $Q^{-1}(0.001) = 3.0902$. $P_D = 0.5$: $Q^{-1}(0.5) = 0$.

$$\mu = 3.0902 - 0 = 3.0902$$
$$\mathrm{SNR} = \mu^2 = 3.0902^2 = 9.5496 \approx 9.55.$$

That is the "required SNR 9.55" of the original note[C]. For $P_D = 0.9$ instead: $Q^{-1}(0.9) = -1.2816$, so $\mu = 3.0902 + 1.2816 = 4.3718$ and $\mu^2 = 19.11$. Going from a coin toss to 90% detection doubles the SNR you need.

11.4 Thresholds by percentile, and false alarms per scene

A false-alarm rate is per pixel. A PRISMA scene has on the order of a million pixels, so $P_{\mathrm{FA}} = 10^{-3}$ means about a thousand false alarms per scene. In practice the product never sets η from a formula; it flags a fixed share of the scene, for example its top 1% of scores ("Thresholding is percentile, never absolute", `docs/05-detectors.md`). Chapter 27 explains why.

11.5 The ROC curve and pAUC

Definition: ROC curve, pAUC

Sliding the threshold from high to low traces the **ROC curve**: detection rate against false-alarm rate. The **partial area under the curve** $\mathrm{pAUC}_{\leq 1\%}$ is the area under the ROC between false-alarm rates 0 and 0.01, divided by 0.01. It is 1 for a perfect detector and 0.005 for a random one (`app/evaluation/metrics.py`, pauc).

Why 0.005 for random? A random detector's ROC is the diagonal, $P_D = P_{\mathrm{FA}}$. The area under it from 0 to 0.01 is a triangle, $\frac{1}{2} \times 0.01 \times 0.01 = 0.00005$; divided by 0.01 that is 0.005.

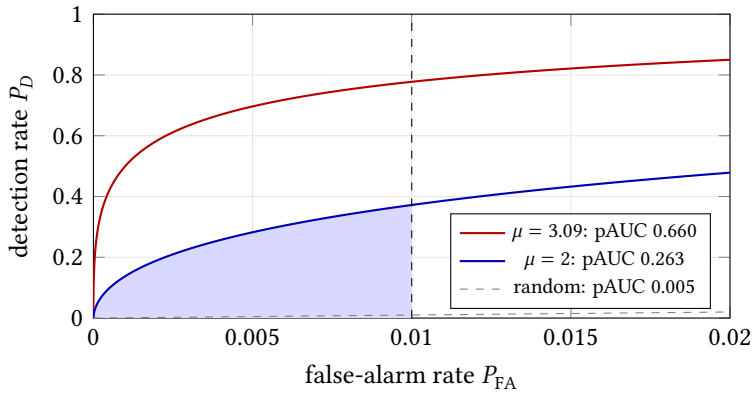


Figure 11.2: ROC curves of the two-bell test for two signal strengths, false-alarm rates up to 2%. The shaded area left of 1%, divided by 0.01, is the pAUC of the $\mu = 2$ detector[C] (`figdata/roc_toy.dat`, written by `compute_step_by_step.py`).

The project reports pAUC on four labelled PRISMA scenes, together with the share of objects "caught" (an object counts if one of its pixels lands in the top 0.1% or top 1% of the scene's scores). For scale: the current release candidate, Indradhanu Heavy fine-tuned (FTH3 epoch 5), has $\mathrm{pAUC}_{\leq 1\%} = 0.189$ against 0.170 for the shipped FT04 model, and catches 16.0% of objects at the top 0.1% and 49.4% at the top 1% (`research/experiments/indradhanu-finetune/FTH3-heavy-joint-more2/README.md`)[M].

11.6 Likelihood ratios and the generalised likelihood ratio test

Which statistic T should you compute? If you know the densities under both hypotheses, the best choice (for any fixed false-alarm rate) is the **likelihood ratio**, $p(\text{data} \mid H_1)/p(\text{data} \mid H_0)$. That fact is the Neyman-Pearson lemma; this book uses it without proof.

Usually H_1 contains unknown numbers: the fill, the target's brightness. The standard fix is to plug in the best-fitting values.

Definition: GLRT

The **generalised likelihood ratio test** (GLRT) uses

$$\Lambda = \frac{\max_{\text{unknowns under } H_1} \; p(\text{data} \mid H_1)}{\max_{\text{unknowns under } H_0} \; p(\text{data} \mid H_0)},$$

usually through $2 \ln \Lambda$: twice the best log-likelihood under H_1 minus twice the best under H_0.

Worked example 11.3: The GLRT for an unknown mean in one band

One reading $x \sim \mathcal{N}(\mu, 1)$. H_0: $\mu = 0$. H_1: μ is anything. Log-likelihood (dropping the constant $-\ln\sqrt{2\pi}$, which cancels in the difference): $\ell(\mu) = -\frac{1}{2}(x-\mu)^2$.

$$\text{best under } H_1: \quad \mu = x, \qquad \ell = -\tfrac{1}{2}(x-x)^2 = 0$$
$$\text{under } H_0: \quad \ell = -\tfrac{1}{2}x^2$$
$$2\ln\Lambda = 2\big(0 - (-\tfrac{1}{2}x^2)\big) = x^2.$$

The GLRT squares the reading. With $x = 2.5$: $2\ln\Lambda = 6.25$.

11.7 The chi-square family

Definition: chi-square distribution

If $Z_1, \ldots, Z_q$ are independent standard normals, $Z_1^2 + \cdots + Z_q^2$ has the **chi-square distribution with q degrees of freedom**, written χ_q^2. Its mean is q, because each $\mathbb{E}[Z_i^2] = \mathrm{Var}(Z_i) = 1$.
If instead each Z_i is shifted by μ_i, the sum $\sum(Z_i+\mu_i)^2$ is **noncentral chi-square** with **noncentrality** $\delta = \sum_i \mu_i^2$. Its mean is $q + \delta$: expanding, $\mathbb{E}[(Z+\mu)^2] = \mathbb{E}[Z^2] + 2\mu\mathbb{E}[Z] + \mu^2 = 1 + 0 + \mu^2$.

In worked example 11.3 the statistic x^2 is χ_1^2 under H_0. Chapters 17 to 21 meet χ_q^2 for $q = 1$, 3 and 30. The thresholds at $P_{\mathrm{FA}} = 10^{-3}$ and the noncentrality needed for $P_D = 0.5$ come from tables[C]:

degrees of freedom q	1	3	10	30
χ_q^2 threshold at $P_{\mathrm{FA}} = 10^{-3}$	10.83	16.27	29.59	59.70
noncentrality δ for $P_D = 0.5$	10.83	14.24	20.53	30.59

Two entries you can check by hand. For $q = 1$, $Z^2 > \eta$ means $|Z| > \sqrt{\eta}$, which has probability $2Q(\sqrt{\eta})$. Setting $2Q(\sqrt{\eta}) = 0.001$ needs $Q(\sqrt{\eta}) = 0.0005$, so $\sqrt{\eta} = 3.29$ and $\eta = 3.29^2 = 10.83$. For $q = 2$ there is an exact formula, $P(\chi_2^2 > \eta) = e^{-\eta/2}$; setting it to 0.001 gives $\eta = -2\ln 0.001 = -2 \times (-6.9078) = 13.82$, which the script confirms.

Analogy: a smoke alarm

A smoke alarm's sensitivity is its threshold. Too sensitive and it shrieks at burnt toast (false alarms); too dull and it sleeps through a fire (missed detections). Moving the dial trades one for the other, and the ROC curve is the full list of trades the alarm can offer. A better alarm (a better test statistic) is one whose whole curve sits higher: more fires caught at the same toast rate.

Common confusions

Is $P_{\mathrm{FA}} = 10^{-3}$ "a one in a thousand chance this detection is false"? No. It is the share of *background* pixels flagged. If targets are rare, most flags can still be false: 1000 false alarms against 20 targets.
Why 9.55 here and 10.83 in the chi-square table for one dimension? 9.55 is for a one-sided test, $T > \eta$, which looks only for positive signals. The chi-square test $T^2 > \eta$ is two-sided and spends half its false alarms on the negative side, so it needs a bit more. Chapter 19 uses the one-sided version because a fill cannot be negative.
Is a high pAUC the same as catching most objects? No: pAUC counts pixels, "caught" counts objects. They usually move together but are different measures.
Does the GLRT always give the best test? Not always, but it is the standard recipe and it behaves

well when the data are plentiful. Every detector in Part IV is a GLRT.

Check yourself

1. With threshold $\eta = 2.326$ on a standard normal T, what is P_{FA}?
2. With $\eta = 3.09$ and $\mu = 4$, show that $P_D = 0.819$.
3. Compute the SNR needed for $P_{\text{FA}} = 10^{-3}$ and $P_D = 0.9$.
4. A scene has 2 million valid pixels. How many false alarms do you expect at $P_{\text{FA}} = 10^{-3}$?
5. Find the χ^2_2 threshold for $P_{\text{FA}} = 0.01$ using $P(\chi^2_2 > \eta) = e^{-\eta/2}$.

Check yourself: worked answers

1. $Q(2.326) = 0.01$, one pixel in a hundred.
2. $P_D = Q(3.09 - 4) = Q(-0.91) = \Phi(0.91) = 0.819$.
3. $\mu = Q^{-1}(0.001) - Q^{-1}(0.9) = 3.0902 - (-1.2816) = 4.3718$; SNR $= 4.3718^2 = 19.11$.
4. $2{,}000{,}000 \times 0.001 = 2000$.
5. $e^{-\eta/2} = 0.01$; take logs: $-\eta/2 = \ln 0.01 = -4.6052$; multiply by -2: $\eta = 9.21$.

Next: Part II, the physical problem, starting with Chapter 12.

Part II

The problem

Chapter 12

The mixing model and the fill

The one thing this chapter teaches

A pixel holding a small target is modelled as $\mathbf{x} = a\,\mathbf{t} + (1-a)\,\mathbf{b} + \mathbf{n}$: fill a of target, the rest background, plus sensor noise; subtracting the background leaves the fill times the target-minus-background difference.

12.1 The problem: targets are smaller than pixels

PRISMA and EnMAP pixels are 30 m across. Most things worth finding (a shed roof, a tarpaulin, a fuel spill, a small greenhouse) are smaller. A 10 m by 10 m roof covers $10 \times 10 = 100$ of the pixel's $30 \times 30 = 900$ square metres, a share of $100/900 = 0.111$. The rest of the pixel is grass, soil or road. So the pixel's spectrum is not the roof's spectrum; it is a blend dominated by whatever surrounds the roof. Every method in this book is a way of seeing a little target through a lot of background.

12.2 The model

Definition: the linear mixing model

At pixel p,

$$\mathbf{x}_p = a_p\,\mathbf{t} + (1 - a_p)\,\mathbf{b}_p + \mathbf{n}_p, \qquad 0 \le a_p \le 1,$$

where $\mathbf{t}$ is the **target** spectrum (the same everywhere), $\mathbf{b}_p$ the **local background** (what pixel p would show if the target were not there), a_p the **fill** at that pixel, and $\mathbf{n}_p$ **sensor noise**, random with mean zero and covariance Σ_n. The original note labels this "Model 1.3, linear sub-pixel mixing"; it is an assumption[A], not a law.

It is the mixing of Chapter 1 with two additions: the background changes from pixel to pixel (hence the subscript p), and the sensor adds noise. The model is linear because each component of $\mathbf{x}_p$ is a weighted average of the corresponding components of $\mathbf{t}$ and $\mathbf{b}_p$. Light that bounces between materials before reaching the sensor (inside a tree canopy, between grains of a sand-oil mixture) breaks the straight-line rule; the note, and this book, assume it away.

Worked example 12.1: A noisy mixed pixel

Target metal $\mathbf{t} = (0.3,\ 0.3,\ 0.3)$, background grass $\mathbf{b} = (0.1,\ 0.1,\ 0.5)$, fill $a = 0.2$, noise this time $\mathbf{n} = (0.01,\ -0.02,\ 0.00)$. From worked example 1.2, $0.2\,\mathbf{t} + 0.8\,\mathbf{b} = (0.14,\ 0.14,\ 0.46)$. Add the noise

band by band:

$$\begin{aligned} \text{band 1:} &\quad 0.14 + 0.01 = 0.15 \\ \text{band 2:} &\quad 0.14 - 0.02 = 0.12 \\ \text{band 3:} &\quad 0.46 + 0.00 = 0.46 \end{aligned}$$

The sensor reports $\mathbf{x} = (0.15,\ 0.12,\ 0.46)$.

12.3 The difference form

You saw in Chapter 1 that subtracting the background from a noiseless mixture leaves $a(\mathbf{t} - \mathbf{b})$. With noise, repeat the steps:

$$\begin{aligned} \mathbf{x}_p - \mathbf{b}_p &= a_p\mathbf{t} + (1 - a_p)\mathbf{b}_p + \mathbf{n}_p - \mathbf{b}_p \\ &= a_p\mathbf{t} + \mathbf{b}_p - a_p\mathbf{b}_p + \mathbf{n}_p - \mathbf{b}_p && \textit{multiply out } (1 - a_p)\mathbf{b}_p \\ &= a_p\mathbf{t} - a_p\mathbf{b}_p + \mathbf{n}_p && \mathbf{b}_p - \mathbf{b}_p \textit{ cancels} \\ &= a_p(\mathbf{t} - \mathbf{b}_p) + \mathbf{n}_p && \textit{take out the factor } a_p \end{aligned}$$

The signal is the fill times the difference spectrum, and the noise sits on top of it. In worked example 12.1: $\mathbf{x} - \mathbf{b} = (0.05,\ 0.02,\ -0.04)$, which is $0.2 \times (0.2,\ 0.2,\ -0.2) = (0.04,\ 0.04,\ -0.04)$ plus the noise $(0.01,\ -0.02,\ 0)$.

Look at the sizes. The signal is 0.04 per band; the noise here is up to 0.02. At a fill of 0.05 the signal would be 0.01 per band, the same size as the noise. Small fills are hard, and a large part of this book measures exactly how hard.

12.4 What you know and what you do not

In a real pixel you observe only $\mathbf{x}_p$. Of the other four quantities:

- $\mathbf{t}$ is a candidate: you will try each library spectrum in turn (Part IV);
- $\mathbf{b}_p$ is unknown and must be estimated, either from a ring of neighbouring pixels (the classical way) or by a learned model that predicts it from the surroundings (Part III);
- a_p is what you want, and it is estimated last (Chapter 18);
- $\mathbf{n}_p$ is never known, only its covariance.

12.5 The fill is not the area fraction

It is tempting to read a_p as "the share of the pixel's ground covered by the target" (0.111 for the 10 m roof). That is the right first guess, and Chapter 13 explains why it is not the whole story: the sensor's blur moves some of the roof's light into neighbouring pixels, and moves some of the neighbours' light into this one. One object then produces a whole *field* of fills a_q over the pixels around it.

Analogy: milk in coffee

Pour a little milk into a cup of coffee. The colour of the result lies on the straight line between "coffee" and "milk", at a point set by the share of milk. If you know the colour of black coffee and the colour of milk, the colour of the cup tells you the share. That is the mixing model. It also shows the difficulty: with a teaspoon of milk in a large mug the colour barely moves, and a slightly different

batch of coffee (an error in the background) moves it as much.

Common confusions

Is a_p the target's brightness? No. A shaded roof is $g\,\mathbf{t}$ with a brightness factor g; a roof covering part of the pixel is $a\,\mathbf{t} + (1-a)\mathbf{b}$. They are different, and Chapter 22 shows that from one pixel only the product $a\,g$ can be recovered.
Is $\mathbf{t}$ different at each pixel? In the model, no: one material, one spectrum. In reality the deployed material is weathered, dusty or painted, which is the "library-to-image gap" of Chapter 26.
Is the noise the same thing as the background's variation? No. Sensor noise $\mathbf{n}$ is small (Chapter 16). The big uncertainty is in $\mathbf{b}_p$, the background you have to guess.
Does mixing still hold after the product's normalisation? Yes; Chapter 15 proves it.

Check yourself

1. Compute the noiseless mixture with 50% metal and 50% soil.
2. A 15 m by 15 m roof sits wholly inside a 30 m pixel. What area fraction does it cover?
3. For $a = 0.3$, metal over grass, compute $\mathbf{x} - \mathbf{b}$ without noise.
4. At $a = 0.05$, metal over grass, how big is the signal per band? Compare it with noise of size 0.01.
5. Which of $\mathbf{x}_p$, $\mathbf{t}$, $\mathbf{b}_p$, a_p, $\mathbf{n}_p$ does the sensor measure?

Check yourself: worked answers

1. $0.5\,\mathbf{t} = (0.15,\ 0.15,\ 0.15)$, $0.5\,\mathbf{s} = (0.05,\ 0.10,\ 0.15)$; sum $(0.20,\ 0.25,\ 0.30)$.
2. $15 \times 15 = 225$; $225/900 = 0.25$.
3. $0.3 \times (0.2,\ 0.2,\ -0.2) = (0.06,\ 0.06,\ -0.06)$.
4. $0.05 \times 0.2 = 0.01$ per band (with signs $+, +, -$): the same size as the noise.
5. Only $\mathbf{x}_p$.

Next: Chapter 13, how one object spreads its light.

Chapter 13

The point-spread function

The one thing this chapter teaches

The sensor blurs: a target's light spreads over a neighbourhood of pixels according to the point-spread function, so with the project's Gaussian of $\sigma = 1.4$ pixels a one-pixel target keeps only about 7.5% of its light in its own pixel.

13.1 The problem: pixels are not sharp squares

A pixel does not see a crisp 30 m square of ground. The telescope optics blur the image, the detector averages over its area, and the satellite moves during the exposure. Together these spread the light from any one point on the ground over several pixels. For a small target this matters twice: its own pixel gets less of the target than its area suggests, and the neighbouring pixels get some. The second effect returns in Chapter 24, where the neighbours' share is what lets the background model "see" a target it was supposed to have hidden.

Definition: point-spread function

The **point-spread function** (PSF) h describes how the light from a single point on the ground is shared among nearby positions on the image: $h(\mathbf{u})$ is the weight at offset $\mathbf{u}$, and the weights add to 1. The fill of pixel p by a target occupying ground region Ω is then the share of the target's blurred light that falls in pixel p:

$$a_p = \int_{\Omega} h(\mathbf{c}_p - \mathbf{u})\, d\mathbf{u},$$

with $\mathbf{c}_p$ the centre of pixel p. Read the integral as "add up the PSF weight over every bit of the target".

The project's implant model uses a Gaussian PSF with standard deviation $\sigma_{\text{PSF}} = 1.4$ pixels (`app/evaluation/implant.py`, default `psf_sigma = 1.4`).

13.2 One dimension first

A Gaussian blur of standard deviation σ centred at position c puts a share of its light into an interval $[u, v]$ equal to the area under the Gaussian over that interval. By standardising (Chapter 10):

$$\text{share in } [u, v] = \Phi\Big(\frac{v - c}{\sigma}\Big) - \Phi\Big(\frac{u - c}{\sigma}\Big).$$

Worked example 13.1: How a point at a pixel centre shares its light along a row

Put the point at $c = 0$, the centre of pixel 0, which spans $[-0.5,\ 0.5]$. $\sigma = 1.4$.

$$\begin{aligned}
\text{own pixel } [-0.5, 0.5]: &\quad \Phi(0.5/1.4) - \Phi(-0.5/1.4) = \Phi(0.3571) - \Phi(-0.3571) \\
&\quad = 0.6395 - 0.3605 = 0.2790 \\
\text{next pixel } [0.5, 1.5]: &\quad \Phi(1.5/1.4) - \Phi(0.5/1.4) = \Phi(1.0714) - 0.6395 \\
&\quad = 0.8580 - 0.6395 = 0.2185 \\
\text{two away } [1.5, 2.5]: &\quad \Phi(2.5/1.4) - \Phi(1.5/1.4) = 0.9629 - 0.8580 = 0.1049 \\
\text{three away } [2.5, 3.5]: &\quad \Phi(3.5/1.4) - \Phi(2.5/1.4) = 0.9938 - 0.9629 = 0.0309
\end{aligned}$$

By symmetry the left side matches the right. Within two pixels either side (five pixels in all): $0.2790 + 2 \times 0.2185 + 2 \times 0.1049 = 0.2790 + 0.4370 + 0.2098 = 0.9258$.

Only 28% of the light stays in the pixel the point sits in, along one direction.

13.3 Two dimensions: multiply

The 2-D Gaussian PSF is $h(x, y) \propto e^{-(x^2+y^2)/(2\sigma^2)}$. Because $e^{u+v} = e^u \times e^v$,

$$e^{-(x^2+y^2)/(2\sigma^2)} = e^{-x^2/(2\sigma^2)} \times e^{-y^2/(2\sigma^2)}\ :$$

a 2-D Gaussian is a row Gaussian times a column Gaussian. So the share landing in a square pixel is the row share times the column share.

Worked example 13.2: The 7.5% figure

Point at the centre of its pixel. Own pixel: $0.2790 \times 0.2790 = 0.0778$. Edge neighbour (one step right): $0.2185 \times 0.2790 = 0.0610$. Corner neighbour: $0.2185 \times 0.2185 = 0.0477$. Within the 5×5 block: $0.9258^2 = 0.857$.

0.0477	0.0610	0.0477
0.0610	**0.0778**	0.0610
0.0477	0.0610	0.0477

The original note's figure is 7.5% for a target that *fills* the pixel[C]. That is a little lower than 7.8% because a target filling the pixel is a spread of points, and those near the pixel's edges lose more to the neighbours than the centre point does. Either way: **a one-pixel target keeps under a tenth of its light in its own pixel**, and the rest spreads over roughly a 5×5 neighbourhood.

13.4 Width: standard deviation and FWHM

Optical engineers describe a blur by its **full width at half maximum** (FWHM): the width of the curve at half its peak height. For a Gaussian, find where $e^{-x^2/(2\sigma^2)} = \frac{1}{2}$:

$$\begin{aligned}
-\frac{x^2}{2\sigma^2} &= \ln\tfrac{1}{2} = -\ln 2 && \textit{take logs of both sides} \\
x^2 &= 2\sigma^2 \ln 2 && \textit{multiply both sides by } -2\sigma^2 \\
x &= \sigma\sqrt{2\ln 2} = 1.1774\,\sigma && \textit{square root; } \ln 2 = 0.6931 \\
\text{FWHM} &= 2x = 2.3548\,\sigma.
\end{aligned}$$

For $\sigma = 1.4$ pixels: FWHM $= 2.3548 \times 1.4 = 3.30$ pixels, about $3.30 \times 30 = 99$ m. The same relation converts a spectral band's FWHM into a Gaussian width in Chapter 14.

13.5 The project's implant weights

`psf_weights` in `implant.py` builds the fill field for an implanted object: it blurs the object's footprint with the Gaussian and then divides by the blurred value's average over the footprint, "so the footprint's mean weight is 1 (the object keeps its contrast; neighbours get the blur's tail)". The weight on the one-pixel *halo* around the object then measures how much contrast leaks into the neighbours.

Worked example 13.3: The halo weight of a one-pixel object, by hand

`scipy.ndimage.gaussian_filter` uses a sampled Gaussian kernel: the weight at offset k pixels is proportional to $e^{-k^2/(2\sigma^2)}$. For a one-pixel object the blurred value at offset (i, j) is proportional to $e^{-i^2/(2\sigma^2)}e^{-j^2/(2\sigma^2)}$. Dividing by the value at the object itself (offset $(0, 0)$, where both factors are 1), an edge neighbour, offset $(1, 0)$, gets

$$2\sigma^2 = 2 \times 1.4^2 = 2 \times 1.96 = 3.92$$
$$e^{-1/3.92} = e^{-0.2551} = 0.7748.$$

The "one-pixel halo" in the note's script is the four edge neighbours (a one-step growth with `binary_dilation`'s default cross-shaped element), so the mean halo weight is 0.775, the note's 0.78[C]. Bigger objects have relatively less edge: 0.64 for 2×2 and 0.49 for 4×4[C].

How does this compare with real objects? The comment above the bench-v2 code in `implant.py` records a measurement on PRISMA:

> "real objects' difference from their ring ... bleeds into the 1-px halo (halo / object contrast 0.74 vs 0.60). v2 matches both."

So $\sigma = 1.4$ reproduces the halo of small real objects (0.74 measured[M]). The original note adds a caution you should keep: read literally as the sensor's optics, a FWHM of about 100 m is pessimistic for PRISMA, and part of the measured halo is probably label misregistration (the bench ground truth may be offset by about a pixel). The note, and this book, treat $\sigma = 1.4$ as an *effective* spread, not a claim about the telescope.

Analogy: spray paint

Aim a spray can at one tile of a tiled floor. Most of the paint lands near the aim point, but the mist drifts onto the neighbouring tiles, and a little reaches tiles two away. The PSF is the pattern of the mist. A "one-tile target" leaves its own tile only partly painted and its neighbours tinted; you cannot paint one tile cleanly with a can.

Common confusions

Is 7.5% the fill of a one-pixel target? It is the share of the target's light in its own pixel. Under `psf_weights`' convention the object's pixels are rescaled to mean weight 1, so "fill" in an implant means the contrast you chose, and the halo gets 0.775 of it. The two conventions answer different questions; the note uses both.

σ or FWHM? The code's `psf_sigma` is a standard deviation. FWHM is 2.35 times bigger.

Why do the edge-neighbour and halo numbers differ (0.061 versus 0.775)? 0.061 is a share of the total light (out of 1); 0.775 is relative to the object's own pixel (which is 1 by construction). $0.0610/0.0778 = 0.78$: the same ratio, up to the integration over the pixel.

Does the blur change the spectrum? No: the PSF mixes light between pixels with the same weights in every band (in this model), so it changes fills, not spectral shapes.

Check yourself

1. Using worked example 13.1, what share of a point's light lands in the pixel two to the right, in one dimension?
2. What is the FWHM of a Gaussian blur with $\sigma = 2$ pixels?
3. In two dimensions, what share lands in the pixel diagonally two away (two right, two up)?
4. Under `psf_weights` with $\sigma = 1.0$, what is the halo weight of a one-pixel object?
5. Give two reasons the fill of a 10 m roof's pixel is not 0.111.

Check yourself: worked answers

1. 0.1049.
2. $2.3548 \times 2 = 4.71$ pixels.
3. $0.1049 \times 0.1049 = 0.0110$.
4. $e^{-1/(2\times1^2)} = e^{-0.5} = 0.6065$.
5. The blur moves some of the roof's light into neighbouring pixels (lowering this pixel's fill), and the result also depends on where the roof sits in the pixel. (A third: the roof's own brightness differs from the library's, Chapter 22.)

Next: Chapter 14, turning a lab spectrum into sensor bands.

Chapter 14

Resampling the library to a sensor

The one thing this chapter teaches

Each sensor band sees a weighted average of the lab spectrum over a narrow range of wavelengths, with Gaussian weights set by the band's FWHM; because the weights add to one, resampling is a linear averaging that commutes with mixing.

14.1 The problem: the library and the sensor speak different languages

The shipped spectral library (`data/family_library/`) stores each material as a reflectance curve measured in a laboratory or field spectrometer, on 2151 channels at 1 nm steps from 350 to 2500 nm (the "ASD axis"). A satellite band is about 10 nm wide and centred somewhere in between. To compare a pixel with a library entry you must first compute what that band would have reported for the library material. That is resampling.

The library holds 411 spectra in 17 families, 176 of them man-made in 10 families (counted from the directory; it matches the low-level design). After resampling to the 165-band common grid at 10 nm FWHM, 352 of the 411 are finite in every band[C]; the rest have gaps.

14.2 The spectral response of a band

Definition: spectral response function

The **spectral response function** (SRF) $s_n(\lambda)$ of band n says how strongly the band responds to light of wavelength λ. The band reports the response-weighted average of the incoming reflectance $\rho(\lambda)$:

$$t_n = \frac{\sum_\lambda s_n(\lambda)\rho(\lambda)}{\sum_\lambda s_n(\lambda)},$$

the sums running over the library's 1 nm samples. (The note writes integrals; with 1 nm samples they are these sums.)

The product models each SRF as a Gaussian centred at the band centre λ_n:

$$s_n(\lambda) = \exp\Big(-\frac{(\lambda-\lambda_n)^2}{2\varsigma_n^2}\Big), \qquad \varsigma_n = \frac{w_n}{2\sqrt{2\ln 2}} = \frac{w_n}{2.3548},$$

where w_n is the band's FWHM. (The relation is the one derived in Chapter 13; ς is a "variant sigma", used so it is not confused with the noise σ.) For a 10 nm band, $\varsigma = 10/2.3548 = 4.247$ nm. The code truncates

the weights at $3\varsigma = 12.74$ nm either side, and returns "not a number" (NaN) for a band with fewer than three library samples in that window (`app/spectral_match/resample.py`, `n_sigma_window = 3.0`, `min_lib_points_per_band = 3`).

Worked example 14.1: A coarse resampling by hand

Pretend the library had only three samples near a band centred at 500 nm with FWHM 10 nm: reflectance 0.20 at 495 nm, 0.30 at 500 nm, 0.50 at 505 nm. Weights:

$$\begin{aligned} \text{at 500 nm (offset 0)} &: \quad e^{0} = 1 \\ \text{at 495 or 505 nm (offset 5)} &: \quad e^{-25/(2\times 4.247^2)} = e^{-25/36.07} = e^{-0.6931} = 0.5 \end{aligned}$$

(Half weight at half the FWHM from the centre: that is what "half maximum" means.) Weighted average:

$$\begin{aligned} \text{numerator} &= 0.5\times 0.20 + 1\times 0.30 + 0.5\times 0.50 = 0.10 + 0.30 + 0.25 = 0.65 \\ \text{denominator} &= 0.5 + 1 + 0.5 = 2 \\ t &= 0.65/2 = 0.325. \end{aligned}$$

The real code does the same with every 1 nm sample within ±12.74 nm, 25 of them.

14.3 Resampling as a matrix, and why it commutes with mixing

Divide each weight by the row's total and resampling becomes a matrix $\mathcal{S}$ with one row per band and one column per library wavelength: $\mathbf{t} = \mathcal{S}\boldsymbol{\rho}$. In worked example 14.1 the row is (0.25, 0.5, 0.25). Every row adds up to 1; such a matrix is called **row-stochastic**. Two consequences:

- a flat spectrum stays flat: if $\rho = 0.4$ everywhere, every band reports $0.4 \times$ (sum of the row) $= 0.4$;
- resampling a mixture equals mixing the resampled spectra, because a matrix is linear: $\mathcal{S}\big(a\boldsymbol{\rho}_A + (1-a)\boldsymbol{\rho}_B\big) = a\,\mathcal{S}\boldsymbol{\rho}_A + (1-a)\,\mathcal{S}\boldsymbol{\rho}_B$.

Worked example 14.2: Mix then resample, or resample then mix

Two materials on the three samples of worked example 14.1: $\boldsymbol{\rho}_A = (0.20,\ 0.30,\ 0.50)$ and $\boldsymbol{\rho}_B = (0.60,\ 0.60,\ 0.60)$; fill $a = 0.2$ of A.
Resample, then mix. $\mathcal{S}\boldsymbol{\rho}_A = 0.325$ (above). $\mathcal{S}\boldsymbol{\rho}_B = 0.25\times 0.6 + 0.5\times 0.6 + 0.25\times 0.6 = 0.6$. Mix: $0.2\times 0.325 + 0.8\times 0.6 = 0.065 + 0.48 = 0.545$.
Mix, then resample. $0.2\boldsymbol{\rho}_A + 0.8\boldsymbol{\rho}_B = (0.04 + 0.48,\ 0.06 + 0.48,\ 0.10 + 0.48) = (0.52,\ 0.54,\ 0.58)$. Resample: $0.25\times 0.52 + 0.5\times 0.54 + 0.25\times 0.58 = 0.13 + 0.27 + 0.145 = 0.545$.
The same.

This is why the library can be resampled once and the mixing model of Chapter 12 applied afterwards.

14.4 What if the band is not where you think?

Sensors drift: a band's true centre may be $\lambda_n + \Delta$ instead of λ_n. The resampled value then changes by about Δ times the local slope of the spectrum. A reflectance that rises by 0.2 over 100 nm has slope $0.2/100 = 0.002$ per nm; a 2 nm error changes the band by about $2\times 0.002 = 0.004$. The note measured the effect on all 352 complete library entries[C]:

perturbation	angle change, median	angle change, max	largest relative change, median
centre shift 1 nm	0.11°	0.52°	1.2%
centre shift 2 nm	0.21°	1.04°	2.5%
centre shift 5 nm	0.52°	2.55°	6.1%
FWHM 8 or 12 nm instead of 10	0.03°	0.20°	-
FWHM 15 nm instead of 10	0.06°	0.50°	-

A one- or two-nanometre calibration error costs a tenth to a fifth of a degree for a typical entry. Keep that size in mind: families are a few degrees apart (Chapter 21), and Chapter 25 shows that an atmospheric gain error costs ten times more. Two practical failures matter more than accuracy: a band with no FWHM (PRISMA's null FWHM makes the band invalid) and a pixel with missing data on a kept band (such a pixel is not scored).

Analogy: a radio dial

Tune a radio to 100.0 MHz and you hear that station loudest, but a weak station at 100.2 MHz bleeds in faintly. The radio's response curve is the SRF: strongest at the dial setting, falling off on either side. A sensor band "listens" to the spectrum the same way, and resampling computes what it would hear. A radio slightly mistuned (a centre shift) hears a slightly different mix.

Common confusions

Is resampling picking the library value at the band centre? No, it averages over a window. Picking would ignore the band's width and add noise from the library's fine structure.
Why do some library spectra become NaN? Many lab spectra have gaps, stored in the files as huge negative numbers that the code turns into NaN. A band with fewer than three valid samples in its window is NaN. Oil keeps 30 of its 40 entries complete on the 165-band grid[C].
Is the 165-band grid a sensor? No. It is the product's common grid, 10 nm spacing at 10 nm FWHM. A native sensor (PRISMA, EnMAP) has its own centres and widths, and the same resampling applies with those.

Check yourself

1. What Gaussian width ς corresponds to a 12 nm FWHM?
2. Resample the flat library values 0.4, 0.4, 0.4 with the weights of worked example 14.1.
3. For a 10 nm FWHM band, what is the weight of a sample 2.5 nm from the centre?
4. Repeat worked example 14.2 with $a = 0.5$.
5. Using the table, how much does a 2 nm centre shift typically change a spectral angle?

Check yourself: worked answers

1. $12/2.3548 = 5.096$ nm.
2. $(0.5 \times 0.4 + 1 \times 0.4 + 0.5 \times 0.4)/2 = 0.8/2 = 0.4$. Flat stays flat.
3. $e^{-2.5^2/(2\times 4.247^2)} = e^{-6.25/36.07} = e^{-0.1733} = 0.841$.
4. Resample then mix: $0.5 \times 0.325 + 0.5 \times 0.6 = 0.1625 + 0.3 = 0.4625$. Mix then resample: mixture $(0.40,\ 0.45,\ 0.55)$; $0.25 \times 0.40 + 0.5 \times 0.45 + 0.25 \times 0.55 = 0.10 + 0.225 + 0.1375 = 0.4625$. Same.
5. The median is 0.21°, the worst entry 1.04°.

Next: Chapter 15, the units the model works in.

Chapter 15

Normalisation by wavelength

The one thing this chapter teaches

The product converts every spectrum, image and library alike, to standard scores band by band using one fixed mean and standard-deviation curve; because the same affine map is applied to everything, mixtures and differences survive it intact.

15.1 The problem: one set of units for everyone

Indradhanu, the foundation model that predicts the background (Chapter 16), does not work in raw reflectance. It works in normalised units, in which every band has a comparable spread. A library spectrum must be converted to the same units before it can be compared with a residual. That conversion must not break the mixing model: a mixed pixel must still be a mixture after normalising. Not every normalisation has that property.

15.2 The normalisation

Definition: normalisation by wavelength

Let $\mu(\lambda)$ and $\sigma(\lambda)$ be curves giving the mean and the standard deviation of surface reflectance at each wavelength, measured once on SpectralEarth's training data and stored in the model checkpoint (`WavelengthStats`, `app/foundation_models/components/pixel_normalization.py`). For a list of bands, $\boldsymbol{\mu} = (\mu(\lambda_1), \dots, \mu(\lambda_N))$ and $\mathbf{D} = \operatorname{diag}(\sigma(\lambda_1), \dots, \sigma(\lambda_N))$; between measured wavelengths the curves are interpolated in a straight line, and outside them the end values are held. The **normalised spectrum** is

$$\mathbf{z}(\mathbf{x}) = \mathbf{D}^{-1}(\mathbf{x} - \boldsymbol{\mu}), \qquad \text{that is,} \qquad z_n = \frac{x_n - \mu(\lambda_n)}{\sigma(\lambda_n)}.$$

It is the standard score of Chapter 7, band by band, with a fixed global mean and standard deviation. It never mixes bands.

For scale, on the 165-band grid the reflectance standard deviation runs from 0.035 (blue) to 0.100, median 0.077, and the mean curve has median 0.155 (`app/constants/hyperspectral.json`)[C]. **One normalised unit is therefore about 0.08 reflectance.** (That file is the 165-grid statistic, a stand-in for SpectralEarth's `stats.json`, which is not in the repository.)

Worked example 15.1: Normalising one value, and interpolating the curves

A band where $\mu = 0.155$ and $\sigma = 0.077$, reflectance $x = 0.30$:

$$x - \mu = 0.30 - 0.155 = 0.145$$
$$z = 0.145/0.077 = 1.883.$$

Interpolation: if the stored curve has $\mu(460) = 0.10$ and $\mu(470) = 0.12$, then at 465 nm, halfway, $\mu = 0.10 + \frac{1}{2}(0.12 - 0.10) = 0.11$.

15.3 Mixing survives normalisation

Definition: affine map

A map of the form $\mathbf{z} = \mathbf{A}\mathbf{x} + \mathbf{c}$, a matrix followed by a fixed shift, is **affine**. Normalisation is affine with $\mathbf{A} = \mathbf{D}^{-1}$ and $\mathbf{c} = -\mathbf{D}^{-1}\boldsymbol{\mu}$.

Rule 15.1: Mixing survives the normalisation

If $\mathbf{x} = a\mathbf{t} + (1-a)\mathbf{b} + \mathbf{n}$, then

$$\mathbf{z}(\mathbf{x}) = a\,\mathbf{z}(\mathbf{t}) + (1-a)\,\mathbf{z}(\mathbf{b}) + \mathbf{D}^{-1}\mathbf{n}, \qquad \mathbf{z}(\mathbf{t}) - \mathbf{z}(\mathbf{b}) = \mathbf{D}^{-1}(\mathbf{t} - \mathbf{b}).$$

Proof. The trick is to write $\boldsymbol{\mu}$ as $a\boldsymbol{\mu} + (1-a)\boldsymbol{\mu}$, which is allowed because $a + (1-a) = 1$.

$$\begin{aligned}
\mathbf{z}(\mathbf{x}) &= \mathbf{D}^{-1}\big(a\mathbf{t} + (1-a)\mathbf{b} + \mathbf{n} - \boldsymbol{\mu}\big) \\
&= \mathbf{D}^{-1}\big(a\mathbf{t} + (1-a)\mathbf{b} + \mathbf{n} - a\boldsymbol{\mu} - (1-a)\boldsymbol{\mu}\big) && \boldsymbol{\mu} = a\boldsymbol{\mu} + (1-a)\boldsymbol{\mu} \\
&= \mathbf{D}^{-1}\big(a(\mathbf{t} - \boldsymbol{\mu}) + (1-a)(\mathbf{b} - \boldsymbol{\mu}) + \mathbf{n}\big) && \textit{regroup} \\
&= a\,\mathbf{D}^{-1}(\mathbf{t} - \boldsymbol{\mu}) + (1-a)\,\mathbf{D}^{-1}(\mathbf{b} - \boldsymbol{\mu}) + \mathbf{D}^{-1}\mathbf{n} && \textit{a matrix distributes over a sum} \\
&= a\,\mathbf{z}(\mathbf{t}) + (1-a)\,\mathbf{z}(\mathbf{b}) + \mathbf{D}^{-1}\mathbf{n}.
\end{aligned}$$

For the difference, $\mathbf{z}(\mathbf{t}) - \mathbf{z}(\mathbf{b}) = \mathbf{D}^{-1}(\mathbf{t} - \boldsymbol{\mu}) - \mathbf{D}^{-1}(\mathbf{b} - \boldsymbol{\mu}) = \mathbf{D}^{-1}(\mathbf{t} - \mathbf{b})$: the mean cancels.

Worked example 15.2: Checking it with numbers

Two bands with $\boldsymbol{\mu} = (0.1,\ 0.2)$ and standard deviations $(0.05,\ 0.1)$. Target $\mathbf{t} = (0.3,\ 0.3)$, background $\mathbf{b} = (0.1,\ 0.5)$, $a = 0.25$, no noise.
Mix first: $\mathbf{x} = 0.25\mathbf{t} + 0.75\mathbf{b} = (0.075 + 0.075,\ 0.075 + 0.375) = (0.15,\ 0.45)$. Normalise: $\big((0.15 - 0.1)/0.05,\ (0.45 - 0.2)/0.1\big) = (0.05/0.05,\ 0.25/0.1) = (1,\ 2.5)$.
Normalise first: $\mathbf{z}(\mathbf{t}) = \big(0.2/0.05,\ 0.1/0.1\big) = (4,\ 1)$, $\mathbf{z}(\mathbf{b}) = \big(0/0.05,\ 0.3/0.1\big) = (0,\ 3)$. Mix: $0.25 \times (4, 1) + 0.75 \times (0, 3) = (1 + 0,\ 0.25 + 2.25) = (1,\ 2.5)$. The same.
Difference: $\mathbf{z}(\mathbf{t}) - \mathbf{z}(\mathbf{b}) = (4,\ -2)$, and $\mathbf{D}^{-1}(\mathbf{t} - \mathbf{b}) = (0.2/0.05,\ -0.2/0.1) = (4,\ -2)$. The same.

15.4 Why per-scene min-max scaling fails

Another common normalisation stretches each band of each scene so its smallest value becomes 0 and its largest 1: $z = (x - \min)/(\max - \min)$. Within one scene that is affine too, so mixing survives *within* the scene. But its coefficients depend on that scene's extremes.

Reflectance 0.25 in a scene whose band runs from 0.05 to 0.45 becomes $(0.25 - 0.05)/0.40 = 0.5$. The same reflectance in a scene whose band runs from 0.10 to 0.30 becomes $(0.25 - 0.10)/0.20 = 0.75$. A library spectrum belongs to no scene, so it has no defined normalised version at all. The project's earlier proposal records this as the reason the external HyperSL model, which uses per-scene min-max normalisation,

cannot be used for library matching. **A normalisation fixed once for everything is what makes matching in the model's units possible at all.**

15.5 Noise in normalised units

Noise is divided by σ too ($\mathbf{D}^{-1}\mathbf{n}$ in rule 15.1). A reflectance noise of 0.0004 in a band with $\sigma = 0.077$ is $0.0004/0.077 = 0.0052$ normalised units. Chapter 16 uses this to show that sensor noise is tiny compared with the model's errors.

Analogy: a fixed exchange rate

Converting prices from rupees to euros with one published exchange rate keeps every relationship intact: the average of two prices converts to the average of their conversions, and a price difference converts by the same rate. Now imagine each shop setting its own rate from its own cheapest and dearest items that day (min-max). Prices inside one shop still compare correctly, but a catalogue price that belongs to no shop cannot be converted at all, and the same item gets different euro prices in different shops.

Common confusions

Is this the whitening of Chapter 7? No. Normalisation uses fixed global curves, one band at a time. Whitening uses the scene's residual covariance and mixes bands. The product normalises first; the detectors whiten afterwards.

Does normalisation remove brightness differences? No. A roof at half brightness is still at half brightness relative to the mean curve; the shift by $\boldsymbol{\mu}$ even makes it no longer a simple multiple. Brightness is handled in Chapter 22.

Do differences depend on $\boldsymbol{\mu}$? No, the mean cancels in any difference (rule 15.1). Residuals are differences, so for them only $\mathbf{D}$ matters.

Check yourself

1. Normalise reflectance 0.20 in a band with $\mu = 0.155$ and $\sigma = 0.077$.
2. Two reflectances, 0.30 and 0.20, in a band with $\sigma = 0.05$. What is their normalised difference? Does it depend on μ?
3. Repeat worked example 15.2 with $a = 0.5$, checking both orders.
4. Why can a library spectrum not be min-max normalised?
5. About how much reflectance is one normalised unit on the 165-band grid?

Check yourself: worked answers

1. $(0.20 - 0.155)/0.077 = 0.045/0.077 = 0.584$.
2. $(0.30 - 0.20)/0.05 = 0.10/0.05 = 2.0$. No; μ cancels.
3. Mix first: $\mathbf{x} = 0.5(0.3, 0.3) + 0.5(0.1, 0.5) = (0.2,\ 0.4)$; normalise: $(0.1/0.05,\ 0.2/0.1) = (2,\ 2)$. Normalise first: $0.5(4, 1) + 0.5(0, 3) = (2,\ 2)$. Same.
4. Min-max needs a scene's minimum and maximum; a library spectrum has no scene.
5. About 0.08 (the median σ is 0.077).

Next: Part III, the learned background, starting with Chapter 16.

Part III

The learned background

Chapter 16

The residual

The one thing this chapter teaches

Indradhanu predicts each pixel's background from its surroundings with the pixel hidden; the residual $\mathbf{r} = \mathbf{x} - \hat{\mathbf{b}}$ equals the fill times the target-minus-prediction difference plus the model's own error, and that model error, not sensor noise, is what limits everything downstream.

16.1 The problem: the background is unknown

The mixing model needs $\mathbf{b}_p$, what pixel p would show without the target. Nobody measured that. The classical estimate is the average of a ring of nearby pixels: the shipped family matcher uses the 13×13 window minus the central 5×5. A ring average is easy, but it blurs edges and texture, so the "residual" it leaves contains a lot of ordinary landscape. Allotrope's approach is to let a foundation model, Indradhanu, look at the pixel's surroundings and paint what it expects there.

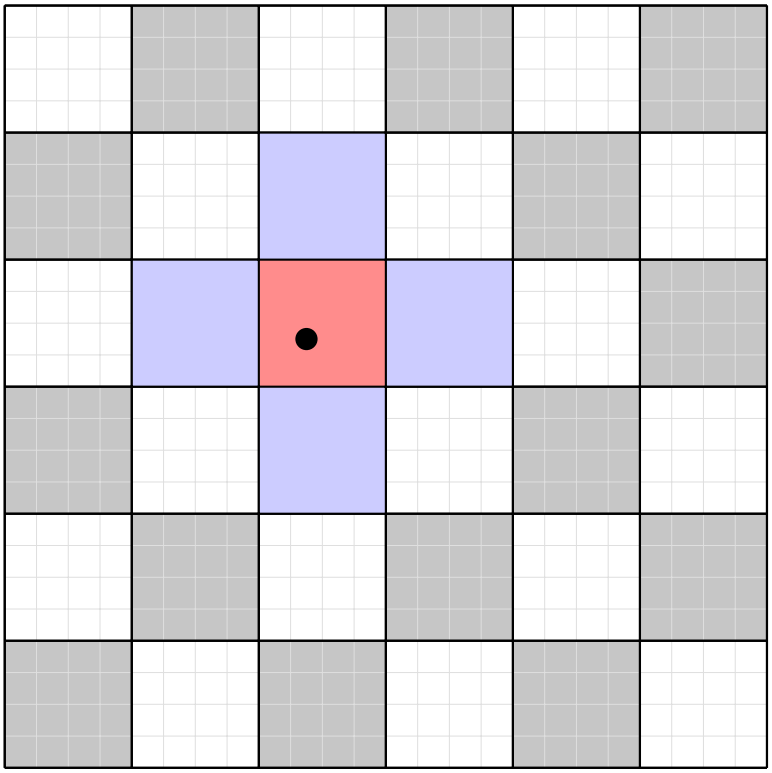
16.2 How Indradhanu predicts the background

Indradhanu is a **masked autoencoder**: during training, parts of each image are hidden and the network learns to reconstruct them from what is visible. In the product it predicts every pixel in one of two complementary **checkerboard** passes over 4×4-pixel **tokens** (small square tiles): in the first pass the "white" tokens are hidden and predicted from the "black" ones, in the second the other way round (`app/foundation_models/inferencers/spectral_stem_inferencer.py`, `_checkerboard_keep_mask`; the decoder-capacity study calls this "the product two-pass"). A pixel's own token is always hidden when that pixel is predicted.

Write the prediction for pixel p as

$$\hat{\mathbf{b}}_p = f_\theta(\mathbf{x}_{C(p)}),$$

where $C(p)$ is the visible context (every visible token), f_θ the network with its trained weights θ, and the hat on $\hat{\mathbf{b}}$ marks an estimate. The model in this book is **Indradhanu Heavy**, the release candidate FTH3 epoch 5 (the shipped model is FT04).



red: the hidden token containing pixel p (dot)

blue: its four edge-neighbour tokens, visible in this pass

grey/white: the two checkerboard colours

Figure 16.1: The checkerboard passes over 4×4-pixel tokens. When pixel p's token (red) is hidden, its four edge neighbours (blue) are visible. Chapter 24 asks how much of a small target's light falls in the blue tokens.

16.3 The residual and the model's error

Definition: residual, background-prediction error

The **residual** is $\mathbf{r}_p = \mathbf{x}_p - \hat{\mathbf{b}}_p$, what the model could not explain. The **background-prediction error** is $\mathbf{e}_p = \mathbf{b}_p - \hat{\mathbf{b}}_p$, how wrong the prediction would be about the target-free pixel. Its covariance over background pixels is Σ_e.

Substitute the mixing model $\mathbf{x}_p = a\mathbf{t} + (1-a)\mathbf{b}_p + \mathbf{n}_p$ (dropping the subscript p on a):

$$\begin{aligned}
\mathbf{r}_p &= a\mathbf{t} + (1-a)\mathbf{b}_p + \mathbf{n}_p - \hat{\mathbf{b}}_p \\
&= a\mathbf{t} - a\mathbf{b}_p + \mathbf{b}_p + \mathbf{n}_p - \hat{\mathbf{b}}_p && \textit{multiply out } (1-a)\mathbf{b}_p \\
&= a(\mathbf{t} - \mathbf{b}_p) + (\mathbf{b}_p - \hat{\mathbf{b}}_p) + \mathbf{n}_p && \textit{regroup} \\
&= a(\mathbf{t} - \mathbf{b}_p) + \mathbf{e}_p + \mathbf{n}_p. && \textit{definition of } \mathbf{e}_p
\end{aligned}$$

That is the note's equation (2.1). It contains the unknown $\mathbf{b}_p$. A second form avoids it. Replace $\mathbf{b}_p$ by $\hat{\mathbf{b}}_p + \mathbf{e}_p$ (the definition of $\mathbf{e}_p$, rearranged by adding $\hat{\mathbf{b}}_p$ to both sides):

$$\begin{aligned}
\mathbf{r}_p &= a\mathbf{t} + (1-a)(\hat{\mathbf{b}}_p + \mathbf{e}_p) + \mathbf{n}_p - \hat{\mathbf{b}}_p \\
&= a\mathbf{t} + (1-a)\hat{\mathbf{b}}_p + (1-a)\mathbf{e}_p + \mathbf{n}_p - \hat{\mathbf{b}}_p && \textit{multiply out} \\
&= a\mathbf{t} - a\hat{\mathbf{b}}_p + (1-a)\mathbf{e}_p + \mathbf{n}_p && (1-a)\hat{\mathbf{b}}_p - \hat{\mathbf{b}}_p = -a\hat{\mathbf{b}}_p \\
&= a(\mathbf{t} - \hat{\mathbf{b}}_p) + (1-a)\mathbf{e}_p + \mathbf{n}_p.
\end{aligned}$$

Define $\mathbf{d}_p = \mathbf{t} - \hat{\mathbf{b}}_p$, the target minus the predicted background:

$$\boxed{\mathbf{r}_p = a\,\mathbf{d}_p + (1-a)\,\mathbf{e}_p + \mathbf{n}_p}$$

This is the form to compute with: once you pick a library spectrum $\mathbf{t}$, $\mathbf{d}_p$ is known.

Worked example 16.1: Both forms on three bands

Metal $\mathbf{t} = (0.3,\ 0.3,\ 0.3)$, true background $\mathbf{b} = (0.1,\ 0.1,\ 0.5)$, the model's prediction $\hat{\mathbf{b}} = (0.12,\ 0.09,\ 0.48)$, fill $a = 0.2$, no noise. The pixel is $\mathbf{x} = (0.14,\ 0.14,\ 0.46)$ (worked example 1.2).
Residual directly: $\mathbf{r} = \mathbf{x} - \hat{\mathbf{b}} = (0.14 - 0.12,\ 0.14 - 0.09,\ 0.46 - 0.48) = (0.02,\ 0.05,\ -0.02)$.
Model error: $\mathbf{e} = \mathbf{b} - \hat{\mathbf{b}} = (0.1 - 0.12,\ 0.1 - 0.09,\ 0.5 - 0.48) = (-0.02,\ 0.01,\ 0.02)$.
First form: $a(\mathbf{t}-\mathbf{b}) = 0.2\times(0.2,\ 0.2,\ -0.2) = (0.04,\ 0.04,\ -0.04)$; add $\mathbf{e}$: $(0.04-0.02,\ 0.04+0.01,\ -0.04+0.02) = (0.02,\ 0.05,\ -0.02)$. Matches.
Second form: $\mathbf{d} = \mathbf{t} - \hat{\mathbf{b}} = (0.18,\ 0.21,\ -0.18)$; $a\mathbf{d} = (0.036,\ 0.042,\ -0.036)$; $(1-a)\mathbf{e} = 0.8 \times (-0.02,\ 0.01,\ 0.02) = (-0.016,\ 0.008,\ 0.016)$; sum $(0.020,\ 0.050,\ -0.020)$. Matches.

On a background pixel ($a = 0$), $\mathbf{r} = \mathbf{e} + \mathbf{n}$. The model error and the sensor noise come from unrelated causes, so they are independent and their covariances add: $\Sigma_r = \Sigma_e + \Sigma_n$.

16.4 Clutter-limited, not noise-limited

How big is each part? EnMAP's stated signal-to-noise ratio (a sensor property, the ratio of signal to noise in one band, not the detection SNR of Chapter 11) is 354 in the visible and near-infrared and 260 in the short-wave infrared (LLD section 4). At reflectance 0.15:

$$\text{noise sd} = 0.15/354 = 0.00042 \quad \text{and} \quad 0.15/260 = 0.00058$$
$$\text{normalised} = 0.00042/0.077 = 0.0055 \quad \text{and} \quad 0.00058/0.077 = 0.0075.$$

So sensor noise is 0.005 to 0.008 normalised units[C]. The measured hidden-pixel background error of FT04 on the four PRISMA bench scenes is 0.057 to 0.139 mean absolute normalised units per band (`research/experiments/decoder-capacity/README.md`, table 2)[M]: ten to twenty-five times bigger, and in variance (squared) one to two orders of magnitude. **Every detector in this book is limited by how well the model predicts the background, not by photon noise.** The jargon is "clutter-limited". In practice you can treat $\Sigma_r \approx \Sigma_e$ and forget Σ_n.

Analogy: the countryside painter

Picture a painter who has spent years painting fields, roads and villages. Cover one tile of a landscape photograph and ask her to paint what belongs there; she will paint a convincing patch of field. If the covered tile actually held a crashed aeroplane, her painting will not contain it, and the difference between the photograph and her painting will show the aeroplane plainly. The difference also shows her ordinary mistakes (a hedge slightly misplaced), and those mistakes, not the graininess of the photograph, are what make small aeroplanes hard to see.

Common confusions

Is e the same as n? No. $\mathbf{n}$ is sensor noise, tiny. $\mathbf{e}$ is the model's mistake about the background, large.
Is the residual the anomaly score? No. The residual is a vector (165 numbers). The score is one number computed from it, the subject of Chapter 17.
If p's token is hidden, can the model see the target at all? Not directly. But the sensor blur puts part of a small target's light into the visible neighbouring tokens (Chapter 13), and the model can partly copy it. That is leakage, Chapter 24.
Why two forms of the residual? The first, with $\mathbf{t} - \mathbf{b}_p$, describes the physics but contains the unknown $\mathbf{b}_p$. The second, with $\mathbf{d}_p = \mathbf{t} - \hat{\mathbf{b}}_p$, contains only things you know once you choose a candidate $\mathbf{t}$, at the price of the factor $(1 - a)$ on $\mathbf{e}$.

Check yourself

1. Pixel $\mathbf{x} = (0.15,\ 0.12,\ 0.46)$ (the noisy pixel of worked example 12.1) and prediction $\hat{\mathbf{b}} = (0.12,\ 0.09,\ 0.48)$. Compute $\mathbf{r}$.
2. With $\mathbf{b} = (0.1,\ 0.1,\ 0.5)$, what is $\mathbf{e}$?
3. Check question 1 against the first form $a(\mathbf{t} - \mathbf{b}) + \mathbf{e} + \mathbf{n}$ with $a = 0.2$ and $\mathbf{n} = (0.01,\ -0.02,\ 0)$.
4. A sensor has SNR 300 at reflectance 0.15. What is its noise in normalised units (use $\sigma = 0.077$)?
5. What is the residual of a pixel with no target?

Check yourself: worked answers

1. $(0.15 - 0.12,\ 0.12 - 0.09,\ 0.46 - 0.48) = (0.03,\ 0.03,\ -0.02)$.
2. $(0.1 - 0.12,\ 0.1 - 0.09,\ 0.5 - 0.48) = (-0.02,\ 0.01,\ 0.02)$.
3. $(0.04,\ 0.04,\ -0.04) + (-0.02,\ 0.01,\ 0.02) + (0.01,\ -0.02,\ 0)$: band 1 $0.04 - 0.02 + 0.01 = 0.03$; band 2 $0.04 + 0.01 - 0.02 = 0.03$; band 3 $-0.04 + 0.02 + 0 = -0.02$. Matches.
4. $0.15/300 = 0.0005$; $0.0005/0.077 = 0.0065$.
5. $\mathbf{r} = \mathbf{e} + \mathbf{n}$: the model's error plus sensor noise.

Next: Chapter 17, turning a residual into one number.

Chapter 17

Residual Mahalanobis: surprise in any direction

The one thing this chapter teaches

Residual Mahalanobis whitens the residual against the scene's own residual covariance (top 30 directions), subtracts the local mean, and takes the squared length; it is the likelihood-ratio test for "unusual in any direction", and knowing the direction in advance would need less signal.

17.1 The problem: one number per pixel

The residual is a vector of 165 numbers. A detector must reduce it to one number, large for anomalies. The obvious choice, its length, fails: the module's docstring explains why.

> "A scene's normal residuals are large along a few shared directions - overall brightness, haze, detector striping, dark water - and tiny along the rest. Projecting r onto its top-d principal components, whitening each by its spread, and scoring the squared length (Mahalanobis distance) discounts those common directions and rewards errors in directions the background never takes." (`app/utils/anomaly_detection/residual_mahalanobis.py`)

You already have every piece: covariance (Chapter 5), principal components (Chapter 6), whitening and Mahalanobis distance (Chapter 7).

17.2 The recipe

Definition: residual Mahalanobis (RM)

1. Estimate the residual covariance $\hat{\Sigma}_r$ from up to 200,000 valid pixels of the scene (a fixed random seed, so the result is repeatable).
2. Take its eigendecomposition $\hat{\Sigma}_r = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$ and keep the top $d = 30$ components: $\mathbf{U}_d$ (the first 30 eigenvectors) and $\mathbf{\Lambda}_d$.
3. Whiten: $\mathbf{W} = \mathbf{\Lambda}_d^{-1/2}\mathbf{U}_d^\top$, a $30 \times N$ matrix.
4. For each pixel, subtract the mean residual over a ring around it: the 61×61 window minus the central 21×21 (the PRISMA setting). If the ring has fewer than $2d = 60$ valid pixels, use the scene mean instead.
5. Score: $\mathrm{RM}(p) = \|\mathbf{W}(\mathbf{r}_p - \bar{\mathbf{r}}_{\mathrm{ring}(p)})\|^2$.

All constants were checked against the code: `PCA_COMPONENTS = 30`, `_SAMPLE = 200_000`, `LOCAL_WINDOW = (21, 61)`, `MIN_RING_FACTOR = 2`. The code subtracts the ring mean after whitening rather than before; because $\mathbf{W}$ is a matrix, $\mathbf{W}\mathbf{r} - \mathbf{W}\bar{\mathbf{r}} = \mathbf{W}(\mathbf{r} - \bar{\mathbf{r}})$, so it is the same.

Why the ring mean? The model's error is not zero on average everywhere; a whole field may be predicted slightly too bright. Subtracting the local average residual removes that slow drift, so the score measures what is unusual *here*. The docstring records the effect on the four labelled PRISMA scenes (mean ROC-AUC): 0.662 for the older combined score, 0.840 with the global mean, 0.854 with the local mean[M].

17.3 Worked on three bands

To keep the arithmetic small, take a residual covariance whose eigenvectors are the band axes (so $\mathbf{U} = \mathbf{I}$ and the rotation step does nothing; Chapter 7 showed the rotation): $\Sigma_r = \mathrm{diag}(4,\ 1,\ 0.25)$. Its standard deviations are 2, 1, 0.5, so $\mathbf{W} = \mathrm{diag}(\frac{1}{2},\ 1,\ 2)$.

Worked example 17.1: Two residuals, and what truncation hides

Residual A: $\mathbf{r} = (2,\ 1,\ 0.5)$, ring mean zero.

$$\mathbf{W}\mathbf{r} = (2/2,\ 1 \times 1,\ 0.5 \times 2) = (1,\ 1,\ 1)$$
$$\mathrm{RM} = 1 + 1 + 1 = 3 \qquad \text{(Euclidean squared length } 4 + 1 + 0.25 = 5.25\text{)}$$

Residual B: $\mathbf{r} = (0,\ 0,\ 1.5)$.

$$\mathbf{W}\mathbf{r} = (0,\ 0,\ 3)$$
$$\mathrm{RM} = 9 \qquad \text{(Euclidean squared length } 2.25\text{)}$$

B is the shorter vector and three times more surprising, because it points where the background never goes.
Now keep only $d = 2$ components, dropping the smallest-variance axis (band 3). A: $(1,\ 1)$, RM $= 2$. B: $(0,\ 0)$, RM $= 0$. **Truncation makes B invisible.**
Finally, a ring mean: if the ring's average residual is $(1,\ 0,\ 0)$, A becomes $(1,\ 1,\ 0.5)$, whitened $(0.5,\ 1,\ 1)$, RM $0.25 + 1 + 1 = 2.25$.

17.4 RM is a likelihood-ratio test

Why is the squared whitened length the right number? Because it is the GLRT (Chapter 11) for a specific question.

Write $\mathbf{y} = \mathbf{W}(\mathbf{r}_p - \mathbf{m})$, the whitened, centred residual, with $\mathbf{m}$ the local mean. Assume that on background pixels $\mathbf{y}$ is Gaussian with covariance $\mathbf{I}_d$ (that is what whitening achieved) and mean zero. Under H_1 a signal $\mathbf{s}$ has been added to the residual, of *any* shape. So

$$H_0 : \mathbf{y} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d), \qquad H_1 : \mathbf{y} \sim \mathcal{N}(\mathbf{W}\mathbf{s}, \mathbf{I}_d) \text{ for some unknown } \mathbf{s}.$$

From Chapter 10 the log-likelihood of $\mathbf{y}$ with mean $\boldsymbol{\nu}$ and covariance $\mathbf{I}$ is a constant minus $\frac{1}{2}\|\mathbf{y} - \boldsymbol{\nu}\|^2$.

$$\text{under } H_0 : \quad \ell_0 = \text{const} - \tfrac{1}{2}\|\mathbf{y}\|^2$$
$$\text{under } H_1, \text{ best } \mathbf{s} : \quad \text{choose } \mathbf{s} \text{ with } \mathbf{W}\mathbf{s} = \mathbf{y}, \text{ so } \ell_1 = \text{const} - \tfrac{1}{2}\|\mathbf{y} - \mathbf{y}\|^2 = \text{const}$$
$$2 \ln \Lambda = 2(\ell_1 - \ell_0) = \|\mathbf{y}\|^2 = \mathrm{RM}(p).$$

(Some $\mathbf{s}$ with $\mathbf{W}\mathbf{s} = \mathbf{y}$ always exists, because the 30 rows of $\mathbf{W}$ are independent.) Under H_0, $\mathbf{y}$ has d independent standard normal components, so RM is χ^2_d (Chapter 11): mean 30, and at $P_{\mathrm{FA}} = 10^{-3}$

threshold 59.70[C]. The docstring's warning applies: real RM scores are "chi-squared-like, heavy right tail", which is one reason the product thresholds them by percentile.

So RM answers the question **"is this residual unusual in *any* of the 30 leading directions?"**

17.5 The price of not knowing the direction

Suppose you knew more: the signal lies in a particular q-dimensional subspace, the span of the columns of some $\mathbf{H}$ (for example a material family, Chapter 21). Then under H_1, $\mathbf{s} = \mathbf{H}\boldsymbol{\theta}$ for unknown $\boldsymbol{\theta}$, and the best fit under H_1 can only cancel the part of $\mathbf{y}$ inside the whitened subspace. By Chapter 9,

$$2 \ln \Lambda = \|\mathbf{P}_{\mathbf{WH}}\, \mathbf{y}\|^2,$$

the explained energy of $\mathbf{y}$ in that subspace. Under H_0 it collects the noise energy of only q directions, so it is χ^2_q; under H_1 it is noncentral with $\delta = \|\mathbf{W}\mathbf{s}\|^2$. Fewer noise directions means a lower threshold for the same signal:

dimension q	1	3	10	30 (RM)
threshold at $P_{\mathrm{FA}} = 10^{-3}$	10.83	16.27	29.59	59.70
required δ for $P_D = 0.5$	10.83	14.24	20.53	30.59

Worked example 17.2: What knowing the family is worth

Going from "any of 30 directions" to a 3-dimensional family subspace:

$$\text{required signal energy ratio} = 30.59/14.24 = 2.148$$
$$\text{ratio in fill} = \sqrt{2.148} = 1.466.$$

The square root is there because the signal energy is proportional to the fill squared ($\|\mathbf{W}(a\mathbf{d})\|^2 = a^2\|\mathbf{W}\mathbf{d}\|^2$). So a family test can detect a fill about 1.47 times smaller than RM, under the Gaussian model.

Put differently, a signal with $\delta = 9$ is caught with probability 0.229 by the 3-dimensional test and 0.028 by RM[C]: eight times more often.

Family matching is RM restricted to library directions. The restriction buys sensitivity and, more importantly, a name.

17.6 What RM does not see

RM keeps the 30 *largest*-variance residual directions. Within them, the smaller-variance ones are amplified by the whitening, which is the sense in which it "rewards errors in directions the background never takes". But anything in the remaining 135 or more directions of the 165-band grid contributes nothing, as residual B showed with $d = 2$. That truncation is deliberate: full-band RX was near-singular and was dropped in 2026-05. It is harmless for detection when targets share the leading residual directions, and real objects mostly do (Chapter 19).

For *naming* it may not be harmless. Chapter 21 shows that what distinguishes one family from another is a few per cent of a spectrum's energy, and nothing guarantees that those few per cent lie inside RM's 30 directions. A family test should therefore use its own whitening (as `family_match.py` does: all bands, with a ridge) rather than inherit RM's truncation. Measuring how much family-separating energy RM discards is the first open experiment in the original note.

Analogy: two security guards

One guard is told "report anything unusual". He must watch every corridor, and since something odd happens somewhere most nights, he only reports what is very odd. A second guard is told "watch for someone carrying a ladder". She can ignore most of what happens and react to a fainter hint of a ladder. RM is the first guard; a family test is the second. The second guard is more sensitive, and her report comes with a description.

Common confusions

Is RM the same as RX? RX is the Mahalanobis distance of raw pixels from a background mean, usually with all bands. RM uses the model's residual, 30 components and a ring mean. Plain RX on hyperspectral bands was dropped from the product.
Why 30? A choice found to work on the bench (`PCA_COMPONENTS = 30`), well conditioned where the full covariance is not. It is not derived from theory.
Is the ring mean subtracted from the residual or the pixel? From the (whitened) residual.
Does a high RM mean a target? It means unusual. Field edges, glint, shorelines and haze are unusual too; on the bench, false alarms outnumber targets about 50 to 1 among high-scoring pixels (Chapter 27).

Check yourself

1. With $\mathbf{W} = \mathrm{diag}(\frac{1}{2}, 1, 2)$, compute RM for $\mathbf{r} = (2,\ 0,\ 0)$.
2. Compute RM for $\mathbf{r} = (0,\ 1,\ 1)$ with $d = 3$ and with $d = 2$.
3. What are the mean and the 10^{-3} threshold of RM's null distribution?
4. By what factor in fill does a 10-dimensional subspace test beat RM? (Use the table.)
5. Why does RM fall back to the scene mean when the ring has fewer than 60 valid pixels?

Check yourself: worked answers

1. $\mathbf{Wr} = (1,\ 0,\ 0)$; RM $= 1$.
2. $\mathbf{Wr} = (0,\ 1,\ 2)$; with $d = 3$, RM $= 0 + 1 + 4 = 5$; with $d = 2$ (drop band 3), $(0,\ 1)$, RM $= 1$.
3. Mean $d = 30$; threshold 59.70.
4. $30.59/20.53 = 1.490$; $\sqrt{1.490} = 1.221$.
5. A mean of 30-dimensional vectors from too few samples is itself noisy; about $2d$ samples keeps it stable (the code's comment on `MIN_RING_FACTOR`).

Next: Part IV, matching on the residual, starting with Chapter 18.

Part IV

Matching on the residual

Chapter 18

The fill estimate

The one thing this chapter teaches

If you assume the material is $\mathbf{t}$, the best estimate of its fill is $\hat{a} = \mathbf{d}^\top \mathrm{S}^{-1}\mathbf{r} \,/\, \mathbf{d}^\top \mathrm{S}^{-1}\mathbf{d}$: a weighted projection of the residual onto the target difference, unbiased when the library spectrum is right, with variance $1/\mathbf{d}^\top \mathrm{S}^{-1}\mathbf{d}$, the smallest any unbiased estimate can have.

18.1 The problem: how much metal is in this pixel?

Pick one candidate material, say galvanised metal, with resampled, normalised spectrum $\mathbf{t}$. Then $\mathbf{d} = \mathbf{t} - \hat{\mathbf{b}}$ is known, and the boxed form of the residual from Chapter 16 reads

$$\mathbf{r} = a\,\mathbf{d} + \mathbf{w}, \qquad \mathbf{w} = (1-a)\mathbf{e} + \mathbf{n}.$$

The unknown is the single number a. The vector $\mathbf{w}$ is "everything else". The question is how to combine the residual's bands into one estimate of a, given that some bands are much noisier than others and that the noise in different bands is correlated.

18.2 The noise model

Treat $\mathbf{w}$ as Gaussian with mean zero and covariance S. By rule 5.2 (with the matrix $(1-a)\mathbf{I}$) and the independence of $\mathbf{e}$ and $\mathbf{n}$,

$$\mathrm{S} = \mathrm{Cov}\big((1-a)\mathbf{e} + \mathbf{n}\big) = (1-a)^2\Sigma_e + \Sigma_n.$$

For the small fills of interest $(1-a)^2$ is close to 1 (0.81 at $a = 0.1$, 0.9025 at $a = 0.05$), and Σ_n is negligible (Chapter 16), so in practice $\mathrm{S} \approx \Sigma_r$, the residual covariance of the scene's background pixels.

Write $\langle \mathbf{u}, \mathbf{v} \rangle_{\mathrm{S}} = \mathbf{u}^\top \mathrm{S}^{-1}\mathbf{v}$ and $\|\mathbf{u}\|_{\mathrm{S}}^2 = \mathbf{u}^\top \mathrm{S}^{-1}\mathbf{u}$, the whitened dot product and squared length of Chapter 7.

18.3 Deriving the estimate, one step at a time

Step 1: write the likelihood. By the multivariate Gaussian of Chapter 10, with mean $a\mathbf{d}$ and covariance S,

$$p(\mathbf{r} \mid a) = \text{const} \times \exp\Big(-\tfrac{1}{2}(\mathbf{r} - a\mathbf{d})^\top \mathrm{S}^{-1}(\mathbf{r} - a\mathbf{d}) \Big).$$

Step 2: take logs. By rule 10.1, $\ell(a) = \text{const} - \tfrac{1}{2} J(a)$ with

$$J(a) = (\mathbf{r} - a\mathbf{d})^\top \mathrm{S}^{-1}(\mathbf{r} - a\mathbf{d}),$$

a *weighted squared error*: the squared Mahalanobis length of what is left after removing $a\mathbf{d}$. Maximising ℓ is minimising J.

Step 3: expand J term by term. The transpose of a difference is the difference of transposes, and a is a number, so $(\mathbf{r} - a\mathbf{d})^\top = \mathbf{r}^\top - a\mathbf{d}^\top$. Multiply out the brackets as you would $(p-q)(p-q)$, keeping the order of the matrix products:

$$\begin{aligned} J(a) &= (\mathbf{r}^\top - a\mathbf{d}^\top)\,\mathbf{S}^{-1}\,(\mathbf{r} - a\mathbf{d}) \\ &= \mathbf{r}^\top\mathbf{S}^{-1}\mathbf{r} - a\,\mathbf{r}^\top\mathbf{S}^{-1}\mathbf{d} - a\,\mathbf{d}^\top\mathbf{S}^{-1}\mathbf{r} + a^2\,\mathbf{d}^\top\mathbf{S}^{-1}\mathbf{d}. \end{aligned}$$

The two middle terms are equal: each is a single number, and by rule 4.2 with the symmetric $\mathbf{S}^{-1}$, $\mathbf{r}^\top\mathbf{S}^{-1}\mathbf{d} = \mathbf{d}^\top\mathbf{S}^{-1}\mathbf{r}$. So

$$J(a) = \mathbf{r}^\top\mathbf{S}^{-1}\mathbf{r} - 2a\,\mathbf{d}^\top\mathbf{S}^{-1}\mathbf{r} + a^2\,\mathbf{d}^\top\mathbf{S}^{-1}\mathbf{d}.$$

Step 4: differentiate with respect to a. Everything except a is a fixed number. Using rule 8.1 term by term:

$$J'(a) = 0 - 2\,\mathbf{d}^\top\mathbf{S}^{-1}\mathbf{r} + 2a\,\mathbf{d}^\top\mathbf{S}^{-1}\mathbf{d}.$$

Step 5: set to zero and solve.

$$\begin{aligned} -2\,\mathbf{d}^\top\mathbf{S}^{-1}\mathbf{r} + 2a\,\mathbf{d}^\top\mathbf{S}^{-1}\mathbf{d} &= 0 \\ 2a\,\mathbf{d}^\top\mathbf{S}^{-1}\mathbf{d} &= 2\,\mathbf{d}^\top\mathbf{S}^{-1}\mathbf{r} && \text{add } 2\mathbf{d}^\top\mathbf{S}^{-1}\mathbf{r} \text{ to both sides} \\ a &= \frac{\mathbf{d}^\top\mathbf{S}^{-1}\mathbf{r}}{\mathbf{d}^\top\mathbf{S}^{-1}\mathbf{d}} && \text{divide both sides by } 2\mathbf{d}^\top\mathbf{S}^{-1}\mathbf{d} \end{aligned}$$

Step 6: check it is a minimum. $J''(a) = 2\,\mathbf{d}^\top\mathbf{S}^{-1}\mathbf{d}$, which is positive (a squared Mahalanobis length). A minimum.

Rule 18.1: The fill estimate

$$\hat{a} = \frac{\mathbf{d}^\top\mathbf{S}^{-1}\mathbf{r}}{\mathbf{d}^\top\mathbf{S}^{-1}\mathbf{d}} = \frac{\langle \mathbf{d}, \mathbf{r}\rangle_{\mathbf{S}}}{\|\mathbf{d}\|_{\mathbf{S}}^2}.$$

It is the "minimiser = B/C" of Chapter 8, and it is the coefficient of the projection of $\mathbf{r}$ onto the line through $\mathbf{d}$ (Chapter 9), taken in the whitened metric.

Worked example 18.1: The fill estimate on three bands, checked by trying values

Three normalised bands. Target difference $\mathbf{d} = (2,\ 2,\ -2)$. Residual $\mathbf{r} = (0.5,\ 0.3,\ -0.6)$ (made from a true fill of 0.2, $0.2\mathbf{d} = (0.4,\ 0.4,\ -0.4)$, plus noise $(0.1,\ -0.1,\ -0.2)$). Noise covariance $\mathbf{S} = \mathrm{diag}(0.01,\ 0.01,\ 0.04)$: band 3 is twice as noisy (sd 0.2 against 0.1).
$\mathbf{S}^{-1} = \mathrm{diag}(100,\ 100,\ 25)$.

$$\begin{aligned} \mathbf{S}^{-1}\mathbf{r} &= (100\times 0.5,\ 100\times 0.3,\ 25\times(-0.6)) = (50,\ 30,\ -15) \\ \mathbf{d}^\top\mathbf{S}^{-1}\mathbf{r} &= 2\times 50 + 2\times 30 + (-2)\times(-15) = 100 + 60 + 30 = 190 \\ \mathbf{S}^{-1}\mathbf{d} &= (100\times 2,\ 100\times 2,\ 25\times(-2)) = (200,\ 200,\ -50) \\ \mathbf{d}^\top\mathbf{S}^{-1}\mathbf{d} &= 2\times 200 + 2\times 200 + (-2)\times(-50) = 400 + 400 + 100 = 900 \\ \hat{a} &= 190/900 = 0.2111. \end{aligned}$$

Check by trying values of a in $J(a)$. At $a = 0.2$: $\mathbf{r} - 0.2\mathbf{d} = (0.5-0.4,\ 0.3-0.4,\ -0.6+0.4) = (0.1,\ -0.1,\ -0.2)$; weighted squares $100\times 0.01 + 100\times 0.01 + 25\times 0.04 = 1+1+1 = 3$. The same way:

a	0.15	0.20	0.2111	0.25
$\mathbf{r} - a\mathbf{d}$	$(0.2,\ 0,\ -0.3)$	$(0.1,\ -0.1,\ -0.2)$	$(0.078,\ -0.122,\ -0.178)$	$(0,\ -0.2,\ -0.1)$
$J(a)$	$4 + 0 + 2.25 = 6.25$	3.00	2.889	$0 + 4 + 0.25 = 4.25$

The smallest is at 0.2111. Also, with $\mathbf{r}^\top \mathbf{S}^{-1}\mathbf{r} = 0.5 \times 50 + 0.3 \times 30 + (-0.6) \times (-15) = 25 + 9 + 9 = 43$, the expanded form is $J(a) = 43 - 380a + 900a^2$; at $a = 0.2$ that is $43 - 76 + 36 = 3$. Consistent. For comparison, the unweighted estimate $\mathbf{d}^\top \mathbf{r}/\mathbf{d}^\top \mathbf{d} = (1 + 0.6 + 1.2)/(4 + 4 + 4) = 2.8/12 = 0.2333$ trusts noisy band 3 as much as the others, and lands farther from the truth here.

18.4 Is it right on average? Unbiasedness

Imagine repeating the measurement many times with fresh noise. Since $\mathbf{r} = a\mathbf{d} + \mathbf{w}$ and $\mathbb{E}[\mathbf{w}] = \mathbf{0}$, $\mathbb{E}[\mathbf{r}] = a\mathbf{d}$. The estimate is a fixed row vector times $\mathbf{r}$, so the expectation passes inside:

$$\mathbb{E}[\hat{a}] = \frac{\mathbf{d}^\top \mathbf{S}^{-1}\mathbb{E}[\mathbf{r}]}{\mathbf{d}^\top \mathbf{S}^{-1}\mathbf{d}} = \frac{\mathbf{d}^\top \mathbf{S}^{-1}(a\mathbf{d})}{\mathbf{d}^\top \mathbf{S}^{-1}\mathbf{d}} = a\,\frac{\mathbf{d}^\top \mathbf{S}^{-1}\mathbf{d}}{\mathbf{d}^\top \mathbf{S}^{-1}\mathbf{d}} = a.$$

On average it is exactly right: **unbiased**, provided the library spectrum is the deployed material (the last section of this chapter drops that proviso).

18.5 How precise is it? The variance

Write $\hat{a} = \mathbf{c}^\top \mathbf{r}$ with $\mathbf{c} = \mathbf{S}^{-1}\mathbf{d}/\mathcal{J}$, where $\mathcal{J} = \mathbf{d}^\top \mathbf{S}^{-1}\mathbf{d}$. The fixed part $a\mathbf{d}$ does not wobble, so $\mathbf{r}$ wobbles with covariance $\mathbf{S}$, and rule 5.1 gives

$$\begin{aligned}
\operatorname{Var}(\hat{a}) &= \mathbf{c}^\top \mathbf{S}\mathbf{c} \\
&= \frac{(\mathbf{S}^{-1}\mathbf{d})^\top\, \mathbf{S}\, (\mathbf{S}^{-1}\mathbf{d})}{\mathcal{J}^2} \\
&= \frac{\mathbf{d}^\top \mathbf{S}^{-1}\mathbf{S}\mathbf{S}^{-1}\mathbf{d}}{\mathcal{J}^2} && (\mathbf{S}^{-1}\mathbf{d})^\top = \mathbf{d}^\top \mathbf{S}^{-1}\text{: rule 4.1, } \mathbf{S}^{-1} \text{ symmetric} \\
&= \frac{\mathbf{d}^\top \mathbf{S}^{-1}\mathbf{d}}{\mathcal{J}^2} && \mathbf{S}^{-1}\mathbf{S} = \mathbf{I} \\
&= \frac{\mathcal{J}}{\mathcal{J}^2} = \frac{1}{\mathcal{J}} = \frac{1}{\mathbf{d}^\top \mathbf{S}^{-1}\mathbf{d}}.
\end{aligned}$$

Worked example 18.2: The precision of the three-band estimate

$\mathcal{J} = 900$, so $\operatorname{Var}(\hat{a}) = 1/900 = 0.00111$ and the standard deviation is $1/30 = 0.0333$. Directly: $\mathbf{c} = (200,\ 200,\ -50)/900 = (0.2222,\ 0.2222,\ -0.0556)$, and $\mathbf{c}^\top \mathbf{S}\mathbf{c} = 0.01 \times 0.2222^2 + 0.01 \times 0.2222^2 + 0.04 \times 0.0556^2 = 0.000494 + 0.000494 + 0.000124 = 0.00111$. The estimate 0.2111 is 0.0111 from the truth, a third of a standard deviation: ordinary luck.

The number $\mathcal{J} = \mathbf{d}^\top \mathbf{S}^{-1}\mathbf{d}$ is the hero of the next three chapters. A long target difference, or one pointing where the background rarely goes, makes it large and the estimate precise.

18.6 Could any estimate do better? The Cramér-Rao bound

This section is harder; you can skip to the next on a first reading. It shows that no unbiased estimate of a can have a smaller variance than $1/\mathcal{J}$, so $\hat{a}$ is as good as it gets.

Definition: score, Fisher information

The **score** is the slope of the log-likelihood, $s(a) = \ell'(a)$. The **Fisher information** is its variance, $\mathcal{J}(a) = \operatorname{Var}(s)$; it also equals $-\mathbb{E}[\ell''(a)]$, how sharply curved the log-likelihood is on average. A sharply peaked log-likelihood pins a down well.

For our model, $\ell(a) = \text{const} - \frac{1}{2}J(a)$, so using Step 4,

$$\begin{aligned} s(a) &= -\tfrac{1}{2}J'(a) = \mathbf{d}^\top \mathbf{S}^{-1}\mathbf{r} - a\,\mathbf{d}^\top \mathbf{S}^{-1}\mathbf{d} = \mathbf{d}^\top \mathbf{S}^{-1}(\mathbf{r} - a\mathbf{d}) \\ \mathrm{Var}(s) &= \mathbf{d}^\top \mathbf{S}^{-1}\,\mathbf{S}\,\mathbf{S}^{-1}\mathbf{d} = \mathbf{d}^\top \mathbf{S}^{-1}\mathbf{d} = \mathcal{I} \qquad \textit{rule 5.1 with weights } \mathbf{S}^{-1}\mathbf{d} \\ -\ell''(a) &= \mathbf{d}^\top \mathbf{S}^{-1}\mathbf{d} = \mathcal{I}. \end{aligned}$$

So the Fisher information is the same $\mathcal{I}$ that appeared as $1/\mathrm{Var}(\hat{a})$.

Rule 18.2: The Cramér-Rao bound

For any estimate $\tilde{a}$ that is unbiased whatever the true a, $\mathrm{Var}(\tilde{a}) \geq 1/\mathcal{I}$.
Why. Two facts. (i) For an unbiased estimate, $\mathrm{Cov}(\tilde{a}, s) = 1$; this comes from differentiating $\mathbb{E}[\tilde{a}] = a$ with respect to a, and this book takes it as given. (ii) For any two random quantities, $\mathrm{Cov}(X, Y)^2 \leq \mathrm{Var}(X)\,\mathrm{Var}(Y)$; this is the Cauchy-Schwarz inequality of Chapter 3 with covariance playing the dot product and standard deviation playing length (a correlation cannot exceed 1). Put $X = \tilde{a}$, $Y = s$: $1^2 \leq \mathrm{Var}(\tilde{a}) \times \mathcal{I}$, so $\mathrm{Var}(\tilde{a}) \geq 1/\mathcal{I}$.
Our $\hat{a}$ meets it. $\mathrm{Cov}(\hat{a}, s) = \mathrm{Cov}(\mathbf{c}^\top \mathbf{r},\ \mathbf{d}^\top \mathbf{S}^{-1}\mathbf{r}) = \mathbf{c}^\top \mathbf{S}\mathbf{S}^{-1}\mathbf{d} = \mathbf{c}^\top \mathbf{d} = \mathbf{d}^\top \mathbf{S}^{-1}\mathbf{d}/\mathcal{I} = 1$, and $\mathrm{Var}(\hat{a}) = 1/\mathcal{I}$ exactly. An estimate that reaches the bound is called **efficient**.

For the metal-over-grass example of the next chapter at the middle background-error level, the note computes $\mathrm{sd}(\hat{a}) = 1/\|\mathbf{d}\|_\mathbf{S} = 0.012^{[\mathrm{C}]}$. **Precise in principle; the real failures of fill estimation are bias, not variance.**

18.7 Keeping the fill between 0 and 1

$\hat{a}$ can come out negative or above 1, which no physical fill can be. The natural fix is to clip: $\tilde{a} = \min(1, \max(0, \hat{a}))$. Clipping never moves the estimate farther from a true fill that lies in $[0, 1]$, so it never increases the error. But near $a = 0$ it biases upward: negative estimates are raised to 0 while positive ones are kept.

Worked example 18.3: The upward bias of clipping at zero fill

True $a = 0$, $\mathcal{I} = 900$, so $\hat{a} \sim \mathcal{N}(0,\ (1/30)^2)$; write $\hat{a} = Z/30$ with Z standard normal. The clipped estimate's average is

$$\begin{aligned} \mathbb{E}[\max(0, \hat{a})] &= \tfrac{1}{30}\,\mathbb{E}[\max(0, Z)] = \tfrac{1}{30}\int_0^\infty z\,\varphi(z)\,dz \\ &= \tfrac{1}{30} \times 0.3989 \qquad \textit{the integral from Chapter 10} \\ &= 0.0133. \end{aligned}$$

An empty pixel reads, on average, as a fill of 0.013. The note's general formula is $\mathbb{E}\tilde{a} - a = \mathcal{I}^{-1/2}[\varphi(\kappa) - \kappa\,\Phi(-\kappa)]$ with $\kappa = a\sqrt{\mathcal{I}}$ (ignoring the clip at 1); at $\kappa = 0$ it gives $\varphi(0)/30 = 0.3989/30$, as above; at $\kappa = 1$ (true $a = 1/30$), $(0.2420 - 1 \times 0.1587)/30 = 0.0833/30 = 0.0028$.

18.8 A wrong library spectrum

Real roofs are weathered, dusty and painted. Suppose the deployed material is $\mathbf{t}' = \mathbf{t} + \boldsymbol{\delta}_t$ but you estimate with $\mathbf{t}$. Then the true residual is $\mathbf{r} = a(\mathbf{d} + \boldsymbol{\delta}_t) + \mathbf{w}$ (because $\mathbf{t}' - \hat{\mathbf{b}} = \mathbf{d} + \boldsymbol{\delta}_t$), and

$$\mathbb{E}[\hat{a}] = \frac{\mathbf{d}^\top \mathbf{S}^{-1}\,a(\mathbf{d} + \boldsymbol{\delta}_t)}{\mathbf{d}^\top \mathbf{S}^{-1}\mathbf{d}} = a\Big(\frac{\mathbf{d}^\top \mathbf{S}^{-1}\mathbf{d}}{\mathbf{d}^\top \mathbf{S}^{-1}\mathbf{d}} + \frac{\mathbf{d}^\top \mathbf{S}^{-1}\boldsymbol{\delta}_t}{\mathbf{d}^\top \mathbf{S}^{-1}\mathbf{d}}\Big) = a\Big(1 + \frac{\langle \mathbf{d}, \boldsymbol{\delta}_t \rangle_\mathbf{S}}{\|\mathbf{d}\|_\mathbf{S}^2}\Big).$$

Worked example 18.4: A darker deployed material

Same three bands. The deployed material is darker than the library in bands 1 and 2: $\boldsymbol{\delta}_t = (-0.4,\ -0.4,\ 0)$. True fill 0.2.

$$\langle \mathbf{d}, \boldsymbol{\delta}_t \rangle_{\mathbf{S}} = 2 \times 100 \times (-0.4) + 2 \times 100 \times (-0.4) + (-2) \times 25 \times 0 = -80 - 80 + 0 = -160$$
$$\text{ratio} = -160/900 = -0.178$$
$$\mathbb{E}[\hat{a}] = 0.2 \times (1 - 0.178) = 0.2 \times 0.822 = 0.164.$$

The estimate is 18% low on average, however many times you repeat it.

Two cases are worth separating. A mismatch *perpendicular* to $\mathbf{d}$ in the whitened metric does not bias the fill (the ratio is zero) but turns the residual away from $\mathbf{d}$, which lowers the match scores of the next chapters. A mismatch *along* $\mathbf{d}$ is indistinguishable from a different fill. The low-level design records both effects on real data[M]: the finite-target matched filter's fill estimate was "unbiased and at the theoretical limit on synthetic data", yet "collapsed towards 0 on the Gudalur sheets and underestimated oil by about 0.8". There the library spectrum was not the deployed material and $\langle \mathbf{d}, \boldsymbol{\delta}_t \rangle_{\mathbf{S}} < 0$.

Analogy: three scales, one wobbly

You weigh a parcel on three kitchen scales. Two are steady to within 10 g; one wobbles by 20 g. A sensible average gives the steady scales more say: weight each reading by one over its variance ($1/10^2$, $1/10^2$, $1/20^2$, so 4, 4 and 1 in proportion). That is what $\mathbf{S}^{-1}$ does to the bands, and with correlated scales it also accounts for scales that err together. And if the parcel's label is wrong (a wrong library spectrum), no amount of careful averaging fixes the answer.

Common confusions

Can $\hat{a}$ be negative? Yes, when the residual points away from $\mathbf{d}$. A strongly negative $\hat{a}$ is evidence against this material.
Is a small $\mathrm{Var}(\hat{a})$ **a sign the estimate is right?** No. Variance measures repeatability. A wrong library spectrum gives a precise, repeatable, wrong answer (worked example 18.4).
Why $\mathbf{S}^{-1}$ and not $\mathbf{S}$? Noisy directions should count less. Dividing by variance (multiplying by the inverse) does that.
Which $\mathbf{S}$ is used in practice? The scene's residual covariance $\hat{\Sigma}_r$ estimated from background pixels, which stands in for $(1-a)^2\Sigma_e + \Sigma_n$.

Check yourself

1. With $\mathbf{d}$ and $\mathbf{S}$ of worked example 18.1, compute $\hat{a}$ for the noiseless residual $\mathbf{r} = (0.4,\ 0.4,\ -0.4)$.
2. If $\mathbf{S}$ were $\mathbf{I}$, what would $\hat{a}$ be for $\mathbf{r} = (0.5,\ 0.3,\ -0.6)$?
3. If every variance in $\mathbf{S}$ doubled, what would the standard deviation of $\hat{a}$ become?
4. What is the average clipped estimate at true fill 0 when $\mathcal{I} = 900$?
5. Show that $\boldsymbol{\delta}_t = (0.1,\ -0.1,\ 0)$ causes no bias with this $\mathbf{d}$ and $\mathbf{S}$.

Check yourself: worked answers

1. $\mathbf{d}^\top \mathbf{S}^{-1} \mathbf{r} = 200 \times 0.4 + 200 \times 0.4 + (-50) \times (-0.4) = 80 + 80 + 20 = 180$; $180/900 = 0.2$, exactly the true fill.
2. The unweighted $\mathbf{d}^\top \mathbf{r} / \mathbf{d}^\top \mathbf{d} = 2.8/12 = 0.2333$.
3. $\mathbf{S}^{-1}$ halves, so $\mathcal{I} = 450$; sd $= 1/\sqrt{450} = 0.0471$.

4. $0.3989/30 = 0.0133$.
5. $\langle \mathbf{d}, \boldsymbol{\delta}_t \rangle_{\mathrm{S}} = 200 \times 0.1 + 200 \times (-0.1) + (-50) \times 0 = 20 - 20 + 0 = 0$. No bias (but the residual's direction changes).

Next: Chapter 19, deciding whether the material is there at all.

Chapter 19

The matched filter and the smallest detectable fill

The one thing this chapter teaches

The matched filter divides the fill estimate by its standard deviation, giving a statistic that is standard normal on background and shifted by $a\sqrt{\mathbf{d}^\top \mathbf{S}^{-1}\mathbf{d}}$ on target; from it follows the smallest fill you can detect, which for real objects hiding in the model's own error directions is a few per cent in theory and much more in practice.

19.1 The problem: is the material there?

Chapter 18 estimated how much metal there is, assuming there is some. Now ask the prior question: is there any? That is a hypothesis test (Chapter 11): H_0, $a = 0$; H_1, $a > 0$.

19.2 The statistic

Standardise the fill estimate: divide by its standard deviation $1/\sqrt{\mathcal{I}}$.

$$T_{\mathrm{MF}} = \hat{a}\sqrt{\mathcal{I}} = \frac{\mathbf{d}^\top \mathbf{S}^{-1}\mathbf{r}}{\mathbf{d}^\top \mathbf{S}^{-1}\mathbf{d}} \times \sqrt{\mathbf{d}^\top \mathbf{S}^{-1}\mathbf{d}} = \frac{\mathbf{d}^\top \mathbf{S}^{-1}\mathbf{r}}{\sqrt{\mathbf{d}^\top \mathbf{S}^{-1}\mathbf{d}}}.$$

This is the **matched filter**. From the previous chapter, $\hat{a}$ is Gaussian with mean a and variance $1/\mathcal{I}$. Multiplying by $\sqrt{\mathcal{I}}$ multiplies the mean by $\sqrt{\mathcal{I}}$ and the variance by $\mathcal{I}$:

$$T_{\mathrm{MF}} \sim \mathcal{N}\big(a\sqrt{\mathcal{I}},\ 1\big).$$

Under H_0 ($a = 0$) it is standard normal; under H_1 the bell is shifted by $\mu = a\sqrt{\mathcal{I}}$. That is exactly the two-bell picture of Figure 11.1. The detection SNR is $\mu^2 = a^2\,\mathbf{d}^\top \mathbf{S}^{-1}\mathbf{d}$.

In whitened terms, T_{MF} is the length of the projection of the whitened residual $\mathbf{W}\mathbf{r}$ onto the whitened target direction $\mathbf{W}\mathbf{d}$, with a sign: $\mathbf{W}\mathbf{d}^\top \mathbf{W}\mathbf{r}/\|\mathbf{W}\mathbf{d}\|$.

Worked example 19.1: The matched filter on three bands

From worked example 18.1: $\mathbf{d}^\top \mathbf{S}^{-1}\mathbf{r} = 190$ and $\mathbf{d}^\top \mathbf{S}^{-1}\mathbf{d} = 900$, so

$$T_{\mathrm{MF}} = 190/\sqrt{900} = 190/30 = 6.33.$$

Threshold for $P_{\mathrm{FA}} = 10^{-3}$: 3.09. $6.33 > 3.09$: detected.

Whitened check, with $\mathbf{W} = \text{diag}(1/0.1,\ 1/0.1,\ 1/0.2) = \text{diag}(10,\ 10,\ 5)$: $\mathbf{Wd} = (20,\ 20,\ -10)$, $\mathbf{Wr} = (5,\ 3,\ -3)$; dot product $100 + 60 + 30 = 190$; $\|\mathbf{Wd}\| = \sqrt{400 + 400 + 100} = \sqrt{900} = 30$. Same. At the true fill 0.2 the shift is $\mu = 0.2 \times 30 = 6$, the SNR 36, and the detection rate $P_D = Q(3.09 - 6) = Q(-2.91) = 0.998$.

19.3 Detection rate and the smallest detectable fill

From Chapter 11, $P_D = Q(Q^{-1}(P_{\text{FA}}) - \mu)$ with $\mu = a\|\mathbf{d}\|_{\text{S}}$. Solve for the fill that reaches a chosen P_D:

$$a\|\mathbf{d}\|_{\text{S}} = Q^{-1}(P_{\text{FA}}) - Q^{-1}(P_D)$$

$$a_{\min} = \frac{Q^{-1}(P_{\text{FA}}) - Q^{-1}(P_D)}{\|\mathbf{d}\|_{\text{S}}} \qquad \textit{divide both sides by } \|\mathbf{d}\|_{\text{S}}$$

At $P_{\text{FA}} = 10^{-3}$ and $P_D = 0.5$ the numerator is 3.09, and $a_{\min}^2\|\mathbf{d}\|_{\text{S}}^2 = 9.55$: the required SNR of worked example 11.2.

Worked example 19.2: Smallest detectable fill on three bands

$\|\mathbf{d}\|_{\text{S}} = 30$. $a_{\min} = 3.09/30 = 0.103$ for a coin-toss detection, and $(3.0902 + 1.2816)/30 = 4.3718/30 = 0.146$ for $P_D = 0.9$.

Two technical remarks. Because a fill cannot be negative, the GLRT for $a \geq 0$ is $\max(0, T_{\text{MF}})^2$: a negative statistic counts as zero. That is why the shipped ACE (Chapter 20) is signed. And when $\mathbf{S}$ is estimated from K background pixels instead of known, the statistic becomes Kelly's GLRT or the adaptive matched filter; the loss is a factor of roughly $(K - N)/K$ in SNR, which for $K = 200{,}000$ and $N \leq 372$ is $(200{,}000 - 372)/200{,}000 = 0.998$, negligible.

19.4 A real example: galvanised metal over grass

The note works one material through on the 165-band grid. Target: galvanised sheet metal (USGS GDS334). Background: 70% cheatgrass plus 30% an alfisol soil, all from the shipped library. Their difference:

- mean absolute value 0.170 reflectance per band, 2.18 in normalised units[C];
- squared length in normalised units $\|\mathbf{D}^{-1}(\mathbf{t} - \mathbf{b})\|^2 = 904.1$[C].

To compute $\|\mathbf{d}\|_{\text{S}}^2$ you need the residual covariance, which is not isotropic: the whole point of RM is that "a scene's normal residuals are large along a few shared directions". The note uses a two-level model[A].

Definition: two-level covariance, foreignness

A fraction $1 - \varepsilon$ of the background error's energy lies in k shared directions (the columns of $\mathbf{U}_k$), spread evenly, and the rest, ε, is spread evenly over the other $N - k$ directions. With total error energy $N\sigma_e^2$ (that is, σ_e^2 per band on average), the eigenvalues are

$$\lambda_{\text{in}} = \frac{(1-\varepsilon)N\sigma_e^2}{k} \text{ (shared directions)}, \qquad \lambda_{\text{out}} = \frac{\varepsilon N\sigma_e^2}{N-k} \text{ (the rest)}.$$

The **foreignness** φ of a target difference $\mathbf{d}$ is the share of its energy $\|\mathbf{d}\|^2$ that lies outside the shared directions.

Check the energy adds up: $k\lambda_{\text{in}} + (N-k)\lambda_{\text{out}} = (1-\varepsilon)N\sigma_e^2 + \varepsilon N\sigma_e^2 = N\sigma_e^2$.

By the eigen form of the Mahalanobis distance (Chapter 7: each squared component divided by its eigenvalue), the inside part contributes $(1-\varphi)\|\mathbf{d}\|^2/\lambda_{\text{in}}$ and the outside part $\varphi\|\mathbf{d}\|^2/\lambda_{\text{out}}$:

$$\|\mathbf{d}\|_S^2 = \|\mathbf{d}\|^2\Big[\frac{1-\varphi}{\lambda_{\text{in}}} + \frac{\varphi}{\lambda_{\text{out}}}\Big].$$

The inputs. $N = 165$, $k = 10$, $\varepsilon = 0.1$[A]. σ_e from the measured mean absolute errors 0.05, 0.10, 0.14 via worked example 10.2: 0.063, 0.125, 0.175. Foreignness: real objects' difference from their ring, relative to the scene's top-10 spectral subspace, has median $\varphi = 0.006$ (the bench-v2 comment in `implant.py`)[M]; library families against the top-10 subspace of the library's natural spectra give 0.000 to 0.024[C], in agreement.

Worked example 19.3: The smallest detectable fill of metal over grass, middle error level

Mean absolute error 0.10, $\varphi = 0.006$.

$$\begin{aligned}
\sigma_e &= 0.10 \times 1.2533 = 0.12533, \qquad \sigma_e^2 = 0.015708\\
\lambda_{\text{in}} &= 0.9 \times 165 \times 0.015708/10 = 148.5 \times 0.015708/10 = 2.33264/10 = 0.233264\\
\lambda_{\text{out}} &= 0.1 \times 165 \times 0.015708/155 = 16.5 \times 0.015708/155 = 0.259182/155 = 0.00167214\\
\text{inside part} &= (1-0.006) \times 904.1/0.233264 = 898.675/0.233264 = 3852.6\\
\text{outside part} &= 0.006 \times 904.1/0.00167214 = 5.4246/0.00167214 = 3244.1\\
\|\mathbf{d}\|_S^2 &= 3852.6 + 3244.1 = 7096.7\\
a_{\min} &= \sqrt{9.5496/7096.7} = \sqrt{0.0013456} = 0.0367.
\end{aligned}$$

The note's value is 7097.1 and 0.037[C]; the small difference is rounding of σ_e. And $\text{sd}(\hat{a}) = 1/\sqrt{7096.7} = 1/84.24 = 0.012$, the number quoted in Chapter 18.

Look at the two parts. The outside part comes from only 0.6% of the difference's energy, yet it supplies almost half the total. The reason is $\lambda_{\text{in}}/\lambda_{\text{out}} = 0.233264/0.00167214 = 139.5$: a unit of signal in a quiet direction is worth 140 units in a busy one. **Detectability lives in the directions the background error rarely takes.**

Table 19.1: Smallest detectable fill $a_{\min}$ ($P_{\text{FA}} = 10^{-3}$, $P_D = 0.5$, required SNR 9.55) for metal over grass, two-level covariance with $\varepsilon = 0.1$, $k = 10$[C]. Measured inputs[M]: the error levels and $\varphi = 0.006$.

mean $\lvert e\rvert$ (normalised)	$\varphi = 0.006$	$\varphi = 0.05$	$\varphi = 0.2$	$\varphi = 1$
0.05 (set1-like)	0.018	0.009	0.005	0.002
0.10	0.037	0.018	0.009	0.004
0.14 (set2/set4-like)	0.051	0.025	0.013	0.006

19.5 The reality check

Theory says a fill of 2 to 5% of metal is detectable. Does the bench agree? The note estimates how much signal real bench targets carry. FT04's hidden-pixel error is 0.18 to 0.22 on target pixels against 0.06 to 0.14 on background (`decoder-capacity/README.md`)[M]. If the target's part and the background part add "in quadrature" (like perpendicular vectors: squares add; exact for root-mean-square values of independent parts, a heuristic for mean absolute values), the target part on set 1 is

$$\begin{aligned}
0.218^2 &= 0.047524, \qquad 0.057^2 = 0.003249\\
0.047524 - 0.003249 &= 0.044275\\
\sqrt{0.044275} &= 0.210.
\end{aligned}$$

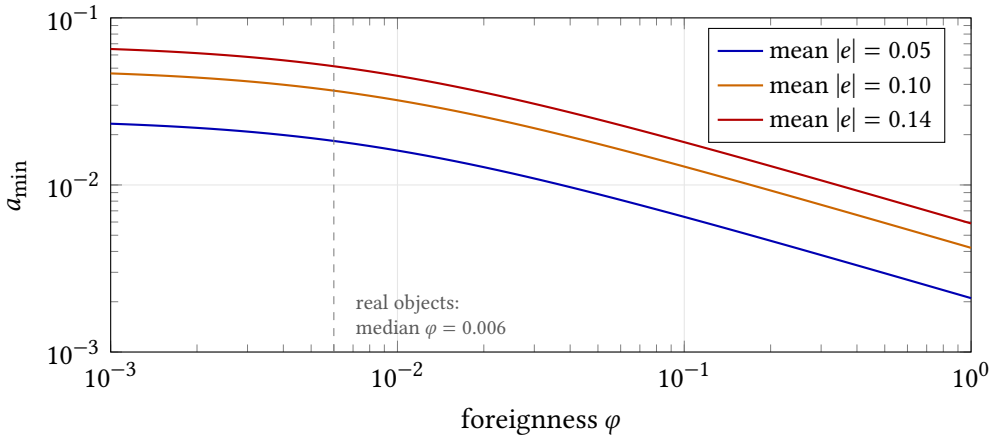


Figure 19.1: Gaussian-theory smallest detectable fill against foreignness (the original note's figure, from `figdata/amin_phi.dat`). Real objects sit at the far left, where the target hides in the same directions as the model's own error.

The note gives 0.17 (set 2), 0.12 (set 3), 0.13 (set 4) the same way. Divided by the metal example's 2.18 per band, the average bench target is "worth" a metal-over-grass fill of 0.06 to 0.10 (set 1: $0.210/2.18 = 0.096$).

Even at $\varphi = 0.006$, Table 19.1 then predicts $P_D \geq 0.75$ at $P_{\mathrm{FA}} = 10^{-3}$ in every scene. Take the worst, set 3: $a \approx 0.057$ against $a_{\min} \approx 0.047$ at its error level of 0.13. Since $a_{\min}\|\mathbf{d}\|_{\mathbf{S}} = 3.09$, the shift is $\mu = 3.09 \times 0.057/0.047 = 3.75$, and $P_D = Q(3.09 - 3.75) = Q(-0.66) = 0.74$.

The measurement says otherwise: Heavy (FTH3 epoch 5) catches 16.0% of objects at the top 0.1% and 49.4% at the top 1%, with $\mathrm{pAUC}_{\leq 1\%} = 0.189$ (`FTH3-heavy-joint-more2/README.md`)[M]. **The Gaussian $a_{\min}$ is a lower bound that is wrong by an order of magnitude**, for reasons Chapter 26 takes apart one by one: heavy tails, uneven clutter, alignment with the error directions, leakage and blur. The practical conclusion is already clear: detectability must be calibrated empirically, by implanting targets at controlled fill, and thresholds must be set per scene.

Analogy: listening for a known tune

In a noisy café you can pick out a song you know well even when it is played quietly, because your brain matches what it hears against the tune it expects. A stranger who does not know the song hears only noise. The matched filter is the listener who knows the tune: it correlates the residual against the expected difference $\mathbf{d}$. And if the café noise happens to contain the same notes as the song (an error in the same directions as the target), even the expert struggles.

Common confusions

Is this SNR the sensor's SNR of 354? No. The sensor SNR is signal over noise in one band of one pixel. The detection SNR $a^2\mathbf{d}^\top\mathbf{S}^{-1}\mathbf{d}$ is for the whole test, against the model's error.

Is $a_{\min}$ a hard limit? It is the fill detected half the time at the chosen false-alarm rate, under the Gaussian model. Change P_{FA} or P_D and it changes.

Why does a tiny foreignness matter so much? Because whitening divides by the eigenvalue, and the outside eigenvalue is 140 times smaller.

Is the two-level covariance measured? No, it is an assumption[A] with measured inputs. It captures "a few loud directions, many quiet ones"; the real spectrum of eigenvalues is a smoother decline.

Check yourself

1. Compute T_{MF} for the noiseless residual $\mathbf{r} = (0.4,\ 0.4,\ -0.4)$ of Chapter 18.
2. With $\|\mathbf{d}\|_{\mathrm{S}} = 30$, what fill is detected 90% of the time at $P_{\mathrm{FA}} = 10^{-3}$?
3. In a two-level model, $\|\mathbf{d}\|^2 = 100$, $\varphi = 0.1$, $\lambda_{\mathrm{in}} = 1$, $\lambda_{\mathrm{out}} = 0.01$. Compute $\|\mathbf{d}\|_{\mathrm{S}}^2$ and say which part dominates.
4. At a fill of twice $a_{\min}$ (with $P_{\mathrm{FA}} = 10^{-3}$, $P_D = 0.5$ defining $a_{\min}$), what is the detection rate?
5. In one sentence, why does the reality check fail?

Check yourself: worked answers

1. $180/30 = 6$.
2. $(3.0902 + 1.2816)/30 = 0.146$.
3. $100\times(0.9/1+0.1/0.01) = 100\times(0.9+10) = 1090$. The outside part (1000 of the 1090) dominates.
4. $\mu = 2 \times 3.09 = 6.18$; $P_D = Q(3.09 - 6.18) = Q(-3.09) = 0.999$.
5. The Gaussian model assumes thin-tailed, stationary noise and a target the background model cannot copy; real residuals have heavy tails, are noisiest where targets are, and partly contain the target (Chapter 26).

Next: Chapter 20, matching shape while ignoring scale.

Chapter 20

ACE: matching the shape, ignoring the scale

The one thing this chapter teaches

When the overall size of the clutter is unknown, the likelihood-ratio test reduces to ACE, the squared cosine between the whitened residual and the whitened target direction; it does not change if the clutter is louder or quieter, which is what makes per-pixel scores comparable.

20.1 The problem: the clutter's loudness varies

The matched filter assumes you know $\mathbf{S}$ exactly. In practice the residual error is larger in some scenes than others (a calm lake against a burnt forest) and larger in some places within a scene than others. If the true covariance is $4\mathbf{S}$ instead of $\mathbf{S}$, every matched-filter value on background is twice as big as you think, and the false-alarm rate explodes (Chapter 27 puts numbers on it). You want a statistic that ignores the overall scale of the clutter and looks only at its shape.

20.2 The model with unknown scale

Let $\mathbf{S} = \sigma^2 \mathbf{S}_0$, where $\mathbf{S}_0$ (the shape) is known and σ^2 (the loudness) is unknown under both hypotheses. Whiten with $\mathbf{S}_0$: write $\tilde{\mathbf{r}} = \mathbf{S}_0^{-1/2}\mathbf{r}$ and $\tilde{\mathbf{d}} = \mathbf{S}_0^{-1/2}\mathbf{d}$ (the tilde marks whitened quantities; any whitening matrix works, Chapter 7). Then

$$H_0 \;:\; \tilde{\mathbf{r}} \sim \mathcal{N}(\mathbf{0},\, \sigma^2\mathbf{I}), \qquad H_1 \;:\; \tilde{\mathbf{r}} \sim \mathcal{N}(a\tilde{\mathbf{d}},\, \sigma^2\mathbf{I}),$$

with a and σ^2 unknown. N is the number of bands.

20.3 Deriving the GLRT, one step at a time

The log-likelihood of an N-band Gaussian with covariance $\sigma^2\mathbf{I}$ and mean $\boldsymbol{\nu}$ is, from Chapter 10 (the determinant of $\sigma^2\mathbf{I}$ is $(\sigma^2)^N$):

$$\ell = -\frac{N}{2}\ln(2\pi) - \frac{N}{2}\ln\sigma^2 - \frac{\|\tilde{\mathbf{r}} - \boldsymbol{\nu}\|^2}{2\sigma^2}.$$

Step 1: H_0, best σ^2. Here $\boldsymbol{\nu} = \mathbf{0}$. Write $Q_0 = \|\tilde{\mathbf{r}}\|^2$. As a function of $v = \sigma^2$ this is $-\frac{N}{2}\ln v - \frac{Q_0}{2v}$ plus a constant, maximised (Chapter 8) at $v = Q_0/N$. Substituting back:

$$\ell_0^{\max} = -\frac{N}{2}\ln(2\pi) - \frac{N}{2}\ln\frac{Q_0}{N} - \frac{N}{2}.$$

(The last term: $Q_0/(2 \times Q_0/N) = N/2$.)

Step 2: H_1, best a. For any fixed σ^2, maximising ℓ over a means minimising $\|\tilde{\mathbf{r}} - a\tilde{\mathbf{d}}\|^2$, the projection problem of Chapter 9. What is left is the remainder's energy:

$$Q_1 = \|\tilde{\mathbf{r}}\|^2 - \frac{(\tilde{\mathbf{d}}^\top \tilde{\mathbf{r}})^2}{\|\tilde{\mathbf{d}}\|^2}.$$

Step 3: H_1, best σ^2. The same algebra as Step 1 with Q_1 in place of Q_0: $\sigma^2 = Q_1/N$ and $\ell_1^{\max} = -\frac{N}{2}\ln(2\pi) - \frac{N}{2}\ln\frac{Q_1}{N} - \frac{N}{2}$.

Step 4: subtract. Everything cancels except the logarithms:

$$\ell_1^{\max} - \ell_0^{\max} = -\frac{N}{2}\ln\frac{Q_1}{N} + \frac{N}{2}\ln\frac{Q_0}{N} = \frac{N}{2}\Big(\ln\frac{Q_0}{N} - \ln\frac{Q_1}{N}\Big) = \frac{N}{2}\ln\frac{Q_0}{Q_1} \qquad \textit{rule 10.1}$$

$$\Lambda = \Big(\frac{Q_0}{Q_1}\Big)^{N/2}.$$

Step 5: rewrite with the cosine. Let $c^2 = (\tilde{\mathbf{d}}^\top \tilde{\mathbf{r}})^2/(\|\tilde{\mathbf{d}}\|^2\|\tilde{\mathbf{r}}\|^2)$, the squared cosine between the two whitened vectors. Then $Q_1 = Q_0 - c^2 Q_0 = Q_0(1-c^2)$, so $Q_0/Q_1 = 1/(1-c^2)$. As c^2 grows from 0 to 1, $1/(1-c^2)$ grows, and so does Λ. Thresholding Λ is the same as thresholding c^2.

Rule 20.1: ACE

$$T_{\text{ACE}} = \frac{(\mathbf{d}^\top \mathbf{S}_0^{-1}\mathbf{r})^2}{(\mathbf{d}^\top \mathbf{S}_0^{-1}\mathbf{d})(\mathbf{r}^\top \mathbf{S}_0^{-1}\mathbf{r})} = \cos^2 \angle(\tilde{\mathbf{d}}, \tilde{\mathbf{r}}),$$

the **adaptive coherence estimator**: the squared cosine of the angle between residual and target in the whitened space. It is the squared projection angle of Chapter 9.

Worked example 20.1: ACE on three bands

Take $\mathbf{S}_0 = \text{diag}(0.01,\ 0.01,\ 0.04)$ from worked example 18.1. Whitened vectors (from worked example 19.1): $\tilde{\mathbf{d}} = (20,\ 20,\ -10)$, $\tilde{\mathbf{r}} = (5,\ 3,\ -3)$.

$$\begin{aligned} Q_0 = \|\tilde{\mathbf{r}}\|^2 &= 25 + 9 + 9 = 43 \\ \tilde{\mathbf{d}}^\top \tilde{\mathbf{r}} = 100 + 60 + 30 &= 190, \qquad \|\tilde{\mathbf{d}}\|^2 = 400 + 400 + 100 = 900 \\ \text{explained energy} &= 190^2/900 = 36100/900 = 40.111 \\ Q_1 &= 43 - 40.111 = 2.889 \\ T_{\text{ACE}} &= 40.111/43 = 0.9328 \\ \text{angle} &= \arccos\sqrt{0.9328} = \arccos(0.9658) = 15.0^\circ. \end{aligned}$$

Check Step 5: $Q_0/Q_1 = 43/2.889 = 14.88$ and $1/(1-0.9328) = 1/0.0672 = 14.88$. The likelihood ratio is $14.88^{3/2} = 57.4$. And the best variances are $Q_0/N = 43/3 = 14.333$ (worked example 8.3) and $Q_1/N = 0.963$.

20.4 Why the scale drops out

Multiply the residual by any number k (louder clutter, or a brighter target, or both). The numerator of T_{ACE} gains k^2, from $(\mathbf{d}^\top \mathbf{S}_0^{-1} k\mathbf{r})^2$, and the denominator gains k^2, from $(k\mathbf{r})^\top \mathbf{S}_0^{-1}(k\mathbf{r})$. They cancel. With $k = 10$ in worked example 20.1: numerator $1900^2 = 3{,}610{,}000$, denominator $900 \times 4300 = 3{,}870{,}000$, ratio 0.9328. Unchanged. Likewise replacing $\mathbf{S}_0$ by $4\mathbf{S}_0$ multiplies both by $1/16$. A test whose false-alarm rate does not depend on the noise level is called **CFAR** (constant false-alarm rate).

20.5 How ACE behaves on background

On pure Gaussian background the whitened residual points in a random direction. How close does a random direction in N dimensions come to a fixed one? Its squared cosine averages $1/N$ (each of the N directions gets an equal share of the energy on average), and its exact distribution is called Beta($\frac{1}{2}$, $\frac{N-1}{2}$); you only need its numbers[C]. With $N = 165$: mean $1/165 = 0.006$, 99.5th percentile 0.047. A random residual almost never lines up with a given target by chance, so even a modest ACE is informative, provided the pixel was anomalous to begin with.

20.6 The shipped matcher

The shipped family matcher computes ACE against $\mathbf{t} - \mathbf{m}$, where $\mathbf{m}$ is the mean of the 13×13-minus-5×5 ring around the pixel, with S_0 the whole-scene pixel covariance plus a ridge of 10^{-3} times its mean eigenvalue (`app/spectral_match/family_match.py`, `whitener`, `ace`). It is a classical detector: the ring is the background estimate, no learned model. It is **signed**: only residuals that point towards the target, not away, can score (a fill cannot be negative). The residual-domain version this book is building towards replaces $\mathbf{m}$ by Indradhanu's $\hat{\mathbf{b}}_p$ and the scene covariance by the residual covariance $\boldsymbol{\Sigma}_r$. Chapter 24 is about what that substitution costs.

Analogy: recognising a tune at any volume

You recognise a song whether it is played softly or loudly, because you attend to the melody, not the volume. ACE attends to the direction of the residual, not its length. The price is the same as for a listener who ignores volume: a faint, random hum that happens to resemble the melody for a moment can fool you, which is why ACE is applied to pixels already flagged as anomalous.

Common confusions

Is ACE an angle? It is a squared cosine, between 0 and 1. $T_{\mathrm{ACE}} = 0.93$ is an angle of 15°.
Does ACE ignore the size of the residual entirely? Yes, which is a strength (CFAR) and a weakness: a tiny residual that happens to point the right way scores high. Detection (is the pixel unusual?) must come first.
Is ACE invariant to everything about the null? Only to its overall scale. It still depends on the shape of S_0, the number of bands, and how heavy the tails are (Chapter 27).
Matched filter or ACE? With a known, reliable $\mathbf{S}$ the matched filter is more sensitive. With a scale that varies from place to place, ACE's false-alarm rate is the one you can trust.

Check yourself

1. Compute ACE for $\tilde{\mathbf{d}} = (1,\ 0)$ and $\tilde{\mathbf{r}} = (1,\ 1)$, and the angle.
2. In worked example 20.1, double $\tilde{\mathbf{r}}$. Recompute Q_0, the explained energy and ACE.
3. What is the average ACE of a random residual against a fixed target in 165 whitened bands?
4. From $Q_0 = 43$ and $Q_1 = 2.889$, compute the likelihood ratio for $N = 3$.
5. Why is the shipped ACE signed?

Check yourself: worked answers

1. Dot product 1; squared lengths 1 and 2. ACE $= 1^2/(1 \times 2) = 0.5$; cos $= 0.7071$; angle 45°.
2. $Q_0 = 4 \times 43 = 172$; dot product 380, explained $380^2/900 = 144400/900 = 160.44$; ACE $= 160.44/172 = 0.9328$. Unchanged.

3. $1/165 = 0.006$.
4. $(43/2.889)^{3/2} = 14.88^{1.5} = 14.88 \times \sqrt{14.88} = 14.88 \times 3.858 = 57.4$.
5. A residual pointing away from the target would mean a negative fill, which is impossible, so it should not count as a match.

Next: Chapter 21, from one spectrum to a family of them.

Chapter 21

Families as subspaces

The one thing this chapter teaches

A material family is nearly one direction plus a little, so it is matched as a low-dimensional subspace; families come within a few degrees of each other, and what tells them apart is only a few per cent of a spectrum's energy.

21.1 The problem: one material, many spectra

"Metal" is not one spectrum. The library's metal family holds 13 complete entries: galvanised sheet, aluminium, rusted steel, painted and unpainted. The roof in the image is probably none of them exactly. Matching against each entry separately and taking the best both wastes the family's shared structure and gives a family with more entries more chances to win by luck. The alternative is to treat the family as a subspace: the span of its spectra (Chapter 9).

21.2 How many directions does a family need?

Stack family k's resampled, normalised spectra as the columns of $\mathbf{H}_k$. The uncentred energy analysis of Chapter 6 (worked example 6.3) asks how much of the family's energy its top direction holds, and how many directions hold 99%.

Table 21.1: Man-made families of the shipped library on the 165-band grid, normalised units[C]. m_k: entries finite on all 165 bands. Rank_{99}: number of directions holding 99% of the (uncentred) energy.

family	m_k	top-1 share	rank_{99}	family	m_k	top-1 share	rank_{99}
building	11	0.940	2	paint	8	0.965	2
glass	1	1.000	1	pavement	15	0.977	2
metal	13	0.949	2	plastic	38	0.930	4
oil	30	0.947	3	roofing	27	0.917	4
textile	14	0.893	4	wood/paper	7	0.987	2

Oil keeps 30 of 40 entries, plastic 38 of 39, wood/paper 7 of 8.

Every man-made family is nearly rank one: the top direction holds 0.89 (textile) to 0.99 (wood/paper) of the energy, and two to four directions hold 99%. **A family is "one shape at varying brightness, plus a little".** That is good for detection (a low-dimensional subspace, cheap in the sense of worked example 17.2) and it is also the root of the confusion problem, because the shapes of different families are close.

21.3 The subspace detector

The test for "some member of family k, in some combination, at some fill" is the subspace GLRT you met in Chapter 17: whiten, project onto the whitened family subspace $\tilde{\mathbf{H}}_k = \mathbf{W}\mathbf{H}_k$, and measure the explained energy,

$$T_k = \|\mathbf{P}_{\tilde{\mathbf{H}}_k}\tilde{\mathbf{r}}\|^2.$$

With the noise scale known it is $\chi^2_{q_k}$ under H_0, with q_k the family's dimension. The derivation is Chapter 17's with $\mathbf{s} = \mathbf{H}_k\boldsymbol{\theta}$.

21.4 Explained energy, unexplained energy and the projection angle

By Pythagoras (Chapter 9), the whitened residual splits into

$$\|\tilde{\mathbf{r}}\|^2 = \underbrace{\|\mathbf{P}_{\tilde{\mathbf{H}}_k}\tilde{\mathbf{r}}\|^2}_{E_k^{\text{expl}}} + \underbrace{\|\mathbf{P}^{\perp}_{\tilde{\mathbf{H}}_k}\tilde{\mathbf{r}}\|^2}_{E_k^{\text{unex}}}, \qquad \cos^2\theta_k = \frac{E_k^{\text{expl}}}{\|\tilde{\mathbf{r}}\|^2} = \text{ACE}_k.$$

The share explained is the subspace version of ACE (Chapter 20), and θ_k is the projection angle onto the family. Three quantities, three jobs:

- RM is the total whitened energy in 30 leading directions (Chapter 17): is anything there?
- ACE_k is the share explained by $q_k \le 4$ directions: does it look like family k?
- E_k^{unex} is the lack of fit: is anything left that family k cannot explain? (Chapter 27 uses it to abstain.)

Worked example 21.1: The energy split on the plane of worked example 9.2

Treat the plane spanned by $(1, 1, 0)$ and $(0, 1, 1)$ as a two-member family in already-whitened units, and $\tilde{\mathbf{r}} = (1, 2, 3)$. From worked example 9.2: $E^{\text{expl}} = 12.667$, $E^{\text{unex}} = 1.333$, total 14. ACE $= 12.667/14 = 0.905$, projection angle 18.0°.

21.5 Why raw family scores cannot be compared

On pure Gaussian background, how much of a random whitened residual does a q-dimensional subspace explain by chance? Each of the N whitened directions holds, on average, an equal share of the energy, so a q-dimensional subspace collects q/N of it. (Exactly, the share follows a Beta$(q/2,\ (N-q)/2)$ distribution.) On 165 bands[C]:

subspace dimension q	mean null share q/N	99.5th percentile
1	$1/165 = 0.006$	0.047
3	$3/165 = 0.018$	0.076

A family of larger effective rank collects more null energy, so a raw score of 0.07 is unremarkable for a rank-3 family and remarkable for a rank-1 family. That is why the product calibrates each family against its own null (Chapter 27) rather than comparing raw scores.

21.6 How far apart are families?

The smallest principal angle between two family subspaces (Chapter 9) says how close the families come. The note computes it between the top-3 subspaces of pairs of man-made families, raw and after projecting

out a 10-dimensional "background" subspace (the top-10 principal directions of the library's centred natural spectra, the analogue of `implant.background_subspace`)[C]:

Table 21.2: Smallest principal angle between rank-3 family subspaces, normalised units[C].

pair	raw	separating share $\sin^2$	after removing background-10
metal vs roofing	3.70°	0.0042	49.8°
metal vs plastic	2.04°	0.0013	43.2°
plastic vs textile	4.53°	0.0062	32.7°
paint vs plastic	5.92°	0.0106	19.4°
pavement vs building	0.51°	0.00008	10.0°

Worked example 21.2: The share of energy that separates two families

From Chapter 9: a unit vector at angle θ to a subspace has $\sin^2\theta$ of its energy outside it. For paint against plastic, $\theta = 5.92°$:

$$5.92° = 5.92 \times \pi/180 = 0.1033 \text{ rad}$$
$$\sin(5.92°) = 0.1031$$
$$\sin^2 = 0.0106.$$

For pavement against building, $\sin(0.51°) = 0.0089$ and $\sin^2 = 0.00008$. So between 8×10^{-5} and 0.011 of a unit target's energy is what distinguishes its family from the nearest other family.

Two readings of Table 21.2. Raw family subspaces are a few degrees apart: the part of a target that says "metal, not roofing" is under 1% of its energy. Once the directions natural surfaces share are removed, the families are well separated in angle, 10° to 50°. But that remaining part carries only 0 to 2.4% of a family's difference from the natural mean (its foreignness: 0.000 for glass and pavement up to 0.024 for plastic and textile[C]). **Identification lives in a few per cent of the signal energy.** Every error source of Part V has to be compared with that few per cent, not with the whole signal.

There is a direct link to Chapter 17. RM keeps the 30 largest residual directions. If the few per cent that separates families lies outside them, RM cannot see it, which is why family tests use their own whitening.

Analogy: telling families of faces apart

All human faces share most of their structure: two eyes, a nose, a mouth, in the same places. Two families' faces are "a few degrees apart": most of what you see is the shared human template. To tell the Smiths from the Joneses you must discount the shared template and look at the small family traits (a chin, an eyebrow), which are distinct once isolated but carry little of the picture. A blurred or badly lit photograph (error sources) can wipe out exactly those traits while leaving "a face" perfectly recognisable.

Common confusions

Is the principal angle the SAM angle between two spectra? It is the smallest angle between *any* spectrum in one family's subspace and *any* in the other's, including mixtures and brightness changes, so it can be much smaller than the angle between any two library entries.

Why rank 3 for the angles when rank_{99} is 2 to 4? The note fixes rank 3 to compare like with like.

Does removing the background directions help? It increases angles (good for naming per unit of energy) and removes most of the energy (bad for detection). Chapter 24 meets the same trade in another guise.

Is "uncentred" a mistake? No. For a family the brightness direction is the main thing the members

share, and it should count.

Check yourself

1. From Table 21.1, which family is least like "one shape", and what share does its top direction hold?
2. Compute the separating share $\sin^2$ for metal against roofing (3.70°).
3. What share of a random whitened residual does a rank-2 family explain on average, on 165 bands? On 234 bands?
4. In worked example 21.1, what is the lack of fit E^{unex}?
5. Why must family scores be calibrated separately before being compared?

Check yourself: worked answers

1. Textile, 0.893.
2. $3.70^\circ = 0.0646$ rad; $\sin = 0.0645$; $\sin^2 = 0.0042$.
3. $2/165 = 0.0121$; $2/234 = 0.0085$.
4. 1.333.
5. A family with more dimensions explains more background energy by chance (q/N), so equal raw scores do not mean equal evidence.

Next: Chapter 22, brightness, haze and mixtures inside a family.

Chapter 22

Nuisances and the family cone

The one thing this chapter teaches

Things you do not care about but cannot ignore (target brightness, a haze offset, the background's own shape) can be written as a known interference subspace and projected out before matching; the shipped Simplex ACE instead lets the target be any non-negative mixture of the family's spectra.

22.1 The problem: the target is not exactly the library

A roof facing away from the sun is darker than the lab sample (a **gain** $g < 1$). Light scattered from nearby fields or haze adds a little to every band (an **offset** c). A deployed material can be a blend of several library entries. A matcher that insists on one exact library spectrum at one exact brightness will miss all of these. There are two standard remedies: describe the nuisance as extra directions and remove them, or widen the family to a cone of mixtures.

22.2 Matched subspace detection with interference

Definition: signal and interference subspaces

Write the residual as

$$\mathbf{r} = \mathbf{H}\boldsymbol{\theta} + \mathbf{J}\boldsymbol{\psi} + \mathbf{w},$$

with $\mathbf{H}$ the **signal subspace** (rank q: the family), $\mathbf{J}$ the **interference subspace** (rank j: the nuisances), unknown amplitudes $\boldsymbol{\theta}$ and $\boldsymbol{\psi}$, and noise $\mathbf{w} \sim \mathcal{N}(\mathbf{0},\ \sigma^2\mathbf{S}_0)$. Whiten by $\mathbf{S}_0$ and mark whitened quantities with a tilde.

The test is a GLRT (Chapter 11). Interference is allowed under both hypotheses; signal only under H_1.

Step 1, under H_0. The best $\boldsymbol{\psi}$ removes everything in the span of $\tilde{\mathbf{J}}$ (a projection, Chapter 9). What remains is $Q_0 = \|\mathbf{P}^{\perp}_{\tilde{\mathbf{J}}}\tilde{\mathbf{r}}\|^2$.

Step 2, under H_1. The best $\boldsymbol{\theta}, \boldsymbol{\psi}$ remove everything in the span of both: $Q_1 = \|\mathbf{P}^{\perp}_{[\tilde{\mathbf{H}}\ \tilde{\mathbf{J}}]}\tilde{\mathbf{r}}\|^2$.

Step 3, the difference. Let $\mathbf{H}_{\perp} = \mathbf{P}^{\perp}_{\tilde{\mathbf{J}}}\tilde{\mathbf{H}}$, the part of the signal subspace that is *not* already interference. The span of $[\tilde{\mathbf{H}}\ \tilde{\mathbf{J}}]$ equals the span of $[\mathbf{H}_{\perp}\ \tilde{\mathbf{J}}]$, and $\mathbf{H}_{\perp}$ is perpendicular to $\tilde{\mathbf{J}}$, so projecting onto the combined span is projecting onto $\tilde{\mathbf{J}}$ plus projecting onto $\mathbf{H}_{\perp}$. Hence

$$Q_0 - Q_1 = \|\mathbf{P}_{\mathbf{H}_{\perp}}\mathbf{P}^{\perp}_{\tilde{\mathbf{J}}}\tilde{\mathbf{r}}\|^2 \ :$$

the energy, after removing interference, that the clean part of the signal subspace explains.

Rule 22.1: The matched subspace detector

With σ known, $(Q_0 - Q_1)/\sigma^2$ is χ^2_q under H_0. With σ unknown, the GLRT is

$$T_{\text{MSD}} = \frac{Q_0 - Q_1}{Q_1} \cdot \frac{N - q - j}{q},$$

which under H_0 follows an $F_{q,\,N-q-j}$ distribution (a ratio of two independent chi-squares, each divided by its degrees of freedom). Dividing by Q_0 instead of Q_1 gives the adaptive subspace detector, the subspace version of ACE.

Worked example 22.1: An offset as interference, three bands

Already-whitened units ($\mathbf{S}_0 = \mathbf{I}$). Interference: a constant offset in every band, $\mathbf{J} = \mathbf{1} = (1,\ 1,\ 1)$. Signal: $\mathbf{h} = (1,\ 0,\ 0)$. Residual $\mathbf{r} = (1,\ 2,\ 3)$.
Remove the offset. Projecting onto $\mathbf{1}$ gives the mean of the components times $\mathbf{1}$: mean $(1+2+3)/3 = 2$, so

$$\mathbf{P}_{\mathbf{1}}^{\perp}\mathbf{r} = (1-2,\ 2-2,\ 3-2) = (-1,\ 0,\ 1), \qquad Q_0 = 1 + 0 + 1 = 2$$
$$\mathbf{h}_{\perp} = \mathbf{P}_{\mathbf{1}}^{\perp}\mathbf{h} = (1 - \tfrac{1}{3},\ 0 - \tfrac{1}{3},\ 0 - \tfrac{1}{3}) = (\tfrac{2}{3},\ -\tfrac{1}{3},\ -\tfrac{1}{3})$$

Project the cleaned residual onto $\mathbf{h}_{\perp}$:

$$\mathbf{h}_{\perp}^{\top}(-1, 0, 1) = -\tfrac{2}{3} + 0 - \tfrac{1}{3} = -1, \qquad \|\mathbf{h}_{\perp}\|^2 = \tfrac{4}{9} + \tfrac{1}{9} + \tfrac{1}{9} = \tfrac{6}{9} = \tfrac{2}{3}$$
$$Q_0 - Q_1 = (-1)^2/\tfrac{2}{3} = 1.5$$
$$Q_1 = 2 - 1.5 = 0.5$$
$$T_{\text{MSD}} = \frac{1.5}{0.5} \times \frac{3 - 1 - 1}{1} = 3.$$

With $\sigma = 1$ known, the chi-square statistic is 1.5, far below the χ^2_1 threshold of 10.83: no detection.

22.3 Brightness and haze are a known interference subspace

Let the observed target be $g\mathbf{t} + c\mathbf{1}$: gain g (slope, shadow, surface angle), offset c (adjacency, haze). The mixed pixel is $\mathbf{x} = a(g\mathbf{t} + c\mathbf{1}) + (1 - a)\mathbf{b} + \mathbf{n}$. Derive its residual, writing $\mathbf{b} = \hat{\mathbf{b}} + \mathbf{e}$ as in Chapter 16:

$$\begin{aligned}
\mathbf{x} - \hat{\mathbf{b}} &= ag\mathbf{t} + ac\mathbf{1} + (1-a)\mathbf{b} + \mathbf{n} - \hat{\mathbf{b}} \\
&= ag\mathbf{t} + ac\mathbf{1} + (1-a)\hat{\mathbf{b}} + (1-a)\mathbf{e} + \mathbf{n} - \hat{\mathbf{b}} && \mathbf{b} = \hat{\mathbf{b}} + \mathbf{e} \\
&= ag\mathbf{t} - a\hat{\mathbf{b}} + ac\mathbf{1} + (1-a)\mathbf{e} + \mathbf{n} && (1-a)\hat{\mathbf{b}} - \hat{\mathbf{b}} = -a\hat{\mathbf{b}} \\
&= ag(\mathbf{t} - \hat{\mathbf{b}}) + ag\hat{\mathbf{b}} - a\hat{\mathbf{b}} + ac\mathbf{1} + (1-a)\mathbf{e} + \mathbf{n} && \textit{add and subtract } ag\hat{\mathbf{b}} \\
&= ag\,(\mathbf{t} - \hat{\mathbf{b}}) + a(g-1)\,\hat{\mathbf{b}} + ac\,\mathbf{1} + (1-a)\mathbf{e} + \mathbf{n}.
\end{aligned}$$

The first term is the target direction with amplitude ag. The next two point along $\hat{\mathbf{b}}$ and $\mathbf{1}$, both known at every pixel. So $\mathbf{J} = [\hat{\mathbf{b}},\ \mathbf{1}]$ (mapped by $\mathbf{D}^{-1}$ into normalised units) is an interference subspace and rule 22.1 applies.

Worked example 22.2: Checking the identity with numbers

$\mathbf{t} = (0.3,\ 0.3,\ 0.3)$, $\mathbf{b} = (0.1,\ 0.1,\ 0.5)$, $\hat{\mathbf{b}} = (0.12,\ 0.09,\ 0.48)$, $a = 0.2$, $g = 0.8$, $c = 0.02$, no noise.
Left side: $g\mathbf{t} + c\mathbf{1} = (0.24 + 0.02,\ \ldots) = (0.26,\ 0.26,\ 0.26)$; times a: $(0.052,\ 0.052,\ 0.052)$; $(1-a)\mathbf{b} = (0.08,\ 0.08,\ 0.40)$; so $\mathbf{x} = (0.132,\ 0.132,\ 0.452)$ and $\mathbf{x} - \hat{\mathbf{b}} = (0.012,\ 0.042,\ -0.028)$.

Right side, band by band. $ag = 0.16$, $\mathbf{t} - \hat{\mathbf{b}} = (0.18,\ 0.21,\ -0.18)$, product $(0.0288,\ 0.0336,\ -0.0288)$. $a(g-1) = -0.04$, times $\hat{\mathbf{b}}$: $(-0.0048,\ -0.0036,\ -0.0192)$. $ac = 0.004$ in each band. $(1-a)\mathbf{e} = 0.8 \times (-0.02,\ 0.01,\ 0.02) = (-0.016,\ 0.008,\ 0.016)$.

$$\begin{aligned} \text{band 1:} \quad & 0.0288 - 0.0048 + 0.004 - 0.016 = 0.012 \\ \text{band 2:} \quad & 0.0336 - 0.0036 + 0.004 + 0.008 = 0.042 \\ \text{band 3:} \quad & -0.0288 - 0.0192 + 0.004 + 0.016 = -0.028 \end{aligned}$$

Both sides agree.

Only the product ag can be recovered. Take $(a, g) = (0.2,\ 0.8)$ and $(0.16,\ 1.0)$: both have $ag = 0.16$, so the target term is identical. They differ only in the amplitudes along $\hat{\mathbf{b}}$ and $\mathbf{1}$ (-0.04 against 0, 0.004 against 0.0032), which are exactly what the free nuisance amplitudes $\boldsymbol{\psi}$ absorb. From one pixel, "20% of a roof in slight shadow" and "16% of a roof in full sun" are the same.

Projecting out $\hat{\mathbf{b}}$ costs something: it removes the target's component along its own background, and it uses up one degree of freedom. In the toy, the share of $\mathbf{t} - \hat{\mathbf{b}}$ lying along $\hat{\mathbf{b}}$ is $(\hat{\mathbf{b}}^\top(\mathbf{t} - \hat{\mathbf{b}}))^2/(\|\hat{\mathbf{b}}\|^2\|\mathbf{t} - \hat{\mathbf{b}}\|^2) = 0.0459^2/(0.2529 \times 0.1089) = 0.0021/0.0275 = 0.077$, about 8%. For a dark target on a bright background it can be most of the target. This is the trade every brightness-robust detector makes, made explicit. The cheaper alternative is to rely on the family itself containing spectra at several brightnesses, which Table 21.1 says it effectively does.

22.4 Simplex ACE: the family as a cone

The shipped man-made family score takes the whitened difference $\mathbf{z} = \mathbf{W}(\mathbf{x} - \mathbf{m})$ ($\mathbf{m}$ the ring mean) and projects it onto the **cone**

$$\left\{ \sum_j \alpha_j \, \mathbf{W}(\mathbf{t}_{kj} - \mathbf{m}) \ : \ \alpha_j \geq 0 \right\},$$

the set of non-negative combinations of the family's whitened differences, found by non-negative least squares (NNLS). The score is the explained share, $\|\text{projection}\|^2/\|\mathbf{z}\|^2$ (`family_scores` in `family_match.py`). It is called Simplex ACE (Ziemann and Theiler, as cited by the code and the LLD).

What does the cone mean? Take two members and non-negative weights with total $A = \alpha_1 + \alpha_2 > 0$. Then

$$\begin{aligned} \alpha_1(\mathbf{t}_1 - \mathbf{m}) + \alpha_2(\mathbf{t}_2 - \mathbf{m}) &= \alpha_1\mathbf{t}_1 + \alpha_2\mathbf{t}_2 - (\alpha_1 + \alpha_2)\mathbf{m} \\ &= A\Big(\frac{\alpha_1}{A}\mathbf{t}_1 + \frac{\alpha_2}{A}\mathbf{t}_2\Big) - A\mathbf{m} && \textit{take out } A \\ &= A(\bar{\mathbf{t}} - \mathbf{m}), \qquad \bar{\mathbf{t}} = \frac{\alpha_1}{A}\mathbf{t}_1 + \frac{\alpha_2}{A}\mathbf{t}_2. \end{aligned}$$

The weights α_1/A and α_2/A are non-negative and add to 1, so $\bar{\mathbf{t}}$ is a mixture (a "convex combination") of the members, and A plays the role of the fill. **Simplex ACE lets the deployed material be any mixture of the family's lab spectra, at any non-negative fill**, and scores the best such mixture with (signed) ACE. It can never score below the best single member, because each single member is one of the mixtures (the LLD's unit test checks exactly this).

Worked example 22.3: The cone in two whitened dimensions

Two members with whitened differences $\mathbf{v}_1 = (2,\ 0)$ and $\mathbf{v}_2 = (0,\ 2)$; the cone is the quarter-plane where both coordinates are ≥ 0.
Pixel $\mathbf{z} = (1,\ 0.5)$: it is inside the cone, so its projection is itself and the score is $1.25/1.25 = 1$. NNLS weights: $(1,\ 0.5) = 0.5\mathbf{v}_1 + 0.25\mathbf{v}_2$, so $A = 0.75$ and the mixture is $\frac{0.5}{0.75} = \frac{2}{3}$ of member 1 and $\frac{1}{3}$ of member 2. Single members: ACE with $\mathbf{v}_1$ is $2^2/(4 \times 1.25) = 0.8$, with $\mathbf{v}_2$ is $1^2/(4 \times 1.25) = 0.2$. The

cone beats both.
Pixel $\mathbf{z} = (1,\ -0.5)$: the nearest point of the cone is $(1,\ 0)$ (the weight on $\mathbf{v}_2$ would have to be negative, so it is set to 0). Score $1/1.25 = 0.8$, the same as member 1 alone.

The cone does not allow a brightness change *relative to* its mixture weights unless the family holds spectra at several brightnesses; Table 21.1 says the families effectively do.

Analogy: a paint shop

A paint shop's "blues" are a few base tins. Any colour you can mix from non-negative amounts of those tins is "a blue from this shop"; that set of mixable colours is the cone. Asking "could this sample have been mixed here?" is Simplex ACE. Removing nuisances is a different strategy: first ignore how much white or grey has been added to the sample (interference), then compare what is left with the bases.

Common confusions

Is the cone the same as the subspace? No. The subspace allows negative combinations ("metal minus a bit of other metal"), the cone does not. The cone is a stricter, more physical model.
Does projecting out $\hat{\mathbf{b}}$ remove the background? It removes the direction of the predicted background from the residual, including any part of the target along it.
Is NNLS's mass A the fill? It is a fill estimate for the best mixture, entangled with brightness like any fill estimate here.
Why not use both nuisance removal and the cone? Possible; each costs degrees of freedom and changes the null distribution, which then needs its own calibration.

Check yourself

1. In worked example 22.1, recompute with $\mathbf{r} = (2,\ 2,\ 2)$. What does the detector say, and why?
2. A roof at $g = 0.5$ fills $a = 0.3$. Which other fill gives the same target term at $g = 1$?
3. In worked example 22.3, find the score and NNLS weights for $\mathbf{z} = (2,\ 2)$.
4. For $\mathbf{z} = (-1,\ 0.5)$ in worked example 22.3, what is the projection onto the cone and the score?
5. Show that $0.3(\mathbf{t}_1 - \mathbf{m}) + 0.1(\mathbf{t}_2 - \mathbf{m}) = 0.4(\bar{\mathbf{t}} - \mathbf{m})$ and give $\bar{\mathbf{t}}$'s mixture weights.

Check yourself: worked answers

1. The mean is 2; $\mathbf{P}_1^{\perp}\mathbf{r} = (0,\ 0,\ 0)$, so $Q_0 = 0$ and there is nothing left to explain. The residual is pure offset: interference, not signal.
2. $ag = 0.15$, so $a = 0.15$ at $g = 1$.
3. $(2, 2) = 1\mathbf{v}_1 + 1\mathbf{v}_2$; inside the cone; score 1; weights $1, 1$; $A = 2$, mixture half and half.
4. The nearest point of the cone is $(0,\ 0.5)$ (the first weight clipped to 0). Explained energy 0.25, total 1.25, score 0.2.
5. $0.3\mathbf{t}_1 + 0.1\mathbf{t}_2 - 0.4\mathbf{m} = 0.4(0.75\mathbf{t}_1 + 0.25\mathbf{t}_2) - 0.4\mathbf{m} = 0.4(\bar{\mathbf{t}} - \mathbf{m})$ with weights $0.3/0.4 = 0.75$ and $0.1/0.4 = 0.25$.

Next: Chapter 23, why the simplest matcher fails on mixed pixels.

Chapter 23

Why SAM fails on mixed pixels

The one thing this chapter teaches

A mixed pixel lies in the plane of its target and background, and for small fills it stays within a few degrees of the background; SAM on raw pixels therefore names the background (or whatever library entry resembles it), while matching on the residual removes the problem at its source.

23.1 The problem

SAM (Chapter 3) works well on pure pixels. The low-level design measured it on planted man-made targets at 30% fill: it named the right family 6% of the time[M]. This chapter derives why, first on the toy spectra and then on the real example the note uses, galvanised metal over grass.

23.2 The geometry of a two-component mixture

Let $\mathbf{x} = a\mathbf{t} + (1-a)\mathbf{b}$ (no noise) and let θ_{bt} be the angle between $\mathbf{b}$ and $\mathbf{t}$. Both $\mathbf{t}$ and $\mathbf{b}$ start at the origin, so they span a plane, and $\mathbf{x}$, a combination of them, lies in that plane too. Use the trick of Chapter 3: coordinates in the plane with $\mathbf{b}$ along the first axis.

$$\begin{aligned}\mathbf{b} &= (\|\mathbf{b}\|,\ 0)\\ \mathbf{t} &= (\|\mathbf{t}\|\cos\theta_{bt},\ \|\mathbf{t}\|\sin\theta_{bt})\\ \mathbf{x} &= a\mathbf{t} + (1-a)\mathbf{b} = \big((1-a)\|\mathbf{b}\| + a\|\mathbf{t}\|\cos\theta_{bt},\ \ a\|\mathbf{t}\|\sin\theta_{bt}\big).\end{aligned}$$

The angle of $\mathbf{x}$ above the first axis, which is its angle to $\mathbf{b}$, has tangent equal to the second coordinate over the first:

Rule 23.1: SAM on a two-component mixture

$$\tan\angle(\mathbf{x},\mathbf{b}) = \frac{a\|\mathbf{t}\|\sin\theta_{bt}}{(1-a)\|\mathbf{b}\| + a\|\mathbf{t}\|\cos\theta_{bt}}, \qquad \angle(\mathbf{x},\mathbf{t}) = \theta_{bt} - \angle(\mathbf{x},\mathbf{b}).$$

For small a the denominator is about $\|\mathbf{b}\|$ and tan of a small angle is about the angle in radians (Chapter 3), so

$$\angle(\mathbf{x},\mathbf{b}) \approx a\,\frac{\|\mathbf{t}\|}{\|\mathbf{b}\|}\,\sin\theta_{bt} \quad \text{(radians)}.$$

The pixel turns away from its background in proportion to the fill, the target-to-background brightness ratio, and how different the two are. A dark target on a bright background barely moves it.

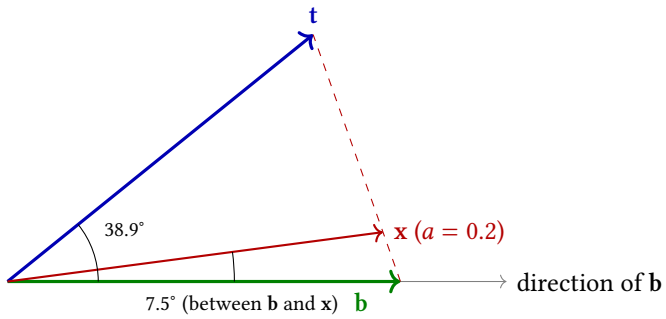


Figure 23.1: The toy metal **t** and grass **b** in their own plane. The mixing line (dashed) joins their tips. At 20% fill the pixel **x** is 7.5° from grass and 31.4° from metal.

Worked example 23.1: The toy: 20% metal over grass

From earlier chapters: $\|\mathbf{t}\| = \|\mathbf{g}\| = 0.5196$, $\theta_{bt} = 38.94^\circ$, $\cos\theta_{bt} = 0.7778$, $\sin\theta_{bt} = 0.6285$. With $a = 0.2$:

$$\begin{aligned} \text{numerator} &= 0.2 \times 0.5196 \times 0.6285 = 0.06532 \\ \text{denominator} &= 0.8 \times 0.5196 + 0.2 \times 0.5196 \times 0.7778 = 0.41568 + 0.08083 = 0.49651 \\ \tan\angle(\mathbf{x},\mathbf{b}) &= 0.06532/0.49651 = 0.13155 \\ \angle(\mathbf{x},\mathbf{b}) &= \arctan(0.13155) = 7.49^\circ \\ \angle(\mathbf{x},\mathbf{t}) &= 38.94 - 7.49 = 31.45^\circ. \end{aligned}$$

Check directly with rule 3.1: $\mathbf{x} = (0.14,\ 0.14,\ 0.46)$, $\mathbf{x}^\top\mathbf{g} = 0.014 + 0.014 + 0.23 = 0.258$, $\|\mathbf{x}\| = \sqrt{0.0196 + 0.0196 + 0.2116} = \sqrt{0.2508} = 0.5008$, $\cos = 0.258/(0.5008 \times 0.5196) = 0.258/0.26022 = 0.9915$, angle 7.49°. Agrees. The small-fill approximation gives $0.2 \times 1 \times 0.6285 = 0.1257$ rad $= 7.2^\circ$, close.

SAM, offered grass and metal, says grass by 7.5° against 31.4°. Only beyond $a = 0.5$ (where the two angles are equal, 19.47° each) does it say metal.

23.3 The real example: galvanised metal over grass and soil

The note's script takes galvanised sheet metal (USGS GDS334) as **t** and 0.7× cheatgrass +0.3× an alfisol soil as **b**, resampled to the 165-band grid, in reflectance. The quantities the formula needs:

- $\theta_{bt} = 23.44^\circ$ (`compute_numbers_output.txt`)[C]; $\cos\theta_{bt} = 0.91746$, $\sin\theta_{bt} = 0.39783$;
- $\|\mathbf{t}\| = 1.3067$ and $\|\mathbf{b}\| = 3.5957$ (`compute_step_by_step_output.txt`)[C], a ratio of 0.363. The metal is dark (mean reflectance 0.099) against a bright vegetated background (mean 0.265).

Worked example 23.2: Re-deriving the note's angles at 10% and 50% fill

$a = 0.1$:

$$\begin{aligned} \text{numerator} &= 0.1 \times 1.3067 \times 0.39783 = 0.05198 \\ \text{denominator} &= 0.9 \times 3.5957 + 0.1 \times 1.3067 \times 0.91746 = 3.23613 + 0.11988 = 3.35601 \\ \tan\angle(\mathbf{x},\mathbf{b}) &= 0.05198/3.35601 = 0.01549 \\ \angle(\mathbf{x},\mathbf{b}) &= 0.887^\circ \\ \angle(\mathbf{x},\mathbf{t}) &= 23.443 - 0.887 = 22.56^\circ. \end{aligned}$$

$a = 0.5$:

$$\text{numerator} = 0.5 \times 1.3067 \times 0.39783 = 0.25992$$

$$\text{denominator} = 0.5 \times 3.5957 + 0.5 \times 1.3067 \times 0.91746 = 1.79785 + 0.59942 = 2.39727$$
$$\tan \angle(\mathbf{x}, \mathbf{b}) = 0.25992/2.39727 = 0.10842$$
$$\angle(\mathbf{x}, \mathbf{b}) = 6.19^\circ, \qquad \angle(\mathbf{x}, \mathbf{t}) = 23.44 - 6.19 = 17.25^\circ.$$

The note's script, which computes the angles directly on the 165 bands, reports 22.56° and 17.25°[C]. The formula reproduces them.

fill a	0.1	0.2	0.3	0.5	0.85
angle to background	0.89°	1.91°	3.10°	6.19°	15.83°
angle to the true target	22.56°	21.53°	20.34°	17.25°	7.61°
nearest library entry	NPV, 2.87°	NPV, 2.62°	NPV, 2.79°	NPV, 2.50°	metal family

(Angles from the formula, `compute_step_by_step_output.txt`[C]; nearest entries searched over all 352 complete library spectra by the note's script[C]. NPV is non-photosynthetic vegetation: dry grass, straw.)

The background itself is not in the library (it is a 70/30 blend), but entries resembling it are. Dry vegetation sits 2.5° to 2.9° from the pixel for every fill up to 0.5, while the true target is 17° to 23° away. Some man-made entry becomes nearest only from $a = 0.775$, and the target's own family only from $a = 0.85$[C].

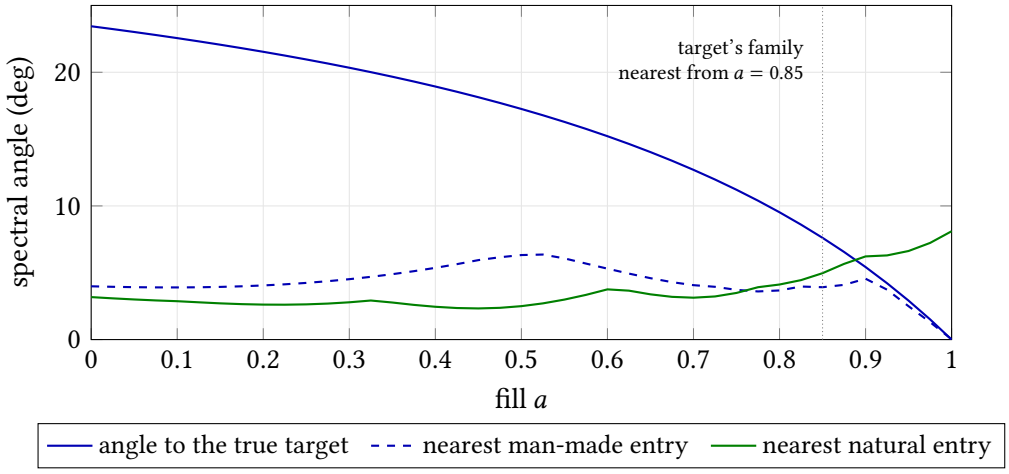


Figure 23.2: SAM on galvanised metal mixed into a grass/soil background, against all 352 complete library entries on the 165-band grid (the original note's figure, from `figdata/sam_mixing.dat`)[C].

23.4 The fix: match the residual

Subtract a background estimate first. From Chapter 16,

$$\mathbf{x} - \hat{\mathbf{b}} = a(\mathbf{t} - \hat{\mathbf{b}}) + (1 - a)\mathbf{e} + \mathbf{n},$$

which points along the target difference *whatever* the fill, up to the model's error. In the toy with a perfect background estimate, $\mathbf{x} - \mathbf{g} = (0.04,\ 0.04,\ -0.04) = 0.2(\mathbf{t} - \mathbf{g})$: angle 0° to $\mathbf{t} - \mathbf{g}$ at any fill (worked example 1.3). That is the case for matching on the residual, and for the ring-mean subtraction the shipped matcher already does. The fill no longer decides the direction; it only decides the length, and the length competes with the error $\mathbf{e}$ (Chapter 19).

Analogy: ink in a bucket

Drop a spoonful of blue ink into a bucket of water. The water turns faintly blue, but if you ask "which colour on this chart is the bucket closest to?", the answer is "clear water" (or "very pale grey"), not "blue ink". To see the ink you must compare the bucket with what it looked like before, and look at the difference.

Common confusions

Is SAM wrong? It answers "what does this pixel look like overall?" correctly. The question "is there a little of something else here?" is a different one.
Would a better library fix it? No. Even a library containing the exact background and the exact target would name the background for small fills, as the toy shows.
Why does the dark metal make it worse? The rotation is proportional to $\|\mathbf{t}\|/\|\mathbf{b}\|$, here 0.36: a dark target barely moves a bright pixel.
Does matching the residual remove all difficulty? It removes the geometric one. It leaves the error $\mathbf{e}$, leakage (Chapter 24) and the gain error (Chapter 25).

Check yourself

1. Using rule 23.1, compute the toy pixel's angle to grass at $a = 0.1$ (use $\|\mathbf{t}\| = \|\mathbf{b}\| = 0.5196$, cos = 0.7778, sin = 0.6285).
2. Use the small-fill approximation for the real example at $a = 0.1$. Compare with 0.887°.
3. At what fill are the toy pixel's angles to metal and grass equal?
4. Re-derive the real example's angle to the target at $a = 0.3$.
5. Why does subtracting the background change the picture?

Check yourself: worked answers

1. Numerator $0.1 \times 0.5196 \times 0.6285 = 0.03266$. Denominator $0.9 \times 0.5196 + 0.1 \times 0.5196 \times 0.7778 = 0.46764 + 0.04041 = 0.50805$. tan = 0.06428, angle 3.68°.
2. $0.1 \times 0.3634 \times 0.39783 = 0.01446$ rad = 0.828°, a little under the exact 0.887°.
3. At $a = 0.5$: with equal lengths, the mixture is the midpoint and bisects the angle, $38.94/2 = 19.47°$ each.
4. Numerator $0.3 \times 1.3067 \times 0.39783 = 0.15595$. Denominator $0.7 \times 3.5957 + 0.3 \times 1.3067 \times 0.91746 = 2.51699 + 0.35965 = 2.87664$. tan = 0.05421, angle 3.10°; to the target $23.44 - 3.10 = 20.34°$, as in the table.
5. The residual $a(\mathbf{t} - \hat{\mathbf{b}}) + \ldots$ points along the target difference at every fill; only its length depends on the fill.

Next: Part V, what goes wrong, starting with Chapter 24.

Part V

What goes wrong

Chapter 24

Leakage

The one thing this chapter teaches

Because a small target's blurred light reaches the visible neighbouring tokens, the background prediction partly copies the target; copying a fraction l multiplies the detection SNR by $(1-l)^2$ and pulls the fill estimate down to $a(1-l)/(1-la)$, and for a one-pixel target about 40% of its light sits in the visible tokens.

24.1 The problem: the painter can peek

Chapter 16 said the model predicts pixel p with p's token hidden. But Chapter 13 showed that a one-pixel target keeps under a tenth of its light in its own pixel; the rest spreads over a 5×5 neighbourhood, and much of that neighbourhood lies in tokens that are *visible* when p's token is hidden (Figure 16.1). The model sees a faint halo of the target around the hidden tile and, being a good painter, may reproduce some of it in its prediction. Whatever it reproduces is subtracted from the residual. That is **leakage**.

24.2 Scalar leakage: the model

Definition: proportional leakage

The prediction copies a fraction l (between 0 and 1) of the target's contribution:

$$\hat{\mathbf{b}}_p = \mathbf{b}_p + l\, a_p(\mathbf{t} - \mathbf{b}_p) - \mathbf{e}_p.$$

With $l = 0$ this is the leak-free $\hat{\mathbf{b}}_p = \mathbf{b}_p - \mathbf{e}_p$ of Chapter 16.

The residual. Substitute into $\mathbf{r}_p = \mathbf{x}_p - \hat{\mathbf{b}}_p$ with $\mathbf{x}_p = a\mathbf{t} + (1-a)\mathbf{b}_p + \mathbf{n}_p$ (drop the subscripts on a, $\mathbf{b}$):

$$\begin{aligned}
\mathbf{r} &= a\mathbf{t} + (1-a)\mathbf{b} + \mathbf{n} - \mathbf{b} - la(\mathbf{t} - \mathbf{b}) + \mathbf{e} \\
&= a(\mathbf{t} - \mathbf{b}) + \mathbf{n} - la(\mathbf{t} - \mathbf{b}) + \mathbf{e} && a\mathbf{t} + (1-a)\mathbf{b} - \mathbf{b} = a(\mathbf{t} - \mathbf{b}), \textit{ as in Chapter 12} \\
&= a(1-l)(\mathbf{t} - \mathbf{b}) + \mathbf{e} + \mathbf{n} && \textit{take out } a(\mathbf{t} - \mathbf{b})
\end{aligned}$$

The target difference the matcher uses.

$$\mathbf{d} = \mathbf{t} - \hat{\mathbf{b}} = \mathbf{t} - \mathbf{b} - la(\mathbf{t} - \mathbf{b}) + \mathbf{e} = (1 - la)(\mathbf{t} - \mathbf{b}) + \mathbf{e}.$$

24.3 Three consequences

(i) Detection loses SNR. The signal along $\mathbf{u} = \mathbf{t} - \mathbf{b}$ has amplitude $a(1-l)$ instead of a. SNR is proportional to the amplitude squared, so it is multiplied by $(1-l)^2$, and the smallest detectable fill is multiplied by $1/(1-l)$.

(ii) The fill estimate is biased down. With no noise and no model error ($\mathbf{e} = \mathbf{n} = \mathbf{0}$), $\mathbf{r} = a(1-l)\mathbf{u}$ and $\mathbf{d} = (1-la)\mathbf{u}$. By rule 18.1:

$$\hat{a} = \frac{\mathbf{d}^\top \mathbf{S}^{-1}\mathbf{r}}{\mathbf{d}^\top \mathbf{S}^{-1}\mathbf{d}} = \frac{(1-la)\,a(1-l)\,\mathbf{u}^\top \mathbf{S}^{-1}\mathbf{u}}{(1-la)^2\,\mathbf{u}^\top \mathbf{S}^{-1}\mathbf{u}} \qquad \textit{pull the numbers out of each product}$$

$$= \frac{a(1-l)}{1-la} \qquad \textit{cancel } (1-la) \textit{ and } \mathbf{u}^\top \mathbf{S}^{-1}\mathbf{u}$$

Since $1-l < 1-la$ whenever $a < 1$, this is below a.

(iii) The direction survives. $\mathbf{r}$ still points along $\mathbf{u}$; only its length shrank. Scale-invariant statistics (ACE, Simplex ACE, the family percentiles of Chapter 27) lose only through the lower SNR, not through a wrong direction.

Worked example 24.1: Thirty per cent leakage at 20% fill

$a = 0.2$, $l = 0.3$.

$$\text{SNR factor} = (1-0.3)^2 = 0.7^2 = 0.49$$

$$a_{\min}\ \text{factor} = 1/0.7 = 1.43$$

$$\hat{a} = \frac{0.2 \times 0.7}{1 - 0.3 \times 0.2} = \frac{0.14}{1-0.06} = \frac{0.14}{0.94} = 0.149.$$

Check with vectors, $\mathbf{u} = (2,\ 2,\ -2)$ and $\mathbf{S} = \mathbf{I}$: $\mathbf{r} = 0.14\mathbf{u} = (0.28,\ 0.28,\ -0.28)$, $\mathbf{d} = 0.94\mathbf{u} = (1.88,\ 1.88,\ -1.88)$. $\mathbf{d}^\top\mathbf{r} = 3 \times 1.88 \times 0.28 = 1.5792$, $\mathbf{d}^\top\mathbf{d} = 3 \times 1.88^2 = 10.6032$, $\hat{a} = 1.5792/10.6032 = 0.149$.

For $l = 0.1,\ 0.3,\ 0.5$ the SNR falls to $0.81,\ 0.49,\ 0.25$ of its leak-free value and $a_{\min}$ rises by $1.11,\ 1.43,\ 2.00$[C].

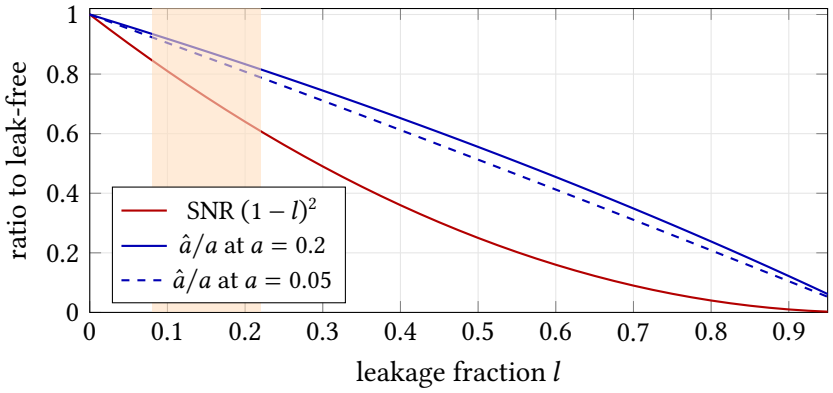


Figure 24.1: The cost of proportional leakage (the original note's figure, from `figdata/leakage.dat`). The shaded band is the estimate for a one-pixel target with 4×4-pixel tokens (worked example 24.2)[C].

24.4 Leakage as a matrix

A learned predictor need not copy every band by the same fraction. Describe its sensitivity by the **Jacobian** $\mathbf{J}_{pq}$: the matrix that says how much $\hat{\mathbf{b}}_p$ changes per unit change of the visible pixel $\mathbf{x}_q$ (one

matrix per visible pixel q; this $\mathbf{J}$ has nothing to do with the interference subspace of Chapter 22). If the target adds $a_q(\mathbf{t}-\mathbf{b}_q) \approx a_q(\mathbf{t}-\mathbf{b}_p)$ at each visible neighbour q, then to first order

$$\hat{\mathbf{b}}_p \approx \hat{\mathbf{b}}_p^{(0)} + a_p \mathbf{L}_p(\mathbf{t}-\mathbf{b}_p), \qquad \mathbf{L}_p = \sum_{q \in C(p)} \mathbf{J}_{pq} \frac{a_q}{a_p},$$

where $\hat{\mathbf{b}}_p^{(0)}$ is the prediction without the target, and the residual becomes $\mathbf{r}_p = a_p(\mathbf{I}-\mathbf{L}_p)(\mathbf{t}-\mathbf{b}_p)+\mathbf{e}_p+\mathbf{n}_p$. Scalar leakage is the special case $\mathbf{L}_p = l\,\mathbf{I}$.

Because p is hidden, $\mathbf{J}_{pp} = 0$: **leakage happens only through the fills a_q at visible neighbours**, that is, through the PSF halo and any part of the object larger than a token. If $\mathbf{L}_p$ is not a multiple of $\mathbf{I}$, leakage also turns the target's direction, and a matcher should compare against the "leaked library" $(\mathbf{I}-\mathbf{L}_p)(\mathbf{t}-\hat{\mathbf{b}}_p)$ rather than $\mathbf{t}-\hat{\mathbf{b}}_p$. Two families stay distinguishable only if they still differ after $(\mathbf{I}-\mathbf{L}_p)$ is applied.

24.5 The 40% halo, re-derived

How much of a one-pixel target's light lands in the four edge-neighbour tokens, which are visible when its own token is hidden? Use the 1-D shares of Chapter 13, now for tokens 4 pixels wide.

Step 1: one dimension, a pixel at the edge of its token. Put the token at $[0,4]$ and the point at the centre of its first pixel, $c = 0.5$. With $\sigma = 1.4$:

$$\begin{aligned}
\text{own token } [0,4] &: \quad \Phi\tfrac{4-0.5}{1.4}) - \Phi\tfrac{0-0.5}{1.4}) = \Phi(2.500) - \Phi(-0.357) = 0.9938 - 0.3605 = 0.6333 \\
\text{left token } [-4,0] &: \quad \Phi(-0.357) - \Phi\tfrac{-4-0.5}{1.4}) = 0.3605 - \Phi(-3.214) = 0.3605 - 0.0007 = 0.3598 \\
\text{right token } [4,8] &: \quad \Phi\tfrac{8-0.5}{1.4}) - \Phi(2.500) = 1.0000 - 0.9938 = 0.0062 \\
\text{side tokens together} &: \quad 0.3598 + 0.0062 = 0.3660
\end{aligned}$$

Step 2: one dimension, an inner pixel, $c = 1.5$:

$$\begin{aligned}
\text{own token} &: \quad \Phi\tfrac{2.5}{1.4}) - \Phi\tfrac{-1.5}{1.4}) = \Phi(1.786) - \Phi(-1.071) = 0.9629 - 0.1420 = 0.8209 \\
\text{left token} &: \quad 0.1420 - \Phi(-3.93) = 0.1420 - 0.0000 = 0.1419 \\
\text{right token} &: \quad 1.0000 - 0.9629 = 0.0371 \\
\text{side tokens together} &: \quad 0.1419 + 0.0371 = 0.1790
\end{aligned}$$

By symmetry, $c = 2.5$ behaves like 1.5 and $c = 3.5$ like 0.5.

Step 3: average over the four positions along a row.

$$\begin{aligned}
\text{own} &= (0.6333 + 0.8209 + 0.8209 + 0.6333)/4 = 2.9084/4 = 0.7271 \\
\text{side} &= (0.3660 + 0.1790 + 0.1790 + 0.3660)/4 = 1.0900/4 = 0.2725
\end{aligned}$$

Step 4: two dimensions. The 2-D share is a product of a row share and a column share (Chapter 13). Light lands in the *left or right* edge-neighbour token when it is in a side token horizontally and in the own token vertically; in the *upper or lower* token, the other way round. So, for a pixel at position (c_x, c_y),

$$\text{edge tokens} = \text{own}(c_x)\,\text{side}(c_y) + \text{own}(c_y)\,\text{side}(c_x), \qquad \text{own token} = \text{own}(c_x)\,\text{own}(c_y).$$

Averaging over the 16 positions of the target in its token, the horizontal and vertical positions vary independently, so the average of each product is the product of the averages:

$$\begin{aligned}
\text{own token} &= 0.7271 \times 0.7271 = 0.529 \\
\text{edge-neighbour tokens} &= 2 \times 0.7271 \times 0.2725 = 0.396.
\end{aligned}$$

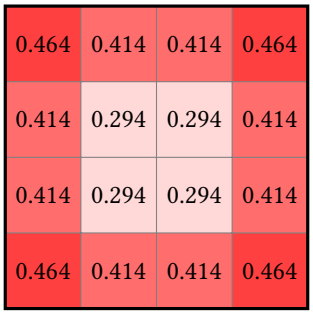


Figure 24.2: Hand-computed halo exposure. Corners leak most ($2 \times 0.6333 \times 0.3660 = 0.464$), the four central pixels least ($2 \times 0.8209 \times 0.1790 = 0.294$); the average of the 16 is 0.396.

What the project measured or computed

The note's script does the same calculation for a target that *fills* its pixel (each pixel split into 5×5 sub-points) and finds 52% in the own token and 40% (range 30 to 47%) in the four edge-neighbour tokens[C]. The hand version above, with a point at each pixel centre, gives 53% and 40% (range 29 to 46%) (`compute_step_by_step_output.txt`). With 8×8-pixel cells (FT08's 2×2-token blocks) the visible share falls to 24% on average but ranges from 1.6% to 46%: a target near a cell edge leaks as much as before[C].

About 40% of a one-pixel target's light sits in tokens the model can see while predicting it.

24.6 From halo exposure to a leakage fraction

Exposure is not leakage: what matters is how much of that visible halo the model copies into its prediction. Two simple linear predictors bracket it.

Worked example 24.2: Leakage of two linear predictors

If the prediction were the mean of the 16 pixels bordering the hidden token, its leakage, the target's average fill on those 16 pixels divided by its fill at p, averaged over the 16 positions, is $l = 0.218$ (range 0.19 to 0.23)[C]. If it were the mean of all 64 pixels of the four neighbour tokens, $l = 0.084$ (0.06 to 0.10)[C]. The SNR cost by consequence (i):

$$l = 0.084: \quad (1 - 0.084)^2 = 0.916^2 = 0.839, \text{ a 16\% loss}$$
$$l = 0.218: \quad (1 - 0.218)^2 = 0.782^2 = 0.612, \text{ a 39\% loss}$$

A learned predictor sits somewhere in or near that range: it costs, by this estimate, 16 to 39% of the SNR. There is direct evidence that Heavy-type models do leak. FT06 trained a head to predict the expected error scale at hidden pixels; on targets that predicted scale exceeds its own 3 to 8 px surroundings by factors of 1.05/1.12/1.04 (epoch 1) and 1.07/1.19/1.13 (epoch 3) on sets 1, 2, 4 (`FT06-uncertainty/README.md`)[M]. A hidden target the context could not see would give a ratio of exactly 1; the excess shows the context does see it.

24.7 When do wider masks pay?

Hiding bigger blocks (8-pixel cells instead of 4) reduces leakage but makes the background harder to predict. Which wins? Along a fixed target direction the SNR is $a^2(1-l)^2\|\mathbf{t}-\mathbf{b}\|^2/\sigma_e^2$, with σ_e^2 the background-

error variance in that direction. Write it for cell sizes c_1 and c_2 and divide:

$$\frac{\mathrm{SNR}_{c_2}}{\mathrm{SNR}_{c_1}} = \left(\frac{1-l_{c_2}}{1-l_{c_1}}\right)^2 \times \frac{\sigma^2_{e,c_1}}{\sigma^2_{e,c_2}} \qquad a \text{ and } \|\mathbf{t}-\mathbf{b}\| \text{ cancel}$$

This exceeds 1 exactly when

$$\left(\frac{1-l_{c_2}}{1-l_{c_1}}\right)^2 > \frac{\sigma^2_{e,c_2}}{\sigma^2_{e,c_1}}.$$

Worked example 24.3: Should FT08's 8-pixel blocks have helped?

Take $l_4 = 0.22$ and scale leakage with halo exposure[A]: $l_8 \approx 0.22 \times 0.24/0.40 = 0.22 \times 0.6 = 0.132$. Then

$$\left(\frac{1-0.132}{1-0.22}\right)^2 = \left(\frac{0.868}{0.78}\right)^2 = 1.113^2 = 1.24.$$

FT08 would gain if hiding 8-pixel blocks raised the background-error variance by less than 24%. FT08 measured a resolved loss, pAUC −0.0058 [−0.0106, −0.0010] (`FT07-gated/README.md`)[M], and its README attributes it to more background residual ("normal texture inside a block is rebuilt less exactly"). That is consistent with the inequality failing, but $\sigma_{e,8}/\sigma_{e,4}$ was not measured directly; it is one of the note's cheap open experiments.

24.8 A part of the residual that cannot leak

The reduced-attention stem's last layer is linear: each normalised output band is $\hat{z}_n = \mathbf{u}(\lambda_n)^\top \boldsymbol{\zeta} + c(\lambda_n)$, where $\boldsymbol{\zeta}$ is a D-dimensional code the network computes and $\mathbf{u}(\lambda_n)$, $c(\lambda_n)$ are fixed (`WavelengthLinearDecoder` in `app/foundation_models/components/spectral_reduced.py`). In reflectance, $\hat{\mathbf{b}} = \boldsymbol{\mu} + \mathbf{D}(\mathbf{U}\boldsymbol{\zeta} + \mathbf{c})$: every possible prediction is a fixed vector plus something in the span of the columns of $\mathbf{DU}$. Let $\mathbf{Q}$ project onto the directions perpendicular to that span. Two predictions differ by $\mathbf{DU}(\boldsymbol{\zeta} - \boldsymbol{\zeta}')$, which $\mathbf{Q}$ sends to zero, so $\mathbf{Q}$ removes any leaked term, and

$$\mathbf{Q}\mathbf{r}_p = \mathbf{Q}(\mathbf{x}_p - \boldsymbol{\mu} - \mathbf{D}\mathbf{c})$$

whatever the network does. $D = 32$ for FT04 and $D = 64$ for Heavy (`compressed_channels` in the configs; checked by the note).

Worked example 24.4: A one-dimensional decoder in three bands

Suppose the decoder can only output $\hat{\mathbf{b}} = \zeta(1,\ 1,\ 0)$ for some number ζ (take $\boldsymbol{\mu} = \mathbf{c} = \mathbf{0}$, $\mathbf{D} = \mathbf{I}$). The projection away from $(1, 1, 0)$ is

$$\mathbf{Q} = \mathbf{I} - \tfrac{1}{2}\begin{pmatrix} 1 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix} = \begin{pmatrix} 0.5 & -0.5 & 0 \\ -0.5 & 0.5 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

For any ζ, $\mathbf{Q}\hat{\mathbf{b}} = \zeta(0.5 - 0.5,\ -0.5 + 0.5,\ 0) = \mathbf{0}$. So for a pixel $\mathbf{x} = (0.3,\ 0.1,\ 0.2)$, $\mathbf{Qr} = \mathbf{Qx} = (0.15 - 0.05,\ -0.15 + 0.05,\ 0.2) = (0.1,\ -0.1,\ 0.2)$, whatever the network predicted.

So the residual splits into a part where the model both suppresses the background and can leak (the span of $\mathbf{DU}$) and a part where it does neither and the "learned" residual is the raw pixel less a fixed offset. FT04's measured floor, the residual energy outside the span, is 2 to 10% on background and on targets (`decoder-capacity/README.md`, table 1)[M]: the leak-proof part exists but carries little target. Heavy's floor ($D = 64$) has not been measured.

24.9 The classical ring leaks too

The ring estimate is not leak-free either. Under `psf_weights`, the mean weight on the 2 to 3 px ring that `contrast_z` uses is 0.32 for a 1×1 object, 0.20 for 2×2 and 0.12 for 4×4[C], so ring-based contrast sees $(1 - l_{\text{ring}})$ of the signal with l_{ring} of that size. The shipped matcher's 13×13-minus-5×5 ring starts 3 px out, so it leaks less. Learned and ring backgrounds trade leakage against how well they predict $\mathbf{b}_p$.

Analogy: the painter sees the splashes

Return to the countryside painter of Chapter 16. The tile she must repaint is covered, but the crashed aeroplane splashed debris onto the neighbouring tiles, which she can see. A careful painter continues the patterns she sees into the covered tile, so she paints in a little of the debris. The difference between photograph and painting then shows less aeroplane than was really there. She has not cheated; she has done her job too well.

Common confusions

Is leakage a bug in the model? No; it is what a good interpolator does with a blurred target. It can be reduced (wider masks, leak-free subspaces) at a cost.
Is 40% the leakage fraction? No. 40% is the share of the target's light that is *visible*. The leakage fraction l depends on how much of it the predictor copies: 0.08 to 0.22 for the two linear predictors.
Does leakage change which family is named? Scalar leakage only shrinks the residual. Matrix leakage ($\mathbf{L}_p$ not a multiple of $\mathbf{I}$) can also turn it, which is why a matcher may need the leaked library.
Why not hide a bigger area always? The background error grows; worked example 24.3 gives the break-even condition.

Check yourself

1. With $l = 0.5$ and $a = 0.2$, compute the SNR factor and the noise-free $\hat{a}$.
2. In Step 1, what share of the edge pixel's light lands in its own token in 2-D if it is at a token corner?
3. Compute the edge-token share for a pixel on the middle of a token edge, $(c_x, c_y) = (0.5, 1.5)$.
4. If $l_4 = 0.22$ and $l_8 = 0.15$, by how much may the background-error variance grow for 8-pixel cells to still pay?
5. Why is $\mathbf{J}_{pp} = 0$?

Check yourself: worked answers

1. $(1 - 0.5)^2 = 0.25$. $\hat{a} = 0.2 \times 0.5/(1 - 0.1) = 0.1/0.9 = 0.111$.
2. $0.6333 \times 0.6333 = 0.401$.
3. $0.6333 \times 0.1790 + 0.8209 \times 0.3660 = 0.1134 + 0.3004 = 0.414$.
4. $\big((1 - 0.15)/(1 - 0.22)\big)^2 = (0.85/0.78)^2 = 1.0897^2 = 1.188$: by less than about 19%.
5. Pixel p is hidden when it is predicted, so its own value cannot influence its prediction.

Next: Chapter 25, errors that come from the atmosphere.

Chapter 25

Atmospheric gain

The one thing this chapter teaches

An uneven per-band gain from imperfect atmospheric correction rotates the target direction by at most $\arccos\left(2\sqrt{\kappa}/(1+\kappa)\right)$, where κ is the largest gain over the smallest; at the gain errors the project measures that is 3° to 5°, as large as the gaps between families.

25.1 The problem: reflectance is itself an estimate

The satellite measures radiance at the top of the atmosphere. Reflectance at the ground is computed from it by an atmospheric-correction processor, which has to guess the haze, water vapour and illumination. Its mistakes are smooth across wavelength: a little too high in the blue, a little too low in the short-wave infrared. Model the corrected reflectance as

$$\mathbf{x}^{\text{obs}} = \mathbf{G}\mathbf{x} + \mathbf{o}, \qquad \mathbf{G} = \operatorname{diag}\left(g(\lambda_1), \dots, g(\lambda_N)\right),$$

a per-band **gain** g (near 1) and an **offset** $\mathbf{o}$ (near 0), both smooth in wavelength.

25.2 The offset cancels; the gain does not

Assume the learned background responds to such errors the same way the pixel does ("approximately equivariant", plausible because it was trained on many scenes with different residual corrections)[A]: $\hat{\mathbf{b}}(\mathbf{G}\mathbf{x} + \mathbf{o}) \approx \mathbf{G}\hat{\mathbf{b}}(\mathbf{x}) + \mathbf{o}$. Then

$$\begin{aligned}\mathbf{r}^{\text{obs}} &= \mathbf{x}^{\text{obs}} - \hat{\mathbf{b}}^{\text{obs}} \approx (\mathbf{G}\mathbf{x} + \mathbf{o}) - (\mathbf{G}\hat{\mathbf{b}} + \mathbf{o}) = \mathbf{G}(\mathbf{x} - \hat{\mathbf{b}}) = \mathbf{G}\mathbf{r} \\ &= a\,\mathbf{G}(\mathbf{t} - \mathbf{b}) + \mathbf{G}\mathbf{e} + \mathbf{G}\mathbf{n}.\end{aligned}$$

The offset cancels in the residual (it does not cancel for SAM on raw pixels: Chapter 3 showed a 0.1 offset turning grass by 9.45°). The gain stays and multiplies each band of the target difference by a slightly different number, which turns its direction.

25.3 How far can a gain turn a vector? Two bands first

The angle between $\mathbf{G}\mathbf{d}$ and $\mathbf{d}$ does not change if all gains are scaled together, so set the smallest gain to 1 and the largest to $\kappa = g_{\max}/g_{\min}$. In two bands, $\mathbf{G} = \operatorname{diag}(1,\ \kappa)$ and $\mathbf{d} = (u,\ v)$ with $u, v > 0$ (opposite signs can only reduce the turn). Then $\mathbf{G}\mathbf{d} = (u,\ \kappa v)$ and

$$\cos\angle(\mathbf{G}\mathbf{d}, \mathbf{d}) = \frac{u^2 + \kappa v^2}{\sqrt{u^2 + v^2}\,\sqrt{u^2 + \kappa^2 v^2}}.$$

Which $\mathbf{d}$ is turned the most? Square, divide top and bottom by v^4, and write $s = u^2/v^2$:

$$\cos^2 = \frac{(s+\kappa)^2}{(s+1)(s+\kappa^2)}.$$

Minimise over s. The logarithm has the same minimiser (rule 10.1) and is easier to differentiate:

$$\ln\cos^2 = 2\ln(s+\kappa) - \ln(s+1) - \ln(s+\kappa^2)$$
$$\frac{d}{ds}: \quad \frac{2}{s+\kappa} - \frac{1}{s+1} - \frac{1}{s+\kappa^2} = 0.$$

Multiply through by $(s+\kappa)(s+1)(s+\kappa^2)$:

$$2(s+1)(s+\kappa^2) - (s+\kappa)(s+\kappa^2) - (s+\kappa)(s+1) = 0.$$

Expand each product:

$$2(s+1)(s+\kappa^2) = 2s^2 + 2(1+\kappa^2)s + 2\kappa^2$$
$$(s+\kappa)(s+\kappa^2) = s^2 + (\kappa+\kappa^2)s + \kappa^3$$
$$(s+\kappa)(s+1) = s^2 + (1+\kappa)s + \kappa$$

Collect by powers of s. The s^2 terms: $2-1-1 = 0$. The s terms: $2+2\kappa^2-\kappa-\kappa^2-1-\kappa = 1-2\kappa+\kappa^2 = (\kappa-1)^2$. The constants: $2\kappa^2 - \kappa^3 - \kappa = -\kappa(\kappa^2 - 2\kappa + 1) = -\kappa(\kappa-1)^2$. So

$$(\kappa-1)^2\, s - \kappa(\kappa-1)^2 = 0$$
$$s = \kappa \qquad \textit{divide by } (\kappa-1)^2\textit{, non-zero when } \kappa \neq 1$$

The worst direction has $u/v = \sqrt{\kappa}$. Substitute $s = \kappa$:

$$\cos^2 = \frac{(2\kappa)^2}{(\kappa+1)(\kappa+\kappa^2)} = \frac{4\kappa^2}{(\kappa+1)\,\kappa\,(1+\kappa)} = \frac{4\kappa}{(1+\kappa)^2}$$
$$\cos = \frac{2\sqrt{\kappa}}{1+\kappa}.$$

Rule 25.1: The largest turn from a gain (Kantorovich)

For any positive per-band gain with $\kappa = g_{\max}/g_{\min}$ and any non-zero $\mathbf{d}$, in any number of bands,

$$\cos\angle(\mathbf{Gd}, \mathbf{d}) \geq \frac{2\sqrt{\kappa}}{1+\kappa}, \qquad \sin^2\angle(\mathbf{Gd}, \mathbf{d}) \leq \left(\frac{\kappa-1}{\kappa+1}\right)^2.$$

The two-band derivation above is the heart of it: with more bands the worst case puts all of $\mathbf{d}$'s weight on the band with the smallest gain and the band with the largest, which is the two-band problem. (The general proof is Kantorovich's inequality, cited in the original note.)

Worked example 25.1: The 2.7° bound at $\kappa = 1.1$

$$\sqrt{1.1} = 1.04881$$
$$2\sqrt{1.1} = 2.09762$$
$$\cos \geq 2.09762/2.1 = 0.998866$$
$$1 - \cos = 0.001134$$
$$\theta \approx \sqrt{2 \times 0.001134} = \sqrt{0.002268} = 0.04763 \text{ rad} \qquad \textit{small angle: } \cos\theta \approx 1 - \theta^2/2$$
$$= 0.04763 \times 180/\pi = 2.73^\circ.$$

Check with the worst vector $\mathbf{d} = (\sqrt{1.1},\ 1) = (1.04881,\ 1)$: $\mathbf{Gd} = (1.04881,\ 1.1)$; $\mathbf{d}^\top\mathbf{Gd} = 1.1 + 1.1 = 2.2$; $\|\mathbf{d}\|^2 = 1.1 + 1 = 2.1$; $\|\mathbf{Gd}\|^2 = 1.1 + 1.21 = 2.31$; $\cos = 2.2/\sqrt{2.1 \times 2.31} = 2.2/\sqrt{4.851} = 2.2/2.20250 = 0.998866$. The same. And the sine form: $((1.1-1)/(1.1+1))^2 = (0.1/2.1)^2 = 0.00227 = \sin^2(2.73^\circ)$.

For $\kappa = 1.02,\ 1.05,\ 1.1,\ 1.2$ the worst-case angle is $0.57^\circ,\ 1.40^\circ,\ 2.73^\circ,\ 5.22^\circ$[C]. Real gains are not worst-case: a random smooth ±5% gain (a cubic curve squashed into $[0.95,\ 1.05]$, so κ up to $1.05/0.95 = 1.105$) applied to 200 library-minus-background differences gave a median of 0.61° and a 95th percentile of 1.29°[C], well inside the bound. One caution: the bound is for ordinary angles. In whitened coordinates the quiet directions are stretched, and the angle can be larger.

25.4 How big are the gains we actually have?

The atmospheric-correction work measured the mean absolute difference (MAE) between our processor's land reflectance and the vendor's L2 product: 0.0146 for EnMAP (against PACO), 0.0241 for PRISMA (against ASI L2C) and 0.0081 for AVIRIS-NG (against JPL); harmonisation reduced EnMAP and PRISMA to 0.0094 and 0.0128 (`research/experiments/atm-correction/README.md`)[M]. Against a median reflectance of 0.155:

$$0.015/0.155 = 0.097,\ \text{about } 10\%.$$

Read as a pure gain error[A], a 10% error spread unevenly across bands is $\kappa \approx 1.1$ to 1.2, a worst-case rotation of 3° to 5°.

Compare with Table 21.2: metal and roofing are 3.7° apart, metal and plastic 2.0°, pavement and building 0.5°. **An atmospheric-correction gain error at the level we measure is enough, by itself, to move a target across a family boundary.** These MAEs are one processor against another, not against ground truth; since the models were trained on vendor L2, they are the relevant error for deploying from L1.

For contrast, Chapter 14 found that band-centre errors of 1 to 2 nm cost only 0.1° to 0.2° typically: an order of magnitude below the gain error.

Analogy: tinted sunglasses

Grey sunglasses darken every colour equally; colours keep their relationships (a uniform gain, $\kappa = 1$, no rotation). Amber sunglasses cut blue more than red, and every colour shifts towards orange (an uneven gain, $\kappa > 1$). Two shades of blue-green that you could tell apart without glasses may become hard to tell apart with them. The amount by which the glasses can shift a colour depends on how uneven the tint is, not on how dark it is.

Common confusions

Is κ the size of the gain error? It is the ratio of the largest gain to the smallest. A uniform 20% error ($\kappa = 1$) turns nothing; ±5% ($\kappa = 1.105$) can turn by up to 2.87°.
Does the offset really cancel? In the residual, under the equivariance assumption. On raw pixels, no.
Is the worst case likely? No; the random smooth gains gave a median of 0.61°. But a family boundary 0.5° to 4° away does not need the worst case.
Why compare with vendor products, not ground truth? Ground truth reflectance is rarely available; and the models learned from vendor L2, so departures from it are what the models will see.

Check yourself

1. Compute the worst-case angle for $\kappa = 1.05$, step by step.
2. For $\kappa = 1.1$, compute the angle between $\mathbf{Gd}$ and $\mathbf{d}$ for $\mathbf{d} = (1,\ 1)$. Compare with the worst case.
3. For $\mathbf{d} = (1,\ 0)$, what is the angle? Why?
4. An MAE of 0.0241 against reflectance 0.155 is what percentage?
5. Why does an offset error hit SAM on raw pixels but not the residual?

Check yourself: worked answers

1. $\sqrt{1.05} = 1.02470$; $2 \times 1.02470 = 2.04939$; $/2.05 = 0.999703$; $1 - \cos = 0.000297$; $\theta \approx \sqrt{0.000594} = 0.02437$ rad $= 1.40^\circ$.
2. $\mathbf{Gd} = (1,\ 1.1)$; dot $1+1.1 = 2.1$; lengths $\sqrt{2} = 1.41421$ and $\sqrt{2.21} = 1.48661$; $\cos = 2.1/2.10238 = 0.998868$; about 2.73°, almost the worst case (the worst has $u/v = 1.049$, close to 1).
3. 0°: $\mathbf{G}(1, 0) = (1, 0)$. A vector in one band is only scaled.
4. $0.0241/0.155 = 0.155$, about 16%.
5. The residual is a difference of two things that both carry the offset, so it cancels; a raw pixel carries it alone.

Next: Chapter 26, why the clean theory over-promises.

Chapter 26

Why the Gaussian theory is too optimistic

The one thing this chapter teaches

The Gaussian model predicts that bench targets should be caught three times out of four at a 0.1% false-alarm rate; the bench catches one in six, because real residuals have heavy tails, are noisiest exactly where targets are, share the target's directions, partly contain the target, and because the deployed material is not the library spectrum.

26.1 The problem: theory and measurement disagree

Chapter 19 ended with a clash. Gaussian theory, fed with measured error levels and measured target strengths, predicts $P_D \geq 0.75$ at $P_{\mathrm{FA}} = 10^{-3}$ on every PRISMA bench scene. Heavy (FTH3 epoch 5) catches 16.0% of objects at the top 0.1% and 49.4% at the top 1%[M]. This chapter lists the reasons, puts a number on each where the project has one, and then collects every error source in one table.

26.2 Reason 1: heavy tails

The Gaussian curve's tails fall off extremely fast (Chapter 10). Real residual statistics do not: RM is "chi-squared-like, heavy right tail" (its docstring). A heavier-tailed null needs a higher threshold for the same false-alarm rate, and a higher threshold needs more signal.

Definition: Student-t distribution

The Student-t distribution with ν (nu) degrees of freedom is a bell curve with heavier tails than the Gaussian; the smaller ν, the heavier. Its variance is $\nu/(\nu-2)$ (for $\nu > 2$), so dividing a t_ν variable by $\sqrt{\nu/(\nu-2)}$ gives unit variance, comparable with $\mathcal{N}(0,1)$.

Worked example 26.1: The threshold under a t_3 null

The t_3 value exceeded with probability 10^{-3} is 10.215. Scale to unit variance: $\sqrt{(3-2)/3} = \sqrt{1/3} = 0.5774$, so the threshold is $10.215 \times 0.5774 = 5.90$, against the Gaussian 3.09. Required SNR scales with the threshold squared:

$$(5.90/3.09)^2 = 1.909^2 = 3.64.$$

For t_5: $5.893 \times \sqrt{3/5} = 5.893 \times 0.7746 = 4.57$, and $(4.57/3.09)^2 = 2.18$[C].

A t_3 null costs a factor 3.6 in SNR, $\sqrt{3.64} = 1.91$ in fill.

26.3 Reason 2: the error is largest where targets are

The Gaussian model uses one covariance for the whole scene. But FT06's predicted error scale on target pixels sits at the 77th to 90th percentile of the scene (`FT06-uncertainty/b_at_targets_output.txt`)[M]: targets tend to be where the model is least sure (edges, built-up areas, mixed land cover). A global S understates the local clutter there, so the real SNR at targets is lower than the scene-wide calculation says. (Per-pixel error scales are one remedy; FT06 is that experiment.)

26.4 Reason 3: real objects hide in the error's own directions

Real objects have a foreignness of $\varphi \approx 0.006$ (Chapter 19): 99.4% of their difference from the background lies in the directions the model's error already occupies. Figure 19.1 shows $a_{\min}$ rising steeply towards the left edge, and real objects sit at that edge. The two-level model already includes this, but its ε and k are assumptions[A]; if the quiet directions are less quiet than assumed, the outside part of worked example 19.3 shrinks and $a_{\min}$ grows.

26.5 Reason 4: leakage and spread

The model partly reconstructs the target ($l \approx 0.08$ to 0.22, Chapter 24), and the PSF spreads a small target so that its own pixel keeps under a tenth of its light (Chapter 13).

Worked example 26.2: Putting two measured factors together

Tails (t_3): fill $\times\sqrt{3.64} = 1.91$. Leakage at $l = 0.218$: fill $\times 1/(1 - 0.218) = 1/0.782 = 1.28$. Together: $1.91 \times 1.28 = 2.44$. The set-3 prediction of Chapter 19 had $a/a_{\min} = 0.057/0.047 = 1.21$; dividing by 2.44 gives 0.50, so $\mu = 3.09 \times 0.50 = 1.54$ and $P_D = Q(3.09 - 1.54) = Q(1.55) = 0.06$, a long way down from 0.74, before the local-clutter and alignment effects are counted. This combination is this book's illustration, not a measurement.

26.6 The largest error of all: the library is not the material

Write the deployed material as $\mathbf{t}' = \mathbf{t} + \boldsymbol{\delta}_t$: weathering, dust, mixtures within the object, viewing angle. Chapter 18 showed its effect on the fill and the cosine. Its size is best read from the one real test with field spectra. On the Gudalur sheets, Simplex ACE names 27% of answered pixels correctly with the library alone and about 78% with the sheets' own field spectra added; the absorption-feature check that was right 62% of the time on planted targets was right 0% of the time on the real sheets (LLD section 5)[M]. **Of all error sources this is the largest, and it is the one synthetic tests cannot measure.**

26.7 The decoder's output floor

From Chapter 24, the part of the background the model can never predict lies outside the span of its decoder. For FT04 ($D = 32$) it is 2 to 10% of residual energy on background and on targets; mean absolute floor 0.013 to 0.033 (background) and 0.022 to 0.043 (targets), against actual residuals of 0.048 to 0.122 and 0.149 to 0.204 (`decoder-capacity/README.md`)[M]. The decoder-capacity study's conclusion follows: over 90% of the error is in *which* code the decoder predicts, not in what it *can* render. For matching, the floor is a nuisance subspace of known range: project it out and it cannot be mistaken for a family.

26.8 All the error sources in one table

Each source enters the residual either as a turn of the target direction (which costs identification) or as extra residual energy (which costs detection). The yardstick is the separation between families from Chapter 21: a few degrees, a few per cent of the energy.

Table 26.1: Error sources against the identification yardstick (family separation 0.5° to 6° raw).

source	how it enters	size
sensor noise	residual energy	0.005 to 0.008 normalised[C]; negligible
background-prediction error	residual energy; sets $\mathbf{S}$	0.05 to 0.14 mean abs[M]
leakage	SNR $\times(1-l)^2$; direction via $\mathbf{I}-\mathbf{L}$	$l \approx 0.08$ to 0.22 (estimate)[C]
atmospheric gain	turns $\mathbf{t}-\mathbf{b}$	$\leq 2.7°$ at $\kappa = 1.1$; $\kappa \approx 1.1$ to 1.2[A]
atmospheric offset	cancels in the residual	(hits SAM on raw pixels)
band centre / width	turns $\mathbf{t}$	0.1° to 0.2° typical at 1 to 2 nm[C]
library-to-image gap	turn and bias	dominant: 27% vs 78% right at Gudalur[M]
decoder floor	fixed nuisance subspace	2 to 10% of residual energy (FT04)[M]

The practical conclusion of the original note: **detectability must be calibrated empirically**, by implanting targets at controlled fill (bench-v2), and per-scene percentile thresholds are a necessity rather than a convenience. Part VI is about doing exactly that.

Analogy: the brochure fuel economy

A car's advertised fuel economy is measured on a rolling road at a steady speed, in mild weather, with nobody in the back. Real driving is worse, and for reasons you can list: hills, cold mornings, a loaded boot, stop-start traffic. Each reason has a size, and together they explain the gap. The brochure figure is still useful as a best case; it is not a forecast. The Gaussian $a_{\min}$ is the brochure figure.

Common confusions

Is the Gaussian theory useless then? No. It ranks designs (which subspace, which whitening, how much leakage costs) and gives a best case. It does not predict absolute detection rates.
Are the tail factors measured? The t_3 and t_5 nulls are illustrations of plausible tails, computed exactly. The heaviness of the real tail is measured only indirectly ("chi-squared-like, heavy right tail").
Which error matters most for naming? The library-to-image gap, then the atmospheric gain; both are comparable with the separation between families.
Can more training fix the domain gap? Not by itself; the library is the reference. Field spectra of the deployed materials, as at Gudalur, fix it locally.

Check yourself

1. Under a unit-variance t_5 null, by what factor does the fill needed for detection grow?
2. From Table 26.1, which two sources can by themselves move a target across a family boundary?
3. Redo worked example 26.2 with t_5 tails and $l = 0.084$.
4. Why does a global covariance overstate the SNR at targets?
5. What were the Gudalur naming rates with the library alone and with field spectra?

Check yourself: worked answers

1. $\sqrt{2.18} = 1.48$.
2. The atmospheric gain (3° to 5° worst case at the measured level) and the library-to-image gap.
3. $\sqrt{2.18} = 1.476$; $1/(1-0.084) = 1.092$; product 1.61. $a/a_{\min} = 1.21/1.61 = 0.75$; $\mu = 3.09 \times 0.75 = 2.32$; $P_D = Q(3.09 - 2.32) = Q(0.77) = 0.22$.
4. Targets sit where the model's error is larger than average (77th to 90th percentile), so the local clutter exceeds the global one.
5. 27% and about 78% of answered pixels.

Next: Part VI, deciding and measuring, starting with Chapter 27.

Part VI

Deciding and measuring

Chapter 27

Abstention and per-scene calibration

The one thing this chapter teaches

Instead of trusting likelihoods it cannot compute, the product turns each family score into a percentile among this scene's ordinary pixels and names a family only above $p^* = 1 - \alpha/F$; that bounds false names on ordinary pixels by about α, but not on anomaly-flagged false alarms, which need their own null.

27.1 The problem: when to say "I don't know"

Most anomalies are in no library: field edges, glint, shorelines, haze. A matcher that always names the closest family will put a confident wrong name on nearly every false alarm. The product needs a rule that names a family only when the evidence is strong, and otherwise says "man-made, type unconfirmed" or nothing. That is called **abstention**.

27.2 The ideal: posterior probabilities

Definition: Bayes' rule, prior, posterior

For explanations $k = 0, 1, \dots, K$ (with $k = 0$ meaning "none of the library"), each with a **prior** probability π_k (how common it is before looking), the **posterior** probability after seeing the residual $\mathbf{r}$ is

$$P(k \mid \mathbf{r}) = \frac{\pi_k \, p(\mathbf{r} \mid k)}{\sum_j \pi_j \, p(\mathbf{r} \mid j)} \; :$$

prior times likelihood, divided by the total so the posteriors add to 1.

Worked example 27.1: Rare classes need strong evidence

Three explanations: none, metal, plastic, with priors 0.98, 0.01, 0.01 (false alarms far outnumber targets). Suppose the likelihoods of the observed residual are in the ratio $1 : 30 : 5$ (metal explains it thirty times better than "none").

$$\text{prior} \times \text{likelihood} = (0.98 \times 1,\ 0.01 \times 30,\ 0.01 \times 5) = (0.98,\ 0.30,\ 0.05)$$

$$\text{total} = 0.98 + 0.30 + 0.05 = 1.33$$

$$\text{posterior} = (0.98/1.33,\ 0.30/1.33,\ 0.05/1.33) = (0.737,\ 0.226,\ 0.038).$$

Even with thirty-to-one evidence, "none" remains the most probable explanation.

The likelihood $p(\mathbf{r} \mid k)$ itself must account for the unknown fill and amplitudes. Averaging over them (a "Laplace approximation" does it roughly) gives the GLRT statistic minus a penalty of about $\frac{q_k}{2}\log(\cdot)$ for the family's dimension q_k. Without the penalty, a family with more freedom wins on noise alone. The LLD saw exactly this: "uncapped, about 1,900 minerals out-vote the about 180 man-made spectra by sheer number of chances", fixed there by capping each family at 40 spaced entries and never forming a cone of natural spectra.

Definition: Chow's rule

Name the most probable explanation if its posterior is at least $1 - c$; otherwise abstain. Here c is the cost of abstaining relative to a wrong name. With an explicit "none" class, abstaining can also take the form of naming class 0.

With $c = 0.3$: in worked example 27.1 the top posterior $0.737 \geq 0.7$, so name "none". A pixel with posteriors $(0.45,\ 0.40,\ 0.15)$ has top $0.45 < 0.7$: abstain.

The trouble is that the densities $p(\mathbf{r} \mid k)$ are not known well enough to compute these posteriors (Chapter 26), and the null changes a great deal from scene to scene. So the shipped matcher replaces likelihoods by *calibrated* scores.

27.3 Per-scene calibration

Definition: calibrated family score

For each family k, compute its score T_k on N_0 background pixels sampled from the same scene (anomalies grown by 3 px excluded; $N_0 = 10{,}000$; `NULL_N`, `EXCLUDE_BUFFER_PX` in `family_match.py`). The **calibrated score** of pixel p is the share of those null scores strictly below its own:

$$\pi_k(p) = \frac{\#\{i \,:\, T_k(\mathbf{y}_i) < T_k(p)\}}{N_0} \qquad \texttt{(calibrate)}.$$

Name the best family if $\max_k \pi_k(p) \geq p^* = 1 - \alpha/F$, with F the number of man-made families and α the target false-name rate.

Worked example 27.2: A calibrated score from a tiny null

Null scores of ten background pixels, sorted: 0.01, 0.02, 0.02, 0.03, 0.04, 0.05, 0.06, 0.08, 0.10, 0.15. A pixel scores 0.09: eight null scores are below it, so $\pi = 8/10 = 0.8$. A pixel scoring 0.2 beats all ten: $\pi = 1.0$.
The product's settings: $\alpha = 5\%$, $F = 10$, so $p^* = 1 - 0.05/10 = 1 - 0.005 = 0.995$. Out of 10,000 null pixels, a pixel must beat at least 9,950: the bar sits at 50 null exceedances.

The percentile is a "conformal p-value" in the statistics literature: it makes no assumption about the shape of the null, only that the null pixels are a fair sample of ordinary pixels.

27.4 The false-name bound, derived

Definition: exchangeable

Pixel p is **exchangeable** with the null sample if, before looking, there is no reason to expect its score to be ranked differently from any null pixel's: it is an ordinary background pixel of the same scene, drawn the same way.

Step 1. Put p's score among the N_0 null scores: $N_0 + 1$ values. If p is exchangeable (and there are no

ties), its rank from the bottom is equally likely to be any of $1, 2, \ldots, N_0 + 1$.

Step 2. $\pi_k(p) \geq p^*$ means at least $p^* N_0$ null scores are below it, so its rank is among the top $N_0(1-p^*)+1$ positions (rounding aside). Its probability is at most

$$\frac{N_0(1-p^*)+1}{N_0+1} \leq (1-p^*) + \frac{1}{N_0}.$$

(Divide top and bottom by N_0 and drop the $+1$ in the denominator, which can only make the fraction larger.)

Worked example 27.3: The bound with ten null pixels

$N_0 = 10$, $p^* = 0.8$. $\pi \geq 0.8$ needs at least 8 null scores below p, so rank 9, 10 or 11 of 11: probability $3/11 = 0.273$. The formula: $(10 \times 0.2 + 1)/11 = 3/11$; the simpler bound $0.2 + 0.1 = 0.3$.

Step 3: all families at once. A pixel gets *some* man-made name if any of the F families clears the bar. The **union bound** says the probability that at least one of several events happens is at most the sum of their probabilities (if rain on Monday has probability 0.3 and on Tuesday 0.3, rain on at least one of the two days has probability at most 0.6; less if the two overlap). So

$$P(\text{some man-made name}) \leq F\Big(1 - p^* + \frac{1}{N_0}\Big) = F \cdot \frac{\alpha}{F} + \frac{F}{N_0} = \alpha + \frac{F}{N_0}.$$

With the product's numbers: $0.05 + 10/10{,}000 = 0.05 + 0.001 = 0.051$.

27.5 How precise is the bar?

Each null pixel exceeds the true 99.5th percentile with probability $q = 0.005$, like a biased coin. The share of N_0 coins that come up heads has variance $q(1-q)/N_0$ (one coin has variance $q(1-q)$; independent variances add, and dividing by N_0 divides the variance by N_0^2). So the realised per-family false-name rate wobbles with standard deviation

$$\sqrt{0.005 \times 0.995/10{,}000} = \sqrt{0.004975/10{,}000} = \sqrt{4.975 \times 10^{-7}} = 7.05 \times 10^{-4},$$

which is $7.05 \times 10^{-4}/0.005 = 14\%$ of the target rate[C]. With $N_0 = 10^5$ it would be 4%. The LLD measured the realised rate at a quarter to a half of α[M]: the union bound is loose, because family scores are strongly correlated (families are a few degrees apart, Table 21.2), so the events overlap heavily.

27.6 The bound says nothing about false alarms

The exchangeability in the bound is with *ordinary* pixels. The pixels the matcher is actually asked about are anomaly-flagged, unusual by construction, and the bound does not apply to them. On the four PRISMA scenes, 9 to 24% of false alarms were named man-made at $\alpha = 5\%$ (LLD section 5)[M].

A calibration that does apply draws the null from the *candidate* population itself (the top-percentile pixels), which is about 98% non-target (false alarms outnumber targets about 50 to 1). Suppose a fraction π of that null sample is target. For any threshold η, the null's exceedance rate mixes the two kinds:

$$\begin{aligned} P_{\text{null}}(T > \eta) &= (1-\pi)\, P_{\text{non-target}}(T > \eta) + \pi\, P_{\text{target}}(T > \eta) \\ &\geq (1-\pi)\, P_{\text{non-target}}(T > \eta) && \textit{the second term is not negative} \\ P_{\text{non-target}}(T > \eta) &\leq \frac{P_{\text{null}}(T > \eta)}{1-\pi} && \textit{divide by } 1-\pi \end{aligned}$$

At $\pi = 0.02$, $1/(1-0.02) = 1.0204$: contamination inflates the guaranteed rate by at most 2%. **This conditional calibration controls false names on false alarms, which is the product's actual failure.**

27.7 Why a fixed absolute threshold fails

Suppose one scene's residual error is c times another's. A matched-filter threshold of 3.09 set for $c = 1$ (false-alarm rate $Q(3.09) = 0.001$) gives, in a scene with $c = 2$, a false-alarm rate of $P(2Z > 3.09) = Q(3.09/2) = Q(1.545) = 0.061$: sixty times more. In a scene with $c = 0.5$ it gives $Q(6.18) = 3 \times 10^{-10}$: practically blind. ACE removes c (Chapter 20) but not the dependence on the null's shape (dimension, tail weight, unevenness). The project rule is the same statement for anomaly scores: "Thresholding is percentile, never absolute" (`docs/05-detectors.md`; a calm lake has residuals near 0.01, a fire-affected scene near 0.1). Family matching inherits it, per family, because the null also depends on the family's effective rank (Chapter 21).

27.8 Open-set naming: three tests

Naming is **open-set**: most anomalies belong to no library family. The note proposes three calibrated tests, plus a margin to the runner-up family:

1. *detection*: the pixel is anomalous (RM percentile, upstream);
2. *attribution*: $\max_k \pi_k \geq p^*$ (the shipped rule);
3. *fit*: the unexplained whitened energy E_k^{unex} (Chapter 21) is no larger than the best family leaves on implanted targets of that family at similar fill (a lack-of-fit test; the reference must come from implants).

A pixel that passes 1 and 2 but fails 3 is "man-made-like, not this family"; one that fails 2 is "man-made, type unconfirmed". One asymmetry in the product: the worker shows natural-family names whether or not they pass p^* (`named = answered or group == "natural"` in `backend/allotrope/action_types/_spectral_library_match_run.py`).

Analogy: a bird-watcher's field guide

A good bird-watcher with a field guide does three things. She first notices that something unusual is in the hedge (detection). She then checks whether it matches a species in her guide well enough that she would bet on it, given how often she sees that species around here (attribution, with a prior). And she asks whether every feature fits, or whether the bird has a marking her guide does not show (fit). "Some kind of warbler, not sure which" is an honest, useful answer; a confident wrong species is worse than none.

Common confusions

Is $\pi_k(p)$ a probability that the pixel is family k? No. It is the pixel's percentile among background scores for that family. High means "unusual for background in the way family k would be".
Why α/F and not α? Because there are F chances to give a wrong name; the union bound shares the budget among them.
Does $\alpha = 5\%$ mean 5% of names are wrong? No. It bounds wrong names on *ordinary* pixels. On false alarms the measured rate was 9 to 24%, and among named pixels the share wrong can be much higher.
Why sample the null from the same scene? Because the null changes from scene to scene by orders of magnitude (the fixed-threshold calculation).

Check yourself

1. Repeat worked example 27.1 with priors 0.5, 0.25, 0.25. What is named now?
2. With the ten null scores of worked example 27.2, what is the calibrated score of a pixel scoring

0.055?

3. What is p^* for $\alpha = 10\%$ and $F = 10$?
4. Compute the relative sampling error of the realised rate for a tail of 0.001 and $N_0 = 10{,}000$.
5. Why does the false-name bound not cover anomaly-flagged pixels?

Check yourself: worked answers

1. $(0.5 \times 1,\ 0.25 \times 30,\ 0.25 \times 5) = (0.5,\ 7.5,\ 1.25)$; total 9.25; posteriors $(0.054,\ 0.811,\ 0.135)$. Metal is named (if $1 - c \leq 0.811$).
2. Six null scores are below 0.055 (0.01, 0.02, 0.02, 0.03, 0.04, 0.05): $\pi = 0.6$.
3. $1 - 0.10/10 = 0.99$.
4. $\sqrt{0.001 \times 0.999/10{,}000} = \sqrt{9.99 \times 10^{-8}} = 3.16 \times 10^{-4}$; relative $3.16 \times 10^{-4}/0.001 = 32\%$.
5. They are not exchangeable with ordinary pixels: they were selected for being unusual, so their scores rank high more often than a random background pixel's.

Next: Chapter 28, what a trained namer would learn.

Chapter 28

The learned namer

The one thing this chapter teaches

A classifier trained with cross-entropy on synthetic implants learns the posterior probabilities of the generator's world; it is only as good as the match between that world and reality, and it can only be calibrated across scenes if its input carries the scene's residual covariance.

28.1 The problem: could a network do the naming?

The parked plan (`docs/proposals/spectral-matching-fresh-start.md`) would train a network, the **namer**, on synthetic mixtures: plant library spectra into real background patches at known fills, and teach the network to say which family was planted. What would such a network learn, and what could go wrong? Both questions have precise answers.

28.2 The generator

Definition: generator

The **generator** G is the recipe that produces training examples. Each example draws: a family k from prior probabilities π_G ($k = 0$ means "none", with fill 0); a spectrum $\mathbf{t}$ from family k (augmented); a fill a, for instance log-uniform on $[0.05,\ 0.5]$; a real background patch $\mathbf{x}_C$ from the training shards, with the target implanted through the PSF (`implant_tile`); a smooth gain and offset; and noise. The namer sees an input $\boldsymbol{v}$ built from the result, for instance the pixel, Heavy's prediction and the residual in normalised units, $(\mathbf{x}_p,\ \hat{\mathbf{b}}_p,\ \mathbf{r}_p)$, plus a summary of the scene's Σ_r.

"Log-uniform" means the logarithm of a is spread evenly, so each factor of the same size is equally likely: the halfway point is $\sqrt{0.05 \times 0.5} = \sqrt{0.025} = 0.158$, and a is as likely to be between 0.05 and 0.158 as between 0.158 and 0.5.

One detail matters a great deal. To simulate leakage faithfully, $\hat{\mathbf{b}}_p$ must be computed by running Heavy on the *implanted* context. Feeding a $\hat{\mathbf{b}}$ from the clean patch would teach the namer a leak-free world that does not exist (Chapter 24).

28.3 What cross-entropy training learns

Definition: cross-entropy

A classifier outputs probabilities $q(k \mid \boldsymbol{v})$ over the classes. Its **cross-entropy** loss on one example with true class k is $-\ln q(k \mid \boldsymbol{v})$: zero if it gave the true class probability 1, large if it gave it a small probability. Training minimises the average over many examples.

Worked example 28.1: The best guess is the true probability

Two classes. For some input, the generator makes class "metal" with probability 0.7 and "none" with 0.3. The classifier says q for metal and $1-q$ for none. Expected loss: $0.7\times(-\ln q)+0.3\times(-\ln(1-q))$.

q	0.5	0.6	0.7	0.8	0.9
expected loss	0.6931	0.6325	**0.6109**	0.6390	0.7645

(At $q = 0.7$: $-0.7\ln 0.7 - 0.3\ln 0.3 = 0.7\times 0.3567 + 0.3\times 1.2040 = 0.2497 + 0.3612 = 0.6109$.) The minimum is at $q = 0.7$. With calculus: the derivative is $-0.7/q + 0.3/(1-q)$; setting it to zero, $0.3q = 0.7(1-q)$, so $0.3q + 0.7q = 0.7$ and $q = 0.7$.

Rule 28.1: What a perfectly trained namer outputs

Minimising expected cross-entropy gives $q^*(k \mid \boldsymbol{v}) = p_G(k \mid \boldsymbol{v})$, the generator's posterior. A fill head trained with squared error outputs the average fill given the input, $\mathbb{E}_G[a \mid \boldsymbol{v},\ k \neq 0]$. Choosing the largest q^* is the best possible classifier for 0-1 loss *under the generator*, and Chow's rule (Chapter 27) on q^* is the best possible abstention rule.

The squared-error part in one line: if the fill is 0.1 or 0.3 with equal chance, the guess c that minimises $\frac{1}{2}(0.1-c)^2 + \frac{1}{2}(0.3-c)^2$ has derivative $-(0.1-c) - (0.3-c) = 0$, so $2c = 0.4$ and $c = 0.2$, the average.

28.4 What the input must contain

Under the Gaussian residual model $\mathbf{r} \sim \mathcal{N}(a\mathbf{H}_k\boldsymbol{\theta},\ \mathbf{S})$, expand the log-likelihood exactly as in Chapter 18:

$$\ell = \text{const} - \tfrac{1}{2}\mathbf{r}^\top\mathbf{S}^{-1}\mathbf{r} + a\,\boldsymbol{\theta}^\top(\mathbf{H}_k^\top\mathbf{S}^{-1}\mathbf{r}) - \tfrac{1}{2}a^2\,\boldsymbol{\theta}^\top\mathbf{H}_k^\top\mathbf{S}^{-1}\mathbf{H}_k\boldsymbol{\theta}.$$

The residual enters only through the numbers $\mathbf{H}_k^\top\mathbf{S}^{-1}\mathbf{r}$ (one small vector per family) and $\mathbf{r}^\top\mathbf{S}^{-1}\mathbf{r}$. Those are **sufficient statistics**: everything the data can say about k, a and $\boldsymbol{\theta}$ is in them (the Fisher-Neyman factorisation, by name). Every one of them contains $\mathbf{S}^{-1}$. **A namer given r without the scene's covariance cannot form them, and cannot be calibrated across scenes.** So either feed it the whitened residual $\mathbf{S}^{-1/2}\mathbf{r}$, or keep the network as the "score each family" box and put the scene-wise percentile calibration of Chapter 27 after it, as the earlier proposal recommends. The second is safer: the calibration then holds whatever the network learned.

28.5 The gap between the generator and reality

Let P be the real world's joint distribution of inputs and labels. For any classifier h and any loss between 0 and 1, the difference between its real risk and its generator risk is at most the **total variation distance** between P and G:

$$|R_P(h) - R_G(h)| \le \mathrm{TV}(P, G).$$

For distributions over a few outcomes, TV is half the sum of the absolute differences of the probabilities. With class priors alone, generator $(0.5,\ 0.25,\ 0.25)$ and reality $(0.98,\ 0.01,\ 0.01)$: $\frac{1}{2}(0.48+0.24+0.24) = 0.48$.

The bound is loose, but it names the two parts of the gap.

Label shift. Reality is dominated by "none" (about 50 false alarms per target among candidates). If only the priors differ, the posterior can be corrected without retraining: $p_P(k \mid \boldsymbol{v}) \propto p_G(k \mid \boldsymbol{v})\, \pi_P(k)/\pi_G(k)$.

Worked example 28.2: Correcting a namer for real-world priors

The namer says $(0.2,\ 0.6,\ 0.2)$ for (none, metal, plastic), trained with priors $(0.5,\ 0.25,\ 0.25)$. Real priors are $(0.98,\ 0.01,\ 0.01)$.

$$\begin{aligned} \text{none} &: 0.2 \times 0.98/0.5 = 0.392 \\ \text{metal} &: 0.6 \times 0.01/0.25 = 0.024 \\ \text{plastic} &: 0.2 \times 0.01/0.25 = 0.008 \\ \text{total} &= 0.392 + 0.024 + 0.008 = 0.424 \\ \text{corrected} &= (0.392,\ 0.024,\ 0.008)/0.424 = (0.925,\ 0.057,\ 0.019). \end{aligned}$$

A confident-looking 60% metal becomes 5.7% once you account for how rare metal is among candidates.

Conditional shift. Real false alarms (field edges, glint, shoreline, haze) are not produced by the generator, and real deployed materials are not library spectra (the domain gap, Chapter 26). No reweighting fixes this. The cheapest partial fix is to take "none" examples from the real top-percentile candidates of training scenes (about 2% of which are targets, a tolerable label noise) rather than from untouched pixels.

The gap can be measured directly with a **classifier two-sample test**: train a separate network to tell synthetic implants from real candidate pixels; an AUC well above 0.5 on held-out scenes means the namer will be trained on a world it will not see. A namer trained on linear mixtures and scored on linear-mixture plantings flatters itself by construction, which is why the earlier proposal's pass rule weights the Gudalur and oil sets.

Analogy: textbook exercises and the real exam

A student who practises only the textbook's exercises learns the textbook's world very well: its typical numbers, its phrasing, how often each topic appears. On an exam with the same mix of topics but different numbers (label shift), you can adjust for the new mix. On an exam with question types the textbook never had (conditional shift), practice does not transfer, and the only cure is practice on real past papers.

Common confusions

Does a perfect namer output true probabilities? It outputs the generator's probabilities. They are real-world probabilities only if the generator matches the real world.
Is more synthetic data the fix? It reduces training noise, not the generator gap.
Why not train on real labelled objects? There are too few per family (Chapter 30); that is why a generator is attractive at all.
Why does $\hat{\mathbf{b}}$ have to come from the implanted context? Because in reality the model sees the target's halo and leaks; a clean $\hat{\mathbf{b}}$ teaches the namer a residual stronger than any it will meet.

Check yourself

1. With true probability 0.7, compute the expected cross-entropy of guessing $q = 0.6$.
2. A namer trained with priors $(0.5,\ 0.5)$ for (none, metal) outputs $(0.5,\ 0.5)$. Real priors are $(0.98,\ 0.02)$. Correct it.
3. Compute the total variation distance between $(0.6,\ 0.4)$ and $(0.9,\ 0.1)$.

4. What is the median of a fill drawn log-uniformly on $[0.05,\ 0.5]$?
5. What two quantities must the namer's input let it compute, under the Gaussian model?

Check yourself: worked answers

1. $-0.7 \ln 0.6 - 0.3 \ln 0.4 = 0.7 \times 0.5108 + 0.3 \times 0.9163 = 0.3576 + 0.2749 = 0.6325$.
2. $(0.5 \times 0.98/0.5,\ 0.5 \times 0.02/0.5) = (0.98,\ 0.02)$; already adds to 1. So $(0.98,\ 0.02)$.
3. $\frac{1}{2}(|0.6 - 0.9| + |0.4 - 0.1|) = \frac{1}{2}(0.3 + 0.3) = 0.3$.
4. $\sqrt{0.05 \times 0.5} = 0.158$.
5. $\mathbf{H}_k^\top \mathbf{S}^{-1}\mathbf{r}$ for each family, and $\mathbf{r}^\top \mathbf{S}^{-1}\mathbf{r}$.

Next: Chapter 29, measuring how well naming works.

Chapter 29

The benchmark and the paired bootstrap

The one thing this chapter teaches

A naming benchmark reports per-family accuracy and a confusion matrix at equal coverage, and compares two methods with a paired bootstrap: resample the objects with replacement, the same draw for both methods, and read the uncertainty of the difference from the spread of the resampled differences.

29.1 The problem: how do you know a namer is better?

Two namers, A and B, are run on the same labelled objects. A gets 60% right, B 50%. Is A better, or did it get lucky on this particular set of objects? With a handful of objects the answer is almost always "could be luck". You need a benchmark big enough and a way of putting an error bar on the difference.

29.2 An OSM-labelled benchmark

The parked plan's judge is a benchmark built from OpenStreetMap tags over the footprints of held-out scenes: `landuse=greenhouse_horticulture`, `plant:source=solar`, `surface=asphalt/concrete`, `roof:material=*`, harbour and water features. Let $\mathcal{O}_k$ be the objects of family k and n_k their number.

Definition: accuracy, confusion matrix, coverage

The **per-family accuracy** is the share of family k's objects named k: $\mathrm{Acc}_k = \frac{1}{n_k}\#\{o \in \mathcal{O}_k : \hat{k}(o) = k\}$. The **confusion matrix** has one row per true family and one column per possible answer, including "unconfirmed" and "natural"; entry (j, k) is the share of family j's objects given answer k. The **coverage** is the share of objects that receive a name at all.

Worked example 29.1: One row of a confusion matrix

Ten metal objects. The namer answers metal 6 times, roofing 2 times, "unconfirmed" once, a natural family once. The metal row is

$$(\text{metal } 6/10,\ \text{roofing } 2/10,\ \text{unconfirmed } 1/10,\ \text{natural } 1/10) = (0.6,\ 0.2,\ 0.1,\ 0.1),$$

so $\mathrm{Acc}_{\text{metal}} = 0.6$ and coverage on these objects is $8/10$ if "named" means a man-made family.

Two rules from the LLD. Methods are compared **at equal coverage**: a method that abstains more can always look more accurate on what it does answer. And the false-name rate is reported separately on ordinary pixels and on false alarms (Chapter 27).

29.3 Wrong labels shrink differences

OpenStreetMap tags are sometimes wrong. Suppose a fraction ϵ_L of tags is wrong, and on a wrongly tagged object the method's answer matches the (wrong) tag only by chance, probability $1/K$ among K families[A]. Then the observed accuracy is

$$\text{observed} = (1 - \epsilon_L) \times \text{Acc}_k + \epsilon_L \times \frac{1}{K}.$$

The difference between two methods is $(1 - \epsilon_L)$ times the true difference, because the ϵ_L/K term is the same for both and cancels.

Worked example 29.2: Ten per cent wrong tags

$\epsilon_L = 0.1$, $K = 10$. Method A's true accuracy 0.6, B's 0.5.

$$\text{A observed} = 0.9 \times 0.6 + 0.1/10 = 0.54 + 0.01 = 0.55$$
$$\text{B observed} = 0.9 \times 0.5 + 0.01 = 0.45 + 0.01 = 0.46$$
$$\text{difference} = 0.55 - 0.46 = 0.09 = 0.9 \times 0.10.$$

To see a difference shrunk by 0.9 with the same confidence you need $1/0.9^2 = 1/0.81 = 1.23$ times as many objects (Chapter 30 explains why the square).

29.4 The paired bootstrap

Definition: bootstrap, paired bootstrap

The **bootstrap** estimates the uncertainty of a number computed from a sample by recomputing it on many *resamples*: samples of the same size drawn *with replacement* from the original (some objects appear twice, some not at all). The spread of the recomputed numbers stands in for the spread you would see across fresh samples. The **paired** bootstrap, for comparing two methods, uses the same resample for both methods each time and records the difference.

`app/evaluation/significance.py` does this per scene: it resamples each scene's objects with replacement (the same draw for both methods), recomputes both measures, and takes the 2.5th and 97.5th percentiles of the difference as a 95% interval. Background pixels are not resampled. A difference is called **resolved** when its interval excludes zero.

Worked example 29.3: A paired bootstrap on six objects

Six objects; 1 means named correctly.

object	1	2	3	4	5	6
method A	1	1	0	1	0	1
method B	1	0	0	1	1	0

Accuracy A $= 4/6 = 0.667$, B $= 3/6 = 0.5$, difference 0.167.
Resample 1: draw objects 2, 2, 5, 1, 6, 3. A's values: $1, 1, 0, 1, 1, 0$, accuracy $4/6 = 0.667$. B's: $0, 0, 1, 1, 0, 0$, accuracy $2/6 = 0.333$. Difference 0.333.
Resample 2: draw 1, 3, 3, 4, 4, 5. A: $1, 0, 0, 1, 1, 0$, accuracy $3/6 = 0.5$. B: $1, 0, 0, 1, 1, 1$, accuracy $4/6 = 0.667$. Difference -0.167.
Repeat thousands of times. With 20,000 resamples the 2.5th and 97.5th percentiles of the difference are -0.333 and 0.667[C](`compute_step_by_step_output.txt`). The interval contains zero: six objects cannot tell these methods apart.

Why pair? Some objects are easy for every method and some hard. Using the same draw for both methods makes those shared ups and downs cancel in the difference, leaving only the objects where the methods disagree (objects 2, 5 and 6 above) to create uncertainty. The next chapter turns that observation into a formula.

One caveat recorded by the project: the bootstrap resamples objects within scenes, so it does not capture scene-to-scene variation, and its interval is a lower bound on the real uncertainty (`research/experiments/bench-resolution/README.md`)[M]. For per-family naming accuracy the same scheme applies, with objects of family k in place of anomaly objects and "named k" in place of "caught".

For a real example: FT08's change of pAUC against its control was -0.0058 with interval $[-0.0106,\ -0.0010]$ (worked example 24.3)[M]. Both ends are below zero: a resolved loss.

Analogy: replaying a tournament

To know whether player A is really better than player B, you would like to replay the season many times. You cannot, so you do the next best thing: build many imaginary seasons by drawing, with replacement, from the matches that were actually played, and in each imaginary season both players face exactly the same opponents. If A comes out ahead in nearly every imaginary season, A is better; if it flips back and forth, you cannot tell.

Common confusions

Why "with replacement"? Without replacement every resample would be the original set in a different order, with the same accuracy.
Is the bootstrap interval exact? It is an approximation, good when the number of objects is not tiny, and here a lower bound because scenes are not resampled.
Why compare at equal coverage? Accuracy on answered objects rises as a method abstains more; comparing at different coverages rewards timidity.
Does a resolved difference mean a large one? No. With enough objects, small differences resolve. Report the size and the interval together.

Check yourself

1. Twenty plastic objects: 9 named plastic, 5 textile, 4 unconfirmed, 2 natural. Write the plastic row and the accuracy.
2. With $\epsilon_L = 0.2$ and $K = 10$, what is the observed accuracy of a method whose true accuracy is 0.7?
3. In worked example 29.3, compute the difference for the resample 1, 1, 4, 4, 6, 6.
4. What uncertainty does the project's bootstrap leave out, and what does that do to its intervals?
5. Is an interval of $[-0.0106,\ -0.0010]$ resolved? What does it say?

Check yourself: worked answers

1. $(9/20,\ 5/20,\ 4/20,\ 2/20) = (0.45,\ 0.25,\ 0.20,\ 0.10)$; accuracy 0.45.
2. $0.8 \times 0.7 + 0.2/10 = 0.56 + 0.02 = 0.58$.
3. A: objects 1,1,4,4,6,6 give $1, 1, 1, 1, 1, 1$, accuracy 1. B: $1, 1, 1, 1, 0, 0$, accuracy $4/6 = 0.667$. Difference 0.333.
4. Scene-to-scene variation; its intervals are too narrow (a lower bound on the real uncertainty).
5. Yes, it excludes zero: the method is worse than its control, by between 0.001 and 0.011 pAUC.

Next: Chapter 30, how many objects you need.

Chapter 30

The minimum detectable improvement

The one thing this chapter teaches

When two methods are scored on the same objects, the uncertainty of their accuracy difference is about $\sqrt{\pi_d/n}$, set by the share π_d of objects on which they disagree; detecting a 10-point gain with the usual confidence therefore needs about $7.85\,\pi_d/0.01$ objects per family, around 160 at $\pi_d = 0.2$.

30.1 The problem: planning a benchmark

Before collecting labelled objects you want to know how many you need. Too few and real improvements drown in noise; too many wastes effort. The paired bootstrap of Chapter 29 measures uncertainty after the fact; this chapter predicts it in advance.

30.2 The per-object difference

For each object, let D be $+1$ if method A is right and B wrong, -1 if B is right and A wrong, and 0 if they agree (both right or both wrong). The accuracy difference is the average of D over the objects. Let π_d be the **discordance**, the share of objects on which exactly one method is right, and Δ the true accuracy difference.

Mean. $\mathbb{E}[D] = P(D = +1) - P(D = -1) = \Delta$.

Mean of the square. D^2 is 1 on every discordant object and 0 otherwise, so $\mathbb{E}[D^2] = \pi_d$.

Variance. $\mathrm{Var}(D) = \mathbb{E}[D^2] - (\mathbb{E}D)^2 = \pi_d - \Delta^2 \approx \pi_d$ when Δ is small. (The first equality is the variance formula $\mathbb{E}[(D - \mathbb{E}D)^2]$ multiplied out.)

The average over n objects. For independent objects variances add, so $\mathrm{Var}(\sum_i D_i) = n\,\mathrm{Var}(D)$. Dividing by n divides the variance by n^2 (because $\mathrm{Var}(cX) = c^2\,\mathrm{Var}(X)$):

$$\mathrm{Var}(\bar{D}) = \frac{n\,\mathrm{Var}(D)}{n^2} = \frac{\mathrm{Var}(D)}{n} \approx \frac{\pi_d}{n}, \qquad \mathrm{SE}(\bar{D}) \approx \sqrt{\frac{\pi_d}{n}}.$$

Only the disagreements create uncertainty. That is the formal version of "why pair".

30.3 From standard error to the smallest detectable difference

Use the Gaussian approximation for $\bar{D}$. A two-sided test at level 5% declares a difference when $|\bar{D}| > 1.96\,\mathrm{SE}$ (because $Q(1.96) = 0.025$ on each side). If the true difference is $\Delta > 0$, $\bar{D}$ is centred on Δ, and the

test succeeds when $\bar{D}$ lands above 1.96 SE. For that to happen with probability 80% (the usual "power"), the centre must sit 0.8416 SE above the bar, since $\Phi(0.8416) = 0.80$:

$$\Delta \geq 1.96\,\text{SE} + 0.8416\,\text{SE} = 2.8016\,\text{SE}.$$

Rule 30.1: Paired proportions (McNemar)

$$\Delta_{\min} \approx 2.80\sqrt{\frac{\pi_d}{n}}, \qquad n \approx \frac{2.80^2\,\pi_d}{\Delta^2} = \frac{7.85\,\pi_d}{\Delta^2}.$$

The second form comes from the first: set $\Delta = 2.8016\sqrt{\pi_d/n}$, square both sides ($\Delta^2 = 7.849\,\pi_d/n$), multiply both sides by n and divide by Δ^2.

Worked example 30.1: Sizing a family benchmark

First, calibrate π_d against the bench. FTH1 against its control (FTB1): caught-top-1% intervals are about ±1.9 points on 909 objects (`FTH1-heavy-joint/README.md`)[M]. A 95% half-width is 1.96 SE, so

$$\text{SE} = 0.019/1.96 = 0.009694$$
$$\pi_d/909 = 0.009694^2 = 0.00009397$$
$$\pi_d = 0.00009397 \times 909 = 0.085^{[C]}.$$

Two namers will disagree more than two anomaly models; take $\pi_d = 0.2$[A].
Objects needed for a 10-point gain.

$$n = 7.849 \times 0.2/0.1^2 = 1.5698/0.01 = 157.$$

At $\pi_d = 0.1$: 78.5, about 80. At $\pi_d = 0.3$: 235[C].
Smallest detectable difference for a given n, at $\pi_d = 0.2$:

$$n = 30: \quad 2.8016 \times \sqrt{0.2/30} = 2.8016 \times 0.08165 = 0.229$$
$$n = 100: \quad 2.8016 \times \sqrt{0.002} = 2.8016 \times 0.04472 = 0.125$$
$$n = 300: \quad 2.8016 \times \sqrt{0.000667} = 2.8016 \times 0.02582 = 0.072$$

objects per family n_k	30	100	300
$\Delta_{\min}$ at $\pi_d = 0.2$	23 pts	12.5 pts	7.2 pts
95% half-width of one accuracy near 0.5	18 pts	9.8 pts	5.7 pts

The last row is the error bar on a single method's accuracy, $1.96\sqrt{0.5 \times 0.5/n}$: for $n = 30$, $1.96\times\sqrt{0.008333} = 1.96 \times 0.0913 = 0.179$[C].

30.4 Objects from the same scene are not independent

Objects in one scene share its atmosphere, its sensor settings and its background. With m objects per scene and correlation ρ between objects of the same scene, n objects carry the information of only

$$\frac{n}{1 + (m-1)\rho}$$

independent ones (the **design effect**). With $m = 20$ and $\rho = 0.1$[A]: $1 + 19 \times 0.1 = 2.9$, so $n/2.9$. To get the 157 effective objects of worked example 30.1 you would need about $157 \times 2.9 = 455$ objects in clusters of 20.

"Hundreds to thousands of objects per family", as the parked plan hopes for, is the right order; tens per family can only show large effects. Whether OpenStreetMap has that many tagged objects inside held scene footprints is the parked plan's first check, and one of the note's open experiments.

30.5 Where to go from here

The original note ends with a list of cheap experiments, ordered by cost; none needs a large GPU. In this book's terms:

1. How much family-separating energy RM's 30-direction truncation discards (Chapter 17).
2. How much Heavy leaks, by size and position relative to the token grid, by running it with and without an implant (Chapter 24).
3. Background error against mask cell size, to test the wider-mask inequality (worked example 24.3).
4. Heavy's decoder output floor at $D = 64$ (Chapter 24).
5. Family matching on the residual instead of the ring mean (Part IV).
6. Gain sensitivity on the planted set at ±5% and ±10% (Chapter 25).
7. The lack-of-fit test and a candidate-conditional null (Chapter 27).
8. OpenStreetMap object density per family (this chapter).
9. The generator-gap two-sample test (Chapter 28).

What would change the plan: if Heavy leaks $l \gtrsim 0.5$ for small objects, naming should use the ring background or a heavily masked pass; if gain sensitivity is at the level of family separation, atmospheric harmonisation is a prerequisite for naming; if OpenStreetMap yields tens rather than hundreds of objects per family, Gudalur and oil remain the gate.

Analogy: weighing two nearly equal bags on a wobbly scale

Two bags differ by 10 grams and your scale wobbles by 50 grams. One weighing each tells you nothing. Weigh them many times and average, and the wobble in the average shrinks with the square root of the number of weighings: four times as many weighings halve it. Weighing both bags in each session (pairing) removes the wobble that both sessions share.

Common confusions

Why does discordance, not accuracy, set the uncertainty? Objects on which both methods agree contribute $D = 0$ every time and add no noise to the difference.
Why 2.80 **and not** 1.96**?** 1.96 alone gives a 50% chance of declaring a true difference of that size; the extra 0.84 buys 80% power.
Is $\pi_d = 0.2$ **measured?** No, assumed[A]; the bench's anomaly models gave 0.085, and namers are expected to disagree more.
Do more scenes help more than more objects per scene? Yes, when objects within a scene are correlated: the design effect grows with m.

Check yourself

1. How many objects are needed to detect a 15-point gain at $\pi_d = 0.2$?
2. What is the smallest detectable difference with 50 objects at $\pi_d = 0.2$?
3. An interval of ±3 points on 400 objects implies what discordance?
4. 200 objects in clusters of 10 with $\rho = 0.2$: what is the effective number?

5. Why do objects on which both methods are wrong not add uncertainty to the difference?

Check yourself: worked answers

1. $7.849 \times 0.2 / 0.15^2 = 1.5698 / 0.0225 = 69.8$, about 70.
2. $2.8016 \times \sqrt{0.2/50} = 2.8016 \times 0.06325 = 0.177$, about 18 points.
3. SE $= 0.03 / 1.96 = 0.01531$; $\pi_d = 0.01531^2 \times 400 = 0.0002343 \times 400 = 0.094$.
4. Design effect $1 + 9 \times 0.2 = 2.8$; $200 / 2.8 = 71$.
5. On those objects $D = 0$ for every resample: they move neither method's accuracy relative to the other.

Part VII

Natural first, then the class

Chapter 31

Two questions instead of one

The one thing this chapter teaches

Naming splits into "could nature explain this pixel?" and, only if not, "which man-made class is it?"; the probability of a class is then the product $P(M)\,P(k \mid M)$, which follows in two lines from the definition of conditional probability and holds exactly on counted data.

31.1 Why the families had to change

The shipped family library (`data/family_library/`, 411 spectra) was labelled by `scripts/curate_family_library.py`, which assigns a group and a family to each library file by matching text patterns against its file name, first match wins[M]. The patterns sort materials by *use*: an asphalt shingle is "roofing" and an asphalt road "pavement"; one concrete sample is "pavement" and every other concrete "building". The family sizes are whatever the libraries held, capped at 40 (`FAMILY_CAP`): glass has one spectrum, oil forty. And Table 21.2 showed pavement and building 0.51° apart: a share $\sin^2(0.51^\circ) = 0.00008$ of a unit target's energy tells them apart, so they are one material with two names.

On 31 October 2026 the user and Claude discussed the naming step afresh. The design was first recorded in `research/experiments/spectral-namer/README.md` ("Design so far"), then written up the same day as `docs/proposals/spectral-namer.md` and built (Chapter 35 reads the code, which departs from this Part in places); the parked plan in `docs/proposals/spectral-matching-fresh-start.md` still describes one namer with a "none" output. Two decisions came out of it. First, a class list aimed at what a defence analyst asks, limited by what a 30 m pixel can hold: a 4.5×1.8 m car covers $8.1/900 = 0.009$ of a 30 m pixel and $8.1/25 = 0.32$ of a 5 m one[C], so at PRISMA's and EnMAP's resolution only large objects can be named, while AVIRIS-NG flown low reaches vehicle scale. The classes are camouflage, military coatings, painted metal, bare metal, vessels on water, synthetic fabric and tarpaulins, fuel and oil, concrete and asphalt (one class), disturbed earth, cut vegetation, glass and photovoltaic panels, and none / natural. The first two have no library spectra at all and need field measurements. Second, the user's two-level design, which this chapter derives.

31.2 The two levels

Definition: level 1, level 2, the three answers

Level 1 decides between *natural* and *man-made* (written M). **Level 2**, asked only when level 1 says man-made, decides which man-made class k it is, or that none is confident. The namer gives one of three answers: *natural*; *man-made, class unknown*; or *man-made, class k*.

Analogy: triage, then the specialist

An emergency department does not send every arrival to every specialist. A triage nurse first asks "does this person need a doctor at all?"; only then is the patient sent to the cardiologist or the orthopaedic surgeon. A patient who clearly needs a doctor but fits no specialist is still seen: "needs a doctor, specialty unclear" is a legitimate outcome. Level 1 is triage, level 2 the specialist, and "man-made, class unknown" is the patient no specialist claims.

31.3 Conditional probability, and the product rule

Definition: conditional probability

For two events A and B with $P(B) > 0$, the **conditional probability** of A given B is

$$P(A \mid B) = \frac{P(A \text{ and } B)}{P(B)} \ :$$

among the cases where B happens, the share in which A happens too.

Check it on counts. Of 100 anomalous pixels, 40 are man-made, and 10 of those are painted metal. Then $P(M) = 40/100 = 0.4$, $P(\text{painted metal and } M) = 10/100 = 0.1$, and $P(\text{painted metal} \mid M) = 0.1/0.4 = 0.25$, which is $10/40$: a quarter of the man-made pixels.

Rule 31.1: The two-level product

Every class k is man-made, so the event "class k" and the event "class k and man-made" are the same event. Then

$$\begin{aligned} P(k \mid M) &= \frac{P(k \text{ and } M)}{P(M)} && \textit{definition of conditional probability} \\ P(k \mid M)\,P(M) &= P(k \text{ and } M) && \textit{multiply both sides by } P(M) \\ P(k) &= P(M)\,P(k \mid M) && \textit{"}k \textit{ and } M\textit{" is "}k\textit{"} \end{aligned}$$

Two consequences. The class probabilities add up to $P(M)$, not to 1: $\sum_k P(k) = P(M) \sum_k P(k \mid M) = P(M) \times 1$, provided level 2's probabilities add to 1 over the man-made classes. And adding $P(\text{natural}) = 1 - P(M)$ makes the whole list add to 1.

Worked example 31.1: A confident level 1, then a doubtful one

Level 1 says $P(M) = 0.9$; level 2 says painted metal 0.6, bare metal 0.25, tarpaulin 0.15[A].

$$\begin{aligned} P(\text{painted metal}) &= 0.9 \times 0.6 = 0.54 \\ P(\text{bare metal}) &= 0.9 \times 0.25 = 0.225 \\ P(\text{tarpaulin}) &= 0.9 \times 0.15 = 0.135 \\ P(\text{natural}) &= 1 - 0.9 = 0.10 \\ \text{total} &= 0.54 + 0.225 + 0.135 + 0.10 = 1.00. \end{aligned}$$

With $P(M) = 0.3$ and the same level-2 answer, painted metal is $0.3 \times 0.6 = 0.18$. A class probability of 0.6 from level 2 is worth 0.54 behind a confident level 1 and 0.18 behind a doubtful one[C]. **The product stops a confident-sounding class from hiding a doubtful "is it man-made at all?".**

31.4 From probabilities to the three answers

A decision rule needs two thresholds[A]: τ_1 for level 1 and τ_2 for level 2.

- If $P(M) < \tau_1$: answer *natural*.
- Else, if the largest $P(k \mid M) < \tau_2$: answer *man-made, class unknown*.
- Else: answer *man-made, class k* with the largest $P(k \mid M)$, and report $P(k) = P(M)\,P(k \mid M)$.

This is Chow's rule (Chapter 27) applied twice: abstaining at level 2 does not throw away level 1's answer. With $\tau_1 = 0.5$, $\tau_2 = 0.5$: the pixel of worked example 31.1 is "man-made, painted metal, 0.54"; a pixel with $P(M) = 0.9$ and level-2 probabilities $(0.4,\ 0.35,\ 0.25)$ is "man-made, class unknown"; a pixel with $P(M) = 0.3$ is "natural"[A].

31.5 The identity on measured counts

The separability check (`research/experiments/spectral-namer/separability.py`, Chapter 32 describes it) plants library spectra in real PRISMA pixels and matches them. It reports, per fill, the share of man-made plantings that level 1 called man-made, and the share of those whose class was also right. A class can be right only if level 1 was, so rule 31.1 holds on the counts:

$$\begin{aligned} P(\text{class right}) &= P(\text{L1 right}) \times P(\text{class right} \mid \text{L1 right}) \\ &= 0.6096 \times 0.6309 = 0.3846 \quad \text{at fill } 0.10, \end{aligned}$$

which is exactly the share of man-made plantings that the check reports as named with the right class, 0.3846[M]; the source is `separability_prisma.json`, and the product is computed here[C]. On EnMAP at the same fill: $0.4842 \times 0.4674 = 0.2263$, again the reported value.

Common confusions

Is the "two-level" namer related to the two-level covariance of Chapter 19? No. That is a model of noise, a few loud directions and many quiet ones. Here "two levels" means two questions asked in turn. The words collide; the ideas do not.
Do the class probabilities add to 1? They add to $P(M)$. Add $P(\text{natural})$ to get 1.
If level 2 says 0.6, is the class 60% likely? Only given that the pixel is man-made. Unconditionally it is $0.6\,P(M)$.
Why not one classifier with a "natural" class among the others? Because the two sides are learned from very different data, as the next chapter shows: level 1 can learn nature from billions of real pixels, and only level 2 depends on a library.

Check yourself

1. Of 200 anomalous pixels, 50 are man-made and 20 of those are tarpaulin. Compute $P(M)$, $P(\text{tarpaulin and } M)$ and $P(\text{tarpaulin} \mid M)$.
2. Level 1 gives 0.7; level 2 gives bare metal 0.5, painted metal 0.3, tarpaulin 0.2. Compute every probability, including natural, and check the total.
3. With $\tau_1 = \tau_2 = 0.5$, what does the namer answer for $P(M) = 0.8$ and level-2 probabilities $(0.45,\ 0.45,\ 0.10)$?
4. Using the separability numbers at fill 0.25 (L1 right 0.7724, class right given L1 right 0.7151), compute the share of man-made plantings named with the right class.
5. A car is 4.5×1.8 m. What is its fill in a 10 m pixel?

Check yourself: worked answers

1. $P(M) = 50/200 = 0.25$; $P(\text{tarpaulin and } M) = 20/200 = 0.1$; $P(\text{tarpaulin} \mid M) = 0.1/0.25 = 0.4$, which is $20/50$.

2. $0.7 \times 0.5 = 0.35$; $0.7 \times 0.3 = 0.21$; $0.7 \times 0.2 = 0.14$; natural $1 - 0.7 = 0.30$; total $0.35 + 0.21 + 0.14 + 0.30 = 1.00$.
3. $P(M) = 0.8 \geq 0.5$, so man-made; the largest level-2 probability is $0.45 < 0.5$, so “man-made, class unknown”.
4. $0.7724 \times 0.7151 = 0.5523$.
5. $8.1/100 = 0.081$, about 8%.

Next: Chapter 32, how level 1 can flag what no library holds.

Chapter 32

Level 1 as an open set: not explained by nature

The one thing this chapter teaches

Level 1 is trained to describe nature, from billions of real pixels, and calls "man-made" whatever nature cannot explain; such an open-set test can flag camouflage, which no library holds, and its false-alarm rate on ordinary pixels is fixed by a percentile, exactly as in Chapter 27.

32.1 The problem: the man-made side of any library is incomplete

The obvious level 1 is a two-class classifier, "natural" against "man-made", trained on library spectra of each. A **closed** classifier can only answer with the classes it was trained on, and it learns the boundary between the two training sets. Its man-made side knows library materials only. Camouflage and military coatings, the first two classes on the list, have no library spectra. Wherever such a material falls, a closed classifier puts it on one side of a boundary drawn without it, and camouflage designed to look like vegetation tends to fall on the natural side.

The natural side is the opposite. Every scene is mostly natural pixels, and the project holds billions of them (SpectralEarth and the native EnMAP, PRISMA and AVIRIS-NG shards). So level 1 can learn nature thoroughly and need not learn man-made materials at all.

Definition: open-set recognition

A recognizer is **open-set** if it may meet, at test time, inputs from classes absent in training, and is built to say "none of the known kinds" for them (Scheirer et al. 2013, in the bibliography). Here level 1 knows one kind, nature, and calls man-made whatever is not it.

32.2 Near the average is not the same as explained

Worked example 32.1: A camouflage net between the natural samples

Work in two dimensions[A]. Natural pixels lie on the line $x+y=1$; five samples are $(0.1, 0.9)$, $(0.3, 0.7)$, $(0.5, 0.5)$, $(0.7, 0.3)$, $(0.9, 0.1)$. The library has one man-made spectrum, $(0.9, 0.9)$. A camouflage net sits at $(0.4, 0.4)$.
A closed rule: nearest average.

$$\text{natural average} = \left(\tfrac{0.1+0.3+0.5+0.7+0.9}{5},\ \tfrac{0.9+0.7+0.5+0.3+0.1}{5}\right) = (2.5/5,\ 2.5/5) = (0.5,\ 0.5)$$

$$\text{camouflage to it} = \sqrt{(0.4-0.5)^2 + (0.4-0.5)^2} = \sqrt{0.01+0.01} = \sqrt{0.02} = 0.1414$$

camouflage to the library $= \sqrt{(0.4-0.9)^2+(0.4-0.9)^2} = \sqrt{0.25+0.25} = \sqrt{0.5} = 0.7071$.

Nearest is natural. Worse, the two end samples are $\sqrt{0.4^2+0.4^2} = \sqrt{0.32} = 0.5657$ from the natural average, four times farther than the camouflage, so no threshold on "distance to the natural average" can flag the camouflage without flagging real nature first.

An open rule: can nature explain it? Nature's model here is the line; the part of a pixel nature cannot explain is how far it is off the line, measured by $|x+y-1|$ (the distance itself is this divided by $\sqrt{2}$; the factor does not change the ranking):

$$\begin{aligned} \text{each natural sample} &: |0.1+0.9-1| = 0,\ \ldots,\ |0.9+0.1-1| = 0 \\ \text{camouflage} &: |0.4+0.4-1| = |-0.2| = 0.2 \\ \text{library man-made} &: |0.9+0.9-1| = 0.8. \end{aligned}$$

The camouflage is flagged and the natural samples are not[C].

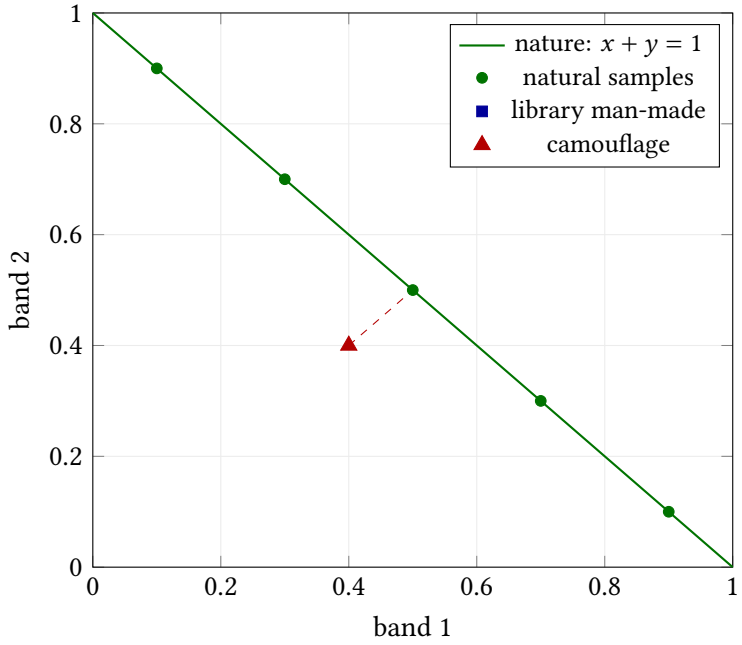


Figure 32.1: Worked example 32.1 to scale. The camouflage (triangle) is next to the natural average $(0.5, 0.5)$ but off the line nature lives on; the dashed segment is its unexplained part, $0.2/\sqrt{2} = 0.141$ long.

Analogy: the doorman who knows the residents

A doorman who knows every resident by face stops anyone he does not recognise, without a list of burglars. A doorman with a list of known burglars waves through any burglar not on it. Level 1 is the first doorman. In a real building the residents number in the thousands, which is why nature needs billions of pixels to learn.

32.3 Setting the threshold without man-made examples

Level 1 produces an **unexplained score** u: how much of the pixel nature cannot account for (a reconstruction residual, a negative log-density, whatever the model offers). Where should the cut be? There are no man-made training examples to tune it on, and none are needed. Score a large sample of ordinary, natural pixels of the scene, and cut at their q-th percentile. By the rank argument of Chapter 27 (worked example 27.3), an ordinary pixel exchangeable with that sample exceeds the cut with probability at most

$1 - q/100$ plus $1/N_0$.

Worked example 32.2: A 90th-percentile cut on ten natural scores

Ten natural pixels score 0.4, 1.1, 0.7, 2.3, 0.2, 0.9, 1.6, 0.5, 3.1, 1.3[A]. Sorted: 0.2, 0.4, 0.5, 0.7, 0.9, 1.1, 1.3, 1.6, 2.3, 3.1. The linear-interpolation percentile sits at position $0.9 \times (10 - 1) = 8.1$, counting from 0, between position 8 (2.3) and position 9 (3.1):

$$\text{cut} = 2.3 + 0.1 \times (3.1 - 2.3) = 2.3 + 0.1 \times 0.8 = 2.3 + 0.08 = 2.38.$$

One natural pixel (3.1) lies above it: 1/10, as designed[C]. A pixel scoring 5.0 is called man-made; one scoring 1.0 natural.

The price of an open level 1. It calls man-made anything nature did not show it: an unusual natural surface, haze it never saw, a sensor artefact. Those land in "man-made, class unknown". That is the right failure for a defence analyst (a false lead to review, not a missed net), but it means level 1's false alarms are only as rare as its training data of nature is complete.

32.4 What the separability check measures, and what it does not

`separability.py` bounds what a classical matcher can do before any training. For each scene (the four PRISMA bench scenes; three implant-bench host scenes each for EnMAP and AVIRIS-NG) it resamples the 411 library spectra to the scene's bands; for each spectrum $\mathbf{t}$ it draws 12 real background pixels $\mathbf{b}$ at least 8 pixels from any object and forms, at fills $a = 0.1,\ 0.25,\ 0.5,\ 1.0$,

$$\mathbf{x} = g \odot \big(a\mathbf{t} + (1 - a)\mathbf{b}\big) + \mathbf{n},$$

with g a smooth multiplicative gain within 10% of 1, $\odot$ band-by-band multiplication, and $\mathbf{n}$ per-band noise estimated from the scene. It then matches $\mathbf{d} = \mathbf{x} - g \odot \mathbf{m}$ ($\mathbf{m}$ the ring mean) by ACE against every *other* library spectrum (leave one spectrum out, Chapter 33), whitened with the scene's covariance, and takes the best match's family as level 2 and that family's group as level 1. Plantings per fill: PRISMA $411 \times 12 \times 4 = 19{,}728$, EnMAP and AVIRIS-NG $411 \times 12 \times 3 = 14{,}796$.

Table 32.1: The separability check, final on 31 October 2026[M], from the experiment's README and its three JSON files (paths below the table). L1 is "man-made called man-made / natural called natural"; L2 is "man-made planting named with the right family".

	L1			L2		
fill	PRISMA	EnMAP	AVIRIS-NG	PRISMA	EnMAP	AVIRIS-NG
0.10	0.6096 / 0.7822	0.4842 / 0.7173	0.5870 / 0.7683	0.3846	0.2263	0.3565
0.25	0.7724 / 0.8342	0.6635 / 0.7851	0.7603 / 0.8134	0.5523	0.4396	0.5278
0.50	0.8456 / 0.8551	0.8022 / 0.8434	0.8384 / 0.8459	0.6121	0.5770	0.5908
1.00	0.8703 / 0.8596	0.8711 / 0.8664	0.8696 / 0.8541	0.6374	0.6368	0.6174

Sources: `research/experiments/spectral-namer/README.md` and `separability_prisma.json`, `separability_enmap.json` and `separability_aviris_ng.json` in the same folder.

Read it carefully. **Level 1 in this check is closed**: it is the group of the nearest library spectrum, not a model of nature. So the table measures how often a closed, library-only level 1 gets the group right; the open level 1 of this chapter has not been measured yet, and the level 1 that was built is not open either: it is trained on planted man-made library spectra against natural ones (Chapter 35). At 10% fill, $1 - 0.6096 = 0.3904$ of man-made plantings are called natural on PRISMA and $1 - 0.4842 = 0.5158$ on EnMAP, and most 30 m targets are that small; at full fill all three sensors reach 0.87. Per family at fill 0.25 on PRISMA (the JSON's confusion counts[C]), oil is right 1537/1920 = 80%, plastic 70%, textile 69%,

wood and paper 62%, paint 45%, metal 41%, building 39%, roofing 31%, pavement 75/720 = 10%, and glass 0/48, because with one glass spectrum, leaving it out leaves none. (The experiment README says pavement "~12%"; the JSON says 10.4%, and 12% is pavement called building, 87 of 720.) Materials with their own absorption features separate; the use-based families of Chapter 31 do not.

The README's conclusion is level 2's class list: oil/fuel, plastic, textile/fabric, wood/paper, paint/coatings, metal, and concrete/asphalt (building and pavement merged, roofing dissolved into its materials), with glass/PV once spectra exist and camouflage and military coatings once field spectra arrive. And the planted spectra are library spectra, so the laboratory-to-satellite gap of Chapter 26 is absent: "these classical numbers are the baseline the learned namer must beat, mainly at 10-25% fill".

Common confusions

Is level 1 a man-made detector? It is a nature detector. It calls man-made whatever it cannot explain.
Then how is its threshold set without man-made examples? From natural pixels alone, by percentile, which fixes the false-alarm rate on ordinary pixels.
Does the separability table measure the open level 1? No. Its level 1 is closed (the nearest library spectrum's group). It bounds a library-only design.
Is camouflage always caught? Only if it is off the natural manifold in the bands the sensor measures. Material built to match vegetation across the whole spectrum would not be, and nothing spectral could catch it.

Check yourself

1. In worked example 32.1, a second net sits at $(0.45, 0.5)$. Compute its distance to the natural average and its unexplained score $|x + y - 1|$.
2. Recompute the cut of worked example 32.2 at the 80th percentile, and say how many of the ten exceed it.
3. With $N_0 = 10{,}000$ natural pixels and a 99th-percentile cut, bound the chance that an ordinary pixel is called man-made.
4. From Table 32.1, what share of natural plantings does the closed level 1 call man-made at fill 0.25, on PRISMA and on EnMAP?
5. Why can no amount of data on the natural side teach level 1 the difference between two man-made classes?

Check yourself: worked answers

1. Distance $\sqrt{(0.45-0.5)^2 + (0.5-0.5)^2} = \sqrt{0.0025} = 0.05$; unexplained $|0.45 + 0.5 - 1| = 0.05$. Even closer to the average, and still off the line.
2. Position $0.8 \times 9 = 7.2$, between 1.6 (position 7) and 2.3: $1.6 + 0.2 \times 0.7 = 1.6 + 0.14 = 1.74$. Two exceed it (2.3 and 3.1): 2/10.
3. $(1 - 0.99) + 1/10{,}000 = 0.01 + 0.0001 = 0.0101$.
4. PRISMA $1 - 0.8342 = 0.1658$, about 17%; EnMAP $1 - 0.7851 = 0.2149$, about 21%.
5. Level 1 learns only what nature looks like; telling man-made classes apart is level 2's job and needs man-made spectra.

Next: Chapter 33, matching against spectra instead of classifying into a list.

Chapter 33

Matching instead of classifying: the dual encoder

The one thing this chapter teaches

A closed classifier needs retraining for every new class; a matching model scores a pixel against each library spectrum through two encoders and a cosine, so a new library needs no retraining, and ACE is the special case whose encoders are "subtract the ring and whiten", which is why the fill drops out of it.

33.1 The question

The user asked: "do we have to retrain if we have another library?". For a closed classifier, yes; for a matching model, no. This chapter shows why, and what a matching model can and cannot promise.

33.2 A closed classifier

Definition: embedding, closed classifier

An **embedding** is a short vector a network computes from its input, $\boldsymbol{\zeta} = f(\mathbf{x})$, chosen during training so that useful comparisons become simple geometry. A **closed classifier** with K classes ends in a $K \times D$ matrix $\mathbf{V}$: its scores are $\mathbf{V}\boldsymbol{\zeta}$, one row per class, and it answers with the largest.

With $D = 2$, a pixel embedded at $\boldsymbol{\zeta} = (0.8,\ 0.6)$, and two known classes with rows painted metal $(1, 0)$ and tarpaulin $(0, 1)$, the scores are $\mathbf{V}\boldsymbol{\zeta} = (0.8,\ 0.6)$: painted metal. A third class, glass, has no row. Adding one means a new row of $\mathbf{V}$ whose values must be learned from glass examples: retraining.

33.3 A matching model

Definition: dual encoder

A **dual encoder** has two encoders: f turns a pixel (with its ring) into $\boldsymbol{\zeta} = f(\mathbf{x})$, and g turns a library spectrum into $\boldsymbol{\omega}_j = g(\mathbf{t}_j)$, both of length D. The match score is their cosine (rule 3.1),

$$s_j = \frac{\boldsymbol{\zeta} \cdot \boldsymbol{\omega}_j}{\|\boldsymbol{\zeta}\| \, \|\boldsymbol{\omega}_j\|},$$

and the model names the library entry with the largest s_j.

Worked example 33.1: A new library entry, no retraining

$\zeta = (0.8,\ 0.6)$, length $\sqrt{0.64 + 0.36} = 1$. Library embeddings, each of length 1: painted metal $(1,\ 0)$, tarpaulin $(0,\ 1)$, and a glass spectrum added after training, which g embeds at $(0.6,\ 0.8)$[A].

$$\begin{aligned} s_{\text{painted metal}} &= 0.8 \times 1 + 0.6 \times 0 = 0.80 \\ s_{\text{tarpaulin}} &= 0.8 \times 0 + 0.6 \times 1 = 0.60 \\ s_{\text{glass}} &= 0.8 \times 0.6 + 0.6 \times 0.8 = 0.48 + 0.48 = 0.96. \end{aligned}$$

Glass wins. Nothing was trained: g was run once on the new spectrum[C].

Why a new entry can work. The encoder g is one function shared by every spectrum. If it is smooth, spectra that are close give embeddings that are close: a bound $\|g(\mathbf{t}) - g(\mathbf{t'})\| \le L\,\|\mathbf{t} - \mathbf{t'}\|$ for some constant L is the usual way to say so (a **Lipschitz** bound)[A]. A new spectrum near the training spectra then lands near their embeddings, and its matches behave like theirs. A new spectrum unlike anything in training has no such promise. That is the honest limit of "no retraining": it covers new libraries of familiar kinds of material, not camouflage before anyone has measured it, which is why level 1 exists.

Level 1 is library-independent. Level 1 models nature (Chapter 32); it never sees a library. A new library changes only level 2's candidates.

33.4 ACE is a matching model

Let $\mathbf{W}$ be a whitening matrix (Chapter 7) and $\mathbf{m}$ the ring mean around the pixel. Take

$$f(\mathbf{x}) = \mathbf{W}(\mathbf{x} - \mathbf{m}), \qquad g(\mathbf{t}) = \mathbf{W}(\mathbf{t} - \mathbf{m}).$$

Then the squared cosine of the two embeddings is

$$s_j^2 = \frac{\big(\mathbf{W}(\mathbf{x} - \mathbf{m}) \cdot \mathbf{W}(\mathbf{t}_j - \mathbf{m})\big)^2}{\|\mathbf{W}(\mathbf{x} - \mathbf{m})\|^2\,\|\mathbf{W}(\mathbf{t}_j - \mathbf{m})\|^2},$$

which is ACE (rule 20.1) with $\mathbf{d} = \mathbf{t}_j - \mathbf{m}$ and $\mathbf{r} = \mathbf{x} - \mathbf{m}$, after $\mathbf{W}^\top\mathbf{W} = \mathbf{S}_0^{-1}$. The encoders are fixed, not learned, and g depends on the pixel through $\mathbf{m}$. A learned dual encoder replaces these two fixed maps by networks.

33.5 Why the fill drops out

Under the mixing model with a perfect ring, $\mathbf{x} - \mathbf{m} = a(\mathbf{t} - \mathbf{m})$ for the true target (Chapter 12). Write $\mathbf{u} = \mathbf{W}(\mathbf{t} - \mathbf{m})$. Then $f(\mathbf{x}) = a\mathbf{u}$ and $g(\mathbf{t}) = \mathbf{u}$.

$$\begin{aligned} \cos(a\mathbf{u},\ \mathbf{u}) &= \frac{(a\mathbf{u}) \cdot \mathbf{u}}{\|a\mathbf{u}\|\,\|\mathbf{u}\|} \\ &= \frac{a\,(\mathbf{u} \cdot \mathbf{u})}{|a|\,\|\mathbf{u}\|\,\|\mathbf{u}\|} && \textit{pull the number } a \textit{ out of the dot product and the length} \\ &= \frac{a}{|a|} \cdot \frac{\|\mathbf{u}\|^2}{\|\mathbf{u}\|^2} && \mathbf{u} \cdot \mathbf{u} = \|\mathbf{u}\|^2 \\ &= 1 && a > 0, \textit{ so } a/|a| = 1 \end{aligned}$$

More generally, for any candidate direction $\mathbf{v}$, $\cos(a\mathbf{u},\ \mathbf{v}) = \cos(\mathbf{u},\ \mathbf{v})$ for $a > 0$ by the same two steps. **The fill changes the length of $\mathbf{x} - \mathbf{m}$, not its direction, and a cosine sees only direction.**

Worked example 33.2: Whitened ACE at two fills, then with noise

Ring mean (grass) $(0.05,\ 0.40,\ 0.25)$, roof $(0.30,\ 0.32,\ 0.35)$, soil $(0.20,\ 0.25,\ 0.30)$, band standard deviations $(0.02,\ 0.04,\ 0.01)$, so $\mathbf{W}$ divides each band by its standard deviation[A].

$$\mathbf{u}_{\text{roof}} = \left(\tfrac{0.25}{0.02}, \tfrac{-0.08}{0.04}, \tfrac{0.10}{0.01}\right) = (12.5,\ -2,\ 10), \qquad \|\mathbf{u}_{\text{roof}}\|^2 = 156.25 + 4 + 100 = 260.25$$
$$\mathbf{u}_{\text{soil}} = \left(\tfrac{0.15}{0.02}, \tfrac{-0.15}{0.04}, \tfrac{0.05}{0.01}\right) = (7.5,\ -3.75,\ 5), \qquad \|\mathbf{u}_{\text{soil}}\|^2 = 56.25 + 14.0625 + 25 = 95.3125.$$

Fill 0.2: $f(\mathbf{x}) = 0.2\,\mathbf{u}_{\text{roof}} = (2.5,\ -0.4,\ 2)$, squared length $6.25 + 0.16 + 4 = 10.41$. Against soil, dot product $18.75 + 1.5 + 10 = 30.25$ and

$$s^2 = \frac{30.25^2}{95.3125 \times 10.41} = \frac{915.0625}{992.2031} = 0.9223.$$

Fill 0.5: $f(\mathbf{x}) = (6.25,\ -1,\ 5)$, squared length 65.0625, dot product 75.625, $s^2 = 5719.14/6201.27 = 0.9223$. The same, as derived. Against roof, $s^2 = 1$ at both.
Now add noise of one standard deviation per band, whitened $(1,\ 1,\ -1)$, the same at both fills. Fill 0.2: $f(\mathbf{x}) = (3.5,\ 0.6,\ 1)$, squared length 13.61, roof dot product $43.75 - 1.2 + 10 = 52.55$, $s^2 = 2761.50/3542.00 = 0.7796$. Fill 0.5: $f(\mathbf{x}) = (7.25,\ 0,\ 4)$, squared length 68.5625, roof dot product $90.625 + 0 + 40 = 130.625$, $s^2 = 17062.89/17843.39 = 0.9563$[C]. The fill drops out of the formula, not out of the accuracy: noise of a fixed size turns a small $f(\mathbf{x})$ further than a large one.

That is exactly the falling first column of Table 32.1: 0.87 at full fill, 0.61 at 10%.

33.6 Leave one spectrum out

Definition: leave-one-spectrum-out

To test naming without a separate test library, each library spectrum is planted in turn and matched against every *other* spectrum (**leave-one-spectrum-out**; `ace[e] = -1` in `separability.py`). The question becomes "is the nearest other spectrum from the same family?".

Matching a spectrum against itself would always score $s^2 = 1$ and measure nothing. The test is strict for families with few members: glass, with one spectrum, can never be right, and families 0.5° apart trade members freely.

Common confusions

Does a matching model never need retraining? Not for new spectra of familiar kinds. A spectrum unlike anything g learned from has no guarantee.
Is ACE a neural network? No; it is a matching model with fixed linear encoders. The learned dual encoder generalises it.
Does the fill not matter, then? It does not enter the formula. It does enter the accuracy, through noise.
Why is g allowed to depend on the pixel? Because the target's direction is relative to the background: $\mathbf{t} - \mathbf{m}$ changes with $\mathbf{m}$. A learned g can take the ring as a second input for the same reason.

Check yourself

1. With $\zeta = (0.6,\ 0.8)$, compute the cosine against painted metal, tarpaulin and glass of worked example 33.1. Which is named?
2. A closed classifier has $K = 10$ classes and $D = 64$. How many numbers are in $\mathbf{V}$, and how many must be learned to add one class?

3. Show that $\cos(a\mathbf{u},\ \mathbf{v}) = -\cos(\mathbf{u},\ \mathbf{v})$ when $a < 0$. Why is the shipped ACE signed (Chapter 20)?
4. In worked example 33.2, compute s^2 against soil for a 100% roof without noise.
5. Why can leave-one-spectrum-out never name a family with a single spectrum?

Check yourself: worked answers

1. $0.6 \times 1 + 0.8 \times 0 = 0.6$; $0.6 \times 0 + 0.8 \times 1 = 0.8$; $0.6 \times 0.6 + 0.8 \times 0.8 = 0.36 + 0.64 = 1.0$. Glass (it is the same direction).
2. $10 \times 64 = 640$; one new row, 64 numbers, learned from examples of the new class.
3. With $a < 0$, $a/|a| = -1$ in the second line of the derivation. A negative fill is impossible, so a residual pointing away from the target should not count as a match.
4. $f(\mathbf{x}) = \mathbf{u}_{\text{roof}}$, squared length 260.25; dot product with soil $93.75 + 7.5 + 50 = 151.25$; $s^2 = 151.25^2/(95.3125 \times 260.25) = 22876.56/24804.58 = 0.9223$.
5. Its only spectrum is the one left out, so no candidate of that family remains.

Next: Chapter 34, how a dual encoder is trained.

Chapter 34

Training the matcher: the contrastive objective

The one thing this chapter teaches

A dual encoder is trained by turning each pixel's cosines against a set of candidate spectra into probabilities with a softmax, and lowering minus the log of the true spectrum's probability; the gradient of that loss is "probability minus truth" for every candidate, which pulls the true match up and pushes each wrong one down in proportion to how tempting it was.

34.1 The problem

The encoders f and g of Chapter 33 are networks with millions of numbers. Training must arrange them so that a pixel's embedding is closest to the embedding of the spectrum planted in it. The training data come from the generator of Chapter 28: real background pixels with a library spectrum mixed in at a known fill, so each pixel arrives paired with its true spectrum.

34.2 Softmax with a temperature

Definition: softmax, temperature

Given scores $s_1, \dots, s_J$ and a positive number τ, the **temperature**, the **softmax** turns them into probabilities

$$p_j = \frac{e^{s_j/\tau}}{\sum_{i=1}^{J} e^{s_i/\tau}}.$$

Every p_j is positive and they add to 1. A small τ magnifies differences between scores; a large one flattens them.

Definition: contrastive loss (InfoNCE)

For a pixel whose true spectrum is candidate c, the **contrastive loss** is

$$L = -\ln p_c = -\frac{s_c}{\tau} + \ln \sum_{i=1}^{J} e^{s_i/\tau},$$

the cross-entropy of Chapter 28 with the candidates as classes. (The second form uses $\ln(a/b) = \ln a - \ln b$ and $\ln e^u = u$, rule 10.1.) It is called InfoNCE in the literature (Oord et al. 2018; Radford et al. 2021 used it to train image and text encoders together).

The difference from Chapter 28: there the classes were fixed outputs of a network; here they are whatever spectra are in the candidate set, scored by cosines. The same trained encoders work with a different candidate set tomorrow.

Worked example 34.1: Three candidates at three temperatures

The cosines of worked example 33.1: painted metal 0.80, tarpaulin 0.60, glass 0.96, and the truth is glass.
At $\tau = 0.5$: scores divided by τ are 1.6, 1.2, 1.92.

$$
\begin{aligned}
e^{1.6} &= 4.9530, \qquad e^{1.2} = 3.3201, \qquad e^{1.92} = 6.8210 \\
\text{sum} &= 4.9530 + 3.3201 + 6.8210 = 15.0941 \\
p &= (4.9530,\ 3.3201,\ 6.8210)/15.0941 = (0.3281,\ 0.2200,\ 0.4519) \\
L &= -\ln 0.4519 = 0.7943.
\end{aligned}
$$

The other two temperatures, the same way[C]:

τ	p (metal, tarpaulin, glass)	p_{glass}	L
1.0	(0.3342, 0.2736, 0.3922)	0.3922	0.9360
0.5	(0.3281, 0.2200, 0.4519)	0.4519	0.7943
0.1	(0.1642, 0.0222, 0.8135)	0.8135	0.2064

The same cosines give a confident answer at $\tau = 0.1$ and a timid one at $\tau = 1$. Cosines lie between -1 and 1, so without a small temperature the softmax could never become confident; that is why one is used.

34.3 The gradient: probability minus truth

One more derivative rule, checked: the derivative of e^x is e^x. At $x = 1$: $(e^{1.001} - e^1)/0.001 = (2.72100 - 2.71828)/0.001 = 2.72 \approx e = 2.718$.

Write $z_i = s_i/\tau$ for the scaled scores, and $y_i = 1$ for the true candidate, 0 otherwise. Differentiate $L = -z_c + \ln \sum_i e^{z_i}$ with respect to one z_j:

$$
\begin{aligned}
\frac{\partial}{\partial z_j}\big(-z_c\big) &= -y_j && -1 \textit{ if } j = c, \textit{ else } 0 \\
\frac{\partial}{\partial z_j} \ln \sum_i e^{z_i} &= \frac{1}{\sum_i e^{z_i}} \times e^{z_j} && \textit{chain rule: } \ln \textit{ then the sum; only term } j \textit{ depends on } z_j \\
&= p_j && \textit{the softmax} \\
\frac{\partial L}{\partial z_j} &= p_j - y_j. &&
\end{aligned}
$$

Training moves each candidate's score against the sign of $p_j - y_j$: the true one, where $p_c - 1 < 0$, goes up; every wrong one goes down by an amount equal to its own probability. A tempting wrong candidate is pushed hardest.

Worked example 34.2: The pull and the pushes

From worked example 34.1 at $\tau = 0.5$, $p - y = (0.3281 - 0,\ 0.2200 - 0,\ 0.4519 - 1) = (0.3281,\ 0.2200,\ -0.5481)$[C]. Check: the three add to $0.3281 + 0.2200 - 0.5481 = 0$, as they must, since the p_j add to 1 and the y_j add to 1. The glass score is pulled up by 0.5481; painted metal, the more tempting wrong answer, is pushed down by 0.3281, tarpaulin by 0.2200. The gradient reaches f and g through the chain rule, moving the embeddings themselves.

34.4 A batch: every other spectrum is a negative

In practice a batch holds B pixels and their B true spectra, and each pixel treats the other $B-1$ spectra of the batch as its wrong candidates (**in-batch negatives**). The cosines form a $B \times B$ table whose diagonal holds the true pairs.

Worked example 34.3: Two pixels, two spectra

Cosines (rows: pixels, columns: spectra), true pairs on the diagonal[A]:

$$\begin{pmatrix} 0.9 & 0.2 \\ 0.3 & 0.7 \end{pmatrix}, \qquad \tau = 0.5.$$

Row 1: $e^{1.8} = 6.0496$, $e^{0.4} = 1.4918$, sum 7.5415, $p_{\text{true}} = 6.0496/7.5415 = 0.8022$, $L_1 = -\ln 0.8022 = 0.2204$. Row 2: $e^{0.6} = 1.8221$, $e^{1.4} = 4.0552$, sum 5.8773, $p_{\text{true}} = 4.0552/5.8773 = 0.6900$, $L_2 = -\ln 0.6900 = 0.3711$. Mean loss $(0.2204 + 0.3711)/2 = 0.5915/2 = 0.2958$[C].

A trap specific to this library: false negatives. If the batch holds two spectra of the same material (a pavement and a building concrete, 0.51° apart), the loss tells the encoders to push them apart, which is learning to separate what physics cannot. One remedy is to count every spectrum of the true *class* as correct (sum their probabilities before taking the logarithm); it is the one the built namer uses (Chapter 35). With the defence class list, which merges concrete and asphalt, the worst of these pairs disappear by construction.

Analogy: a line-up with a memory

A witness picks a face from a line-up. Each time, she is told whether she was right, and her memory adjusts: the true face becomes more vivid, and each wrong face fades in proportion to how much she was drawn to it. The faces in the line-up change every time, but her memory of what makes a face recognisable carries over, which is why she can later pick out someone she has never seen in a line-up before.

34.5 What carries over from Chapter 28

The contrastive loss is a cross-entropy, so rule 28.1 applies: a perfectly trained matcher outputs the generator's posterior over the candidates, not the real world's. The label-shift correction and the generator gap of Chapter 28 hold unchanged. A matcher trained on library spectra planted in real pixels is judged, in the end, on real labelled objects (Chapter 29).

Common confusions

Is the temperature a fudge? It sets how sharp the probabilities are. It is chosen (or learned) once; it does not change which candidate is best.
Does a wrong candidate with a small probability get no push? It gets a small one, equal to its probability.
Is InfoNCE different from cross-entropy? It is cross-entropy over a candidate set built from the batch, with cosines as scores.
Why not train g only on the current library? Then a new library would again be unfamiliar. The point of g is to learn what makes spectra similar, from as many varied spectra as can be gathered.

Check yourself

1. Compute p and L for worked example 34.1 at $\tau = 0.5$ if the truth were painted metal instead.
2. In worked example 34.2, check that the gradient entries add to zero, and say what that means

for the scores' sum.

3. Recompute row 1 of worked example 34.3 at $\tau = 1$.
4. Show that adding the same number to every z_i does not change p.
5. Why does a false negative (two spectra of one material in a batch) teach the encoders something harmful?

Check yourself: worked answers

1. The same $p = (0.3281,\ 0.2200,\ 0.4519)$; now $L = -\ln 0.3281 = 1.1144$. A larger loss: the model preferred glass.
2. $0.3281 + 0.2200 - 0.5481 = 0$. A gradient step changes the scaled scores by amounts that add to zero: it redistributes, and adding a constant to all scores does nothing (next question).
3. $e^{0.9} = 2.4596$, $e^{0.2} = 1.2214$, sum 3.6810, $p_{\text{true}} = 2.4596/3.6810 = 0.6682$, $L = -\ln 0.6682 = 0.4032$.
4. $e^{z_i + c} = e^c e^{z_i}$; the factor e^c appears in every term of the top and the bottom and cancels.
5. The loss pushes their embeddings apart, so the encoders learn to separate spectra that physics cannot, which spends capacity on noise and makes real matches less stable.

34.6 Where Part VII leaves the design

The redesign did not stay a plan. It was written up as `docs/proposals/spectral-namer.md` and built on the same day, 31 October 2026, in `app/spectral_match/namer/`; its first run, N001, started training that day on an H100. The next chapter reads the code: what was built, where it departs from this Part, and the test it must pass. Its results were not in when this book was last updated.

Next: Chapter 35, the namer as built.

Chapter 35

The namer as built

The one thing this chapter teaches

The built namer is a two-tower network whose towers meet in one table of scaled cosines s_j, and since 31 October 2026 it reads *both* of its answers from that table: level 2 as a softmax over class scores $S_k = \ln \sum_{j \in k} e^{s_j}$ with the planted spectrum left out, and level 1 as $P(M) = \sigma\big(\ln \sum_{j \in \mathcal{M}} e^{s_j} - \ln \sum_{j \in \mathcal{N}} e^{s_j} + b\big)$, the share of the softmax mass that falls on man-made entries. Every gradient in the model is then a difference of softmax weights over library entries, which is why both levels pull the same embedding the same way.

35.1 What was built, in one table

piece	this Part's design	as built
pixel encoder f	pixel and ring	Indradhanu V2's stem on $\mathbf{x}$ and $\mathbf{r}$, then an MLP on $[\mathbf{z}_x, \mathbf{z}_r, \mathbf{z}_x - \mathbf{z}_r]$
spectrum encoder g	a second network	the same stem, a second MLP
level 1	a model of nature (Chapter 32)	*by matching*: natural library entries pooled as an eighth class, compared with the seven man-made ones (since run N001b; N001 used a linear head)
level 2 loss	InfoNCE over spectra (Chapter 34)	cross-entropy over *class* scores, planted spectrum left out
fill	man-made samples	every sample
held-out	20% of spectra; one class	a hash of the name, about 20% overall; textile/fabric in run N002

Sources: `app/spectral_match/namer/`, `configs/namer/N001b.json` and `N002.json`; the proposal's "As built". Where the proposal's text and the code differ, the code is what runs. **Status:** N001 (linear level 1) was stopped after one epoch; N001b and N002 were training on the H100 when this chapter was written, and **none of their results is in this book**.

Analogy: a quiz marked by grape, not by bottle

A wine course pours table wine, adds a splash of one cellar wine and asks two questions: was anything added that does not grow in a vineyard, and which grape. The bottle used for the splash is never on the answer card, so recognising that one bottle earns nothing. The student answers both questions the same way: by asking which bottles on the shelf the glass tastes most like, and whether those bottles stand on the vineyard shelf or the other one.

35.2 The model as two functions

Definition: Siamese network, two-tower model

A **Siamese network** runs two inputs through the *same* function and compares the outputs. A **two-tower model** has two encoders (towers) whose outputs are compared, usually by a cosine; they may share part of their weights.

Write ϕ_θ for the stem, the only shared part (17,825 numbers, $\mathbb{R}^N \to \mathbb{R}^{64}$ for any band count N, Section 35.3). The two towers are

$$f(\mathbf{x}, \mathbf{r}) = \frac{\mathbf{W}_e\,\mathbf{h}}{\|\mathbf{W}_e\,\mathbf{h}\| + \epsilon} \in \mathbb{R}^{128}, \qquad \mathbf{h} = \mathrm{MLP}_p\big([\phi_\theta(\mathbf{x}),\ \phi_\theta(\mathbf{r}),\ \phi_\theta(\mathbf{x}) - \phi_\theta(\mathbf{r})]\big) \in \mathbb{R}^{256}, \tag{35.1}$$

$$g(\mathbf{t}) = \frac{\mathrm{MLP}_\ell(\phi_\theta(\mathbf{t}))}{\|\mathrm{MLP}_\ell(\phi_\theta(\mathbf{t}))\| + \epsilon} \in \mathbb{R}^{128}, \tag{35.2}$$

with the fill estimate $\hat{a} = \sigma(\mathbf{w}_f \cdot \mathbf{h} + c_f)$ read from the same $\mathbf{h}$ (`model.py`: `forward_pixel`, `forward_library`). They meet in

$$s_j = \frac{f(\mathbf{x}, \mathbf{r}) \cdot g(\mathbf{t}_j)}{\tau} = \frac{\cos\big(f,\ g(\mathbf{t}_j)\big)}{\tau}, \qquad j = 1, \dots, L, \tag{35.3}$$

because both have length 1. **Notation:** in this chapter s_j is the *scaled* score, already divided by τ, as the code's variable `s` in `class_scores`; Chapter 34 wrote s_j for the bare cosine and divided by τ inside the softmax. The stem is Siamese three ways: $\mathbf{x}$, $\mathbf{r}$ and every $\mathbf{t}_j$ go through the same ϕ_θ.

Why the towers differ. The pixel is a mixture in context, $\mathbf{x} = \mathbf{g} \odot (a\mathbf{t} + (1-a)\mathbf{b}) + \mathbf{n}$; the library spectrum is $\mathbf{t}$ alone. Subtract the gained ring $\mathbf{r}' = \mathbf{g} \odot \mathbf{r}$ from the pixel:

$$\begin{aligned} \mathbf{x} - \mathbf{r}' &= \mathbf{g} \odot \big(a\mathbf{t} + (1-a)\mathbf{b}\big) + \mathbf{n} - \mathbf{g} \odot \mathbf{r} && \textit{definitions} \\ &= \mathbf{g} \odot \big(a\mathbf{t} + \mathbf{b} - a\mathbf{b} - \mathbf{r}\big) + \mathbf{n} && \textit{expand } (1-a)\mathbf{b} \\ &= \mathbf{g} \odot \big(a(\mathbf{t} - \mathbf{b}) + (\mathbf{b} - \mathbf{r})\big) + \mathbf{n} && \textit{group the } a \textit{ terms.} \end{aligned}$$

The target enters only through $a(\mathbf{t} - \mathbf{b})$: its contrast with the background, scaled by the fill. The second term $\mathbf{b} - \mathbf{r}$ is how much the pixel's own background differs from its ring, the clutter. So the pixel tower must undo a scaling by a and see through clutter, using $\mathbf{z}_x$, $\mathbf{z}_r$ and their difference; the library tower faces neither. With $\mathbf{b} = (0.10, 0.30, 0.25)$, $\mathbf{r} = (0.11, 0.29, 0.25)$, $\mathbf{t} = (0.40, 0.35, 0.05)$, $a = 0.25$, $\mathbf{g} = (1.02, 1.00, 0.98)$ and no noise[A], both sides give $\mathbf{x} - \mathbf{r}' = (0.0663,\ 0.0225,\ -0.0490)$[C]. Forcing one tower on both inputs would make the library side "un-mix" something never mixed.

Why a new library needs no retraining. g is a function of the spectrum alone. A new spectrum $\mathbf{t}_{L+1}$ adds one row $g(\mathbf{t}_{L+1})$ to the table and one term to the sums below; nothing is refitted. Whether that row lands well is another matter, and the open-set run N002 tests it.

35.2.1 Counting the parameters as a formula

A linear layer $i \to o$ has $io + o$ parameters. With stem width D, hidden width H and embedding E:

$$\begin{aligned} \mathrm{MLP}_p:\ 3D \to H \to H \to H &= (3DH + H) + 2(H^2 + H) = 3DH + 2H^2 + 3H \\ \text{heads } (H \to E,\ H \to 1,\ H \to 1) &= HE + E + 2(H + 1) = HE + E + 2H + 2 \\ \mathrm{MLP}_\ell:\ D \to H \to H \to E &= (DH + H) + (H^2 + H) + (HE + E) \\ &= DH + H^2 + 2H + HE + E \\ \text{towers, summed} &= 4DH + 3H^2 + 7H + 2HE + 2E + 2. \end{aligned}$$

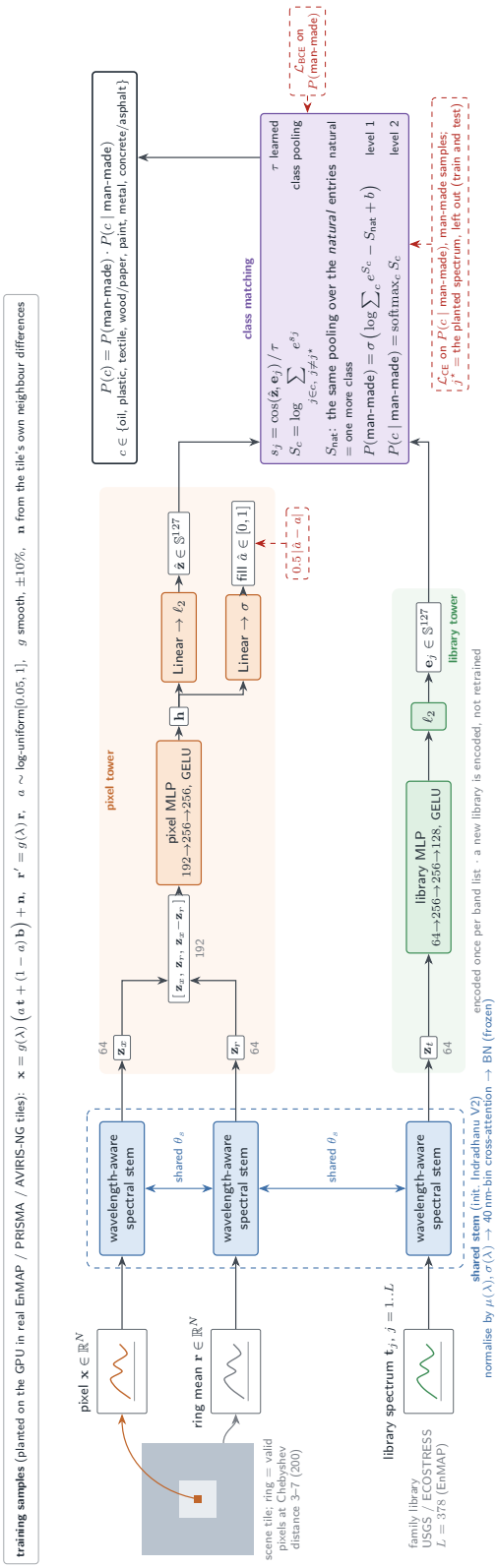


Figure 35.1: The publication schematic of the namer, from `research/experiments/spectral-namer/figures/namer_network.pdf`, redrawn on 31 October 2026: level 1 is computed in the class-matching box, $P(\text{man-made}) = \sigma(\text{LSE}_{c<7} S_c - S_{\text{natural}} + b)$ with the natural entries pooled as an eighth class (Section 35.6, Figure 35.2); the unused linear head is not drawn.

(The last line collects like terms.) The stem at $d = 32$ is $11{,}617 + 97D$ (Table 35.1), the BatchNorm $2D$, and the two scalars $\ln\tau$ and b add 2. At $D = 64$, $H = 256$, $E = 128$:

$$\begin{aligned} 4DH &= 4\times 64\times 256 = 65{,}536, \qquad 3H^2 = 3\times 65{,}536 = 196{,}608,\\ 7H &= 7\times 256 = 1{,}792, \qquad 2HE = 2\times 256\times 128 = 65{,}536, \qquad 2E+2 = 258,\\ \text{towers} &= 65{,}536 + 196{,}608 + 1{,}792 + 65{,}536 + 258 = 329{,}730,\\ \text{total} &= 329{,}730 + 17{,}825 + 128 + 2 = 347{,}685. \end{aligned}$$

The code reports 347,685[C]. (Earlier text, including this chapter's first version, gave 347,684: that was before the level-1 bias b existed.) The proposal's "~1-3 M" is three to nine times the model. The formula also predicts 117,285 for $H = 128$, $E = 64$, which the code confirms[C].

35.3 The stem ϕ_θ, step by step

The stem (`app/foundation_models/components/spectral_reduced.py`, `SpectralReducedCompressor`) is copied from the released Indradhanu V2 (FTH3 epoch 5) with $D = 64$ channels, token size $d = 32$, 4 heads, 40 nm bins and band weighting by width. It reads a spectrum $\mathbf{x} \in \mathbb{R}^N$ with wavelengths $\lambda_1, \dots, \lambda_N$.

Step 0, normalise by wavelength. $v_b = (x_b - \mu(\lambda_b))/\sigma(\lambda_b)$, with μ, σ curves over wavelength stored in the checkpoint and read by linear interpolation (Chapter 15 explains why absolute reflectance must survive). At 864 nm between stored points 860 nm ($\mu = 0.30$, $\sigma = 0.10$) and 870 nm (0.32, 0.12)[A]: $t = 4/10 = 0.4$, $\mu = 0.30 + 0.4\times 0.02 = 0.308$, $\sigma = 0.10 + 0.4\times 0.02 = 0.108$; a reading of 0.35 becomes $(0.35 - 0.308)/0.108 = 0.042/0.108 = 0.389$[C].

Step 1, encode each wavelength. $\mathrm{enc}(\lambda) \in \mathbb{R}^{32}$: sines then cosines of $2\pi\lambda/p_i$ for 16 periods $p_i = 10\times 1000^{i/15}$ nm, a geometric ladder with ratio $1000^{1/15} = 1.5849$ from 10 to 10,000 nm[C]. At 1000 nm the slowest pair is $\sin(0.6283) = 0.5878$, $\cos(0.6283) = 0.8090$; the fastest sine is $\sin(200\pi) = 0$[C].

Step 2, bins. Band b goes to bin $g(b) = \min\big(52, \max(0, \lfloor(\lambda_b - 400)/40\rfloor)\big)$; there are $G = \lceil 2100/40\rceil = \lceil 52.5\rceil = 53$ bins. Write $O_{bg} = 1$ if $g(b) = g$, else 0: a $N \times G$ matrix.

Step 3, band widths as quadrature weights. Band b gets weight c_b, the nanometres it stands for: half of each neighbouring gap, the single gap at either end, gaps capped at 20 nm. A width-weighted mean $\sum_b c_b v_b / \sum_b c_b$ is the trapezoid-like approximation of $\frac{1}{\Lambda}\int v(\lambda)\,d\lambda$ over the covered range Λ, so two sensors with different band spacings average the same curve to nearly the same number. For bands at 410, 430, 445, 470 nm the gaps 20, 15, 25 are capped to 20, 15, 20, and $c = (20,\ 17.5,\ 17.5,\ 20)$[C]. The cap matters at a water gap: bands at 1340 and 1460 nm would otherwise each claim 60 nm they never measured.

Step 4, bin tokens. Each band has a weight vector $\mathbf{k}(\lambda_b) = \mathrm{MLP}_k(\mathrm{enc}(\lambda_b)) \in \mathbb{R}^{32}$, and

$$\boldsymbol{\tau}_g = \frac{\sum_b O_{bg}\, c_b\, v_b\, \mathbf{k}(\lambda_b)}{\sum_b O_{bg}\, c_b} \in \mathbb{R}^{32}. \tag{35.4}$$

Rule 35.1: Folding the bins into one matrix product

Let $\mathbf{K} \in \mathbb{R}^{N\times Gd}$ have entries $K_{b,(g,i)} = O_{bg}\, k_i(\lambda_b)$. Then the numerator of Equation (35.4) for every bin is the row vector $(c \odot v)\,\mathbf{K} \in \mathbb{R}^{Gd}$, read in blocks of d.

$$\begin{aligned}
\big[(c \odot v)\,\mathbf{K}\big]_{(g,i)} &= \sum_b (c_b v_b)\, K_{b,(g,i)} && \textit{row vector times matrix} \\
&= \sum_b c_b v_b\, O_{bg}\, k_i(\lambda_b) && \textit{definition of } \mathbf{K} \\
&= \Big[\sum_b O_{bg}\, c_b\, v_b\, \mathbf{k}(\lambda_b) \Big]_i && \textit{entry } i \textit{ of the bin-}g \textit{ sum.}
\end{aligned}$$

For P pixels this is one product $(P \times N)(N \times Gd)$; nothing of size $P \times N \times d$ is ever built.

Worked example 35.1: Two bins of two bands, $d = 2$

Bands and widths of Step 3; $v = (0.5, -0.2, 1.0, 0.4)$; $\mathbf{k} = (1,0), (0.5,0.5), (0,1), (1,-1)$[A]. Then $c \odot v = (10,\ -3.5,\ 17.5,\ 8)$, the bin weights are $20 + 17.5 = 37.5$ twice, and

$$\mathbf{K} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0.5 & 0.5 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & -1 \end{pmatrix}, \qquad (c \odot v)\,\mathbf{K} = (8.25,\ -1.75,\ 8.0,\ 9.5).$$

Column by column: $10\times1+(-3.5)\times0.5 = 8.25$; $(-3.5)\times0.5 = -1.75$; $8\times1 = 8$; $17.5\times1+8\times(-1) = 9.5$. Dividing by 37.5: $\boldsymbol{\tau}_0 = (0.2200,\ -0.0467)$, $\boldsymbol{\tau}_1 = (0.2133,\ 0.2533)$. A loop over bands gives the same[C].

Each token then gets its bin's position added, $\boldsymbol{\tau}_g \leftarrow \boldsymbol{\tau}_g + \mathrm{Lin}(\mathrm{enc}(420 + 40g))$.

Step 5, attention over bins. 64 learned queries, projected and split into 4 heads of 8 numbers, attend over the 53 tokens: for head h and query c, $\alpha_{cg} = \exp(\mathbf{q}_c \cdot \mathbf{k}_g/\sqrt{8}) \big/ \sum_{g' \in \mathscr{V}} \exp(\mathbf{q}_c \cdot \mathbf{k}_{g'}/\sqrt{8})$ over the bins $\mathscr{V}$ that hold a valid band, and the answer is $\sum_g \alpha_{cg} \mathbf{v}_g$. A masked bin has weight exactly 0, and the surviving weights keep their ratios (Chapter 34's softmax, with $e^{-\infty} = 0$).

Worked example 35.2: A masked bin cannot leak

Head size 2: $\mathbf{q} = (1, 0.5)$; keys $(1,0), (0,1), (2,2)$; values $(0.2, 0.4), (0.6, -0.2), (9, 9)$; the third bin empty[A]. Scores $\mathbf{q} \cdot \mathbf{k}/\sqrt{2} = (1,\ 0.5,\ 3)/1.4142 = (0.7071,\ 0.3536,\ 2.1213)$; masked, the third is $-\infty$. $e^{0.7071} = 2.0281$, $e^{0.3536} = 1.4242$, sum 3.4523; weights $(0.5875,\ 0.4125,\ 0)$; answer $0.5875\,(0.2, 0.4) + 0.4125\,(0.6, -0.2) = (0.3650,\ 0.1525)$, as PyTorch's attention returns[C]. The 9s never enter.

Step 6, channels and skip. Each query's answer, joined over heads to 32 numbers, is mapped by Lin_o and a $32 \to 1$ layer to one number: 64 channels. A skip path adds $\sum_b c_b v_b\, \mathbf{w}(\lambda_b) / \sum_b c_b$ with $\mathbf{w}(\lambda) = \mathrm{MLP}_w(\mathrm{enc}(\lambda)) \in \mathbb{R}^{64}$: every band reaches every channel directly, with a weight computed from its wavelength.

Step 7, frozen BatchNorm. $y_c = \gamma_c (z_c - m_c)/\sqrt{s_c^2 + 10^{-5}} + \beta_c$ with V2's running mean m_c and variance s_c^2 held fixed (`SpectralNamer.train()` keeps the layer in evaluation mode) and γ, β trained. With $m = 0.4$, $s^2 = 0.25$, $\gamma = 1.5$, $\beta = -0.1$, $z = 0.9$[A]: $1.5 \times 0.5/0.50001 - 0.1 = 1.39997$[C]. It is an affine map, the same for pixels, rings and library spectra; in training mode it would have normalised each of the nine encode calls per step by its own batch, and evaluation by a running blend of all nine.

piece	layers	parameters
$\mathbf{k}(\lambda)$	$32 \to 64 \to 32$	$(32 \cdot 64 + 64) + (64 \cdot 32 + 32) = 4{,}192$
$\mathbf{w}(\lambda)$	$32 \to 64 \to D$	$(32 \cdot 64 + 64) + (64D + D) = 2{,}112 + 65D$
bin position	$32 \to 32$	1,056
queries	$D \times 32$	$32D$
$\mathrm{Lin}_{q,k,v,o}$	$4 \times (32 \to 32)$	4,224
to channel	$32 \to 1$	33
total		$4{,}192 + 2{,}112 + 1{,}056 + 4{,}224 + 33 + (65 + 32)D = 11{,}617 + 97D$

Table 35.1: The stem's parameters at token size 32. $D = 64$: 11,617+6,208 = 17,825; $D = 32$: 11,617+3,104 = 14,721. Both agree with the code[C]. No term depends on N: the stem reads any band list.

35.4 The class score

Definition: class score, logsumexp

For one pixel with scaled cosines s_j (Equation (35.3)) against the library entries j, and a class k (a set of entries), the **class score** is

$$S_k = \ln \sum_{j \in k} e^{s_j},$$

the **logsumexp** of its members' scores. Level 2's probability is the softmax over the classes, $P(k \mid M) = e^{S_k} / \sum_{k'} e^{S_{k'}}$.

Logsumexp is a smooth maximum: for m members it lies between the largest s_j and the largest plus $\ln m$. Check with two members at 1.92 and 1.80: $\ln(e^{1.92} + e^{1.80}) = \ln(6.8210 + 6.0496) = \ln 12.8706 = 2.5549$, between 1.92 and $1.92 + \ln 2 = 2.6131$[C]. The temperature starts at $\tau = 0.07$ and is learned as $\ln \tau$.

Rule 35.2: A class's probability is the sum of its members' probabilities

With $q_j = e^{s_j} / \sum_i e^{s_i}$ the softmax over all entries in play, and every entry in exactly one class, $P(k \mid M) = \sum_{j \in k} q_j$.

$$\begin{aligned} P(k \mid M) &= \frac{e^{S_k}}{\sum_{k'} e^{S_{k'}}} && \textit{definition} \\ &= \frac{\sum_{j \in k} e^{s_j}}{\sum_{k'} \sum_{j \in k'} e^{s_j}} && e^{\ln u} = u, \textit{ rule 10.1} \\ &= \frac{\sum_{j \in k} e^{s_j}}{\sum_i e^{s_i}} && \textit{the classes split the entries} \\ &= \sum_{j \in k} q_j. \end{aligned}$$

So the loss $-\ln P(c \mid M)$ for the true class c is minus the log of the *summed* probability of the true class's members: exactly the remedy Chapter 34 proposed for false negatives ("count every spectrum of the true class as correct"). Two concrete spectra of one material are no longer pushed apart.

Leaving the planted spectrum out. Each sample records which library entry was planted; `class_scores(…, exclude=entry)` sets that entry's s_j to $-\infty$ before the logsumexp, in training and in evaluation, for man-made and natural plantings alike. It is the separability check's leave-one-spectrum-out (Chapter 32) used as a training rule.

Worked example 35.3: Glass with a second glass spectrum

Take worked example 34.1's cosines at $\tau = 0.5$ and add a second glass spectrum g_2 at 0.90[A]. The pixel was planted with g_1. Scaled scores: g_1 1.92, g_2 1.80, painted metal 1.60, tarpaulin 1.20; $e^s = 6.8210,\ 6.0496,\ 4.9530,\ 3.3201$.
Per-spectrum InfoNCE, truth g_1, g_2 a "negative":

$$q_{g_1} = \frac{6.8210}{6.8210 + 6.0496 + 4.9530 + 3.3201} = \frac{6.8210}{21.1437} = 0.3226, \qquad L = -\ln 0.3226 = 1.1313.$$

Class level, g_1 kept:

$$P(\text{glass}) = q_{g_1} + q_{g_2} = 0.3226 + 0.2861 = 0.6087 \qquad \textit{rule 35.2}$$
$$L = -\ln 0.6087 = 0.4964.$$

Class level, g_1 left out (what the namer trains on):

$$P(\text{glass}) = \frac{6.0496}{6.0496 + 4.9530 + 3.3201} = \frac{6.0496}{14.3227} = 0.4224, \qquad L = -\ln 0.4224 = 0.8619.$$

All three agree with the code[C]. The per-spectrum loss punishes the pixel for being close to g_2; the class loss rewards it. Leaving g_1 out then removes the easy route: the loss can only fall by bringing g_2, the *other* glass, closer.

35.5 The gradient of the class loss

Write c for the true class and $L_2 = -S_c + \ln \sum_k e^{S_k} = -\ln \sum_{j \in c} e^{s_j} + \ln \sum_i e^{s_i}$ (the second form by rule 35.2's second line). Differentiate with respect to one entry's s_j, using the chain rule exactly as in Chapter 34:

$$\frac{\partial}{\partial s_j} \ln \sum_i e^{s_i} = \frac{e^{s_j}}{\sum_i e^{s_i}} = q_j \qquad \textit{only term } j \textit{ depends on } s_j$$
$$\frac{\partial}{\partial s_j}\Big(-\ln \sum_{i \in c} e^{s_i} \Big) = -\mathbb{1}[j \in c]\, w_j, \qquad w_j = \frac{e^{s_j}}{\sum_{i \in c} e^{s_i}} \qquad \textit{the same, inside class } c$$
$$\frac{\partial L_2}{\partial s_j} = q_j - \mathbb{1}[j \in c]\, w_j.$$

With a one-member class, $w_j = 1$ and this is Chapter 34's "probability minus truth". With several members, the pull of -1 is *shared*, in proportion w_j to how close each member already is. The entries still sum to zero: $\sum_j q_j - \sum_{j \in c} w_j = 1 - 1 = 0$.

Worked example 35.4: Shared pull

In worked example 35.3 with g_1 kept: $q = (0.3226,\ 0.2861,\ 0.2343,\ 0.1570)$; inside glass, $w = (6.8210,\ 6.0496)/12.8706 = (0.5300,\ 0.4700)$. So

$$g_1 : 0.3226 - 0.5300 = -0.2074, \qquad g_2 : 0.2861 - 0.4700 = -0.1839,$$
$$\text{painted metal} : +0.2343, \qquad \text{tarpaulin} : +0.1570,$$

summing to $-0.2074 - 0.1839 + 0.2343 + 0.1570 = 0.0000$[C]. Both glasses are pulled up (negative gradient, so gradient descent raises their scores), g_1 a little more. In the per-spectrum loss of worked example 34.2 g_2 would have been pushed *down* by its $q = 0.2861$.

35.6 Level 1 by matching

35.6.1 The evidence that forced the change

Run N001 used a linear head, $\ell = \mathbf{w} \cdot \mathbf{h} + b_0$, $P(M) = \sigma(\ell)$. Two measurements were reported on 31 October 2026; neither is yet in a repository file[M]:

- N001 after epoch 1 on the H100: level-1 accuracy about 0.50 at every evaluation fill on all three sensors (EnMAP 0.52 at fill 0.25, 0.50 at fill 1.0), while level-2 class accuracy was 0.60 to 0.68.
- A laptop CPU test of 150 steps at full fill on the EnMAP smoke shards, the two level-1 forms side by side:

level 1	BCE	accuracy	man-made recall at ACE's false-alarm rate
chance	$\ln 2 = 0.6931$	0.50	–
linear head on $\mathbf{h}$	0.64	0.65	0.41
by matching	0.43	0.80	0.74

$e^{-\mathrm{BCE}}$ is the geometric mean of the probability given to the truth: $e^{-0.6931} = 0.5000$, $e^{-0.64} = 0.5273$, $e^{-0.43} = 0.6505$[C]. For scale, ACE at fill 1.0 on EnMAP calls 13.36% of natural plantings man-made and recalls 0.8711 of man-made ones (Table 32.1's source JSON)[C]: after 150 steps neither form had reached ACE. N001 was stopped and replaced by N001b, identical except for `"level1": "matching"` (`configs/namer/N001b.json`); N002 uses it too.

The telling detail is the pair at fill 1.0: the planted pixel there *is* a library spectrum, level 2 named its class two times in three from an embedding computed from the same $\mathbf{h}$, and yet a linear read-out of $\mathbf{h}$ could not say "man-made". The information was present; the read-out was wrong.

35.6.2 The rule

Natural library entries get class index 7, and the class scores are pooled over eight classes instead of seven (`train.py`, `forward`). With $S_0, \dots, S_6$ the man-made class scores, $S_7 = S_{\mathrm{nat}}$ and a learned scalar b (`l1_bias`, started at 0), `level1_logit` computes

$$\ell = \ln \sum_{c=0}^{6} e^{S_c} - S_{\mathrm{nat}} + b, \qquad P(M) = \sigma(\ell) = \frac{1}{1 + e^{-\ell}}, \tag{35.5}$$

and level 2 keeps the first seven columns. Two identities turn this into something you can picture.

Rule 35.3: A logsumexp of logsumexps is one logsumexp

If the sets k split a set $\mathscr{M}$, then $\ln \sum_k e^{S_k} = \ln \sum_{j \in \mathscr{M}} e^{s_j}$.

$$\begin{aligned} \ln \sum_k e^{S_k} &= \ln \sum_k \exp\Big(\ln \sum_{j \in k} e^{s_j} \Big) && \text{definition of } S_k \\ &= \ln \sum_k \sum_{j \in k} e^{s_j} && e^{\ln u} = u \\ &= \ln \sum_{j \in \mathscr{M}} e^{s_j} && \text{the sets split } \mathscr{M}. \end{aligned}$$

Check: $(1, 2, 0.5, 3, -1)$ in groups $\{1,2\}, \{0.5\}, \{3,-1\}$ gives 3.4723 both ways[C]. A class with no entry in play has $S_k = -\infty$ and adds $e^{-\infty} = 0$.

Rule 35.4: Level 1 is a share of softmax mass

With $A = \sum_{j\in\mathcal{M}} e^{s_j}$ over the man-made entries in play and $B = \sum_{j\in\mathcal{N}} e^{s_j}$ over the natural ones, $P(M) = \dfrac{e^b A}{e^b A + B}$.

$$\begin{aligned}
\ell &= \ln A - \ln B + b && \textit{rule 35.3 and the definition of } S_{\text{nat}}\\
e^{\ell} &= e^b A/B && \textit{exponentiate}\\
\sigma(\ell) &= \frac{e^{\ell}}{e^{\ell}+1} && \textit{multiply } \tfrac{1}{1+e^{-\ell}} \textit{ above and below by } e^{\ell}\\
&= \frac{e^b A/B}{e^b A/B + 1} = \frac{e^b A}{e^b A + B} && \textit{multiply above and below by } B.
\end{aligned}$$

So $P(M)$ is how much of the pixel's softmax over the whole library sits on man-made entries, with the man-made side weighted by e^b. It is a nearest-prototype rule (as in Snell, Swersky and Zemel 2017, in the bibliography) with every library spectrum a prototype; level 2 is the same rule restricted to the man-made side.

35.6.3 The gradient through the logsumexp

Level 1's loss is the binary cross-entropy $L_1 = -[y \ln p + (1-y)\ln(1-p)]$, $p = \sigma(\ell)$, $y = 1$ for man-made.

Slope in the logit. With $\sigma'(\ell) = p(1-p)$ (Chapter 34):

$$\begin{aligned}
\frac{\partial L_1}{\partial \ell} &= -\Big[y\frac{p(1-p)}{p} - (1-y)\frac{p(1-p)}{1-p}\Big] && \textit{chain rule, slope of } \ln u \textit{ is } 1/u\\
&= -[y(1-p) - (1-y)p] = -[y - yp - p + yp] = p - y && \textit{cancel, expand.}
\end{aligned}$$

Slope of the logit in each entry. Only $\ln A$ contains a man-made s_j; only $-\ln B$ a natural one:

$$\frac{\partial \ell}{\partial s_j} = \frac{e^{s_j}}{A} = u_j \quad (j \in \mathcal{M}), \qquad \frac{\partial \ell}{\partial s_j} = -\frac{e^{s_j}}{B} = -v_j \quad (j \in \mathcal{N}), \qquad \frac{\partial \ell}{\partial b} = 1,$$

where u is the softmax within the man-made entries and v within the natural ones. By the chain rule,

$$\frac{\partial L_1}{\partial s_j} = (p-y)\,u_j \ (j \in \mathcal{M}), \qquad \frac{\partial L_1}{\partial s_j} = -(p-y)\,v_j \ (j \in \mathcal{N}), \qquad \frac{\partial L_1}{\partial b} = p - y. \tag{35.6}$$

They sum to $(p-y)\sum u_j - (p-y)\sum v_j = (p-y)(1-1) = 0$: as in level 2, a step moves score between entries, it cannot lift them all. Compare worked example 35.4: the pull is again shared in proportion to closeness, now within each side.

Into the embedding. $s_j = \mathbf{z}\cdot\mathbf{e}_j/\tau$ gives $\partial s_j/\partial \mathbf{z} = \mathbf{e}_j/\tau$, so

$$\frac{\partial L_1}{\partial \mathbf{z}} = \frac{p-y}{\tau}\Big(\sum_{j\in\mathcal{M}} u_j \mathbf{e}_j - \sum_{j\in\mathcal{N}} v_j \mathbf{e}_j\Big) :$$

for a man-made truth ($p - y < 0$), gradient descent moves $\mathbf{z}$ towards the closeness-weighted average of the man-made entries and away from that of the natural ones.

Into the temperature. With the cosines fixed, $s_j = \cos_j/\tau$, so $\partial s_j/\partial \ln\tau = -s_j$ and

$$\frac{\partial \ell}{\partial \ln \tau} = -\Big(\sum_{j\in\mathcal{M}} u_j s_j - \sum_{j\in\mathcal{N}} v_j s_j\Big),$$

minus the gap between the two sides' softmax-weighted mean scores. τ is shared by both levels, so level 1 now also has a say in how sharp the matching is.

Worked example 35.5: Level 1 by matching with three entries

Truth man-made. Entries in play: plastic $s = 6.0$, metal $s = 5.0$, soil (natural) $s = 5.5$; $b = 0$[A].

$$\ln A = \ln(e^6 + e^5) = 6 + \ln(1 + e^{-1}) = 6 + 0.3133 = 6.3133 \quad \text{(factor out } e^6\text{)}$$

$$\ell = 6.3133 - 5.5 + 0 = 0.8133, \qquad p = \frac{1}{1 + e^{-0.8133}} = \frac{1}{1.4434} = 0.6928$$

$$\text{check, rule 35.4: } p = \frac{403.4288 + 148.4132}{403.4288 + 148.4132 + 244.6919} = \frac{551.8420}{796.5339} = 0.6928$$

$$L_1 = -\ln 0.6928 = 0.3670, \qquad p - y = -0.3072$$

$$u = (403.4288,\ 148.4132)/551.8420 = (0.7311,\ 0.2689), \qquad v = (1)$$

$$\partial L_1/\partial s = (-0.3072 \times 0.7311,\ -0.3072 \times 0.2689,\ +0.3072 \times 1)$$

$$= (-0.2246,\ -0.0826,\ +0.3072)$$

summing to 0. Autograd on `level1_logit` agrees[C]. For the temperature: $\sum u_j s_j = 0.7311 \times 6 + 0.2689 \times 5 = 5.7311$, so $\partial \ell / \partial \ln \tau = -(5.7311 - 5.5) = -0.2311$ and $\partial L_1 / \partial \ln \tau = (-0.3072)(-0.2311) = 0.0710$[C]: a positive slope, so descent lowers $\ln \tau$, sharpening the scores, which widens the man-made lead here.

35.6.4 Why matching helps: three reasons and an offset

A hypothesis consistent with the evidence above, not a proof.

1. A hyperplane can be the wrong shape. The linear head says man-made when $\mathbf{w} \cdot \mathbf{h} + b_0 > 0$, one flat cut. Nature is several clusters (vegetation, soils, rocks, water, snow, dry plant matter) with man-made materials between them.

Worked example 35.6: Four points no line separates

Natural at $(1, 0)$ and $(-1, 0)$; man-made at $(0, 1)$ and $(0, -1)$[A]. If $\mathbf{w} \cdot \mathbf{h} + b_0$ were negative at both natural points and positive at both man-made ones:

$$w_1 + b_0 < 0 \text{ and } -w_1 + b_0 < 0 \;\Rightarrow\; 2b_0 < 0 \qquad \textit{add the two}$$

$$w_2 + b_0 > 0 \text{ and } -w_2 + b_0 > 0 \;\Rightarrow\; 2b_0 > 0 \qquad \textit{add the two}$$

a contradiction. Matching with the four points as entries and $\tau = 0.1$: a pixel at 80° has cosines ± 0.9848 to the man-made pair and ± 0.1736 to the natural pair, so $\ell = \text{LSE}(9.848, -9.848) - \text{LSE}(1.736, -1.736) = 9.848 - 1.767 = 8.081$, man-made; at 190°, $\ell = -8.081$; at 45°, exactly between, $\ell = 0$[C].

The MLP under the head could learn features that make one cut work, but it must find them from one bit per sample. Matching needs no such features.

2. One table for both levels. Level 2's class loss already shapes $\mathbf{z}$ and $\mathbf{e}_j$. Under matching, level 1 reads the same table and its gradient (Equation (35.6)) has the same form, so the two losses cooperate rather than compete for $\mathbf{h}$. The natural entries also receive, for the first time, a training signal: nothing in N001 pulled a pixel toward a natural spectrum.

3. Nature by examples. A new natural spectrum is a new prototype at inference, as for man-made ones; a linear head's idea of nature is frozen in $\mathbf{w}$.

The count offset. If every s_j equals s, then $A = |\mathscr{M}|e^s$ and $B = |\mathscr{N}|e^s$, and by rule 35.4 with $b = 0$, $P(M) = |\mathscr{M}|/(|\mathscr{M}| + |\mathscr{N}|)$. EnMAP's bank in training has 219 natural and $159 - 34 = 125$ man-made entries[C], so a featureless pixel starts at $125/344 = 0.3634$, and $b = \ln(219/125) = 5.3891 - 4.8283 = 0.5608$ would bring it to $\frac{1}{2}$[C]. The learned b can absorb the offset; that is one reason it exists.

35.6.5 What did not change

Natural spectra are planted in both splits, so natural level-1 accuracy is not an unseen-spectrum test; but the planted natural entry is left out of S_{nat}, so a natural mix is never recognised as its own spectrum. At evaluation, planted man-made test spectra are matched only against *training-split* man-made entries. The linear head `to_manmade` is still built; its 257 parameters get no gradient under matching. `load_namer` reads `level1` from the checkpoint's config, so N001's checkpoints still load as linear-head models. The model's docstring, the schematic and the proposal ("Level 1 does not use the library at all") describe the old head.

35.7 The generator as a sampling law

The namer learns only from samples it makes from real tiles (`generator.py`). For a real pixel $\mathbf{b}$ with ring mean $\mathbf{r}$:

$$\mathbf{x} = \mathbf{g} \odot \big(a\mathbf{t} + (1-a)\mathbf{b}\big) + \mathbf{n}, \qquad \mathbf{r}' = \mathbf{g} \odot \mathbf{r}, \tag{35.7}$$

with $\odot$ band-by-band multiplication. Half the samples plant a man-made training-split spectrum, a quarter a natural one, a quarter nothing ($a = 0$); the kind is drawn independently per sample. Compare the generator of Chapter 28: no point-spread function and no halo (the planting is one pixel), the ring is the mean of raw neighbours rather than a model's predicted background $\hat{\mathbf{b}}$, and the fill reaches 1 rather than 0.5.

The ring. The mean of the valid pixels in a 15×15 box minus the central 5×5: the pixels at Chebyshev distance $\max(|\Delta\text{row}|, |\Delta\text{col}|)$ from 3 to 7. The square ring at distance d holds $(2d+1)^2 - (2d-1)^2 = 8d$ pixels, so $8\times(3+4+5+6+7) = 8\times25 = 200$[C], and $15^2 - 5^2 = 225 - 25 = 200$ agrees. (The proposal's "2-7 px" is not what the code does.)

The fill, log-uniform. $a = a_{\min}^{1-u}$ with u uniform and $a_{\min} = 0.05$. Then $\ln a = (1-u)\ln a_{\min}$ is uniform on $[\ln a_{\min}, 0]$, so

$$P(a \le q) = \frac{\ln q - \ln a_{\min}}{0 - \ln a_{\min}} \qquad \textit{length fraction of a uniform}$$
$$= \frac{\ln(q/a_{\min})}{\ln(1/a_{\min})}, \qquad P(a \le 0.25) = \frac{\ln 5}{\ln 20} = \frac{1.6094}{2.9957} = 0.5372.$$

Its density is the derivative in q: $p(a) = \dfrac{1}{a\ln(1/a_{\min})}$ on $[a_{\min}, 1]$ (the derivative of $\ln q$ is $1/q$). Its first two moments:

$$\mathbb{E}[a] = \int_{a_{\min}}^{1} a \cdot \frac{1}{a\ln(1/a_{\min})}\,da = \frac{1-a_{\min}}{\ln(1/a_{\min})} = \frac{0.95}{2.9957} = 0.3171, \qquad \textit{the } a \textit{ cancels}$$

$$\mathbb{E}[a^2] = \int_{a_{\min}}^{1} a^2 \cdot \frac{1}{a\ln(1/a_{\min})}\,da = \frac{1}{\ln(1/a_{\min})}\Big[\frac{a^2}{2}\Big]_{a_{\min}}^{1} = \frac{1-a_{\min}^2}{2\ln(1/a_{\min})} \qquad \textit{one } a \textit{ cancels}$$
$$= \frac{1-0.0025}{2\times2.9957} = \frac{0.9975}{5.9915} = 0.1665,$$
$$\mathrm{Var}(a) = \mathbb{E}[a^2] - \mathbb{E}[a]^2 = 0.1665 - 0.1006 = 0.0659, \qquad \mathrm{sd} = 0.2568.$$

A million draws with the code's formula give mean 0.3173, $\mathbb{E}[a^2] = 0.1666$ and sd 0.2567[C]. More than half the man-made training sits at fills where Table 32.1 shows ACE weakest. The fill head is trained on every sample (target 0 for plain pixels), so its target averages $0.75 \times 0.3171 = 0.2378$[C].

The gain and the noise. $\mathbf{g} = 1 + 0.1\,\mathrm{clip}(c_0 + c_1 t + c_2 t^2, -1, 1)$ over the band index rescaled to $t \in [-1, 1]$, $c_i \sim \mathcal{N}(0, 0.5^2)$: a smooth error within ±10%, applied to the ring too. The noise is Gaussian with a per-tile, per-band σ equal to the root mean square of horizontal neighbour differences over $\sqrt{2}$, because $\mathrm{Var}(n_1 - n_2) = 2\sigma^2$ for independent neighbours (independent variances add, Chapter 10; a difference is a sum with weights 1 and −1, whose squares are both 1). The root mean square is not centred, so texture counts: a noiseless slope rising 0.01 per pixel gives $\sigma = 0.01/\sqrt{2} = 0.00707$[C].

35.8 One training step, as shapes

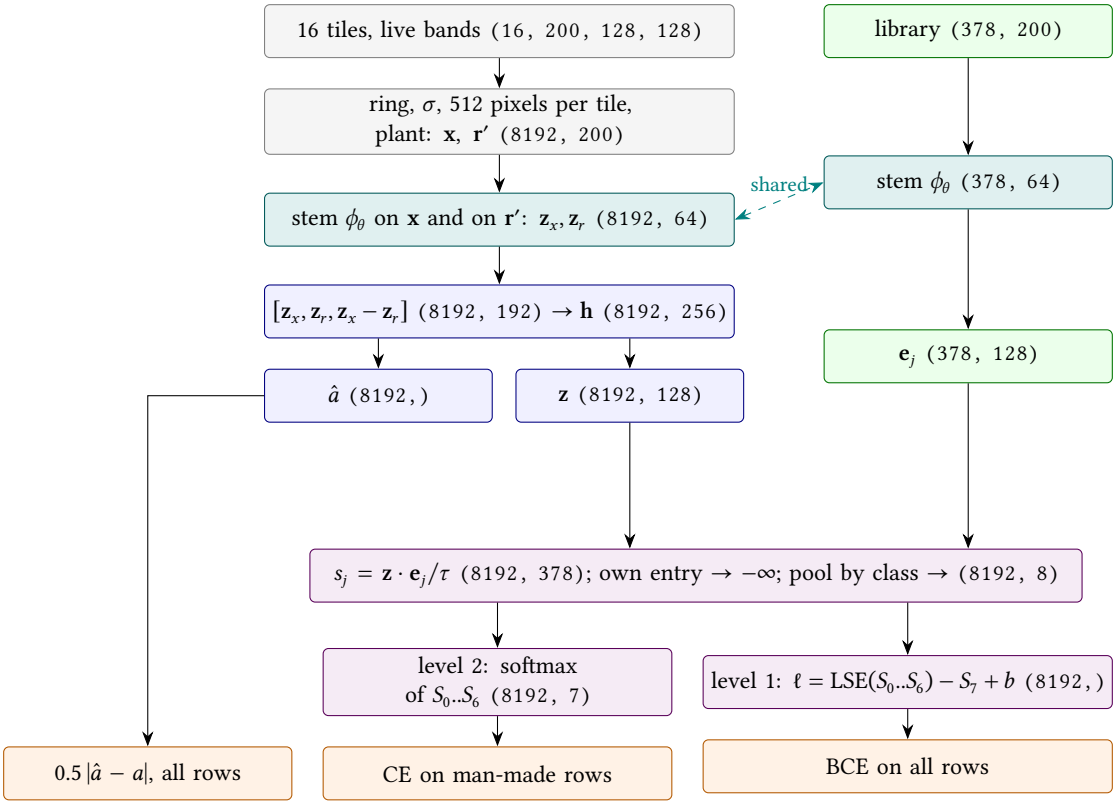


Figure 35.2: Shapes through one training step for one sensor, with level 1 by matching: column 7 of the pooled scores is the natural class. $P = 16 \times 512 = 8192$ samples, $N = 200$ live bands[A], $L = 378$ entries (EnMAP's bank[C]). The stem runs three times per sensor, nine per step; the losses of the three sensors are averaged and back-propagated together.

Worked example 35.7: One row of the score table, both levels

Six entries, scores with τ folded in[A]: plastic 4.0 (planted), plastic 3.0, metal 2.0, concrete 1.0 (test split, no class), soil 2.5, water 0.0. Leaving the planted entry out:

$$S_{\text{plastic}} = 3.0, \quad S_{\text{metal}} = 2.0, \quad S_{\text{nat}} = \ln(e^{2.5} + e^{0}) = \ln 13.1825 = 2.5789,$$
$$\ell = \ln(e^{3} + e^{2}) - 2.5789 = \ln 27.4746 - 2.5789 = 3.3133 - 2.5789 = 0.7344,$$
$$P(M) = \sigma(0.7344) = 1/(1 + 0.4798) = 0.6758, \qquad P(\text{plastic} \mid M) = 20.0855/27.4746 = 0.7311.$$

The test-split entry is encoded but belongs to no class, so it never enters a sum[C].

The total loss and the optimiser. $\mathscr{L} = L_1 + L_2 + 0.5\,L_f$ per sensor, with L_2 over man-made rows only and $L_f = \text{mean}|\hat{a} - a|$ over all rows; the three sensors' losses are each divided by 3 and summed before one AdamW step (weight decay 0.01, gradient norm clipped to 1). The stem and its BatchNorm learn at $0.3 \times 3 \times 10^{-4} = 9 \times 10^{-5}$, everything else at 3×10^{-4}, both times the schedule

$$\lambda(s) = \min\Big(1, \frac{s+1}{500}\Big) \cdot \frac{1}{2}\Big(1 + \cos\frac{\pi \min(s, 20000)}{20000}\Big) :$$

$\lambda(0) = 0.002$, $\lambda(249) = 0.49981$, $\lambda(10000) = \frac{1}{2}(1 + \cos\frac{\pi}{2}) = 0.5$, $\lambda(15000) = \frac{1}{2}(1 - 0.70711) = 0.14645$[C]. 40 epochs of 500 steps, 16 tiles per sensor and 512 pixels per tile: up to $3 \times 8192 = 24{,}576$ samples per step.

35.9 The split as a coin, and the open-set class

A man-made entry is held out when $h/(2^{32} - 1) < 0.2$, h the first 8 hex digits of the SHA-1 of its name. A good hash behaves like an independent coin of probability 0.2 per name, so a class of n spectra has a held-out count with the binomial mean $0.2n$ and standard deviation $\sqrt{n \times 0.2 \times 0.8}$ (Chapter 10). The observed counts on EnMAP's bank (one entry per name; 378 entries, 159 man-made) sit within one standard deviation of that law for every class[C]:

class	n	mean $0.2n$	sd $\sqrt{0.16n}$	held out
oil/fuel	40	8.0	2.53	8
plastic	46	9.2	2.71	8
textile/fabric	14	2.8	1.50	3
wood/paper	11	2.2	1.33	2
paint/coatings	4	0.8	0.80	1
metal	10	2.0	1.26	3
concrete/asphalt	34	6.8	2.33	9
all man-made	159	31.8	5.04	34

So "about 20% per class" is not a promise the hash can keep: 17% to 30% is ordinary for classes this small. For paint, with four spectra, the chance that none would be held out was $0.8^4 = 0.4096$[C]; four spectra are also too few to hold out as a whole class and learn anything from. That is why, on 31 October 2026, the user moved the open-set run N002 from paint/coatings to textile/fabric (14 spectra; `configs/namer/N002.json`). On EnMAP that leaves $125 - 11 = 114$ man-made training entries, and 45 unseen man-made spectra at evaluation (34 held out plus textile's 11 training-split ones)[C]. Pass for N002: textile called man-made at least 70% of the time, given a known class with $P > 0.5$ at most 20%. (The proposal's "As built" and the copy `research/experiments/spectral-namer/N002/config.json` still say paint.)

35.10 The pass test and inference

Level 1 can buy recall by calling more pixels man-made, so the namer is compared with ACE at ACE's own natural false-alarm rate f (one minus the share of natural plantings ACE called natural, Table 32.1): cut the namer's level-1 logits at the $(1 - f)$ quantile of its natural plantings and count the man-made plantings above the cut. At fill 0.25, f is 0.1658 on PRISMA, 0.2149 on EnMAP and 0.1866 on AVIRIS-NG, and the man-made recall must reach ACE's plus 0.10: 0.8724, 0.7635, 0.8603[C]. Level-2 accuracy must also beat ACE by 0.10, after ACE's level 2 has been re-scored on the seven new classes (the N001 README lists that as the first job after launch).

At inference (`infer.py`, `name_pixels`) nothing is left out: the pixel's own spectrum is unknown. The whole library, test split included, is encoded at the scene's bands (bands valid at more than half of the

requested pixels); the ring means are computed band by band with the same box sums; and each pixel gets $P(M) = \sigma(\ell)$ by Equation (35.5) over all man-made and natural entries, the best class with $P(k \mid M)$, and $\hat{a}$. The proposal's combination is then $P(k) = P(M)\,P(k \mid M)$; `scripts/aigc_object_table.py` averages these per detected object.

What the project measured or computed

Built, measured, and not yet known. Settings: `configs/namer/N001b.json` and `N002.json` (40 epochs of 500 steps; AdamW 3×10^{-4}, stem 0.3×; `kinds` 0.5/0.25/0.25; $a_{\min} = 0.05$; gain 0.1; fill weight 0.5; `"level1": "matching"`). Counts[C]: 347,685 parameters (stem 17,825); EnMAP bank 378 entries, 219 natural, 159 man-made, 34 held out. Measured[M]: the CPU smoke run of the first code moved the class cross-entropy from 1.81 to 1.02 with the level-1 cross-entropy still about 0.68 (`research/experiments/spectral-namer/N001/README.md`); N001's epoch 1 and the 150-step laptop comparison above (reported, not yet in a file). Not in the trainer: a bootstrap interval, an "unknown" level-2 output, an unseen-spectrum test of natural recall. Two fixes made before N001 ran (code bundle `namer2`): one library entry per name, and the post-stem BatchNorm frozen at V2's statistics. **N001b's and N002's results were pending when this chapter was written.** Note also that `configs/namer/N001.json` now holds N001b's settings; N001's own config is the copy in `research/experiments/spectral-namer/N001/`.

Common confusions

Is the class loss InfoNCE? It is cross-entropy over classes whose scores are logsumexps of cosines; with one spectrum per class it reduces to InfoNCE.

Is level 1 by matching a new network? No. One new number, b, and a different reading of the same score table. The old linear head is still in the model, unused.

Is $P(M)$ the best man-made cosine against the best natural one? No, it is a share of softmax mass (rule 35.4). Every entry counts, the close ones most, and the side with more entries starts ahead by the log of the count ratio.

Does leaving the planted spectrum out make training harder for no reason? It removes the one shortcut, recognising the sample, so the loss can only fall by learning the material. Under matching it applies to natural plantings too.

Is level 1 the open model of Chapter 32? No. It compares a pixel with the library's man-made and natural spectra; N002 measures how it treats a class it never saw.

Does the namer use the predicted background $\hat{b}$ of Chapter 28? No. It uses a ring of raw neighbours, so the leakage of Chapter 24 does not reach it.

Siamese means two identical towers? Only the stem is shared. After it the pixel tower (ring input, fill head, 214,402 parameters) and the library tower (115,328) differ on purpose.

Check yourself

1. In worked example 35.3, compute $P(\text{glass})$ and the loss with g_1 left out if g_2's cosine were 0.70 instead of 0.90.
2. Show that S_k lies between $\max_{j \in k} s_j$ and $\max_{j \in k} s_j + \ln m$ for a class of m members.
3. With $a_{\min} = 0.1$, compute $P(a \le 0.25)$, $\mathbb{E}[a]$ and $\mathbb{E}[a^2]$.
4. Recompute the gradient of worked example 35.4 with g_1 left out, and check it sums to zero.
5. In worked example 35.5, add a second natural entry, water, at $s = 5.5$. Compute ℓ, $P(M)$, L_1 and the four entry gradients.
6. Use the parameter formula to count a namer with $D = 64$, $H = 128$, $E = 64$.
7. A class of 10 spectra under the 0.2 hash: what is the chance that none is held out? That at least 3 are?

8. Show that $P(M)$ in rule 35.4 does not change if the same constant is added to every s_j, natural and man-made. Why does that matter for τ?

Check yourself: worked answers

1. $s_{g_2} = 0.70/0.5 = 1.40$, $e^{1.40} = 4.0552$. $P(\text{glass}) = 4.0552/(4.0552 + 4.9530 + 3.3201) = 4.0552/12.3283 = 0.3289$; $L = -\ln 0.3289 = 1.1119$. Painted metal now takes the most probability, and the loss tells the encoders to bring the two glasses together.

2. Every term $e^{s_j} \le e^{s_{\max}}$, so $\sum_{j \in k} e^{s_j} \le m\, e^{s_{\max}}$ and $S_k \le s_{\max} + \ln m$; the sum includes $e^{s_{\max}}$ itself, so $S_k \ge \ln e^{s_{\max}} = s_{\max}$.

3. $P(a \le 0.25) = \ln 2.5 / \ln 10 = 0.9163/2.3026 = 0.3979$. $\mathbb{E}[a] = (1 - 0.1)/\ln 10 = 0.9/2.3026 = 0.3909$. $\mathbb{E}[a^2] = (1 - 0.01)/(2 \times 2.3026) = 0.99/4.6052 = 0.2150$.

4. With g_1 out, glass has one member, $w_{g_2} = 1$; over g_2, metal, tarpaulin $q = (0.4224,\ 0.3458,\ 0.2318)$, so the gradient is $(0.4224 - 1,\ 0.3458,\ 0.2318) = (-0.5776,\ 0.3458,\ 0.2318)$, sum 0[C].

5. $S_{\text{nat}} = \ln(2e^{5.5}) = 5.5 + \ln 2 = 6.1931$; $\ell = 6.3133 - 6.1931 = 0.1201$ (from unrounded values); $p = 1/(1 + e^{-0.1201}) = 1/1.8868 = 0.5300$; $L_1 = -\ln 0.5300 = 0.6349$; $p - y = -0.4700$; gradients plastic $-0.4700 \times 0.7311 = -0.3436$, metal $-0.4700 \times 0.2689 = -0.1264$, soil and water $+0.4700 \times 0.5 = +0.2350$ each; sum 0[C]. The extra natural entry lowered $P(M)$ from 0.6928 to 0.5300 without any score changing: the count offset.

6. Towers: $4 \times 64 \times 128 = 32{,}768$; $3 \times 128^2 = 49{,}152$; $7 \times 128 = 896$; $2 \times 128 \times 64 = 16{,}384$; $2 \times 64 + 2 = 130$; sum 99,330. Plus stem 17,825, BatchNorm 128, scalars 2: 117,285, as the code reports[C].

7. None: $0.8^{10} = 0.1074$. At least 3: $1 - P(0) - P(1) - P(2)$ with $P(1) = 10 \times 0.2 \times 0.8^9 = 10 \times 0.2 \times 0.1342 = 0.2684$ and $P(2) = 45 \times 0.04 \times 0.8^8 = 45 \times 0.04 \times 0.1678 = 0.3020$, so $1 - 0.1074 - 0.2684 - 0.3020 = 0.3222$. Metal's 3 of 10 is not unusual.

8. Adding k to every s_j multiplies A and B by e^k: $\dfrac{e^b A e^k}{e^b A e^k + B e^k} = \dfrac{e^b A}{e^b A + B}$. Only differences between scores matter. Dividing by τ is not such a shift: it scales the differences, which is why τ has a gradient (worked example 35.5) while a common offset would not.

35.11 Where Part VII leaves the namer

A two-level namer exists in code, and both levels are now matching: level 2 over seven material classes, level 1 between the man-made and the natural shelves of the same library, both judged against the classical baseline of Table 32.1 at ACE's own false-alarm rate. What it has not done yet, when this book was last updated, is pass, or fail: N001 was stopped for a level 1 at chance, and N001b and N002 were training. The real-object benchmark of Chapter 29 remains the final judge, and the generator gap of Chapter 28 still applies in full, because the planted samples are library spectra mixed linearly into real pixels.

Glossary

Each entry gives a plain-language meaning and the chapter where the term is introduced.

abstention
Declining to name a family when the evidence is weak; reporting "unconfirmed" instead. Chapter 27.

ACE (adaptive coherence estimator)
The squared cosine between the whitened residual and the whitened target direction; the GLRT when the clutter's scale is unknown. Chapter 20.

affine map
A matrix followed by a fixed shift, $\mathbf{A}\mathbf{x} + \mathbf{c}$. Chapter 15.

band
One narrow slice of wavelength measured by the sensor. Chapter 1.

basis
A set of vectors whose combinations make up a subspace, none of them a combination of the others. Chapter 9.

Bayes' rule
Posterior is prior times likelihood, divided by the total. Chapter 27.

bootstrap
Estimating uncertainty by recomputing a number on many resamples drawn with replacement. Chapter 29.

CFAR
Constant false-alarm rate: a test whose false-alarm rate does not depend on the noise level. Chapter 20.

checkerboard pass
The product's way of predicting every pixel: hide alternate 4×4-pixel tokens, predict them, then swap. Chapter 16.

chi-square distribution
The distribution of a sum of q squared independent standard normals; mean q. Chapter 11.

Chow's rule
Name the most probable class if its posterior is at least $1 - c$, else abstain. Chapter 27.

closed classifier
A classifier with a fixed list of classes, one output each; a new class needs a new output and retraining. Chapter 32.

clutter-limited
Limited by how well the background is predicted, not by sensor noise. Chapter 16.

conditional probability
$P(A \mid B) = P(A \text{ and } B)/P(B)$: among the cases where B happens, the share where A does too. Chapter 31.

cone (family cone)
All non-negative combinations of a family's whitened differences; the set Simplex ACE projects onto. Chapter 22.

confusion matrix
Shares of each true family's objects given each answer. Chapter 29.

class score
The logsumexp of a class's scaled cosines, $S_k = \ln \sum_{j \in k} e^{z_j}$; its softmax is the summed softmax of the class's members. Chapter 35.

contrastive loss (InfoNCE)
Minus the log of the true candidate's softmax probability over a set of candidates scored by cosines. Chapter 34.

covariance
Average product of two bands' deviations; positive when they rise and fall together. Chapter 5.

covariance matrix
The matrix of all variances (diagonal) and covariances (off-diagonal). Chapter 5.

Cramér-Rao bound
No unbiased estimate can have variance below one over the Fisher information. Chapter 18.

cross-entropy
Classification loss $-\ln q$(true class). Chapter 28.

derivative
The slope of a curve at a point. Chapter 8.

design effect
How much clustering of objects in scenes reduces the effective sample size, $1 + (m-1)\rho$. Chapter 30.

detection rate P_D
Share of target pixels flagged. Chapter 11.

determinant
For $\left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)$, $ad - bc$; zero means no inverse. Chapter 4.

discordance π_d
Share of objects on which exactly one of two methods is right. Chapter 30.

dot product
Multiply two vectors band by band and add. Chapter 2.

dual encoder
A matching model with one encoder for pixels and one for library spectra, scored by the cosine of their embeddings. Chapter 33.

eigenvector, eigenvalue
A direction a matrix only stretches, and the stretch factor; for a covariance, the variance along that direction. Chapter 6.

embedding
A short vector a network computes from its input, arranged so that comparisons become geometry. Chapter 33.

energy share
An eigenvalue divided by the sum of all eigenvalues. Chapter 6.

exchangeable
A pixel with no reason to rank differently from the null pixels. Chapter 27.

expectation $\mathbb{E}$
Long-run average. Chapter 10.

explained / unexplained energy
Squared lengths of the projection and of the remainder. Chapter 9.

false-alarm rate P_{FA}
Share of background pixels flagged. Chapter 11.

family
A group of library spectra of one kind of material (metal, plastic, roofing, …). Chapter 21.

fill
The share of a pixel's signal that comes from the target. Chapter 12.

Fisher information
The variance of the score; how sharply the log-likelihood is peaked. Chapter 18.

foreignness φ
The share of a target difference's energy outside the background error's shared directions. Chapter 19.

FWHM
Full width at half maximum of a curve; 2.3548σ for a Gaussian. Chapter 13.

gain, offset
Per-band multiplicative and additive errors, here from atmospheric correction. Chapter 25.

Gaussian (normal) distribution
The bell curve; $\mathcal{N}(m, \sigma^2)$. Chapter 10.

GLRT
Generalised likelihood ratio test: plug in the best-fitting unknowns under each hypothesis and compare. Chapter 11.

gradient
The vector of partial derivatives. Chapter 8.

hypothesis (H_0, H_1)
"Nothing there" and "something there". Chapter 11.

identity matrix
The matrix that changes nothing. Chapter 4.

in-batch negatives
The other true spectra of a training batch, used as each pixel's wrong candidates. Chapter 34.

Indradhanu (Heavy)
The project's hyperspectral foundation model that predicts the background; Heavy is its larger release candidate. Chapter 16.

integral
Area under a curve. Chapter 10.

interference subspace
Directions of nuisances (brightness, offset) to be removed before testing. Chapter 22.

inverse
The matrix that undoes another. Chapter 4.

Jacobian (leakage)
How much the prediction at one pixel changes per unit change of a visible pixel. Chapter 24.

leakage
The background prediction partly copying the target. Chapter 24.

leave-one-spectrum-out
Matching a planted library spectrum against every other library spectrum, never itself. Chapter 33.

leave the planted spectrum out
Set the planted library entry's score to $-\infty$ in its own class score, in training and evaluation, so recognising the sample earns nothing. Chapter 35.

level 1, level 2
The two questions of the redesigned namer: could nature explain the pixel; if not, which man-made class. Chapter 31.

level 1 by matching
The built namer's natural-or-man-made answer since run N001b: the natural library entries are pooled as one more class and $P(M) = \sigma(\ln \sum_{\mathcal{M}} e^{s_j} - \ln \sum_{\mathcal{N}} e^{s_j} + b)$, the share of softmax mass on man-made entries. Chapter 35.

likelihood
How well a parameter value explains the observed data. Chapter 10.

Lipschitz bound
$\|g(\mathbf{t}) - g(\mathbf{t}')\| \le L\|\mathbf{t} - \mathbf{t}'\|$: close inputs give close outputs. Chapter 33.

logarithm ln
The power of e that gives a number; turns products into sums. Chapter 10.

Mahalanobis distance
Squared length after whitening, $\mathbf{r}^\top \mathbf{S}^{-1}\mathbf{r}$. Chapter 7.

matched filter
The fill estimate divided by its standard deviation. Chapter 19.

matched subspace detector
The GLRT for a signal in a subspace with nuisances removed. Chapter 22.

matrix
A rectangular table of numbers acting as a machine on vectors. Chapter 4.

mixing model
$\mathbf{x} = a\mathbf{t} + (1 - a)\mathbf{b} + \mathbf{n}$. Chapter 12.

norm (length)
Square root of the sum of squared components. Chapter 2.

normalisation by wavelength
Per-band standard scores with fixed global mean and standard-deviation curves. Chapter 15.

open-set recognition
Recognition built to say "none of the known kinds" for inputs from classes absent in training. Chapter 32.

orthogonal
Perpendicular; dot product zero. Chapter 2.

pAUC
Area under the ROC up to a 1% false-alarm rate, divided by 0.01. Chapter 11.

point-spread function
How a sensor spreads the light of one ground point over nearby pixels. Chapter 13.

posterior, prior
Probability after and before seeing the data. Chapter 27.

principal angle
The smallest angle between any vectors of two subspaces. Chapter 9.

principal components (PCA)
The eigenvectors of a covariance, in order of eigenvalue. Chapter 6.

projection
The closest point in a line or subspace. Chapter 9.

prototype
A stored example in an embedding space that an input is compared with; under level 1 by matching every library spectrum is one. Chapter 35.

quadratic form
$\mathbf{u}^\top \mathbf{A}\mathbf{u}$. Chapter 4.

rank
Number of independent directions a matrix or data set uses. Chapter 6.

reflectance
Fraction of incoming sunlight a surface reflects in a band. Chapter 1.

resampling
Converting a lab spectrum to sensor bands by weighted averaging. Chapter 14.

residual
Pixel minus predicted background, $\mathbf{r} = \mathbf{x} - \hat{\mathbf{b}}$. Chapter 16.

residual Mahalanobis (RM)
The product's anomaly score: whitened squared length of the locally centred residual, 30 components. Chapter 17.

ridge
A small number added to eigenvalues before dividing by them. Chapter 7.

ROC curve
Detection rate against false-alarm rate as the threshold varies. Chapter 11.

SAM
Spectral angle mapper: name a pixel after the library spectrum at the smallest angle. Chapter 3.

score (statistics)
The slope of the log-likelihood. Chapter 18.

Siamese network
A network that runs two inputs through the same weights and compares the outputs; the namer's stem is shared this way by the pixel, its ring mean and every library spectrum. Chapter 35.

Simplex ACE
ACE against the best non-negative mixture of a family's spectra. Chapter 22.

SNR (detection)
Squared shift of the test statistic under H_1, $a^2 \mathbf{d}^\top \mathbf{S}^{-1} \mathbf{d}$. Chapters 11 and 19.

softmax, temperature
$p_j = e^{s_j/\tau} / \sum_i e^{s_i/\tau}$; a small temperature τ sharpens the probabilities. Chapter 34.

spectrum
A surface's list of reflectances, one per band. Chapter 1.

SRF
Spectral response function of a band. Chapter 14.

standard deviation, variance
Typical size of a wobble, and its square. Chapter 5.

standard score (z-score)
Distance from the mean in standard deviations. Chapter 7.

subspace, span
All combinations of some vectors; a flat slab through the origin. Chapter 9.

sufficient statistic
A summary of the data that carries everything relevant to the unknowns. Chapter 28.

threshold η
The bar a test statistic must exceed. Chapter 11.

token
A 4×4-pixel tile, the unit Indradhanu hides and predicts. Chapter 16.

total variation distance
Half the sum of absolute differences between two distributions. Chapter 28.

transpose
Swap rows and columns. Chapter 4.

two-tower model
Two encoders whose outputs are compared, usually by a cosine; another name for a dual encoder, used when the towers differ. Chapter 35.

unbiased
Right on average. Chapter 18.

unexplained score
Level 1's measure of how much of a pixel nature cannot account for. Chapter 32.

union bound
The chance of at least one event is at most the sum of their chances. Chapter 27.

vector
An ordered list of numbers. Chapter 1.

whitening
Rotating onto the principal axes and dividing each by its standard deviation. Chapter 7.

Symbol table

symbol	meaning	first in
N	number of bands (165 on the common grid)	Chapter 1
n	band index; also a sample size (Chapter 30)	Chapter 1
$p,\ q$	pixel indices	Chapter 12
$\mathbf{x},\ \mathbf{x}_p$	pixel spectrum	Chapter 1
$\mathbf{t}$	target (library) spectrum	Chapter 1
$\mathbf{b},\ \mathbf{b}_p$	true local background	Chapter 12
$\hat{\mathbf{b}},\ \hat{\mathbf{b}}_p$	predicted background (Indradhanu)	Chapter 16
$a,\ a_p$	fill	Chapter 12
$\hat{a}$	fill estimate	Chapter 18
$\mathbf{n}$, Σ_n	sensor noise and its covariance	Chapter 12
$\mathbf{r}$, Σ_r	residual $\mathbf{x}-\hat{\mathbf{b}}$ and its covariance	Chapter 16
$\mathbf{e}$, Σ_e	background-prediction error $\mathbf{b}-\hat{\mathbf{b}}$ and its covariance	Chapter 16
$\mathbf{d}$	$\mathbf{t}-\hat{\mathbf{b}}$, target minus predicted background	Chapter 16
$\mathbf{w}$	everything in the residual except the target term	Chapter 18
$\mathbf{S}$	covariance of $\mathbf{w}$ (in practice $\approx \Sigma_r$); $\mathbf{S}_0$ its known shape	Chapters 18 and 20
$\mathbf{u}^\top,\ \mathbf{A}^\top$	transpose	Chapter 4
$\mathbf{A}^{-1}$	inverse	Chapter 4
$\mathbf{I}$	identity matrix	Chapter 4
$\lVert\mathbf{u}\rVert$	length	Chapter 2
$\langle\mathbf{u},\mathbf{v}\rangle_{\mathbf{S}},\ \lVert\mathbf{u}\rVert_{\mathbf{S}}$	whitened dot product $\mathbf{u}^\top\mathbf{S}^{-1}\mathbf{v}$ and length	Chapter 7
$\mathbf{U},\ \mathbf{\Lambda},\ \lambda$	eigenvectors (columns), eigenvalues (diagonal), one eigenvalue	Chapter 6
$\mathbf{W}$	whitening matrix $\mathbf{\Lambda}^{-1/2}\mathbf{U}^\top$ (truncated to d rows in RM)	Chapter 7
$\tilde{}$ (tilde)	a whitened quantity	Chapter 9
d	number of components RM keeps (30)	Chapter 17
$\mathbf{H},\ \mathbf{H}_k$	signal subspace; family k's spectra as columns	Chapters 9 and 21
$\mathbf{J}$	interference subspace (Chapter 22); Jacobian $\mathbf{J}_{pq}$ (Chapter 24)	
$\mathbf{P},\ \mathbf{P}^\perp$	projection and projection away	Chapter 9
$\boldsymbol{\theta},\ \boldsymbol{\psi}$	signal and interference amplitudes	Chapters 9 and 22
$q,\ q_k$	dimension of a signal subspace / family	Chapters 11 and 21
$\mathbf{D},\ \boldsymbol{\mu}$	per-band standard deviations (diagonal) and means	Chapter 15
$\mathbf{z}$	normalised spectrum $\mathbf{D}^{-1}(\mathbf{x}-\boldsymbol{\mu})$; in Chapter 22 a whitened difference	Chapter 15
$\sigma,\ \sigma^2$	standard deviation, variance	Chapter 5
σ_{PSF}	PSF standard deviation (1.4 px)	Chapter 13
$\varsigma_n,\ \mathcal{w}_n$	SRF width and FWHM of band n	Chapter 14
$\varphi(z),\ \Phi(z),\ Q(z)$	standard normal density, left area, right area	Chapter 10
$\mathcal{N}(m,\sigma^2)$	Gaussian distribution	Chapter 10
$\mathbb{E},\ \mathrm{Var},\ \mathrm{Cov}$	expectation, variance, covariance	Chapter 10
ℓ	log-likelihood	Chapter 10
$H_0,\ H_1$	null and alternative hypotheses	Chapter 11
$T,\ \eta$	test statistic, threshold	Chapter 11

symbol	meaning	first in
P_{FA}, P_D	false-alarm and detection rates	Chapter 11
μ	shift of the test statistic under H_1	Chapter 11
Λ	likelihood ratio	Chapter 11
χ^2_q, δ	chi-square with q degrees of freedom; noncentrality	Chapter 11
$\mathcal{J}$	Fisher information $\mathbf{d}^\top \mathbf{S}^{-1}\mathbf{d}$	Chapter 18
$a_{\min}$	smallest detectable fill	Chapter 19
ε, k, λ_{in}, λ_{out}	two-level covariance parameters	Chapter 19
φ (foreignness)	energy share outside the shared error directions	Chapter 19
θ_k, E_k^{expl}, E_k^{unex}	projection angle and energies for family k	Chapter 21
g, c, $\mathbf{1}$	target gain, offset, all-ones vector	Chapter 22
l, $\mathbf{L}_p$	leakage fraction and leakage matrix	Chapter 24
$\mathbf{Q}$, D	projection away from the decoder's range; code width	Chapter 24
$\mathbf{G}$, $\mathbf{o}$, κ	gain matrix, offset, largest-over-smallest gain	Chapter 25
π_k, $\pi_k(p)$	prior of class k; calibrated score of pixel p (context tells which)	Chapter 27
N_0, α, F, p^*	null sample size, false-name target, number of families, bar	Chapter 27
$\boldsymbol{\upsilon}$	the namer's input	Chapter 28
π_d, Δ	discordance, accuracy difference	Chapter 30
ϵ_L, K	wrong-tag rate, number of families	Chapter 29
M, $P(M)$, $P(k \mid M)$	man-made; level 1's and level 2's probabilities	Chapter 31
τ_1, τ_2	level-1 and level-2 thresholds	Chapter 31
u	level 1's unexplained score	Chapter 32
f, g; $\boldsymbol{\zeta}$, $\boldsymbol{\omega}_j$	pixel and spectrum encoders; their embeddings	Chapter 33
$\mathbf{V}$	a closed classifier's class matrix	Chapter 33
s_j, τ, p_j, L	cosine score, temperature, softmax probability, contrastive loss	Chapter 34
s_j, S_k, q_j, w_j	scaled score $\cos/\tau$ (Chapter 35 only), class score, entry softmax, within-class weight	Chapter 35
ℓ, b, A, B, u_j, v_j	level-1 logit, its bias, man-made and natural softmax mass, within-side weights	Chapter 35
ϕ_θ, D, H, E	the shared stem; stem, hidden and embedding widths	Chapter 35
$a_{\min}$	smallest planted fill (0.05)	Chapter 35

Index of worked examples

1.1 A metal roof in half shadow . . . 4
1.2 A pixel that is 20% metal and 80% grass . . . 4
1.3 Metal minus grass, and the mixed pixel minus grass . . . 5
2.1 The dot product of metal and grass . . . 8
2.2 The lengths of metal and grass . . . 8
2.3 Checking the rule on metal and grass . . . 9
3.1 The angle between metal and grass . . . 13
3.2 Brightness does not change the angle . . . 13
3.3 SAM on a soil-like pixel . . . 14
4.1 A 2×2 machine . . . 16
4.2 Products in both orders . . . 17
4.3 Inverting and undoing . . . 19
4.4 A quadratic form two ways . . . 19
5.1 Variance of band 1 . . . 22
5.2 Covariance of band 1 with band 2 . . . 22
5.3 Building $\mathbf{S}$ from outer products . . . 23
5.4 Checking the rule on the four pixels . . . 24
5.5 Covariance of the sum and the difference together . . . 24
6.1 Eigenvalues and eigenvectors of $\mathbf{S}$ . . . 27
6.2 Variance along the long axis, from the data . . . 27
6.3 A two-spectrum family is nearly one direction . . . 29
7.1 Whitening the pixel $(2,\ 1)$ by hand . . . 31
7.2 Two residuals of equal length . . . 33
8.1 The bottom of a quadratic . . . 37
8.2 A minimum in two unknowns . . . 38
8.3 The best variance . . . 39
9.1 Projecting $(3, 1)$ onto the direction $(1, 1)$ . . . 42
9.2 Projecting onto a plane in three bands . . . 43
10.1 How often does noise exceed 0.3? . . . 47
10.2 From a mean absolute error to a standard deviation . . . 48
10.3 Which mean explains three readings best? . . . 49
11.1 False-alarm and detection rates . . . 51

11.2 The required SNR of 9.55 . . . 52
11.3 The GLRT for an unknown mean in one band . . . 54
12.1 A noisy mixed pixel . . . 57
13.1 How a point at a pixel centre shares its light along a row . . . 61
13.2 The 7.5% figure . . . 61
13.3 The halo weight of a one-pixel object, by hand . . . 62
14.1 A coarse resampling by hand . . . 65
14.2 Mix then resample, or resample then mix . . . 65
15.1 Normalising one value, and interpolating the curves . . . 68
15.2 Checking it with numbers . . . 68
16.1 Both forms on three bands . . . 73
17.1 Two residuals, and what truncation hides . . . 76
17.2 What knowing the family is worth . . . 77
18.1 The fill estimate on three bands, checked by trying values . . . 81
18.2 The precision of the three-band estimate . . . 82
18.3 The upward bias of clipping at zero fill . . . 83
18.4 A darker deployed material . . . 84
19.1 The matched filter on three bands . . . 86
19.2 Smallest detectable fill on three bands . . . 87
19.3 The smallest detectable fill of metal over grass, middle error level . . . 88
20.1 ACE on three bands . . . 92
21.1 The energy split on the plane of worked example 9.2 . . . 96
21.2 The share of energy that separates two families . . . 97
22.1 An offset as interference, three bands . . . 100
22.2 Checking the identity with numbers . . . 100
22.3 The cone in two whitened dimensions . . . 101
23.1 The toy: 20% metal over grass . . . 104
23.2 Re-deriving the note's angles at 10% and 50% fill . . . 104
24.1 Thirty per cent leakage at 20% fill . . . 109
24.2 Leakage of two linear predictors . . . 111
24.3 Should FT08's 8-pixel blocks have helped? . . . 112
24.4 A one-dimensional decoder in three bands . . . 112
25.1 The 2.7° bound at $\kappa = 1.1$ . . . 115
26.1 The threshold under a t_3 null . . . 118
26.2 Putting two measured factors together . . . 119
27.1 Rare classes need strong evidence . . . 123
27.2 A calibrated score from a tiny null . . . 124
27.3 The bound with ten null pixels . . . 125
28.1 The best guess is the true probability . . . 129
28.2 Correcting a namer for real-world priors . . . 130

29.1 One row of a confusion matrix . . . 132
29.2 Ten per cent wrong tags . . . 133
29.3 A paired bootstrap on six objects . . . 133
30.1 Sizing a family benchmark . . . 137
31.1 A confident level 1, then a doubtful one . . . 142
32.1 A camouflage net between the natural samples . . . 145
32.2 A 90th-percentile cut on ten natural scores . . . 147
33.1 A new library entry, no retraining . . . 150
33.2 Whitened ACE at two fills, then with noise . . . 151
34.1 Three candidates at three temperatures . . . 154
34.2 The pull and the pushes . . . 154
34.3 Two pixels, two spectra . . . 155
35.1 Two bins of two bands, $d = 2$ . . . 161
35.2 A masked bin cannot leak . . . 161
35.3 Glass with a second glass spectrum . . . 163
35.4 Shared pull . . . 163
35.5 Level 1 by matching with three entries . . . 166
35.6 Four points no line separates . . . 166
35.7 One row of the score table, both levels . . . 168

Annotated bibliography

The references the original note relies on, with what each is for. Details are copied from the original note; where it was unsure, so is this list.

Kelly 1986; Robey et al. 1992
E. J. Kelly, "An adaptive detection algorithm," *IEEE Trans. Aerospace and Electronic Systems* 22(1):115–127, 1986. F. C. Robey, D. R. Fuhrmann, E. J. Kelly, R. Nitzberg, "A CFAR adaptive matched filter detector," *IEEE Trans. AES* 28(1):208–216, 1992. The matched filter with an estimated covariance (Chapter 19).

Kraut and Scharf 1999
"The CFAR adaptive subspace detector is a scale-invariant GLRT," *IEEE Trans. Signal Processing* 47(9):2538–2541. ACE as a GLRT and its Beta null (Chapter 20).

Kraut, Scharf and McWhorter 2001
"Adaptive subspace detectors," *IEEE Trans. Signal Processing* 49(1):1–16. The subspace version of ACE (Chapter 22).

Scharf and Friedlander 1994
"Matched subspace detectors," *IEEE Trans. Signal Processing* 42(8):2146–2157. Signal plus interference subspaces (Chapter 22).

Ziemann and Theiler 2017
"Simplex ACE: a constrained subspace detector," *Optical Engineering*, 2017. Cited by `family_match.py` and the LLD; volume, issue and article number not verified (Chapter 22).

Reed and Yu 1990
"Adaptive multiple-band CFAR detection of an optical pattern with unknown spectral distribution," *IEEE Trans. ASSP* 38(10):1760–1770. The RX detector that RM descends from (Chapter 17).

Kruse et al. 1993
"The spectral image processing system (SIPS)," *Remote Sensing of Environment* 44(2–3):145–163. The spectral angle mapper (Chapters 3 and 23).

Manolakis and Shaw 2002; Manolakis, Marden and Shaw 2003; Manolakis, Lockwood and Cooley 2016
Reviews and a textbook of hyperspectral detection: *IEEE Signal Processing Magazine* 19(1):29–43; *Lincoln Laboratory Journal* 14(1):79–116; *Hyperspectral Imaging Remote Sensing*, Cambridge University Press. The framework of Part IV.

Kay 1993/1998
S. M. Kay, *Fundamentals of Statistical Signal Processing*, vols. I (Estimation) and II (Detection), Prentice Hall. Cramér-Rao bound, GLRT, chi-square tables (Chapters 11 and 18).

Theiler and Foy 2006
"Effect of signal contamination in matched-filter detection of the signal on a cluttered background," *IEEE GRSL* 3(1):98–102. The covariance analogue of leakage (Chapter 24).

Chow 1970
"On optimum recognition error and reject tradeoff," *IEEE Trans. Information Theory* 16(1):41–46. Abstention (Chapter 27).

Vovk, Gammerman and Shafer 2005
Algorithmic Learning in a Random World, Springer. The percentile calibration is a conformal p-value (Chapter 27).

Scheirer et al. 2013
"Toward open set recognition," *IEEE Trans. PAMI* 35(7):1757–1772. Open-set naming (Chapters 27 and 32).

Oord, Li and Vinyals 2018
"Representation learning with contrastive predictive coding," arXiv:1807.03748. The InfoNCE loss (Chapter 34).

Bromley et al. 1993
J. Bromley, I. Guyon, Y. LeCun, E. Säckinger and R. Shah, "Signature verification using a 'Siamese' time delay neural network," *Advances in Neural Information Processing Systems 6* (NIPS 1993). The Siamese network (Chapter 35); page numbers not verified.

Snell, Swersky and Zemel 2017
"Prototypical networks for few-shot learning," *Advances in Neural Information Processing Systems 30* (NIPS 2017). Naming by distance to prototypes in an embedding, the idea behind level 1 by matching (Chapter 35); page numbers not verified.

Radford et al. 2021
"Learning transferable visual models from natural language supervision" (CLIP), *Proc. ICML 2021.* A dual encoder trained contrastively with in-batch negatives (Chapters 33 and 34); page numbers not verified.

Saerens, Latinne and Decaestecker 2002
"Adjusting the outputs of a classifier to new a priori probabilities," *Neural Computation* 14(1):21–41. Label-shift correction (Chapter 28).

Efron and Tibshirani 1993
An Introduction to the Bootstrap, Chapman & Hall (Chapter 29).

McNemar 1947
"Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika* 12(2):153–157 (Chapter 30).

Index of defined terms

addition, 4
affine, 68
alternative, 51

background-prediction error, 72
band, 2
basis, 43
bootstrap, 133

calibrated score, 124
class score, 162
closed classifier, 145
column, 16
component, 3
conditional probability, 142
confusion matrix, 132
contrastive loss, 153
correlation, 22
cosine, 12
covariance, 22
covariance matrix, 22
coverage, 132
cross-entropy, 129

derivative, 36
detection rate, 51
deviation, 21
diagonal matrix, 17
dimension, 3, 43
distance, 9
dot product, 7
dual encoder, 150

eigenvalue, 26
eigenvector, 26
embedding, 149
energy share, 28
exchangeable, 124
expectation, 46
explained energy, 42

false-alarm rate, 51
fill, 4, 57
Fisher information, 82
foreignness, 87

Gaussian, 47
generalised likelihood ratio test, 53
generator, 128

identity, 17
in-batch negatives, 155
independent, 46
index, 3
integral, 47
interference subspace, 99
inverse, 18

leave-one-spectrum-out, 151
length, 8
level 1, 141
level 2, 141
likelihood, 49
linear combination, 4
Lipschitz, 150
local background, 57
log-likelihood, 49
logsumexp, 162

Mahalanobis distance, 33
matrix, 16
maximum-likelihood estimate, 49
mean, 21
mean absolute value, 10
mean vector, 21
mixture, 4

natural logarithm, 49
noncentral chi-square, 54
noncentrality, 54
norm, 8
normalised spectrum, 67
null hypothesis, 51

open-set recognition, 145

orthogonal, 9
outer product, 23

paired, 133
partial area under the curve, 53
per-family accuracy, 132
point-spread function, 60
posterior, 123
principal angle, 44
principal components, 28
prior, 123
probability, 46
projection, 41
projection angle, 42

quadratic form, 19

random variable, 46
range, 43
rank, 28
reflectance, 2
residual, 72
ROC curve, 53
root mean square, 10

scalar, 4
scalar multiplication, 4
score, 82
second derivative, 36
sensor noise, 57
Siamese network, 158
signal subspace, 99
sine, 12
softmax, 153
span, 43
spectral angle mapper, 13
spectral response function, 64
spectrum, 2
standard deviation, 22
standard normal, 47
standard score, 31
subspace, 43
subtraction, 4
symmetric, 17

tail probability, 47
target, 57
temperature, 153
test statistic, 51
threshold, 51
transpose, 17
two-tower model, 158

unexplained energy, 42
unexplained score, 146
unit vector, 9

variance, 22
variances add, 46
vector, 3

wavelength, 2
whitening matrix, 32

z-score, 31